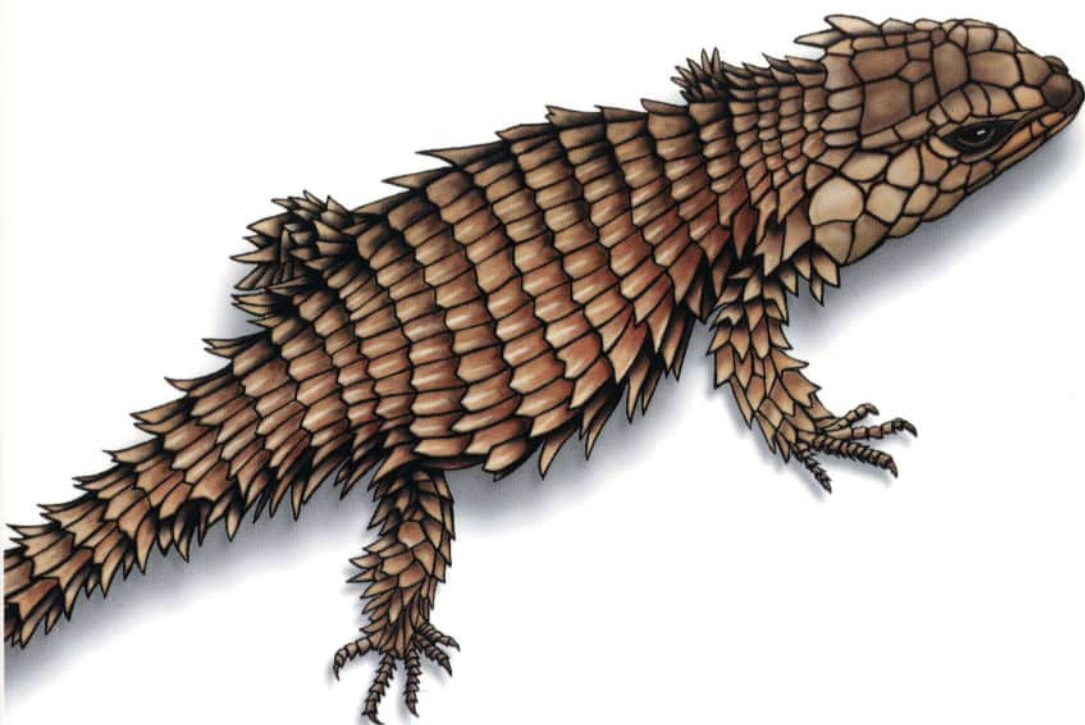


O'REILLY®

Проектирование архитектуры API

Как правильно проектировать,
развивать и эксплуатировать API



alist

Джеймс Гоф,
Дэниэл Брайант,
Мэтью Оберн

Mastering API Architecture

Design, Operate, and Evolve API-Based Systems

James Gough, Daniel Bryant, and Matthew Auburn

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY

Джеймс Гоф, Дэниэл Брайант, Мэтью Оберн

Проектирование архитектуры API

Астана
«АЛИСТ»
2024

УДК 004.438.5
ББК 32.973.26-018.2
Г74

Гоф Дж.

Г74 Проектирование архитектуры API: Пер. с англ. / Дж. Гоф, Д. Брайант, М. Оберн — Астана: АЛИСТ, 2024. — 288 с.: ил.
ISBN 978-601-09-5053-5

Фундаментальная книга о разработке и реализации API (программных интерфейсов приложений). Разобраны базовые вопросы обмена информацией в микросервисной архитектуре, обработка запросов на сайтах и в веб-приложениях (парадигма REST). Показано, как поступательно развивать имеющиеся API, не переписывая их, а также как создать API любой сложности с нуля с учетом возможностей и ограничений конкретной системы. Книга поможет реализовать на предприятии архитектуру сервисной сети и подготовить ресурсы компании к миграции в облако.

Для архитекторов программного обеспечения

УДК 004.438.5
ББК 32.973.26-018.2

© 2024 ALIST LLP

Authorized Russian translation of the English edition of *Mastering API Architecture*. (ISBN 9781492090632)

© 2023 James Gough Ltd, Big Picture Tech Ltd, and Matthew Auburn Ltd.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Авторизованный перевод с английского языка на русский издания *Mastering API Architecture*.

(ISBN 9781492090632) © 2023 James Gough Ltd, Big Picture Tech Ltd, Matthew Auburn Ltd.

Перевод опубликован и продается с разрешения компании-правообладателя O'Reilly Media, Inc.

ISBN 978-1-492-09063-2 (англ.)
ISBN 978-601-09-5053-5 (каз.)

© James Gough Ltd, Big Picture Tech Ltd, Matthew Auburn Ltd, 2023
© Издание на русском языке: ТОО "АЛИСТ", 2024

Оглавление

https://t.me/it_boooks/2

Вступительное слово.....	15
Предисловие	17
Для чего мы написали эту книгу?	17
Почему стоит прочитать эту книгу?	17
Для кого эта книга?	18
Разработчик.....	18
Случайный архитектор	18
Архитектор решений, корпоративный архитектор	19
Чему вы научитесь?.....	19
О чем не расскажет эта книга?	20
Условные обозначения.....	20
Использование примеров кода	21
Платформа онлайн-обучения O'Reilly.....	22
Как с нами связаться?.....	22
Благодарности.....	22
Благодарности Джеймса Гофа.....	23
Благодарности Дэниэла Брайанта.....	23
Благодарности Мэтью Оберна	23
Введение	25
Путешествие по архитектуре.....	25
Что же такое API?.....	26
Практический пример конференц-системы: запуск	27
Типы API в практическом примере конференции.....	29
Причины изменения конференц-системы.....	29
От многоуровневой архитектуры к моделированию API.....	30
Практический пример: эволюционный этап.....	30
Трафик север-юг	31
Трафик восток-запад.....	32
API-инфраструктура и шаблоны трафика	32
Дорожная карта для практического примера конференции	32
Использование диаграмм C4	33
Диаграмма контекста C4.....	33
Диаграмма контейнеров C4	34
Диаграмма компонентов C4	34
Использование записей архитектурных решений	34
ADR-запись эволюции участников.....	35
Осваиваем API: руководства ADR.....	36
Заключение.....	37

ЧАСТЬ I. ПРОЕКТИРОВАНИЕ, СОЗДАНИЕ И ТЕСТИРОВАНИЕ API 39

Глава 1. Проектирование, создание и спецификация API	41
Практический пример: проектирование API Участник	41
Знакомство с REST	42
Знакомство с REST и HTTP на примере	42
Модель зрелости Ричардсона	43
Введение в API удаленного вызова процедур (RPC)	44
Краткое упоминание о языке GraphQL	45
Стандарты и структура REST API	47
Коллекции и пагинация	48
Фильтрация коллекций	49
Обработка ошибок	49
Руководство ADR: выбор стандарта API	50
Определение REST API с помощью OpenAPI	51
Практическое применение спецификаций OpenAPI	51
Генерация кода	52
Валидация OpenAPI	52
Примеры и создание имитаций	53
Обнаружение изменений	54
Версионирование API	54
Семантическое версионирование	55
Спецификация и версионирование OpenAPI	55
Реализация RPC с помощью gRPC	57
Моделирование обмена данными и выбор формата API	59
Сервисы с высоким уровнем трафика	59
Большие полезные нагрузки при обмене данными	59
Преимущества производительности HTTP/2	60
Старые форматы	60
Рекомендация: моделирование обменов	61
Различные спецификации	61
Существует ли «золотая спецификация»?	61
Проблемы комбинированных спецификаций	63
Заключение	63
Глава 2. Тестирование API	65
Сценарий конференц-системы для этой главы	66
Стратегии тестирования	66
Тестовый квадрант	67
Тестовая пирамида	69
Руководство ADR по стратегиям тестирования	71
Контрактное тестирование	72
Почему часто предпочтительнее проводить контрактное тестирование?	72
Порядок реализации контракта	73
Контракты производителя	75
Контракты, ориентированные на потребителя	76
Наш пример: использование CDC	76
Обзор методологии контрактов	76
Фреймворки для контрактного тестирования	77
Хранение и публикация контрактов API	77
Руководство ADR: контрактное тестирование	78

Тестирование компонентов API	79
Контрактное тестирование в сравнении с тестированием компонентов	80
Практический пример: тестирование компонентов для проверки поведения	80
Интеграционное тестирование API	82
Использование серверов-заглушек: почему и как?	82
Руководство ADR: интеграционное тестирование	84
Контейнеризация тестовых компонентов: Testcontainers	85
Практический пример: применение Testcontainers для верификации интеграций	85
Сквозное тестирование	87
Автоматизация сквозной валидации	87
Типы сквозных тестов	88
Руководство ADR: сквозное тестирование	89
Заключение	90
 ЧАСТЬ II. УПРАВЛЕНИЕ ТРАФИКОМ API	91
 Глава 3. API-шлюзы: управление входящим трафиком	93
Является ли API-шлюз единственным решением?	93
Рекомендация: прокси, балансировщик нагрузки или API-шлюз	94
Практический пример: открытие доступа к сервису Участник для потребителей	95
Что такое API-шлюз?	96
Какие функциональные возможности предоставляет API-шлюз?	96
Где выполняется развертывание API-шлюза?	97
Как API-шлюз интегрируется с другими технологиями на периферии?	98
Зачем нужен API-шлюз?	99
Снижение связанности: адаптер/фасад между фронтендами и бэкендами	100
Упрощение использования: агрегирование/преобразование бэкенд-сервисов	101
Защита API от чрезмерного использования и злоупотреблений: обнаружение и устранение угроз	102
Понимание того, как используются API: наблюдаемость	104
Управление API как продуктами: менеджмент жизненного цикла API	104
Монетизация API: управление учетными записями, биллинг и оплата	106
Современная история API-шлюзов	106
1990-е годы и далее: аппаратные балансировщики нагрузки	107
Начало 2000-х годов и далее: программные балансировщики нагрузки	108
Середина 2000-х: контроллеры доставки приложений (ADC)	109
Начало 2010-х: API-шлюзы первого поколения	109
2015 год и далее: API-шлюзы второго поколения	111
Текущая таксономия API-шлюзов	112
Традиционные корпоративные шлюзы	112
Микросервисы/микрошлюзы	113
Шлюзы сервисных сетей	113
Сравнение типов API-шлюзов	113
Практический пример: развитие конференц-системы с помощью API-шлюза	115
Установка Ambassador Edge Stack в Kubernetes	116
Настройка сопоставления URL-путей с бэкенд-сервисами	116
Настройка сопоставлений с использованием маршрутизации на основе хостов	117
Развертывание API-шлюзов: понимание и управление отказами	118
API-шлюз как единая точка отказа	118

Обнаружение и принадлежность проблем	119
Разрешение инцидентов и проблем	119
Снижение рисков	120
Общие ошибки при внедрении API-шлюзов	121
Loorback API-шлюза	121
Шлюз API в качестве корпоративной сервисной шины (ESB, Enterprise Service Bus)	121
Черпахи (API-шлюзы) — и нет им конца	122
Выбор API-шлюза	122
Определение требований	122
Создание или покупка?	123
Руководство ADR: выбор API-шлюза	123
Заключение	124
Глава 4. Сервисные сети: управление трафиком между сервисами	127
Сервисная сеть — это единственное решение?	127
Рекомендация: стоит ли внедрять технологию сервисной сети?	128
Практический пример: извлечение функциональности сессий в сервис	128
Что такое сервисная сеть?	130
Какие функциональные возможности предоставляет сервисная сеть?	132
Где разворачивается сервисная сеть?	133
Как сервисная сеть интегрируется с другими сетевыми технологиями?	134
Зачем нужна сервисная сеть?	135
Тонкий контроль маршрутизации, надежности и управления трафиком	136
Прозрачная маршрутизация и нормализация имен сервисов	136
Надежность	137
Продвинутая маршрутизация трафика: придание формы, контроль, разделение и зеркалирование	138
Придание формы трафику	138
Контроль трафика	139
Обеспечение прозрачной наблюдаемости	139
Обеспечение безопасности: транспортная безопасность, аутентификация и авторизация	140
Поддержка кросс-функционального взаимодействия на разных языках	140
Разделение управления входящим и межсервисным трафиком	141
Эволюция сервисных сетей	142
Ранняя история и мотивация	143
Шаблоны реализации	145
Библиотеки	145
Sidecar	146
Библиотеки gRPC без прокси	148
Без sidecar: реализации ядра операционной системы (eBPF)	150
Таксономия сервисных сетей	151
Практический пример: использование сервисной сети для маршрутизации, наблюдаемости и безопасности	152
Маршрутизация с помощью Istio	152
Наблюдение за трафиком с помощью Linkerd	154
Сегментация сети с помощью Consul	156
Развертывание сервисной сети: понимание и управление отказами	158
Сервисная сеть как единая точка отказа	159

Общие проблемы внедрения сервисной сети.....	159
Сервисная сеть в качестве ESB.....	159
Сервисная сеть в качестве шлюза.....	159
Слишком много сетевых уровней.....	160
Выбор сервисной сети.....	160
Определение требований.....	160
Создание или покупка?.....	160
Руководство ADR: выбор сервисной сети.....	161
Заключение.....	162

ЧАСТЬ III. ЭКСПЛУАТАЦИЯ И БЕЗОПАСНОСТЬ API..... 165

Глава 5. Развертывание и релизы API.....	167
Отделение развертывания от релиза.....	167
Практический пример: установка флагов функций.....	168
Управление трафиком.....	170
Практический пример: моделирование релизов в конференц-системе.....	171
Жизненный цикл API.....	171
Сопоставление стратегий релизов с жизненным циклом.....	172
Руководство ADR: отделение релиза от развертывания с помощью управления трафиком и флагов функций.....	173
Стратегии релизов.....	173
Канареечный релиз.....	174
Зеркалирование трафика.....	176
Сине-зеленое развертывание.....	177
Практический пример: развертывание с помощью Argo Rollouts.....	178
Мониторинг успешного выполнения и выявление отказов.....	181
Три столпа наблюдаемости.....	181
Важные метрики для API.....	183
Считывание сигналов.....	184
Прикладные решения для эффективных релизов программного обеспечения.....	184
Кеширование ответов.....	185
Распространение заголовков на уровне приложений.....	185
Ведение журнала для помощи в отладке.....	185
Рассмотрение субъективной платформы.....	186
Руководство ADR: субъективные платформы.....	186
Заключение.....	187

Глава 6. Операционная безопасность: моделирование угроз для API..... 189

Практический пример: применение OWASP к API участников.....	189
Риск при отсутствии защиты внешних API.....	191
Моделирование угроз 101.....	192
Мыслите как злоумышленник.....	193
Как построить модель угрозы?.....	194
Этап 1: определение целей.....	195
Этап 2: сбор нужной информации.....	195
Этап 3: декомпозиция системы.....	195
Этап 4: выявление угроз и включение их в модель STRIDE.....	196
Spoofing (Спуфинг).....	199
Tampering (Подмена).....	199

Внедрение полезной нагрузки.....	199
Массовое переназначение.....	200
Repudiation (Отрицание).....	201
Information disclosure (Разглашение сведений)	202
Чрезмерное раскрытие данных	202
Некорректное управление активами.....	203
Denial of Service (Отказ в обслуживании)	204
Ограничение количества запросов и сброс нагрузки	204
Elevation of privilege (Расширение полномочий)	206
Неправильная конфигурация безопасности	206
Терминирование TLS-соединений.....	207
Совместное использование ресурсов разными источниками.....	207
Усиление директив безопасности	208
Этап 5: Оценка рисков угроз	208
Этап 6: Валидация	210
Заключение.....	211
Глава 7. Аутентификация и авторизация API.....	213
Аутентификация	213
Аутентификация конечных пользователей на основе токенов	215
Аутентификация между системами.....	216
Почему не следует смешивать ключи и пользователей?	216
OAuth2	217
Роль сервера авторизации при взаимодействии с API.....	218
Веб-токены JSON (JWT).....	218
Кодирование и верификация веб-токенов JSON.....	220
Терминология и механизмы предоставления доступа OAuth2	221
Руководство ADR: стоит ли рассматривать возможность использования OAuth2?	222
Authorization Code Grant: предоставление доступа по коду авторизации.....	223
Authorization Code Grant (+ PKCE).....	225
Практический пример: доступ к API Участник с помощью Authorization Code Grant.....	226
Refresh-токены.....	227
Client Credentials Grant: предоставление доступа к учетным данным клиента	227
Практический пример: доступ к API Участник из CFP-системы при помощи Client Credentials Grant	228
Дополнительные варианты предоставления доступа OAuth2.....	228
Руководство ADR: какой из грантов OAuth2 стоит поддерживать	229
Области видимости OAuth2	229
Практический пример: применение областей видимости OAuth2 к API Участник.....	230
Обеспечение выполнения авторизации.....	231
Вкратце об OIDC	232
Протокол SAML 2.0	234
Заключение.....	234

ЧАСТЬ IV. ЭВОЛЮЦИОННАЯ АРХИТЕКТУРА С ПРИМЕНЕНИЕМ API.....	235
Глава 8. Перепроектирование приложений на архитектуру, управляемую API	237
Зачем использовать API для развития системы?	237
Создание полезных абстракций: повышение связности.....	238
Уточнение границ домена: продвижение слабой связанности	239
Практический пример: установление границ домена сервиса Участник	240
Варианты архитектуры конечного состояния	241
Монолит	241
Сервис-ориентированная архитектура (SOA).....	241
Микросервисы	242
Функции	243
Управление эволюционным процессом	243
Определите свои цели.....	243
Использование функций приспособленности.....	244
Декомпозиция системы на модули	245
Создание API как «швов» для расширения.....	247
Определение точек приложения усилий в системе	248
Непрерывная поставка и верификация	249
Архитектурные паттерны для эволюционирующих систем с API.....	249
Паттерн «Душитель».....	249
Паттерны «Фасад» и «Адаптер»	250
Паттерн «Слоеный пирог API»	251
Определение болевых точек и возможностей.....	252
Вопросы модернизации и технического сопровождения	252
Проблемы с производительностью.....	253
Разрушение зависимостей: API с сильной связанностью	253
Заключение.....	254
Глава 9. Использование инфраструктуры API для эволюции в сторону облачных платформ.....	255
Практический пример: перенос сервиса Участник в облако	255
Выбор стратегии миграции в облако	256
Сохранение или пересмотр.....	257
Перенос	258
Смена платформы.....	258
Новая покупка.....	259
Рефакторинг/смена архитектуры	259
Отключение	260
Практический пример: смена платформы сервиса Участник для переноса его в облако	260
Роль API-менеджмента	260
Север-юг и восток-запад: размывание границ управления трафиком	262
Начните с периферии и работайте вглубь.....	262
Пересекая границы: маршрутизация по сетям.....	262
От зональной архитектуры к принципу нулевого доверия	263
Попадание в зону.....	263
Никому не доверять и проверять	265

Роль сервисной сети в архитектурах с нулевым доверием.....	266
Дополнение сервисной сети сетевыми политиками	267
Заключение.....	269
Глава 10. Подведение итогов.....	271
Практический пример: оглянитесь на пройденный путь	271
API, Закон Конвея и ваша организация	277
Понимание типов решений.....	278
Подготовка к будущему	278
Асинхронное взаимодействие	278
Протокол HTTP/3	279
Сеть на основе платформы	279
Что дальше: как продолжать изучать архитектуру API?	279
Постоянное совершенствование базовых знаний.....	280
Следите за новостями в отрасли	280
Радары, квадранты и трендовые отчеты.....	281
Изучение лучших практик и примеров использования	281
Обучение на практике.....	282
Обучение через преподавание.....	282
Предметный указатель.....	283
Об авторах.....	287
Об изображении на обложке.....	287

*Эта книга посвящена Алексу Блюитту,
который, к сожалению, не дожил до ее выхода.
Мы хотели бы поблагодарить Алекса за его откровенные отзывы,
постоянную поддержку и теплую дружбу на протяжении многих лет.*

– Авторы

Вступительное слово

Когда я, более десяти лет назад, работая в Financial Times, создавала свои первые API, их было сделано не так уж много. Мы строили систему на основе монолитной архитектуры, и API были нужны исключительно для того, чтобы сторонние разработчики могли получить доступ к нашему контенту.

Однако сейчас API повсюду, и они стали ключевым фактором успеха при построении систем.

Это связано с тем, что за последнее десятилетие произошло несколько событий, изменивших подход многих из нас к разработке программного обеспечения.

Прежде всего — изменились доступные нам технологии. С развитием облачных вычислений появились возможности самостоятельного обслуживания и предоставления сервисов по требованию. Автоматизированные конвейеры сборки и развертывания позволяют осуществлять непрерывную интеграцию и развертывание, а контейнеры и связанные с ними технологии оркестровки разрешают запускать значительное количество небольших независимых сервисов в составе распределенной системы.

Почему так происходит? Причиной тому стало второе событие: исследования показывают, что в успешных организациях, занимающихся разработкой программного обеспечения, применяется слабосвязанная архитектура и автономные, наделенные полномочиями команды. Успешность здесь определяется положительным влиянием на бизнес: увеличением доли рынка, ростом производительности и прибыльности.

В настоящее время преобладает тенденция создания архитектур, более слабо связанных, распределенных и построенных на основе API. Необходимо, чтобы API легко обнаруживались, были согласованными и не создавали проблем для потребителей, даже если неожиданно изменятся или исчезнут. Всё прочее будет лишь мешать совместной работе и замедлять работу команд.

Эта книга представляет собой исчерпывающее практическое руководство по построению эффективных архитектур API. Ее авторы, Джеймс, Дэниэл и Мэтью, охватывают широкий спектр вопросов, начиная с создания и тестирования отдельных API, заканчивая экосистемой, в которую они внедряются, способами выполнения релизов и эффективной эксплуатации, и, пожалуй, самое главное — использованием API для развития своей архитектуры. Создание заново, с нуля, тех первых API, над которыми мы работали в Financial Times и которых уже не существует, потребовало больших затрат. Джеймс, Дэниэл и Мэтью дают пример того, как

справляться с неизбежными трудностями и изменениями и развивать свои системы, используя API в качестве основного инструмента.

Архитектура программного обеспечения определяется решениями, которые одновременно важны и трудно изменяемы. Но именно от этих решений зависит успех или неудача вашего проекта.

Основное внимание авторы книги уделяют не абстрактной архитектуре, а тому, как вы применяете архитектуру в своих организациях. Решение о выборе API-шлюза или сервисной сети, а также о том, какой из представленных вариантов выбрать, относится именно к тем серьезным решениям, которые требуют осторожного подхода и тщательной оценки. Джеймс, Дэниэл и Мэтью дают четкие рекомендации в тех случаях, когда они считают это уместным, а в тех случаях, когда варианты не столь однозначны, они предоставляют соображения, дающие возможность сделать наилучший выбор в соответствии с вашими обстоятельствами.

Все предлагаемые авторами решения основаны на практических и реалистичных примерах, в которых рассматриваются концепции и показывается, как их можно реализовать на практике. Приведенный в книге тематический пример конференц-системы развивается на всем ее протяжении точно так же, как это происходит с реальными системами. Авторы показывают, что не обязательно делать всё сразу, а можно развивать архитектуру по частям, извлекая сервисы и добавляя такие инструменты, как API-шлюзы и сервисные сети, по мере возникновения потребности в них.

Создавая свои первые API, я совершила множество ошибок. Хотелось бы в ту пору иметь под рукой такую книгу, способную помочь мне понять, где я могу оступиться, и подсказать разумные решения.

Я рекомендую эту книгу всем, кто руководит работой над системами с ключевой ролью API в них. Опираясь на нее, вы сможете разработать согласованный набор инструментов и стандартов для поддержки каждой команды, создающей API в вашей организации.

*Сара Уэллс,
сопредседатель конференции QCon London,
независимый консультант и бывший технический директор в Financial Times,
Рединг, Великобритания, сентябрь 2022 г.*

Для чего мы написали эту книгу?

В начале 2020 года мы посетили выставку O'Reilly Software Architecture в Нью-Йорке, где Джеймс и Мэтью провели семинар по API и презентацию по API-шлюзам. Джеймс и Дэниэл знают друг друга по лондонскому Java-сообществу, и, как на многих мероприятиях, посвященных архитектуре, мы встретились, чтобы вместе обсудить наши мысли и понимание архитектуры API. Пока мы общались в коридоре, к нам подошли несколько докладчиков конференции и рассказали о своем опыте работы с API. При этом все они просили нас поделиться своими мыслями и рекомендациями по поводу их пути к API. Именно в этот момент мы подумали, что написание книги на тему API поможет донести до других разработчиков архитектур результаты наших обсуждений на конференциях.

Почему стоит прочитать эту книгу?

Эта книга призвана дать полное представление о проектировании, эксплуатации и развитии архитектуры API. Мы поделились в ней своим опытом и советами, сопроводив ее практическим примером, имитирующим реальную систему управления конференциями, позволяющую участникам просматривать и бронировать сессии презентаций. Этот тематический пример прослеживается на протяжении всей книги и помогает понять механизм перехода абстрактных понятий в практическое применение. Общий обзор развития этого примера приведен в *главе 10*. Мы также верим в то, что вы можете принимать собственные решения. Для этого мы:

- ♦ четко сформулируем рекомендации или указания;
- ♦ выделим требующие соблюдения осторожности области, а также потенциальные проблемы в вашем рабочем процессе;
- ♦ предоставим руководство по реестру архитектурных решений (ADR)¹, чтобы помочь вам принять наилучшее возможное решение с учетом особенностей вашей архитектуры, и дадим рекомендации по требующим внимания вопросам (потому что иногда ответ начинается фразой: «Это зависит от...»);

¹ Более подробно об ADR и их значении для принятия и документирования архитектурных решений вы узнаете во *введении*.

- ♦ укажем ссылки на ресурсы и полезные статьи, содержащие более подробную информацию.

Книга представляет собой не просто учебник по новым технологиям. Мы сочли, что описание существующих архитектур с эволюционным подходом к более актуальным архитектурам API принесет вам наибольшую пользу. При этом мы старались не упускать из виду новые технологии и разработки в области архитектуры API.

Для кого эта книга?

Несмотря на то что при создании этой книги мы изначально имели в виду одну персону, в процессе ее написания и рецензирования возникли три ключевых действующих лица: разработчик, случайный (вспомогательный) архитектор и архитектор решений, или корпоративный архитектор. Мы представили их в следующих разделах, чтобы помочь вам в дальнейшем сфокусироваться на позиции не кого-то одного из них, но и взглянуть на каждую главу через призмы их различных ролей.

Разработчик

Скорее всего, вы профессионально занимаетесь программированием уже несколько лет и хорошо знаете общие проблемы разработки программного обеспечения, паттерны и лучшие подходы. Вы всё четче осознаете, что движение индустрии программного обеспечения в сторону построения сервис-ориентированной архитектуры (SOA) и внедрения облачных сервисов означает, что создание и эксплуатация API быстро становятся основным навыком. Вы намерены узнать больше о разработке эффективных API и их тестировании. Хотите изучить различные варианты реализации (например, синхронное и асинхронное взаимодействие) и технологии, а также научиться задавать правильные вопросы и оценивать, какой подход лучше всего пригоден для конкретного контекста.

Случайный архитектор

Скорее всего, вы уже много лет занимаетесь разработкой программного обеспечения и часто работали в качестве руководителя группы или постоянного архитектора программного обеспечения (даже если у вас нет официальных профессиональных званий). Вы в курсе основных архитектурных концепций, таких как проектирование с сильной или слабой связанностью, и применяете их ко всем аспектам разработки программного обеспечения, включая проектирование, тестирование и операционные системы.

Вы понимаете, что ваша роль все больше сводится к объединению систем для удовлетворения требований заказчика. Это могут быть как приложения собственной разработки, так и сторонние предложения типа SaaS. API играют большую роль в успешном объединении ваших систем с внешними. Вы хотите узнать больше о вспомогательных технологиях (например, API-шлюзах, сервисных сетях и т. п.), а также понять, как эксплуатировать и защищать системы на базе API.

Архитектор решений, корпоративный архитектор

Вы уже несколько лет занимаетесь проектированием и созданием корпоративных программных систем, и, скорее всего, в названии вашей должности или описании роли фигурирует слово «архитектор». Вы отвечаете за общую картину поставки программного обеспечения и, как правило, работаете в крупной организации или в ряде крупных взаимосвязанных организаций.

Вы в курсе изменений, вызываемых последними итерациями сервис-ориентированных архитектурных стилей, в процессах разработки, интеграции и управления программным обеспечением и понимаете, что API играют ключевую роль в успехе программной стратегии вашей организации. Вы желаете узнать больше об эволюционных паттернах и понять, как выбор проекта и реализации API повлияет на это. Вы также хотите сосредоточиться на кросс-функциональных «возможностях»: удобстве использования, удобстве сопровождения, масштабируемости и доступности — и понять, как создавать системы на базе API, обладающие этими свойствами, а также обеспечивающие безопасность.

Чему вы научитесь?

Прочитав эту книгу, вы разберетесь:

- ♦ с основами REST API и оптимальными способами создания, формирования версий и тестирования API;
- ♦ с архитектурными паттернами, используемыми при построении платформы API;
- ♦ с различиями в управлении трафиком API на входе и в рамках межсервисного взаимодействия, а также с применением таких моделей и технологий, как API-шлюзы и сервисные сети;
- ♦ с моделированием угроз и ключевыми аспектами безопасности API, такими как аутентификация, авторизация и шифрование;
- ♦ с тем, как осуществить эволюцию существующих систем в сторону API, и различными целевыми развертываниями — например, в облаке.

И вы сможете:

- ♦ проектировать, создавать и тестировать системы на базе API;
- ♦ помогать организации в реализации и управлении программой API с точки зрения архитектуры;
- ♦ развертывать, выполнять релизы и настраивать ключевые компоненты платформы API;
- ♦ выполнять развертывание шлюзов и сервисных сетей на основе конкретных примеров;
- ♦ выявлять уязвимости в архитектуре API и реализовывать взвешенные меры по снижению рисков для защиты;
- ♦ вносить вклад в развитие новых тенденций в области API и соответствующих сообществ.

О чем не расскажет эта книга?

Мы понимаем, что API охватывают огромное пространство рынка, и хотим уточнить, о чем в этой книге не пойдет речь. Это не означает, что мы считаем не освещенные в ней темы неважными. Но если бы мы пытались охватить всё, то не смогли бы эффективно поделиться с вами своими знаниями.

Мы рассмотрим паттерны миграции и модернизации приложений, включающие использование преимуществ облачных платформ, но *книга не будет полностью посвящена облачным технологиям*. Многие из вас работают с гибридной архитектурой или даже размещают все свои системы в центрах обработки данных. Мы хотим обеспечить освещение факторов проектирования и эксплуатации архитектур API, поддерживающих оба подхода.

Книга не привязана к конкретному языку, но на упрощенных примерах демонстрирует подходы к созданию/проектированию API и соответствующей инфраструктуры. В книге большее внимание будет уделено этим подходам, а примеры кода вы всегда найдете в сопутствующем репозитории GitHub².

В книге не отдается предпочтение одному стилю архитектуры перед другим, однако рассматриваются ситуации, когда архитектурные подходы могут приводить к ограничениям представленных API.

Условные обозначения

В этой книге используются следующие типографские обозначения:

◆ *Курсив.*

Им выделяются новые термины.

◆ **Полужирный шрифт.**

Им выделяются интернет-адреса (URL) и адреса электронной почты.

◆ **Рубленый шрифт.**

Им выделяются имена файлов, расширения имен файлов и пути.

◆ **Моноширинный шрифт.**

Им выделяются листинги программного кода, а также встречающиеся внутри абзацев текста ссылки на элементы программ — такие как имена переменных или функций, базы данных, типы данных, переменные среды, операторы и ключевые слова.

◆ **Полужирный моноширинный шрифт.**

Им выделяются команды или другой текст, который должен быть введен пользователем с клавиатуры.

² См. <https://github.com/masteringapi>.

◆ Моноширинный курсивный шрифт.

Им выделяется текст, который следует заменить значениями, заданными пользователем, или значениями, определяемыми контекстом.



Этот значок обозначает совет, подсказку или предложение.



Этот значок обозначает примечание общего характера.



Этот значок обозначает предупреждение или предостережение.

Использование примеров кода

Дополнительный материал (примеры кода, упражнения и т. д.) можно загрузить отсюда: <https://github.com/masteringapi>.

Если у вас возникнут технические вопросы или проблемы при использовании примеров кода, напишите нам по адресу: bookquestions@oreilly.com.

Предназначение этой книги — помочь вам в решении ваших задач. Вы можете использовать любой пример кода, содержащийся в ней, в своих программах и документации. При этом вам не требуется обращаться к нам за разрешением, если только вы не воспроизводите существенную часть кода. Это, например, касается ситуаций, когда вы включаете в свою программу несколько фрагментов кода, приведенного в книге. Однако продажа или распространение примеров из книг издательства O'Reilly требует отдельного разрешения. Вы можете свободно цитировать эту книгу, включая примеры, при ответе на вопрос, но если хотите включить существенную часть приведенного здесь кода в документацию своего продукта, то вам следует связаться с нами.

Ссылка на оригинал приветствуется, но не является обязательной. Указание авторства обычно включает название книги, автора (авторов), издателя и ISBN. Например: «Осваиваем архитектуру API», авторы Джеймс Гоф, Дэниэл Брайант и Мэтью Оберн (O'Reilly). Copyright 2023 James Gough Ltd, Big Picture Tech Ltd и Matthew Auburn Ltd, 978-1-492-09063-2».

Если вы сочтете, что ваше обращение с примерами кода выходит за рамки добросовестного использования или условий, упомянутых ранее, можете обратиться к нам по адресу: permissions@oreilly.com.

Платформа онлайн-обучения O'Reilly

O'REILLY

На протяжении почти 40 лет издательство O'ReillyMedia предоставляет технические и бизнес-тренинги, знания и опыт, чтобы помочь компаниям достичь успеха.

Его уникальное сообщество экспертов и новаторов делится знаниями и опытом с помощью книг, статей, конференций и нашей платформы онлайн-обучения. Эта платформа предоставляет доступ к учебным курсам, проводимым в прямом эфире, углубленные учебные программы, интерактивные среды для написания кода, а также обширную коллекцию текстов и видео от O'Reilly и более чем 200 других издательств. Больше информации можно найти на странице <http://oreilly.com>.

Как с нами связаться?

Пожалуйста, адресуйте все комментарии и вопросы по этой книге издателю:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-998-9938 (в Соединенных Штатах и Канаде)

707-829-0515 (международные или местные звонки)

707-829-0104 (факс)

У этой книги есть веб-страница, где мы приводим ошибки, примеры и любую дополнительную информацию. Она находится по адресу: <https://oreil.ly/Mastering-API-Architecture>.

Для комментариев и технических вопросов об этой книге используйте электронную почту: bookquestions@oreilly.com.

Новости и информацию о наших книгах можно найти сайте <http://www.oreilly.com>.

Найдите нас на LinkedIn: <https://linkedin.com/company/oreilly-media>.

Следите за нами в Twitter: <https://twitter.com/oreillymedia>.

Смотрите нас на YouTube: <https://www.youtube.com/oreillymedia>.

Благодарности

Как и почти во всех технических книгах, на титульном листе этой книги в качестве авторов могут быть указаны только три наши фамилии, но на самом деле свой вклад в ее создание внесли многие люди, как непосредственно — в виде отзывов и советов в процессе подготовки материала книги, так и косвенно — как наши преподаватели и руководители на протяжении многих лет.

Хотя невозможно упомянуть здесь всех, кто помогал нам на этом пути, мы хотели бы выразить особую благодарность тем, кто выделил время из своего плотного рабочего графика, чтобы провести подробное обсуждение материала книги, высказать свое мнение и оказать поддержку.

Наши технические рецензенты: Сэм Ньюмен, Дов Кац, Сара Уэллс, Антуан Кайяу, Стефания Чаплин, Мэтт МакЛарти и Нил Форд.

Предоставляли нам общую информацию, поддержку и советы: Чарльз Хамбл, Ричард Ли, Саймон Браун, Ник Эббит, Джейсон Морган, Ник Джексон, Клифф Тилтман, Элспет Минти, Джордж Болл, Бенджамин Эванс и Мартин Верберг.

Команда O'Reilly: Вирджиния Уилсон, Мелисса Даффилд и Николь Таче.

Благодарности Джеймса Гофа

Я хотел бы поблагодарить свою замечательную семью: Меган, Эмили и Анну. Без их помощи и поддержки написание книги было бы невозможным. Искренняя благодарность моим родителям, Хизер и Полу, за то, что они вдохновляли меня на учебу и постоянно поддерживали.

Хочется поблагодарить моих соавторов Дэниэла и Мэтью, ведь написание книги — это непростая задача, и, как и архитектура, ее решение никогда не бывает идеальным. Это было увлекательное путешествие, в процессе которого мы многому научились друг у друга и у наших замечательных рецензентов. Наконец, хочется выразить слова благодарности Джону Даплину, Эду Сафо, Дэвиду Холливеллу и Дову Кацу за предоставленные мне на протяжении всей моей карьеры поддержку, возможности и ободрение.

Благодарности Дэниэла Брайанта

Я хотел бы сказать спасибо всей своей семье за любовь и поддержку, оказанные мне как в процессе написания книги, так и на протяжении всей моей карьеры. И поблагодарить Джеймса и Мэтью за то, что они были мне прекрасными партнерами в этом писательском путешествии. Мы начали писать эту книгу в начале 2020 года — как раз, когда началась пандемия. Наши еженедельные утренние встречи по средам были не только полезны для совместной работы, но и служили отличным источником веселья и поддержки в этом быстро меняющемся мире. Наконец, я выражаю свою признательность всем участникам лондонского Java-сообщества (LJC), командам InfoQ/QCon, а также всем сотрудникам Ambassador Labs. В этих трех сообществах я нашел доступ к наставникам, рекомендациям и получил множество других возможностей, за что хочу от всего сердца их поблагодарить.

Благодарности Мэтью Оберна

Я хочу поблагодарить мою замечательную жену Ханну: без твоей поддержки, Ханна, я не смог бы написать эту книгу. Спасибо моим родителям — вы показали мне, что всё возможно, и никогда не переставали верить в меня. Эта книга стала для меня удивительным опытом, а Джеймс и Дэниэл — прекрасными наставниками. Вы оба многому меня научили и помогли мне создать лучшее, что я за свою жизнь сделал. Отдельное спасибо Джеймсу — ты познакомил меня с ораторским искусством и помог мне больше, чем кто-либо другой в моей карьере. И, наконец, самое главное, я хочу поблагодарить моего сына Джоши: ты — абсолютное счастье.

https://t.me/it_boooks/2

В этом разделе вы познакомитесь с основами API и их возможностями как части архитектурного пути. Мы введем упрощенное определение для API и их использования в процессе и вне процесса. Для того чтобы продемонстрировать важность API, мы рассмотрим практический пример конференц-системы — действующий пример, получающий развитие на протяжении всей книги. Внепроцессные API позволяют не ограничиваться простой трехуровневой архитектурой — для демонстрации этого мы представим модели трафика и покажем их значение. А также кратко опишем этапы работы над примером, что позволит вам перейти к ним, если какая-то область вас сразу заинтересует.

Для того чтобы показать API и связанную с ними экосистему, мы задействуем ряд важных компонентов, в том числе приведем практический пример с *диаграммами модели C4*¹ и далее будем снова и снова обращаться к специфике и логике этого подхода. Вы также узнаете об использовании *записей архитектурных решений* (Architecture Decision Record, ADR) и о важности четкого определения решений в течение всего жизненного цикла программного обеспечения. В завершение введения мы расскажем о рекомендациях ADR — нашем подходе, который поможет вам принимать решения, когда ответ начинается фразой: «Это зависит от...».

Путешествие по архитектуре

Каждый, кто отправился в длительное путешествие, несомненно, задавал себе вопросы (и, возможно, очень часто): «Мы уже на месте?». При первых нескольких таких вопросах вы смотрите на GPS-навигатор или карту маршрута и приблизительно оцениваете ситуацию, надеясь, что в пути не возникнет никаких задержек. Точно так же путь к созданию архитектуры на основе API может быть весьма сложным для разработчиков и архитекторов. Даже если бы существовал *GPS-навигатор архитектуры*, каким бы был пункт назначения?

Архитектура — это путешествие без цели, и невозможно предсказать, как будут меняться технологии и архитектурные подходы. Например, вы, возможно, не могли предположить, что технология сервисной сети получит столь широкое распространение, но, узнав о ее возможностях, вы, вероятно, задумаетесь об изменении существующей архитектуры. Но не только технологии влияют на изменение архитекту-

¹ См. <https://c4model.com/>.

ры — новые требования и ограничения бизнеса также определяют изменение архитектурного направления.

Комплексный эффект от возрастающей ценности правильного выбора направления движения в сочетании с новыми развивающимися технологиями приводит к концепции *эволюционной архитектуры*. Эволюционная архитектура — это подход к постепенному изменению архитектуры. При его использовании основное внимание уделяется возможностям быстрого изменения подходов и снижению риска негативных последствий. На этом пути мы просим вас не забывать о следующем совете по выбору подходов к архитектуре API:

Хотя архитекторам нравится иметь возможность стратегического планирования на будущее, постоянно меняющаяся экосистема разработки программного обеспечения делает это затруднительным. Поскольку избежать изменений невозможно — необходимо их использовать.

*Книга «Эволюционная архитектура»,
авторы Нил Форд, Ребекка Парсонс и Патрик Куа (Питер, 2019)²*

Во многих проектах API — сами по себе эволюционные механизмы, и они требуют внесения изменений по мере интеграции всё большего числа систем и сервисов. Большинство разработчиков создают сервисы, ориентированные на выполнение одной функции, не задумываясь о более широком повторном использовании API с точки зрения потребителя.

Проектирование API-First — это подход, при котором разработчики и архитекторы учитывают функциональность своего сервиса и проектируют API, ориентируясь на потребителя. В качестве потребителя API может выступать мобильное приложение, другой сервис или даже внешний клиент. В *главе 1* мы рассмотрим методы проектирования, поддерживающие подход API-First, и узнаем, как создаются API, устойчивые к изменениям и пригодные для широкого круга потребителей.

Хорошая новость заключается в том, что начать путь к архитектуре, ориентированной на API, можно в любой момент. Если вы отвечаете за уже существующую техническую инфраструктуру, мы покажем вам методы развития архитектуры, способствующие использованию API в вашей платформе.

С другой стороны, если вам повезет, и у вас будет чистый холст для работы, мы поделимся с вами преимуществами внедрения архитектуры API, основанной на нашем многолетнем опыте, а также выделим ключевые факторы принятия решений.

Что же такое API?

В области архитектуры программного обеспечения существует несколько невероятно сложных для определения терминов. Термин API, обозначающий интерфейс прикладного программирования, относится именно к этой категории, поскольку впервые эта концепция появилась еще 80 лет назад. Термины, используемые в те-

² «Building Evolutionary Architectures» by Neal Ford, Rebecca Parsons, and Patrick Kua (O'Reilly).

чение длительного времени, в итоге оказываются избыточными и приобретают несколько значений в различных проблемных пространствах. Под API мы понимаем следующее:

- ◆ API представляет собой абстракцию от базовой реализации;
- ◆ API представляется спецификацией, в которой вводятся типы. Разработчики могут понимать спецификации и использовать инструментальные средства для генерации кода на различных языках для реализации потребителя API (программного обеспечения, потребляющего API);
- ◆ у API есть определенная семантика или поведение для эффективного моделирования обмена информацией;
- ◆ эффективное проектирование API позволяет расширять возможности клиентов или третьих лиц для интеграции бизнеса.

В широком смысле API можно разделить на две общие категории в зависимости от того, находится ли вызов API *в процессе* или *вне процесса*. Процесс, о котором здесь идет речь, — это процесс операционной системы (ОС). Например, вызов метода Java из одного класса в другой представляет собой внутрипроцессный вызов API, поскольку вызов этот обрабатывается тем же процессом, из которого был сделан. Приложение .NET, вызывающее внешний REST-подобный API с помощью библиотеки HTTP, представляет собой внепроцессный вызов API, т. к. вызов обрабатывается дополнительным внешним процессом, отличным от того, из которого был сделан вызов. Как правило, при вызове API вне процесса данные проходят через сеть: локальную сеть, виртуальное частное облако (Virtual Private Cloud, VPC) или Интернет. Мы сосредоточимся на внепроцессном стиле API, однако архитекторы часто сталкиваются с необходимостью переделать API внутрипроцессного типа в API внепроцессного типа. Для того чтобы продемонстрировать эту (и другие) концепции, мы создадим действующий практический пример, развивающийся на протяжении всей книги.

Практический пример конференц-системы: запуск

Мы выбрали в качестве примера конференц-систему, поскольку эта область легко узнаваема, но при этом обладает достаточной сложностью для моделирования эволюционной архитектуры. На рис. В.1 показан верхний уровень конференц-системы, позволяющий обозначить контекст обсуждаемой архитектуры. С помощью этой системы сторонний заказчик может создать учетную запись участника, ознакомиться с доступными сессиями конференции и забронировать свое участие.

Давайте увеличим масштаб блока конференц-системы (рис. В.2) — это позволит нам более подробно познакомиться с ее основными техническими элементами. Клиент взаимодействует с веб-приложением, вызывающим API конференц-приложения. Приложение для проведения конференции использует команды SQL для запроса к резервному хранилищу данных.

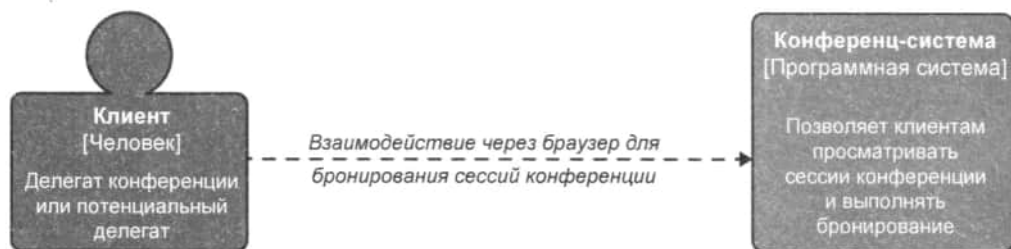


Рис. В.1. Диаграмма C4: контекст конференц-системы



Рис. В.2. Диаграмма C4: контейнер конференц-системы

Из рис. В.2 видно, что с точки зрения API наиболее интересная функциональность находится в контейнере конференц-приложения. Масштаб этого контейнера на рис. В.3 увеличен, что позволяет изучить его структуру и внутренние взаимодействия.

В системе задействованы четыре основных компонента и хранилище данных. *API контроллер* получает весь входящий трафик из пользовательского интерфейса и принимает решение о том, куда направить запрос в системе. Этот компонент также отвечает за перевод из представления на *сетевом* уровне в объект или представление в коде. Компонент API контроллера интересен с точки зрения маршрутизации *внутри процесса* и действует в качестве точки пересечения или паттерна *фронт-контроллера*. Для запросов и обработки API это важный паттерн — все запросы проходят через контроллер, принимающий решение о том, куда направить запрос. В *главе 3* мы рассмотрим возможности выноса контроллера за пределы процесса.

Компоненты *Участник*, *Бронирование* и *Сессия* занимаются переводом запросов (requests) в формализованные запросы (queries) и выполняют команды SQL к базе данных вне процесса. В существующей архитектуре база данных является важным компонентом, потенциально обеспечивающим реализацию взаимосвязей, — например, разграничений между бронированиями и сессиями.

Теперь, когда мы дошли до нужного уровня детализации, давайте еще раз рассмотрим типы взаимодействия с API на нашем практическом примере.

Приложение для конференций [Программная система]

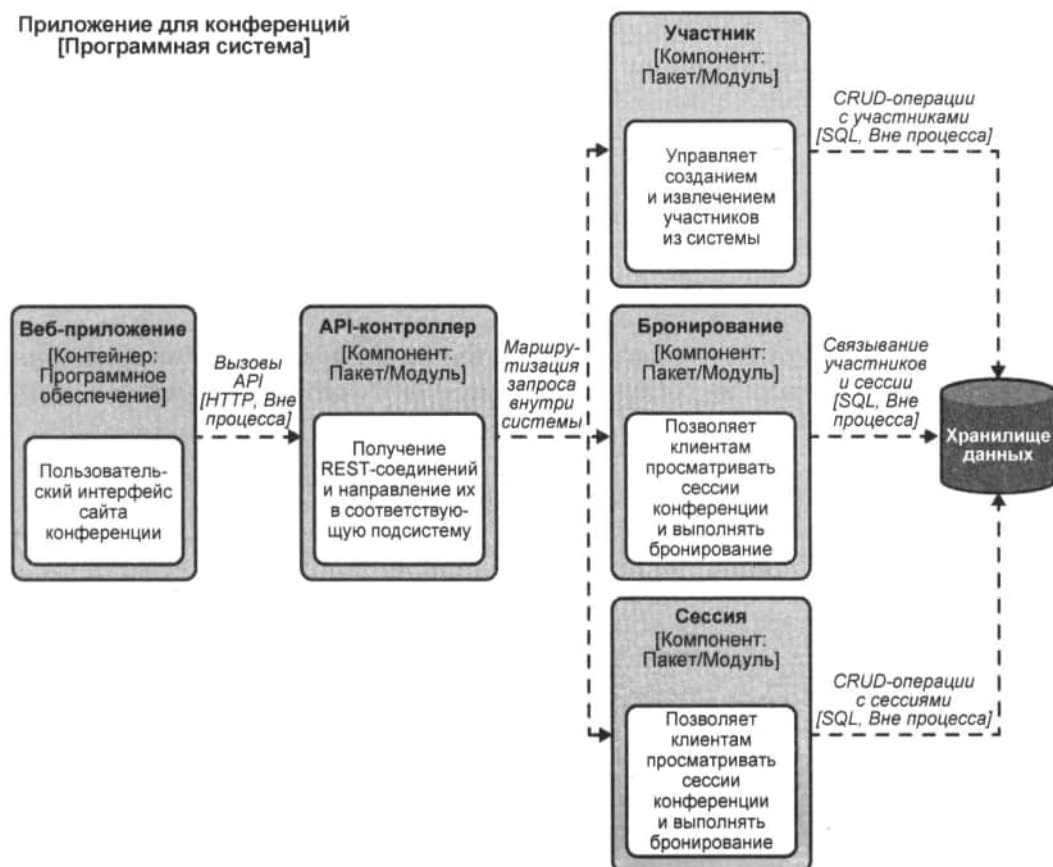


Рис. В.3. Диаграмма C4: компоненты конференц-системы

Типы API в практическом примере конференции

На рис. В.3 стрелка от веб-приложения к API-контроллеру представляет собой вне-процессный вызов, а стрелка от API-контроллера к компоненту Участник — это пример внутрипроцессного вызова. Все взаимодействия в границах приложения для конференций представляют собой примеры вызовов внутри процесса. Такие вызовы четко определены и ограничены языком программирования, используемым для реализации такого приложения. Они безопасны с точки зрения компиляции (условия для работы механизма обмена выполняются в момент написания кода).

Причины изменения конференц-системы

Существующий архитектурный подход работал в конференц-системе в течение многих лет, однако владелец системы попросил внести в нее три усовершенствования, послуживших причиной изменения архитектуры:

- ♦ организаторы конференции хотели бы создать мобильное приложение;
- ♦ организаторы конференции планируют выйти на глобальный уровень, проводя не одну конференцию в год, а десятки. Для того чтобы облегчить это расширение, они предполагают выполнить интеграцию с внешней системой конкурса работ (Call for Papers, CFP) для управления докладчиками и их заявками на выступление на конференции;
- ♦ организаторы конференции хотели бы отказаться от использования частного центра обработки данных и организовать конференц-систему на облачной платформе с глобальным охватом.

Наша задача: привести конференц-систему в соответствие с новыми требованиями, не затрагивая существующую производственную систему и не переписывая все за один раз.

От многоуровневой архитектуры к моделированию API

В качестве исходной точки примера примем типичную трехуровневую архитектуру, состоящую из пользовательского интерфейса (User Interface, UI), серверного уровня обработки и хранилища данных. Чтобы приступить к обсуждению эволюционной архитектуры, нам необходима модель, позволяющая представить, как API-запросы обрабатываются компонентами. То есть модель/абстракция, способная работать как для публичного облака, так и для виртуальных машин в центре обработки данных (ЦОД), а также и для гибридного подхода.

Абстрагирование трафика позволит нам рассматривать внепроцессные взаимодействия между потребителем и сервисом API, иногда называемым *производителем API*. При использовании таких архитектурных подходов, как сервис-ориентированная архитектура (SOA) и архитектура на базе микросервисов, важность моделирования взаимодействия API становится критической. Изучение трафика API и стиля взаимодействия между компонентами позволит найти баланс между реализацией преимуществ усиления развязки и кошмаром сопровождения.



Схемы трафика используются инженерами ЦОД для описания сетевых обменов внутри центра обработки и между низкоуровневыми приложениями. На уровне API задействуются шаблоны трафика для описания потоков между группами приложений. В рамках этой книги мы имеем в виду модели трафика на уровне приложений и API.

Практический пример: эволюционный этап

Для того чтобы приступить к рассмотрению типов трафика, целесообразно сделать небольшой эволюционный шаг в архитектуре нашего примера. На рис. В.4 показан этап рефакторинга компонента *Участник* в самостоятельный сервис, а не в пакет или модуль в рамках *унаследованной конференц-системы*. Теперь в конференц-системе два потока трафика: взаимодействие между клиентом и унаследованной конференц-системой и взаимодействие между унаследованной системой и системой участников.

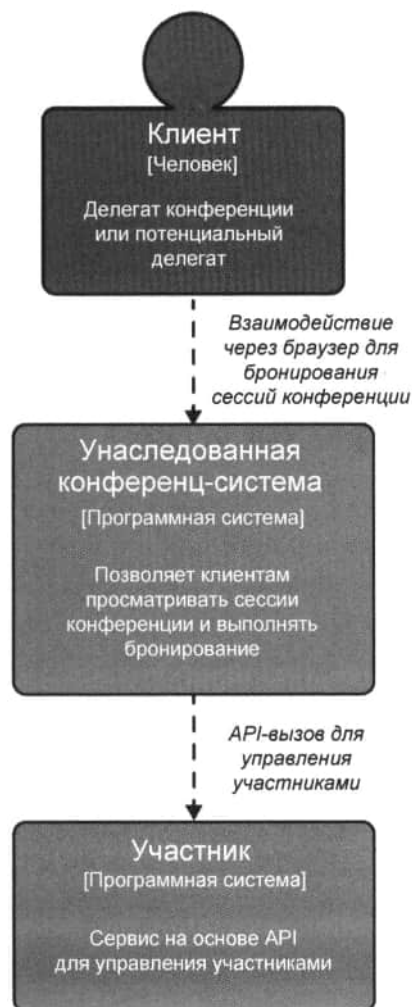


Рис. В.4. Диаграмма C4: контекст эволюционного этапа конференц-системы

Трафик север-юг

На рис. В.4 взаимодействие между клиентом и унаследованной конференц-системой отнесено к трафику север-юг, представляющему собой входящий поток. Клиент использует пользовательский интерфейс, отправляющий запросы в унаследованную конференц-систему через Интернет. Это открытая для публичного доступа точка в нашей сети, доступная пользовательскому интерфейсу³. Это означает, что любой компонент, обрабатывающий трафик север-юг, должен осуществлять конкретную проверку личности клиента, а также включать соответствующие вызо-

³ Предполагается, что это будет пользовательский интерфейс, обращающийся к точке входа. Тем не менее он открыт для потенциального использования.

вы, прежде чем разрешить прохождение трафика через систему. Защита API-трафика север-юг подробно рассматривается в *главе 7*.

Трафик восток-запад

Новое взаимодействие между унаследованной конференц-системой и сервисом участников вводит в нашу систему поток трафика восток-запад. Этот трафик можно рассматривать как *межсервисный* стиль взаимодействия внутри группы приложений. И хотя большей части трафика восток-запад, особенно если его источник находится в вашей более широкой инфраструктуре, можно в той или иной степени доверять, все же необходимо рассмотреть вопрос о защите трафика такого типа.

API-инфраструктура и шаблоны трафика

В архитектурах на базе API присутствуют два ключевых инфраструктурных компонента, представляющих собой основу для управления трафиком. Контроль и координацию движения часто называют *трафик-менеджментом*. Как правило, трафик север-юг будет контролироваться API-шлюзами (это ключевая тема *главы 3*).

Трафик восток-запад часто будет обрабатываться инфраструктурными компонентами, такими как Kubernetes, или сервисной сетью (а это ключевая тема *главы 4*). Указанные инфраструктурные компоненты используют сетевые абстракции для маршрутизации к сервисам, требуя, чтобы эти сервисы работали в управляемой среде. В некоторых системах трафик восток-запад управляется самим приложением, а для определения местоположения других систем применяются методы обнаружения сервисов.

Дорожная карта для практического примера конференции

В ходе работы над книгой вы увидите в нашем конкретном примере следующие изменения или варианты применения технологий:

- ♦ в *главе 1* рассматривается проектирование и спецификация API Участник. Мы также расскажем о важности определения версий и моделирования обмена для производительности этого API;
- ♦ в *главе 2* описано тестирование контрактов и компонентов для проверки поведения сервиса Участник. Вы также увидите, как контейнеры Testcontainers могут помочь в интеграционном тестировании;
- ♦ в *главе 3* мы рассмотрим возможность предоставления сервиса Участник потребителям с помощью API-шлюза. А также продемонстрируем, как развивать конференц-систему с помощью API-шлюза на Kubernetes;
- ♦ в *главе 4* мы выполним рефакторинг функциональности сессий из старой конференц-системы с помощью сервисной сети. Вы также узнаете о том, как сервисная сеть помогает в маршрутизации, наблюдаемости и безопасности;
- ♦ в *главе 5* показано, как с помощью флагов компонентов можно развивать конференц-систему и избегать развертывания и релизов с сильной связанностью.

В ней также освещаются подходы к моделированию релизов в конференц-системе и продемонстрировано использование Argo Rollouts для сервиса Участник;

- ♦ из главы 6 вы узнаете, как применить моделирование угроз и устранить опасения OWASP в сервисе Участник;
- ♦ в главе 7 мы рассмотрим аутентификацию и авторизацию и то, как это реализуется для сервиса Участник;
- ♦ в главе 8 показана установка границ домена сервиса Участник и то, как в этом могут помочь различные шаблоны сервисов;
- ♦ в главе 9 описан переход к облачным технологиям, представлены способы переноса сервиса Участник в облако, а также процесс смены платформы.

Практический пример и планируемая дорожная карта требуют от нас визуализации изменения архитектуры и регистрации решений. Это важные компоненты, помогающие объяснять и планировать изменения в программных проектах. Мы считаем, что диаграммы C4 и записи архитектурных решений (ADR) представляют собой наглядный способ регистрации изменений.

Использование диаграмм C4

В рамках представления практического примера мы задействовали три типа диаграмм C4 из модели C4. Мы считаем, что C4 — это лучший стандарт документации, позволяющий донести архитектуру, контекст и взаимодействие до широкого круга заинтересованных сторон. Возможно, вы зададитесь вопросом, а как же UML? Унифицированный язык моделирования (Unified Modeling Language, UML) представляет собой развитое средство для передачи программных архитектур. Но основная его проблема заключается в том, что большая часть того, что выполняется с помощью UML, не запоминается архитекторами и разработчиками и они быстро возвращаются к квадратам, кругам и ромбам. При этом понять структуру диаграмм, прежде чем перейти к техническому содержанию обсуждения, становится почти невозможно. Многие диаграммы попадают в историю проекта только тогда, когда кто-то по ошибке использует нестираемый маркер вместо маркера, стираемого сухой салфеткой. Модель C4 представляет собой упрощенный набор диаграмм. Они служат ориентиром для архитектуры проекта на различных уровнях детализации.

Диаграмма контекста C4

Диаграмма C4 контекста конференц-системы была представлена на рис. В.1. Цель этой диаграммы — задать контекст как для технической, так и для нетехнической аудитории. Во многих случаях обсуждения архитектуры его участники сразу же погружаются в низкоуровневые детали и упускают из виду контекст высокоуровневых взаимодействий. Задумайтесь о последствиях неправильного построения диаграммы контекста системы — обобщение подхода может сэкономить месяцы работы по устранению недоразумений.

Диаграмма контейнеров C4

Контейнер в C4 определяется как «то, что должно быть запущено для работы всей системы» (например, база данных конференции). Диаграммы контейнеров носят технический характер и основаны на контекстной диаграмме системы более высокого уровня. В то время как на рис. В.1 представлена общая картина конференц-системы, диаграмма контейнеров, показанная на рис. В.2, помогает увидеть техническую структуру основных участников архитектуры и определяет детали взаимодействия клиента с конференц-системой.



Контейнер конференц-приложения представлен на рис. В.2 как просто *программное обеспечение*. Обычно контейнер C4 содержит более подробную информацию о типе контейнера (например, *Java Spring Application*). Однако в этой книге мы стараемся избегать конкретизации технологий, если это не поможет продемонстрировать конкретное решение. Преимущество API и, собственно, современных приложений заключается в том, что в пространстве решений существует значительная гибкость.

Диаграмма компонентов C4

Диаграмма компонентов C4, показанная на рис. В.3, помогает определить роли и обязанности внутри каждого контейнера, а также их взаимодействие между собой. Эта диаграмма дает очень полезную карту кодовой базы и полезна при запросе подробной информации о контейнере. Подумайте в начале работы над новым проектом: просмотр *самодокументируемой* кодовой базы — это один из подходов, но собрать всё воедино может быть непросто. Диаграмма компонентов раскрывает детали языка/стека, используемого для создания программного обеспечения. Для того чтобы не привязываться к определенным технологиям, мы используем термин *пакет/модуль*.

Использование записей архитектурных решений

Как разработчики, архитекторы, да и просто люди, мы все оказывались в ситуации, когда приходилось задавать себе вопрос: «О чем они думали?». Если вы когда-нибудь проезжали по автомагистрали М62 между Лидсом и Манчестером в Великобритании, то, возможно, были озадачены ее конструкцией. По мере подъема на холм трехполосное шоссе начинает уходить в сторону от встречного потока движения, пока в конце концов не появляется ферма Scott Hall, а это примерно 15 акров сельскохозяйственных угодий, приютившаяся между дорожными полосами. По местной легенде, владелец участка упрямылся и отказывался продать свою землю и переехать, поэтому инженеры просто построили дорогу, обходящую его⁴. Спустя пятьдесят лет появился документальный фильм, в котором рассказывается, что истинной причиной явился подземный геологический разлом, и поэтому автомагистраль должна была быть построена именно таким образом. Когда людям приходится

⁴ Упрямый нрав местных жителей подтверждает вероятность такого объяснения.

догадываться, почему что-то было сделано именно так, а не иначе, ожидайте появления слухов, юмора и критики.

В архитектуре программного обеспечения существует множество ограничений, и их все необходимо учитывать. Поэтому важно, чтобы решения были зафиксированы и прозрачны. Записи ADR помогают сделать решения в архитектуре программного обеспечения понятными.

Мотивация определенных решений — один из самых трудных для отслеживания аспектов. Новый человек, приходящий в проект, может пребывать в недоумении, быть озадачен, восхищен или разгневан каким-либо решением, принятым в прошлом.

Майкл Нейгард, создатель концепции ADR

В ADR есть четыре ключевых раздела: статус, контекст, решение и последствия. Запись ADR создается в статусе предложения и по результатам обсуждения обычно либо принимается, либо отклоняется. Не исключено также, что впоследствии решение может быть отменено новым ADR. Контекст помогает определить общую картину и описать проблему или рамки, необходимые для принятия решения. Хотя создание записи в блоге перед ADR и последующее размещение ссылки из ADR помогает сообществу следить за вашей работой, контекст не предназначен для публикации в блоге или подробного описания. В решении четко изложено, что и как вы планируете делать. В архитектуре все решения влекут за собой последствия или компромиссы, и их неправильное принятие иногда обходится невероятно дорого.

При рассмотрении ADR важно понять, согласны ли вы с решением, принятым в ADR, или существует альтернативный подход. Нерассмотренный альтернативный подход может стать причиной отклонения ADR. Отклоненная запись очень полезна, и большинство команд предпочитают сохранять их неизменными, чтобы зафиксировать изменения в перспективе. Записи ADR лучше всего работают, когда они расположены там, где ключевые участники могут их просмотреть, прокомментировать и помочь в продвижении записи к принятию.



Нам часто задают вопрос: в какой момент команда должна создать реестр ADR? Всегда полезно убедиться в том, что перед его созданием было проведено обсуждение, и запись станет результатом коллективного мышления в команде. Публикация ADR для широкой общественности дает возможность получить обратную связь, выходящую за рамки конкретного коллектива.

ADR-запись эволюции участников

На рис. В.4 показано наше решение сделать эволюционный шаг в архитектуре конференц-системы. Это существенное изменение, и оно требует создания записи ADR. В табл. В.1 приведен пример ADR, который мог быть предложен инженерной группой, владеющей конференц-системой.

Некоторые из указанных в ADR последствий достаточно серьезны и, безусловно, требуют дальнейшего обсуждения. Отложим их рассмотрение на последующие главы.

Таблица В.1. Запись ADR001 — отделение участников от унаследованной конференц-системы

Статус	Предложено
Контекст	Владельцы конференции запросили две новые крупные функции для текущей конференц-системы. Их необходимо реализовать, не нарушая работу системы. Конференц-система должна быть усовершенствована для поддержки мобильного приложения и интеграции с внешней системой CFP. И мобильное приложение, и внешняя система CFP должны иметь возможность доступа к участникам для регистрации пользователей в стороннем сервисе
Решение	Мы сделаем эволюционный шаг, показанный на рис. В.4, чтобы выделить компонент Участник в отдельный сервис. Это позволит выполнять разработку с использованием подхода API-First для сервиса Участник и вызывать API из унаследованного сервиса конференции. Такое решение также поддержит возможность разработки прямого доступа к сервису участников для предоставления информации о пользователе во внешнюю систему CFP
Последствия	Вызов сервиса Участник не будет выходить за рамки процесса и может привести к возникновению задержки, так что этот вопрос требует проведения тестирования. Сервис участников может стать единой точкой отказа в архитектуре, и нам, возможно, понадобится принять меры по снижению возможных нежелательных последствий работы одного сервиса Участник. Учитывая планируемую многопользовательскую модель этого сервиса, нам необходимо обеспечить хорошее проектирование, управление версиями и тестирование, чтобы уменьшить количество случайных критических изменений

Осваиваем API: руководства ADR

В рамках нашей книги, описывающей освоение архитектуры API, мы будем приводить руководства ADR. Они помогут осветить важные вопросы, которые необходимо задавать при принятии решений по рассматриваемой теме. Принять решение об архитектуре на основе API может быть очень непросто, и во многих ситуациях ответ на этот вопрос начинается с фразы: «Это зависит от...». Вместо того чтобы говорить «зависит от...» без контекста, руководство ADR поможет описать, от чего зависит решение, и обосновать свои решения. Руководство ADR можно использовать в качестве справочного материала. К нему можно возвращаться и перечитывать его, если вы столкнулись с конкретной проблемой. В табл. В.2 приведен формат руководства ADR и то, что от него следует ожидать.

Таблица В.2. Руководство ADR: формат

Решение	Описывается решение, которое может потребоваться принять при рассмотрении того или иного аспекта из этой книги
Пункты обсуждения	Этот раздел помогает определить ключевые моменты, требующие обсуждения при принятии решения об архитектуре API. В этом разделе мы расскажем о некоторых наших впечатлениях, повлиявших на принятие решения. И поможем вам определить ключевую информацию для принятия решения
Рекомендации	Мы дадим конкретные, необходимые при создании ADR рекомендации, объяснив, почему мы даем ту или иную рекомендацию

Заключение

Во введении мы заложили основу для конкретного практического примера, а также для используемого при обсуждении архитектур и управляемых API подхода:

- ◆ архитектура — это бесконечное путешествие, и API могут сыграть важную роль в ее развитии;
- ◆ API представляют собой абстракцию реализации и могут выполняться как в процессе, так и вне процесса. Часто архитекторы оказываются в ситуации перехода на внепроцессные API. О таких решениях и пойдет речь в этой книге;
- ◆ практический пример конференции предназначен для описания и объяснения существующих концепций. Во введении вам был показан небольшой эволюционный шаг по разворачиванию сервиса Участник в соответствии с возникающими требованиями бизнеса;
- ◆ вы познакомились с первыми тремя уровнями диаграмм C4 и ощутили их важность для совместного использования и передачи информации об архитектуре;
- ◆ записи ADR представляют собой ценный документ для принятия решений и имеют как текущую, так и историческую значимость в течение всего срока реализации проекта;
- ◆ вы ознакомились со структурой руководства ADR, используемого во всей книге для облегчения принятия решений.

Приняв решение о выделении сервиса участников из системы конференции, мы далее рассмотрим варианты разработки и спецификации API Участник.

Проектирование, создание и тестирование API

В этой части книги представлены базовые строительные блоки для архитектур, управляемых API.

- ♦ В *главе 1* вы узнаете об API на базе удаленного вызова процедур (Remote Procedure Call, RPC) и REST. Мы рассмотрим спецификации и схемы, рекомендуемые стандарты, стратегии управления версиями, а также то, как правильно выбрать API для своей системы.
- ♦ В *главе 2* рассказывается о тестировании API и о том, как различные стили тестирования лучше всего применять к архитектурам, управляемым API.

Проектирование, создание и спецификация API

https://t.me/it_boooks/2

При проектировании и создании API вам будет предложено множество вариантов. Построить сервис с помощью современных технологий и фреймворков можно невероятно быстро, но создание долговечного проекта требует тщательного продумывания и взвешивания. В этой главе мы рассмотрим REST и RPC для моделирования отношений производителя и потребителя в рассматриваемом примере.

Вы узнаете, как с помощью стандартов можно сократить время принятия проектных решений и избежать потенциальных проблем совместимости. Познакомьтесь со спецификациями OpenAPI, их практическим применением для команд и оцените важность управления версиями.

Взаимодействия на основе RPC задаются с помощью схем. Для сравнения и сопоставления с REST-подходом мы рассмотрим систему gRPC, а также различные факторы, требующие внимания при моделировании обменов, с учетом вариантов REST и gRPC. Мы узнаем о возможности предоставления REST и RPC API в одном и том же сервисе и выясним, правильно ли это делать.

Практический пример: проектирование API Участник

Во *введении* мы решили изменить нашу унаследованную конференц-систему и перейти к архитектуре, основанной на API. В качестве первого шага к этому изменению мы создадим новый сервис под названием Участник, который будет представлять соответствующий API Участник (Attendee). Мы также привели там узкое определение API. Для эффективного проектирования необходимо более широко рассматривать обмен между производителем и потребителем и, что еще важнее, определить, кто такие производитель и потребитель. Производитель принадлежит команде участников. Эта команда поддерживает два ключевых отношения:

- ♦ команда участников владеет производителем, а команда унаследованной конференц-системы — потребителем. Между этими двумя командами существует тесная связь, и любые изменения в структуре легко координируются. Возможно достижение сильной связности между сервисами производителя и потребителя;
- ♦ команда участников владеет производителем, а команда внешней системы CFP — потребителем. Между командами существует взаимосвязь, но любые изменения должны быть согласованы, чтобы не нарушить интеграцию. Здесь требуется слабая связанность, и необходимо тщательно контролировать вносимые изменения.

На протяжении всей этой главы мы будем сравнивать и сопоставлять подходы к проектированию и созданию API Участник.

Знакомство с REST

Передача репрезентативного состояния (REpresentation State Transfer, REST) представляет собой набор архитектурных ограничений, наиболее часто применяемых при использовании HTTP в качестве базового транспортного протокола. В диссертации «Architectural Styles and the Design of Network-based Software Architectures»¹ Роя Филдинга дается полное определение REST. С практической точки зрения, чтобы считаться RESTful, ваш API должен соответствовать следующим требованиям:

- ♦ обеспечено моделирование взаимодействия между производителем и потребителем, при котором производитель моделирует ресурсы, доступные потребителю для взаимодействия;
- ♦ запросы от производителя к потребителю выполняются без сохранения состояния — т. е. производитель не кеширует информацию о предыдущем запросе. И чтобы выстроить цепочку запросов на тот или иной ресурс, потребитель должен отправить производителю для обработки всю необходимую информацию;
- ♦ запросы кешируются, т. е. производитель может предоставлять потребителю подсказки, где это уместно. В HTTP это часто обеспечивается информацией, содержащейся в заголовке;
- ♦ потребителю передается единый интерфейс. В ближайшее время вы изучите использование глаголов, ресурсов и других шаблонов;
- ♦ это многоуровневая система, абстрагирующаяся от сложности систем, находящихся за REST-интерфейсом. Например, потребитель не должен знать или заботиться о том, взаимодействует ли он с базой данных или другими сервисами.

Знакомство с REST и HTTP на примере

Рассмотрим пример использования REST по протоколу HTTP. Следующий обмен представляет собой GET-запрос, где GET (от *англ.* Get — получить) обозначает метод или глагол. Такой глагол, как GET, описывает действие, выполняемое над определенным ресурсом. В нашем примере мы рассматриваем ресурс участников: `attendees`.

Заголовок `Accept` передается для определения типа желаемого потребителем содержимого. REST определяет понятие представления в теле и позволяет определять метаданные представления в заголовках.

В примерах, приведенных в этой главе, мы представляем запрос над разделителем `---`, а ответ — под ним:

¹ См. <https://oreil.ly/VZ8VV>.

```
GET http://mastering-api.com/attendees
Accept: application/json
---
200 OK
Content-Type: application/json
{
  "displayName": "Jim",
  "id": 1
}
```

Ответ включает в себя код состояния и сообщение от сервера, что позволяет потребителю опросить результат операции на ресурсе на стороне сервера. Код состояния этого запроса: 200 OK. Это означает, что запрос был успешно обработан производителем. В теле ответа возвращается JSON-представление, содержащее информацию об участниках конференции. Многие типы содержимого могут быть возвращены из REST, однако важно учитывать, может ли тип содержимого пройти синтаксический анализ на стороне потребителя. Например, возврат файла `application/pdf` будет считаться допустимым. Но этот формат файла не представляет собой обмен, доступный для простого использования другой системой. Подходы к моделированию типов содержимого, в первую очередь на примере JSON, мы рассмотрим далее в этой главе.



REST относительно прост в реализации, поскольку отношения между клиентом и сервером выполняются без сохранения состояния, т. е. никакое состояние клиента не сохраняется на сервере. Клиент должен передавать контекст обратно серверу при последующих запросах. Например, запрос `http://mastering-api.com/attendees/1` должен получить дополнительную информацию о конкретном участнике.

Модель зрелости Ричардсона

Выступая на конференции QCon² в 2008 году, Леонард Ричардсон рассказал о своем опыте изучения множества REST API и описал уровни принятия, применяемые командами при создании API с точки зрения REST. Мартин Фаулер также рассказал об эвристике зрелости Ричардсона в своем блоге³. В табл. 1.1 приведены различные уровни, представленные в эвристике зрелости Ричардсона, и их применение к RESTful API.

Таблица 1.1. Эвристика зрелости Ричардсона

Уровень 0 — HTTP/RPC	Устанавливает, что API построен с использованием HTTP и представлен единым URI. Если взять предыдущий пример с <code>/attendees</code> и не применять глагол для определения намерения, то мы откроем конечную точку для обмена. По сути, это представляет собой реализацию RPC поверх протокола REST
----------------------	---

² См. <https://oreil.ly/scjnV>.

³ См. <https://oreil.ly/j6U3s>.

Таблица 1.1 (окончание)

Уровень 1 — Ресурсы	Устанавливает использование ресурсов и начинает соответствовать идее моделирования ресурсов в контексте URI. В нашем примере если бы мы добавили GET /attendees/1, возвращающий конкретного участника, то это стало бы похоже на API первого уровня. Мартин Фаупер проводит аналогию с классическим объектно-ориентированным миром по введению идентичности
Уровень 2 — Глаголы (Методы)	Начинается внедрение корректного моделирования URI нескольких ресурсов. Доступ к ним осуществляется различными методами запроса (также известными как HTTP-глаголы), в зависимости от влияния ресурсов на сервер. API уровня 2 может гарантировать, что методы GET не будут влиять на состояние сервера и представлять несколько операций над одним и тем же URI ресурса. В нашем примере добавление DELETE /attendees/1, PUT /attendees/1 представило бы понятие API, соответствующего второму уровню
Уровень 3 — Средства управления гипермедиа	Это воплощение REST-дизайна, предполагающее навигацию по API с помощью архитектурных ограничений HATEOAS (Hypertext As The Engine Of Application State) ⁴ . В нашем примере при вызове GET /attendees/1 ответ будет содержать действия, доступные для выполнения над объектом, возвращенным с сервера. В частности, речь идет о возможности обновления или удаления участника, а также о том, что должен сделать клиент для этого. С практической точки зрения уровень 3 редко используется в современных RESTful HTTP-сервисах, и хотя навигация остается преимуществом в системах с гибким стилем пользовательского интерфейса, она не подходит для межсервисных вызовов API. Использование HATEOAS было бы хлопотным занятием, и его часто сокращают, заранее имея полную спецификацию возможных взаимодействий при программировании для производителя

При проектировании API-обменов важно учитывать различные уровни зрелости Ричардсона. Переход на второй уровень позволит спроектировать для потребителя понятную модель ресурса с соответствующими действиями по отношению к ней. В свою очередь, это уменьшает связанность и скрывает все детали сервиса поддержки. Позже мы также увидим, как эта абстракция применяется к формированию версий.

Если потребителем стала команда CFP, то хорошей отправной точкой будет моделирование обмена с низкой связанностью и проектирование RESTful-модели. Если потребителем стала команда унаследованной конференц-системы, можно выбрать RESTful API, но есть и другой вариант с RPC. Для того чтобы приступить к изучению этого типа традиционного моделирования с подходом восток-запад, рассмотрим RPC.

Введение в API удаленного вызова процедур (RPC)

Удаленный вызов процедур (Remote Procedure Call, RPC) подразумевает вызов метода в одном процессе, но выполнение кода в другом процессе. В то время как REST может проектировать модель домена и предоставляет потребителю абстракцию от базовой технологии, RPC предполагает раскрытие метода из одного процесса и возможность его прямого вызова из другого.

⁴ См. <https://oreil.ly/7F18d>.

Система gRPC — это современный высокопроизводительный RPC с открытым исходным кодом. gRPC находится под управлением Linux Foundation и представляет собой фактический стандарт RPC на большинстве платформ. На рис. 1.1 показан RPC-вызов в системе gRPC, при котором унаследованный сервис конференций вызывает удаленный метод сервиса Участник. gRPC-сервис Участник запускает и раскрывает gRPC-сервер на указанном порту, позволяя вызывать методы удаленно. На стороне клиента (унаследованного сервиса конференций) используется заглушка, абстрагирующая сложность выполнения удаленного вызова от библиотеки. gRPC требует наличия схемы, полностью покрывающей взаимодействие между производителем и потребителем.



Рис. 1.1. Пример диаграммы компонентов C4 с использованием gRPC

Ключевое отличие REST от RPC — состояние. REST по определению не имеет состояния — в случае RPC состояние зависит от реализации. Интеграции на основе RPC в определенных ситуациях также могут наращивать состояние в качестве части обмена. Такое наращивание состояния имеет удобство высокой производительности за счет потенциальной цены надежности и сложности маршрутизации. При использовании RPC модель стремится передать на уровне метода именно требуемую от вторичного сервиса функциональность. Такая необязательность состояния может привести к возможно более сильной связанности между производителем и потребителем. Связанность — это не всегда плохо, особенно в сервисах типа восток-запад, где производительность является ключевым фактором.

Краткое упоминание о языке GraphQL

Прежде чем подробно рассмотреть стили REST и RPC, нельзя не упомянуть о языке GraphQL и о том, какое место он занимает в мире API. RPC предлагает доступ к ряду отдельных функций, предоставляемых производителем, но обычно не рас-

ширяет модель или абстракцию для потребителя. С другой стороны, REST расширяет модель ресурсов для одного API, предоставляемого производителем. С помощью API-шлюзов можно предлагать несколько API на одном базовом URL. Более подробно мы рассмотрим это в главе 3. Если мы предложим таким образом несколько API, то потребителю придется последовательно выполнять запросы для формирования состояния на стороне клиента. Потребителю также необходимо понимать структуру всех сервисов, участвующих в запросе. Такой подход расточителен, если потребителя интересует только подмножество полей в ответе. Мобильные устройства ограничены небольшими экранами и доступностью сети, поэтому GraphQL отлично подходит для этого сценария взаимодействия.

GraphQL представляет собой технологический слой над существующими сервисами, хранилищами данных и API. Он обеспечивает язык запросов к различным источникам. Язык запросов позволяет клиенту запрашивать именно необходимые поля, включая те из них, что охватывают несколько API. В GraphQL используется язык схем GraphQL, позволяющий определять типы в отдельных API и способы объединения API. Одним из основных преимуществ внедрения схемы GraphQL в вашу систему станет возможность предоставления единой версии для всех API, что устраняет необходимость в потенциально сложном управлении версиями на стороне потребителя.

GraphQL отлично подходит в тех случаях, когда потребителю нужен унифицированный API-доступ к широкому спектру взаимосвязанных сервисов. Схема обеспечивает связь и расширяет доменную модель, позволяя заказчику точно указать, что требуется на стороне потребителя. Это весьма пригодно для моделирования пользовательского интерфейса, а также систем отчетности или систем типа хранилищ данных. В системах, где огромные объемы данных хранятся в различных подсистемах, GraphQL может стать идеальным решением для абстрагирования от внутренней сложности системы.

Можно наложить GraphQL на существующие унаследованные системы и использовать его в качестве фасада, чтобы скрыть сложность, хотя предоставление GraphQL поверх слоя хорошо продуманных API часто означает, что фасад проще в реализации и поддержке. GraphQL можно рассматривать как дополнительную технологию. Ее следует учитывать при проектировании и создании API. GraphQL также можно рассматривать как комплексный подход к построению целой экосистемы API.

В определенных сценариях GraphQL проявляет себя с лучшей стороны, и мы рекомендуем вам обратить внимание на следующие книги: «GraphQL: язык запросов для современных веб-приложений» (Питер, 2019)⁵ и «GraphQL in Action» (O'Reilly)⁶ для более глубокого погружения в эту тему.

⁵ См. <https://www.piter.com/product/graphql-yazyk-zaprosov-dlya-sovremennyh-veb-prilozheniy>.

⁶ См. <https://oreil.ly/zLVVG>.

Стандарты и структура REST API

Для REST разработано несколько базовых правил, но по большей части их реализация и проектирование оставлены на усмотрение разработчика. Например, как лучше передать информацию об ошибках? Как должна быть реализована пагинация? Как случайно избежать создания API, в котором часто нарушается совместимость? На текущем этапе полезно сформулировать более практичное определение API, чтобы обеспечить единообразие и ожидания для различных реализаций. Здесь могут помочь стандарты или руководства, однако существует множество источников на выбор.

Для целей обсуждения проекта мы воспользуемся руководствами «Microsoft REST API Guidelines»⁷. Они представляют собой ряд внутренних рекомендаций, поступивших в открытый доступ. В руководстве используется стандарт RFC-2119, определяющий терминологию стандартов, такую как **MUST** (должен), **SHOULD** (следует), **SHOULD NOT** (не следует), **MUST NOT** (не должен) и т. д., позволяющую разработчику определить, являются ли требования факультативными или обязательными.



Поскольку стандарты REST API развиваются, открытый список стандартов API доступен на странице книги в Github⁸. Передавайте нам через запрос на извлечение любые открытые стандарты, которые, по вашему мнению, было бы полезно рассмотреть другим читателям.

Рассмотрим проект API участников с использованием руководства «Microsoft REST API Guidelines» и введем конечную точку для создания нового участника — `attendee`. Если вы знакомы с REST, то у вас сразу возникнет мысль использовать метод `POST`:

```
POST http://mastering-api.com/attendees
{
  "displayName": "Jim",
  "givenName": "James",
  "surname": "Gough",
  "email": "jim@mastering-api.com"
}
---
201 CREATED
Location: http://mastering-api.com/attendees/1
```

Заголовок `Location` раскрывает местоположение нового ресурса, созданного на сервере, и в этом API мы моделируем уникальный идентификатор пользователя. Можно задействовать поле `email` в качестве уникального идентификатора, однако руководство «Microsoft REST API Guidelines» в разделе 7.9 рекомендует не использовать в качестве части URL персональные данные (Personally Identifiable Information, PII).

⁷ См. <https://oreil.ly/H0lfn>.

⁸ См. <https://oreil.ly/jWx2x>.



Причина удаления конфиденциальных данных из URL заключается в том, что пути или параметры запроса могут быть случайно кешированы в сети — например, в журналах сервера или в других местах.

Еще один аспект API, способный вызвать затруднения, — это именование. В разд. «Версионирование API» рассказывается, что такая простая операция, как изменение имени, может нарушить совместимость. В руководстве «Microsoft REST API Guidelines» приведен краткий список рекомендуемых стандартных имен. Однако команды должны расширить его, чтобы иметь общий словарь данных домена, дополняющий стандарты. Во многих организациях очень полезно проактивно изучать требования к проектированию данных и в некоторых случаях к руководству ими. Организации, обеспечивающие согласованность всех API, предлагаемых компанией, демонстрируют единообразие, позволяющее потребителям понимать и связывать ответы. В некоторых областях уже существует широко известная терминология — используйте ее!

Коллекции и пагинация

Представляется целесообразным моделировать запрос `GET /attendees` как ответ, содержащий необработанный массив. Следующий фрагмент исходного кода показывает пример того, как это может выглядеть в качестве тела ответа:

```
GET http://mastering-api.com/attendees
---
200 OK
[
  {
    "displayName": "Jim",
    "givenName": "James",
    "surname": "Gough",
    "email": "jim@mastering-api.com",
    "id": 1,
  },
  ...
]
```

Рассмотрим альтернативную модель запроса `GET /attendees`, в которой массив участников вложен в объект. Может показаться странным, что ответ в виде массива возвращается в объекте, однако причина этого заключается в том, что он позволяет моделировать большие коллекции и пагинацию. Пагинация подразумевает возврат частичного результата с одновременным предоставлением инструкций о том, как потребитель может запросить следующий набор результатов. Это плоды ретроспективного анализа: добавление пагинации позже и преобразование из массива в объект для добавления `@nextLink` (как рекомендуют стандарты) нарушило бы совместимость:

```
GET http://mastering-api.com/attendees
---
200 OK
{
  "value": [
    {
      "displayName": "Jim",
      "givenName": "James",
      "surname": "Gough",
      "email": "jim@mastering-api.com",
      "id": 1,
    }
  ],
  "@nextLink": "{opaqueUrl}"
}
```

Фильтрация коллекций

Наша конференция с одним участником выглядит немного пустой, однако при увеличении размера коллекций нам может понадобиться добавить фильтрацию в дополнение к пагинации. Стандарт фильтрации предоставляет язык выражений в рамках REST для стандартизации поведения запросов фильтрации, основанный на стандарте OData. Например, мы можем найти всех участников по имени Jim с помощью команды `displayName`:

```
GET http://mastering-api.com/attendees?$filter=displayName eq 'Jim'
```

Не обязательно с самого начала выполнять все функции фильтрации и поиска. Однако проектирование API в соответствии со стандартами позволит разработчику поддерживать развивающуюся архитектуру API без нарушения совместимости для потребителей. Фильтрация и формирование запросов — это хорошо представленная в GraphQL функция, особенно если становятся актуальными формирование запросов и фильтрация по многим вашим сервисам.

Обработка ошибок

Важным моментом при расширении API для потребителей станет определение того, что должно происходить в различных сценариях возникновения ошибок. Стандарты ошибок⁹ полезно определить заранее и поделиться ими с производителями для обеспечения согласованности. Важно, чтобы ошибки показывали потребителю, что именно произошло с запросом, т. к. это позволит избежать увеличения объема поддержки, требуемой для API.

В руководстве говорится: «Для неуспешных условий разработчики *должны* иметь возможность написать один фрагмент кода, последовательно обрабатывающий ошибки». Потребителю должен быть предоставлен точный код состояния, т. к. час-

⁹ См. <https://oreil.ly/creK4>.

то потребители строят логику на основе кода состояния, содержащегося в ответе. Мы видели много API, возвращающих ошибки в теле, но при этом ответ относился к типу 2xx, используемому для обозначения успеха. Коды статуса 3xx для перенаправления активно используются некоторыми реализациями потребляющих библиотек, позволяя провайдерам перемещаться и обращаться к внешним источникам. Код 4xx обычно указывает на ошибку на стороне клиента. В этот момент содержимое поля `message` крайне полезно для разработчика или конечного пользователя. Код 5xx обычно свидетельствует о сбое на стороне сервера, и некоторые клиентские библиотеки при таких отказах выполняют повторную попытку запроса. Важно продумать и задокументировать, что произойдет в сервисе в случае непредвиденного сбоя. Например, в платежной системе код 500 означает, прошел платеж или нет.



Убедитесь, что сообщения об ошибках, отправляемые внешнему потребителю, не содержат трассировки стека и другой конфиденциальной информации. Эта информация поможет хакеру, нацеленному на компрометацию системы. В структуре ошибок в руководстве Microsoft есть понятие *InnerError*. Оно может быть полезно для размещения более подробных трассировок стека / описаний проблем. Его использование было бы невероятно полезно для отладки, но эти сообщения необходимо удалить перед отправкой данных потребителю.

Мы лишь слегка коснулись темы создания REST API, но очевидно, что при создании API необходимо принять множество важных решений. Если объединить стремление представить интуитивно понятные API, согласованные и допускающие развитие и совместимость API, то стоит принять стандарт API на ранней стадии развития системы.

Руководство ADR: выбор стандарта API

Важные темы для принятия решения по стандартам API приведены в табл. 1.2, и мы настоятельно рекомендуем их рассмотреть. Существует целый ряд рекомендаций, включая рассмотренные в этом разделе рекомендации Microsoft, и выбор лучшей из них в плане соответствия стилю создаваемых API станет ключевым решением.

Таблица 1.2. Руководство по стандартам API

Решение	Какой стандарт API нам следует принять?
Пункты обсуждения	Существуют ли на текущий момент другие стандарты в организации? Можем ли мы распространить эти стандарты на внешних потребителей? Используем ли мы какие-либо имеющиеся свои стандарты API сторонних производителей, которые мы должны предоставить потребителю (например, Identity Services)? Каковы последствия отсутствия стандарта для наших потребителей?
Рекомендации	Выберите наилучшим образом соответствующий культуре организации и форматам стандарт API. Возможно, таковые уже имеются в инструментарии разработки. Будьте готовы к развитию и дополнению стандарта любыми изменениями, связанными с конкретной областью/отраслью. Начните с чего-то раннего, чтобы впоследствии не пришлось ломать совместимость для достижения постоянства. Критически относитесь к существующим API. Есть ли у них формат, понятный потребителям, или требуются дополнительные усилия для предоставления контента?

Определение REST API с помощью OpenAPI

Как мы уже убедились, проектирование API представляет собой основополагающий фактор успеха API-платформы. Следующий момент, требующий обсуждения, — это совместное использование API с разработчиками, применяющими наши API.

Маркетплейсы API предоставляют публичные или частные списки API, доступных потребителю. Разработчик может просмотреть документацию и быстро опробовать API в браузере, чтобы изучить его поведение и функциональность. Публичные и приватные маркетплейсы API заняли видное место в потребительском пространстве REST API. Успех REST API был обусловлен как техническим ландшафтом, так и низким барьером входа как для клиента, так и для сервера.

По мере роста числа API быстро возникла необходимость в механизме обмена с потребителями информацией о форме и структуре API. Именно поэтому для разработки спецификации OpenAPI Specification (OAS) лидерами индустрии API была создана организация OpenAPI Initiative. Первоначально эталонной реализацией спецификаций OpenAPI был набор инструментов Swagger, но в настоящее время большинство инструментальных средств перешли на использование OpenAPI.

Спецификации OpenAPI — это представления API на основе JSON или YAML. Они описывают структуру объектов домена, участвующих в обмене, и любые требования к безопасности API. Помимо структуры, они передают метаданные об API, включая любые юридические или лицензионные требования, а также содержат документацию и примеры, полезные для разработчиков, использующих API. Спецификации OpenAPI — это важная концепция, связанная с современными REST API, и на основе этих спецификаций создано множество инструментов и продуктов.

Практическое применение спецификаций OpenAPI

Как только OAS становится общей, начинает проявляться мощь спецификации. В OpenAPI.Tools¹⁰ задокументирован весь спектр доступных инструментов с открытым и закрытым исходным кодом. В этом разделе мы рассмотрим некоторые варианты практического применения инструментов, основанные на их взаимодействии со спецификацией OpenAPI.

В ситуации, когда команда CFP выступает в роли потребителя, совместное использование OAS позволяет ей понять структуру API. Использование некоторых из приведенных далее практических приложений может способствовать как улучшению работы разработчиков, так и поддержанию работоспособности обмена данными.

¹⁰ См. <https://oreil.ly/8dFUS>.

Генерация кода

Пожалуй, одной из наиболее полезных функциональных возможностей OAS является возможность генерации клиентского кода для использования API. Как уже говорилось ранее, в код можно включить полную информацию о сервере, безопасности и, конечно же, о самой структуре API. Имея всю эту информацию, становится возможным сгенерировать ряд объектов модели и сервиса, представляющих и вызывающих API. Проект OpenAPI Generator¹¹ поддерживает широкий спектр языков и инструментальных цепочек. Например, в Java можно выбрать Spring или JAX-RS, а в TypeScript — сочетание TypeScript с любимым фреймворком. Также можно сгенерировать заглушки реализации API из OAS.

В связи с этим возникает важный вопрос: что должно быть на первом месте — спецификация или код на стороне сервера? В *главе 2* мы рассмотрим «трассировку контрактов», представляющую собой поведенческий подход к тестированию и созданию API. Проблема спецификаций OpenAPI заключается в том, что сами по себе они передают только форму API. Спецификации OpenAPI не полностью моделируют семантику (или ожидаемое поведение) API в различных условиях. Если вы собираетесь представить API внешним пользователям, важно смоделировать и протестировать все возможные варианты поведения, чтобы впоследствии не пришлось кардинально менять API.

API должны разрабатываться с точки зрения потребителя и учитывать требование абстрагирования базового представления для снижения связанности. Важно иметь возможность свободно выполнять рефракторинг компонентов за кулисами, не нарушая совместимости с API, иначе абстракция API теряет свою ценность.

Валидация OpenAPI

Спецификации OpenAPI полезны для валидации содержимого обмена, чтобы убедиться, что запрос и ответ соответствуют ожиданиям спецификации. На первый взгляд может показаться, что это не очень полезная возможность — если код генерируется, то, конечно, обмен всегда будет правильным? Одним из практических применений валидации OpenAPI стало обеспечение безопасности и инфраструктуры API. Во многих организациях распространена зональная архитектура. При таком подходе для защиты сети от входящего трафика используется понятие *демилитаризованной зоны*, или ДМЗ (DeMilitarized Zone, DMZ). Полезной функциональной возможностью является опрос сообщений в ДМЗ и прерывание трафика в случае несоответствия спецификации. Более подробно вопросы безопасности рассматриваются в *главе 6*.

Примером валидатора может служить инструмент под названием `swagger-request-validator`¹² компании Atlassian. Он способен проверять содержимое файлов JSON REST. В проекте также имеются адаптеры, интегрирующиеся с различными ими-

¹¹ См. <https://oreil.ly/wx0Ce>.

¹² См. <https://oreil.ly/1ILCzT>.

тационными и тестовыми фреймворками для обеспечения соответствия API-спецификациям в процессе тестирования. Инструмент содержит класс `OpenApiInteractionValidator`, используемый для создания отчета о валидации (`ValidationReport`) при обмене данными. Следующий код демонстрирует построение валидатора на основе спецификации, включая все методы `basePathOverrides`. Это может потребоваться при развертывании API на инфраструктуре, изменяющей путь. Отчет о валидации формируется на основе анализа запроса и ответа в точке, где выполняется проверка:

```
//Используя местоположение спецификации, создайте валидатор взаимодействия
//Определение базового пути полезно, если валидатор будет использоваться
//за шлюзом/прокси
final OpenApiInteractionValidator validator = OpenApiInteractionValidator
    .createForSpecificationUrl(specUrl)
    .withBasePathOverride(basePathOverride)
    .build;

//Объекты Request и Response могут быть преобразованы или созданы
//с помощью конструктора
final ValidationReport report = validator.validate(request, response);

if (report.hasErrors()) {
    // Захват или обработка информации об ошибках
}
```

Примеры и создание имитаций

OAS может предоставить примеры ответов для путей, указанных в спецификации. Примеры, как мы уже говорили, полезны для документации — они способны помочь разработчикам понять ожидаемое поведение API. В некоторых продуктах стали использоваться примеры, позволяющие пользователю запрашивать API и возвращать примеры ответов от имитируемого сервиса. Это может быть очень полезно в таких функциях, как портал разработчика, позволяющих разработчикам изучать документацию и вызывать API. Другой полезной функциональной возможностью имитаций и примеров является обмен идеями между производителем и потребителем до того, как они приступят к созданию сервиса. Возможность «опробовать» API часто оказывается более ценной, чем попытка проверить, соответствует ли спецификация вашим требованиям.

Примеры потенциально могут создавать интересную проблему, заключающуюся в том, что эта часть спецификации по сути является строковой (для моделирования XML/JSON и т. д.). `Openapi-examples-validator`¹³ выполняет валидацию на соответствие примера спецификации OAS для соответствующего компонента (`component`) запроса/ответа API.

¹³ См. <https://oreil.ly/bM9fp>.

Обнаружение изменений

Спецификации OpenAPI также могут быть использованы для обнаружения изменений в API, что может оказаться невероятно полезно в рамках конвейера DevOps. Обнаружение изменений для обеспечения обратной совместимости очень важно, но сначала необходимо более подробно разобраться с версиями API.

Версионирование API

Мы изучили преимущества совместного использования спецификации OAS с потребителем, в том числе скорость интеграции. Рассмотрим случай, когда с API начинают работать несколько потребителей. Что произойдет, если в API будут внесены изменения или один из потребителей попросит добавить в API новые функциональные возможности?

Давайте сделаем шаг назад и подумаем, что это могла бы быть библиотека кода, встроенная в наше приложение во время компиляции. Любые изменения в библиотеке будут упакованы в качестве новой версии, и до тех пор, пока код не будет перекомпилирован и протестирован на новой версии, никакого влияния на эксплуатируемые приложения он не окажет. Поскольку API представляют собой работающие сервисы, у нас есть несколько вариантов обновления, и они сразу же становятся доступными при запросе изменений.

- ♦ Выполните релиз и развертывание новой версии в новом местоположении.

Старые приложения продолжают работать со старой версией API. С точки зрения потребителя это вполне нормально, поскольку он переходит на новое местоположение и API только в том случае, если ему нужны новые возможности. Однако владельцу API необходимо поддерживать и управлять несколькими версиями API, включая все необходимые исправления и устранение ошибок.

- ♦ Выполните релиз новой версии API — она должна быть обратно совместима с предыдущей версией.

Это позволяет вносить аддитивные изменения, не затрагивая существующих пользователей API. Изменений со стороны потребителя не требуется, но может потребоваться учесть время простоя или доступность старой и новой версий во время обновления. Если есть небольшое исправление, изменяющее такую мелочь, как неправильное название поля, то это нарушит совместимость.

- ♦ Нарушите совместимость с предыдущим API, и всем потребителям потребуется обновить код для использования нового API.

Поначалу это решение кажется ужасной идеей, т. к. оно приведет к неожиданным поломкам в эксплуатируемой системе¹⁴. Однако может возникнуть ситуация, когда не будет возможности избежать нарушения совместимости со старыми версиями. Подобные изменения могут вызвать блокировку всей системы, что требует координации времени простоя.

¹⁴ Мы много раз оказывались в подобной ситуации, приступая к работе в понедельник!

Проблема заключается в том, что все эти варианты модернизации имеют не только преимущества, но и недостатки как для потребителя, так и для производителя. На самом деле желательно иметь возможность поддерживать комбинацию всех трех вариантов. Для этого необходимо ввести правила, касающиеся формирования версий и того, как версии предоставляются потребителю.

Семантическое версионирование

Семантическое версионирование¹⁵ (Semantic Versioning, или SemVer) предлагает применимый к REST API подход, позволяющий получить комбинацию упомянутых в предыдущем разделе вариантов обновления. Такой метод управления версиями определяет числовое представление, прикрепляемое к релизу API. Число определяется по уровню изменения поведения по сравнению с предыдущей версией с использованием следующих правил:

- ◆ *мажорная* версия вносит несовместимые с предыдущими версиями API изменения. В платформе API переход на новую мажорную версию — это активное решение потребителя. Вероятно, будет подготовлено руководство по миграции и отслеживанию перехода потребителей на новый API;
- ◆ *минорная* версия вносит изменения, совместимые с предыдущей версией API. В платформе API-сервисов допустимо, чтобы потребители получали минорные версии без внесения активных изменений на стороне клиента;
- ◆ *патч-версия* не изменяет существующую функциональность и не вводит новую, а используется для исправления ошибок в существующей Мажор.Минор-версии функциональности.

Формат для семантического версионирования может быть представлен как Мажор.Минор.Патч. Например, 1.5.1 будет представлять мажорную версию 1, минорную версию 5, с обновлением патча 1. В главе 5 рассматривается, как семантическое версионирование связано с концепцией жизненного цикла API и релизов.

Спецификация и версионирование OpenAPI

Теперь, когда мы познакомились с версионированием, можно рассмотреть примеры критических и некритических изменений на примере спецификации API Участник. Существует несколько инструментов для сравнения спецификаций, и в нашем примере мы воспользуемся инструментом `openapi-diff`¹⁶ из состава OpenAPITools.

Начнем с критического изменения — изменим имя поля `givenName` на `firstName`. Это критическое изменение, поскольку потребители будут ожидать синтаксический анализ поля под названием `givenName`, а не `firstName`. Можно запустить инструмент под названием `diff` из контейнера `docker` с помощью следующей команды:

¹⁵ См. <https://semver.org/>.

¹⁶ См. <https://oreil.ly/QrgTf>.

```
$docker run --rm -t \  
-v $(pwd):/specs:ro \  
openapitools/openapi-diff:latest /specs/original.json /specs/first-name.json
```

```
=====  
...  
- GET /attendees  
  Return Type:  
    - Changed 200 OK  
  Media types:  
    - Changed */*  
    Schema: Broken compatibility  
    Missing property: [n].givenName (string)
```

```
-----  
-- Result --  
-----
```

API changes broke backward compatibility

```
-----
```

Можно попробовать добавить новый атрибут к типу возврата `/attendees`, чтобы ввести дополнительное поле возраста под названием `age`. Добавление новых полей не нарушает существующего поведения и, следовательно, не нарушает совместности:

```
$ docker run --rm -t \  
-v $(pwd):/specs:ro \  
openapitools/openapi-diff:latest --info /specs/original.json /specs/age.json
```

```
=====  
...  
- GET /attendees  
  Return Type:  
    - Changed 200 OK  
  Media types:  
    - Changed */*  
    Schema: Backward compatible
```

```
-----  
-- Result --  
-----
```

API changes are backward compatible

```
-----
```

Стоит попробовать, чтобы понять, какие изменения будут совместимы, а какие нет. Внедрение такого инструментария в рамках конвейера API позволит избежать неожиданных несовместимых изменений для потребителей. Спецификации OpenAPI представляют собой важную часть программы API, а в сочетании с инструментарием, версионированием и жизненным циклом они оказывают неоценимую помощь.



Инструменты часто зависят от версии OpenAPI, поэтому важно проверить, поддерживает ли инструмент используемую в текущий момент спецификацию. В приведенном примере мы попробовали использовать инструмент `diff` с более ранней версией спецификации, и никаких критических изменений обнаружено не было.

Реализация RPC с помощью gRPC

У сервисов типа восток-запад, таких как Участник, будет более высокий трафик, и их можно реализовать как микросервисы, используемые в рамках всей архитектуры. gRPC может быть для сервисов восток-запад более подходящим инструментом, чем REST, — в силу меньшего объема передаваемых данных и меньшей скорости внутри экосистемы. Любые решения, касающиеся эффективности, всегда необходимо измерять, чтобы иметь соответствующую информацию.

Давайте рассмотрим использование Spring Boot Starter¹⁷ для быстрого создания gRPC-сервера. Следующий файл .proto моделирует объект участника attendee, описанный в примере спецификации OpenAPI. Как и в случае со спецификациями OpenAPI, генерация кода на основе схемы осуществляется быстро и поддерживается на нескольких языках.

Файл участников .proto определяет пустой запрос и возвращает повторяющийся ответ сервиса Attendee. В протоколах, использующих двоичное представление, важно отметить, что расположение и порядок полей имеют решающее значение, т. е. определяют компоновку сообщения. Добавление нового сервиса или нового метода, как и добавление поля в сообщение, обратно совместимо, однако следует соблюдать осторожность. Все добавляемые поля не должны быть обязательными для заполнения, иначе нарушится обратная совместимость.

Удаление или переименование поля нарушит совместимость, равно как и изменение типа данных поля. Изменение номера поля также станет проблемой, поскольку номера полей используются для идентификации полей через сеть. Ограничения кодирования с помощью gRPC означают, что определение должно быть очень конкретным. REST и OpenAPI достаточно просты, поскольку спецификация служит лишь руководством к действию¹⁸. Дополнительные поля и порядок не имеют значения в OpenAPI, поэтому версионность и совместимость еще более важны, когда речь идет о gRPC:

```
syntax = "proto3";
option java_multiple_files = true;
package com.masteringapi.attendees.grpc.server;
message AttendeesRequest {
}
message Attendee {
  int32 id = 1;
  string givenName = 2;
  string surname = 3;
  string email = 4;
}
```

¹⁷ См. <https://oreil.ly/opOij>.

¹⁸ Валидация спецификаций OpenAPI во время выполнения позволяет добиться большей строгости.

```
message AttendeeResponse {  
    repeated Attendee attendees = 1;  
}  
service AttendeesService {  
    rpc getAttendees(AttendeesRequest) returns (AttendeeResponse);  
}
```

Следующий Java-код демонстрирует простую структуру для реализации поведения на сгенерированных классах gRPC-серверов:

```
@GrpcService  
public class AttendeesServiceImpl extends  
    AttendeesServiceGrpc.AttendeesServiceImplBase {  
  
    @Override  
    public void getAttendees(AttendeesRequest request,  
        StreamObserver<AttendeeResponse> responseObserver) {  
        AttendeeResponse.Builder responseBuilder  
            = AttendeeResponse.newBuilder();  
        // заполнение ответа  
        responseObserver.onNext(responseBuilder.build());  
        responseObserver.onCompleted();  
    }  
}
```

Java-сервис, моделирующий этот пример, можно найти на странице книги в GitHub¹⁹. gRPC не может быть запрошен непосредственно из браузера без дополнительных библиотек, однако вы можете установить gRPC UI²⁰, чтобы использовать браузер для тестирования. Программа `grpcurl` также предоставляет инструмент командной строки:

```
$ grpcurl -plaintext localhost:9090 \  
com.masteringapi.attendees.grpc.server.AttendeesService/getAttendees  
{  
  "attendees": [  
    {  
      "id": 1,  
      "givenName": "Jim",  
      "surname": "Gough",  
      "email": "gough@mail.com"  
    }  
  ]  
}
```

gRPC предоставляет еще один вариант запроса к нашему сервису и определяет спецификацию для потребителя для генерации кода. Спецификация gRPC более стро-

¹⁹ См. <https://oreil.ly/GMy9m>.

²⁰ См. <https://oreil.ly/F4C78>.

гая, чем у OpenAPI, и требует, чтобы методы/интерфейсы были понятны потребителю.

Моделирование обмена данными и выбор формата API

Во *введении* мы рассмотрели концепцию трафика и разницу между запросами, поступающими извне экосистемы, и запросами внутри нее. Модель трафика представляет собой важный фактор при определении подходящего формата API для решения поставленной задачи. Когда мы получаем полный контроль над сервисами и обменами внутри нашей архитектуры, основанной на микросервисах, мы можем начать идти на компромиссы, на которые не смогли бы пойти с внешними потребителями.

Важно понимать, что эксплуатационные характеристики сервиса типа восток-запад, скорее всего, будут более применимы, чем характеристики сервиса север-юг. При обмене север-юг трафик, исходящий из внешней среды производителя, как правило, предполагает использование Интернета. Интернет вносит высокую степень задержки, и архитектура API всегда должна учитывать суммарный эффект от каждого сервиса. В архитектуре, основанной на микросервисах, вполне вероятно, что один запрос север-юг будет включать в себя несколько обменов восток-запад. Высокий обмен трафиком восток-запад должен быть эффективным, чтобы избежать каскадных замедлений, распространяющихся на потребителя.

Сервисы с высоким уровнем трафика

В нашем примере в качестве центрального мы определили сервис Участник. В архитектуре, основанной на микросервисах, компоненты будут отслеживать поле `attendeeId`. API, предлагаемые потребителям, смогут потенциально получать данные, хранящиеся в сервисе участников, и при более крупном масштабе системы это будет компонент с высоким уровнем трафика. Если частота обмена между сервисами высока, то стоимость передачи данных по сети, обусловленная размером полезной нагрузки и ограничениями одного протокола по сравнению с другим, будет становиться все более значительной по мере роста использования. Стоимость может выражаться как в денежных затратах на каждую передачу, так и в общем времени, затраченном на то, чтобы сообщение достигло места назначения.

Большие полезные нагрузки при обмене данными

Большие размеры полезной нагрузки также могут стать проблемой при обмене данными через API и способны привести к снижению производительности передачи данных по сети. JSON, наложенный на REST, пригоден для чтения человеком и часто бывает более многословным, чем фиксированное или двоичное представление, что приводит к увеличению размеров полезной нагрузки.



Распространенным заблуждением стало указание «человекочитаемости» в качестве основной причины использования JSON в процессе передачи данных. Когда мы применяем современные средства трассировки, количество раз, которое разработчику потребуется прочитать сообщение, по сравнению с производительностью

перестает быть весомым аргументом. Кроме того, большие JSON-файлы редко когда читаются от начала до конца. Более эффективное протоколирование и обработка ошибок могут смягчить аргумент «человекочитаемости».

Еще одним фактором при обмене с большой полезной нагрузкой становится время, требуемое компонентам для синтаксического анализа содержимого сообщения на объекте домена на уровне языка. Время выполнения синтаксического анализа форматов данных сильно различается в зависимости от языка реализации сервиса. Многие традиционные языки серверной части могут испытывать трудности при работе с JSON по сравнению, например, с двоичным представлением. Стоит изучить влияние синтаксического анализа и учесть этот момент при выборе формата обмена.

Преимущества производительности HTTP/2

Использование сервисов на базе HTTP/2 может помочь повысить производительность обмена трафиком за счет поддержки двоичного сжатия и кадрирования. Слой двоичного кадрирования (binary framing layer)²¹ прозрачен для разработчика, но за кулисами происходит разбиение и сжатие сообщения на более мелкие фрагменты. Преимущество двоичного кадрирования заключается в том, что оно позволяет полностью мультиплексировать запросы и ответы по одному соединению. Рассмотрим обработку списка в другом сервисе, где требуется получить 20 различных участников. Если бы мы получали их в виде отдельных HTTP/1-запросов, то потребовалось бы создать 20 новых TCP-соединений. Мультиплексирование позволяет нам выполнить 20 отдельных запросов через одно HTTP/2-соединение.

gRPC по умолчанию использует HTTP/2 и уменьшает размер обмена за счет применения двоичного протокола. Если пропускная способность представляет собой проблему или ведет к повышению затрат, то gRPC обеспечит преимущество, особенно при значительном увеличении размера полезной нагрузки контента. gRPC может быть выгоднее REST, если пропускная способность полезной нагрузки становится совокупной проблемой или сервис обменивается большими объемами данных. Если обмен данными в больших объемах происходит часто, стоит также рассмотреть некоторые асинхронные возможности gRPC.



Протокол HTTP/3 уже на подходе, и он изменит всё. В HTTP/3 используется QUIC — транспортный протокол, построенный на основе UDP. Более подробную информацию вы можете найти в документе HTTP/3 Explained²².

Старые форматы

Не все сервисы в архитектуре будут основаны на современных проектах. В главе 8 мы рассмотрим, как изолировать и провести эволюцию старых компонентов, по-

²¹ См. <https://oreil.ly/5Q17R>.

²² См. <https://oreil.ly/DM1j9>.

сколько они будут активно обсуждаться в развивающихся архитектурах. Важно, чтобы те, кто занимается архитектурой API, понимали общее влияние внедрения старых компонентов на производительность.

Рекомендация: моделирование обменов

Когда в роли потребителя выступает команда унаследованной конференц-системы, обмен, как правило, происходит по схеме восток-запад. Когда в роли потребителя выступает команда CFP, обмен, как правило, происходит по схеме север-юг. Разница в требованиях к производительности и связанности потребует от команд рассмотреть способ моделирования обмена. Некоторые аспекты, рекомендуемые для рассмотрения, приведены в табл. 1.3.

Таблица 1.3. Руководство по моделированию обменов

Решение	В каком формате необходимо моделировать API для сервиса?
Пункты обсуждения	Каким будет обмен: север-юг или восток-запад? Контролируем ли мы потребительский код? Существует ли сильная бизнес-область для нескольких сервисов или мы хотим позволить потребителям создавать свои собственные запросы? Какие соображения по поводу версионирования нам необходимо рассмотреть? Какова частота развертывания/изменения базовой модели данных? Если возникли проблемы с пропускной способностью или производительностью, необходимо определить, относится ли этот сервис к компонентам с высоким трафиком
Рекомендации	Если API будет использоваться внешними пользователями, то REST представляет собой низкий барьер для входа и обеспечивает сильную доменную модель. Наличие внешних пользователей также обычно означает, что желателен сервис с низкой связанностью и зависимостью. Если API взаимодействует между двумя сервисами, находящимися под пристальным контролем производителя, или доказано, что у сервиса высокий уровень трафика, рассмотрите вариант gRPC

Различные спецификации

В этой главе мы рассмотрели различные форматы API, которые следует учитывать в архитектуре API, и возникает последний вопрос: «Можем ли мы предоставить все форматы?». Ответ — да, можно поддерживать API, имеющий RESTful-представление, gRPC-сервис и соединения со схемой GraphQL. Однако это будет нелегко и, возможно, не совсем правильно. В этом заключительном разделе главы мы рассмотрим некоторые возможности мультиматного API и вероятные проблемы в его работе.

Существует ли «золотая спецификация»?

Файл .proto для участников и спецификация OpenAPI внешне не слишком отличаются друг от друга — они содержат одинаковые поля и оба имеют типы данных.

Можно ли сгенерировать файл `.proto` из OAS, используя инструмент `openapi2proto`²³? Запуск `openapi2proto--spec spec-v2.json` выведет файл `.proto`, поля которого по умолчанию будут упорядочены в алфавитном порядке. Всё будет хорошо до тех пор, пока мы не добавим в OAS новое обратно-совместимое поле, после чего вдруг ID всех полей изменятся, нарушив обратную совместимость.

Следующий пример файла `.proto` показывает, что поле под названием `a_new_field`, добавленное в начало по алфавиту, изменит двоичный формат и нарушит существующие сервисы:

```
message Attendee {
  string a_new_field = 1;
  string email = 2;
  string givenName = 3;
  int32 id = 4;
  string surname = 5;
}
```



Существуют и другие инструменты, позволяющие решить проблему преобразования спецификации, однако стоит отметить, что некоторые из них поддерживают только спецификацию OpenAPI версии 2. Время, затрачиваемое на переход между версиями 2 и 3 в некоторых инструментах, построенных на основе OpenAPI, требует, чтобы многие продукты поддерживали обе версии OAS.

Альтернативным вариантом служит плагин `grpc-gateway`²⁴. Он генерирует обратный прокси, обеспечивающий REST-фасад перед gRPC-сервисом. Обратный прокси генерируется во время сборки на основе файла `.proto` и обеспечивает наилучшее отображение на REST, аналогичное `openapi2proto`. Кроме того, в файле `.proto` можно указать расширения для сопоставления методов RPC с хорошим представлением в OAS:

```
import "google/api/annotations.proto";
//...
service AttendeesService {
  rpc getAttendees(AttendeesRequest) returns (AttendeeResponse) {
    option (google.api.http) = {
      get: "/attendees"
    };
  }
}
```

Использование плагина `grpc-gateway` дает нам еще один вариант представления как REST-, так и gRPC-сервиса. Однако этот плагин включает в себя несколько команд и настройку, знакомую только разработчикам, работающим с языком Go или средой сборки.

²³ См. <https://oreil.ly/11XL>.

²⁴ См. <https://oreil.ly/u2Em7>.

Проблемы комбинированных спецификаций

Здесь важно сделать шаг назад и подумать, что мы пытаемся сделать. При конвертации из OpenAPI мы фактически пытаемся преобразовать наше REST-представление в серию вызовов gRPC. То есть мы собираемся преобразовать расширенную модель гипермедиа домена в низкоуровневый вызов функций. Это потенциальное смещение различий между RPC и API, скорее всего, приведет к борьбе с совместимостью.

При преобразовании gRPC в OpenAPI мы сталкиваемся с аналогичной проблемой — задача состоит в том, чтобы взять gRPC и сделать его похожим на REST API. Это, скорее всего, создаст сложный ряд проблем при развитии сервиса.

Когда спецификации объединяются или генерируются на основе друг друга, возникает проблема версионирования. Важно помнить о том, что спецификации gRPC и OpenAPI поддерживают свои индивидуальные требования к совместимости. Необходимо принять активное решение о том, имеет ли смысл связывать домен REST с доменом RPC и увеличивает ли это общую ценность.

Вместо того чтобы генерировать RPC для трафика восток-запад из трафика север-юг, более разумным будет тщательное проектирование взаимодействия на основе микросервисов (RPC) независимо от REST-представления, что позволит обоим API свободно развиваться. Именно такой выбор мы сделали для примера конференции, и в проекте он будет отражен как запись ADR.

Заключение

В этой главе мы рассмотрели, как проектировать, создавать и выполнять спецификацию API, а также различные обстоятельства, при которых вы можете выбрать REST или gRPC. Важно помнить, что речь идет не о сравнении REST с gRPC, а о том, что в конкретной ситуации является наиболее подходящим вариантом для моделирования обмена. Основные выводы:

- ♦ в большинстве технологий барьер для создания API на основе REST- и RPC-технологий невысок. Тщательная проработка проекта и структуры — важное архитектурное решение;
- ♦ при выборе между моделями REST и RPC следует учитывать модель зрелости Ричардсона и степень связанности между производителем и потребителем;
- ♦ REST — достаточно свободный стандарт. При создании API необходимо следовать согласованному стандарту API, чтобы обеспечить согласованность API и ожидаемое поведение потребителей. Стандарты API также могут помочь предотвратить возникновение проектных решений, способных привести к несовместимости API;
- ♦ спецификации OpenAPI — это полезный способ совместного использования структуры API и автоматизации многих действий, связанных с программированием. Необходимо активно выбирать функциональные возможности OpenAPI и

определять, какие инструментальные средства или возможности генерации будут применяться в проектах;

- ♦ версионирование — важная тема, усложняющая работу производителя, но необходимая для облегчения использования API потребителем. Не выполнять планирование версионирования в API, доступных потребителям, опасно. Версионность должна быть активным решением в наборе функций продукта, и механизм доведения версионности до потребителей должен быть частью обсуждения;
- ♦ gRPC отлично работает в сетях с высокой пропускной способностью и станет идеальным вариантом для сетей типа восток-запад. Инструментарий для gRPC содержит большее количество возможностей и предоставляет дополнительный вариант выбора при моделировании обменов;
- ♦ моделирование нескольких спецификаций становится довольно сложным, особенно при переходе от одного типа спецификации к другому. Версионирование еще больше усложняет ситуацию, но служит важным фактором, позволяющим избежать критических изменений. Командам следует хорошо подумать, прежде чем комбинировать RPC-представления с RESTful API-представлениями, поскольку существуют принципиальные различия в плане использования и контроля над кодом потребителя.

Задача архитектуры API состоит в том, чтобы удовлетворить требования с точки зрения бизнеса потребителей, создать отличные условия для работы разработчиков с API и избежать неожиданных проблем совместимости. В *главе 2* описывается тестирование, необходимое для обеспечения соответствия сервисов этим целям.

Тестирование API

В главе 1 были рассмотрены различные типы API и их значение для архитектуры. В этой главе, завершающей часть I «Проектирование, создание и тестирование API», исследуются подходы к тестированию. Новый API Участник, созданный во введении, очевидно, должен быть протестирован и проверен. Мы считаем, что тестирование — это важнейшая часть проектирования API. Оно позволяет обеспечить высокий уровень уверенности в том, что ваш сервис работает так, как ожидается, и поможет вам предоставить потребителям вашего API качественный продукт. Только путем тестирования API в различных условиях можно получить уверенность в правильности его работы.

При создании API, как и при создании любого продукта, единственный способ убедиться в том, что продукт работает так, как ожидается, — это его тестирование. В случае с проверкой капы¹ это может означать растягивание, удары, толчки и изгибы изделия, а также проведение моделирования². Как обсуждалось в разд. «Определение REST API с помощью OpenAPI» главы 1, API не должен возвращать ничего отличного от того, что описано в документации. Также неприятно, когда в API вносятся нестандартные изменения или возникают сетевые задержки из-за большого времени получения результата. Подобные проблемы отталкивают клиентов, и их можно полностью предотвратить путем создания качественных тестов для API-сервиса. Любой созданный API должен быть готов выполнять целый ряд требований, включая отправку полезной обратной связи пользователям, предоставившим некачественные данные, быть безопасным и возвращать результаты в рамках заданной цели уровня обслуживания (Service-Level Objective, SLO) на основе согласованных нами индикаторов уровня обслуживания (Service-Level Indicators, SLI)³.

В этой главе мы расскажем о различных видах тестирования, применяемых к API, чтобы избежать возникновения подобных проблем, а также выделим положительные и отрицательные стороны каждого вида тестирования, чтобы вы могли решить, на что лучше всего потратить свое время. Мы сосредоточимся на тестировании API и на тех областях, где, по нашему мнению, вы получите наибольшую пользу. Общее тестирование сервисов мы рассматривать не станем.

¹ Приспособление из гибкой пластмассы, надеваемое на зубы с целью защиты от спортивных травм. — Прим. ред.

² Друг Мэтью владеет компанией по производству кап, и ему приходилось слышать о сложном процессе проверки прочности изделия. Никому не нужна капа, единственное испытание которой происходит во время матча!

³ Более подробно SLO и SLI будут рассмотрены в главе 5.

Глава содержит указания на дополнительные ресурсы для читателей, стремящихся получить значительно более глубокие и специализированные знания о тестировании.

Сценарий конференц-системы для этой главы

В разд. введения «*ADR-запись эволюции участников*» мы объяснили причины отделения API участников от остальной части конференц-системы. Выделение этого API вводит новые взаимодействия. API Участник будет использоваться внешней системой CFP и унаследованной конференц-системой, как показано на рис. 2.1. В этой главе мы рассмотрим тестирование, необходимое для сервиса Участник, и то, как тестирование может помочь проверить взаимодействие между традиционной конференц-системой и API Участник. Мы, как коллектив, видели достаточно много API, становящихся несовместимыми или приводящих к случайным критическим изменениям при релизах новых версий, и это в первую очередь связано с отсутствием тестирования. Для нового API Участник важно избежать этих ловушек, обеспечив уверенность в том, что всегда будут возвращаться правильные результаты, а это возможно только при условии вложения сил и средств в тестирование на должном уровне.

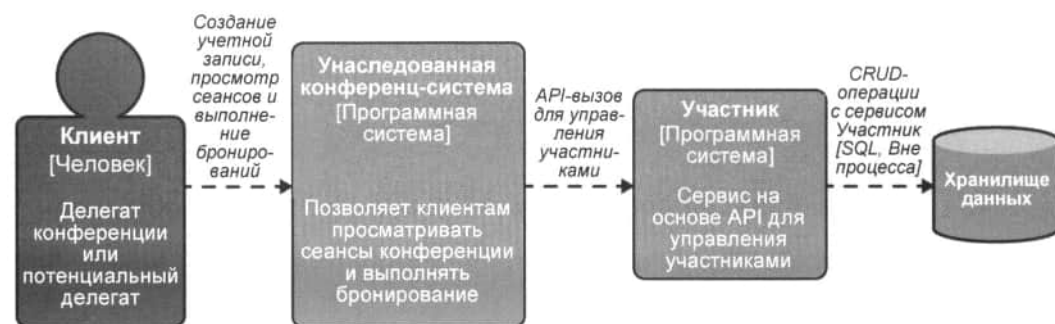


Рис. 2.1. Сценарий для главы

Тестирование может проводиться на разных уровнях API, начиная с отдельных блоков, составляющих сервис, и заканчивая проверкой его работы в составе всей экосистемы. Прежде чем рассказать о некоторых инструментах и фреймворках, доступных для тестирования API, важно понять, какие стратегии могут быть для этого использованы.

Стратегии тестирования

Тестирование очень важно — оно гарантирует, что вы создаете работающее приложение. Однако вам нужно не просто то, что так или иначе работает, а решение с правильным поведением. Однако в реальности время и ресурсы на написание тестов ограничены, поэтому необходимо убедиться, что вы не тратите циклы на написание тестов, не приносящих практически никакой пользы. В конце концов, цен-

ность для клиентов заключается в запуске системы в эксплуатацию. Поэтому необходимо грамотно подходить к выбору охвата и соотношения типов тестов. Избегайте создания неактуальных, дублирующих тестов, а также таких, которые отнимают больше времени и ресурсов, чем приносят пользы (т. е. нестабильных). Для релиза API не обязательно проводить все положенные тесты, поскольку это может быть нецелесообразно из-за нехватки времени и требований бизнеса.

Чтобы сориентировать вас в правильном балансе и правильных тестах для вашего случая, мы приведем тестовый квадрант и тестовую пирамиду. Эти инструменты позволят вам сконцентрироваться на определении необходимых тестов.

Тестовый квадрант

Впервые *квадрант тестирования* был представлен Брайаном Мариком в его серии блогов, посвященных agile-тестированию⁴. Затем он приобрел широкую популярность благодаря книге «Гибкое тестирование»⁵ Лайзы Криппин и Джанет Грегори. Технологическая сторона построения API обеспечивает его правильное создание, чтобы его части (например, функции или конечные точки) работали так, как ожидается, и чтобы он был устойчив и продолжал корректно вести себя в нестандартных ситуациях. Бизнес заботится о том, чтобы разрабатывался правильный сервис (т. е., в нашем случае, чтобы API Участник обеспечивал нужную функциональность). Поясним, что под термином «бизнес» подразумевается кто-то, имеющий четкое представление о продукте, его характеристиках и функциональности, которые необходимо разработать. Но он не обязательно должен иметь техническое представление о программном решении.

Квадрант тестирования (рис. 2.2) объединяет тесты, помогающие заинтересованным сторонам технологического и бизнес-направлений, — хотя каждое направление может иметь свое мнение относительно приоритетов.

Тестовый квадрант *не* обязывает к соблюдению порядка тестов. Его квадранты пронумерованы лишь для удобства, и это распространенный источник путаницы, о котором Лайза рассказывает в одном из своих постов в блоге⁶. В общем виде четыре составляющих квадранта можно представить следующим образом:

◆ Квадрант 1 (Кв. 1).

Модульные и компонентные тесты для технологий. Они помогают убедиться в работоспособности созданного сервиса, и эта проверка должна быть выполнена с помощью автоматического тестирования.

◆ Квадрант 2 (Кв. 2).

Тесты с участием бизнеса. Они позволяют убедиться в том, что строящаяся система служит определенной цели. Это проверяется с помощью автоматизированного тестирования, а также может включать ручное тестирование.

⁴ См. <https://oreil.ly/JAg7C>.

⁵ См. <https://www.dialektika.com/books/978-5-8459-1625-9.html>.

⁶ См. <https://oreil.ly/n2lwu>.

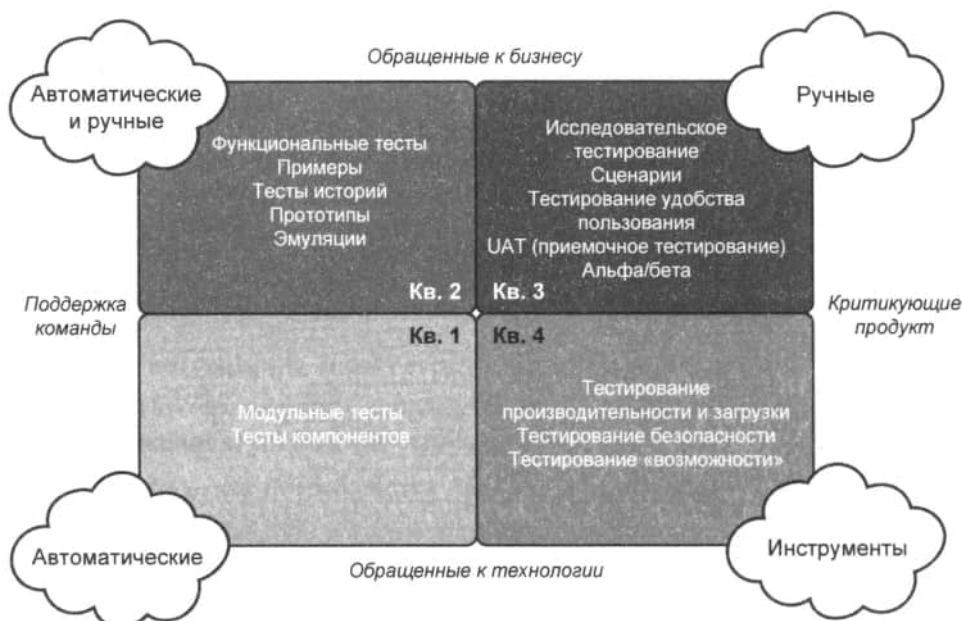


Рис. 2.2. Квадранты гибкого тестирования
(из книги «Гибкое тестирование» Лайзы Криспин и Джанет Грегори)

◆ Квадрант 3 (Кв. 3).

Тесты с участием бизнеса. Речь идет об обеспечении выполнения функциональных требований, а также об исследовательском тестировании. При первоначальном создании рис. 2.2 этот вид тестирования выполнялся вручную, теперь же появилась возможность проводить автоматизированное тестирование и в этой области.

◆ Квадрант 4 (Кв. 4).

Обеспечение работоспособности существующих модулей с технической точки зрения. Из квадранта 1 вы знаете, что созданный продукт работает. Однако, когда продукт используется, работает ли он так, как ожидалось? Примерами корректной работы с технической точки зрения могут служить обеспечение безопасности, целостность SLA и автоматическое масштабирование.

Левая часть квадранта (Кв. 1 и Кв. 2) посвящена поддержке продукта. Эта часть помогает направлять его развитие и предотвращать дефекты. Правая сторона (Кв. 3 и Кв. 4) — это критика продукта и поиск дефектов. Верхняя часть квадранта (Кв. 2 и Кв. 3) — это качество вашего продукта с внешней точки зрения, соответствие его ожиданиям пользователей. Это то, что представители сферы бизнеса считают важным. Нижняя часть квадранта (Кв. 1 и Кв. 4) — это тесты, обращенные к технологии для поддержания внутреннего качества приложения⁷.

⁷ Чтобы узнать больше о гибком тестировании, ознакомьтесь с книгами «Гибкое тестирование» (<https://www.williamspublishing.com/Books/978-5-8459-1625-9.html>), «Agile-тестирование» (<https://www.mann-ivanov-ferber.ru/books/agile-testirovanie/>) или с видеоуроками «Agile Testing Essentials» (<https://oreil.ly/x7goy>).

Квадрант тестирования не говорит о том, с чего следует начинать тестирование, — он помогает сориентироваться в том, какие тесты понадобятся. Решение о выборе тестов должно быть принято вами и должно основываться на важных для вас факторах. Например, система продажи билетов должна выдерживать большие скачки трафика, поэтому, возможно, лучше начать с обеспечения устойчивости системы продажи билетов (например, с тестирования производительности). Это будет частью квадранта 4.

Тестовая пирамида

В дополнение к квадрантам тестирования в рамках стратегии автоматизации тестирования может быть использована *пирамида тестирования* (также известная как пирамида автоматизации тестирования). Впервые тестовая пирамида была представлена в книге «Succeeding with Agile»⁸ Майка Кона. Эта пирамида иллюстрирует представление о том, сколько времени следует тратить на ту или иную область тестирования, о соответствующей сложности ее сопровождения и о ценности, получаемой с точки зрения дополнительной уверенности. Ядро тестовой пирамиды осталось неизменным. В ее основании лежат модульные тесты, в среднем блоке — сервисные тесты, а на вершине пирамиды — UI-тесты. На рис. 2.3 приведены области тестовой пирамиды, которые вам предстоит изучить.

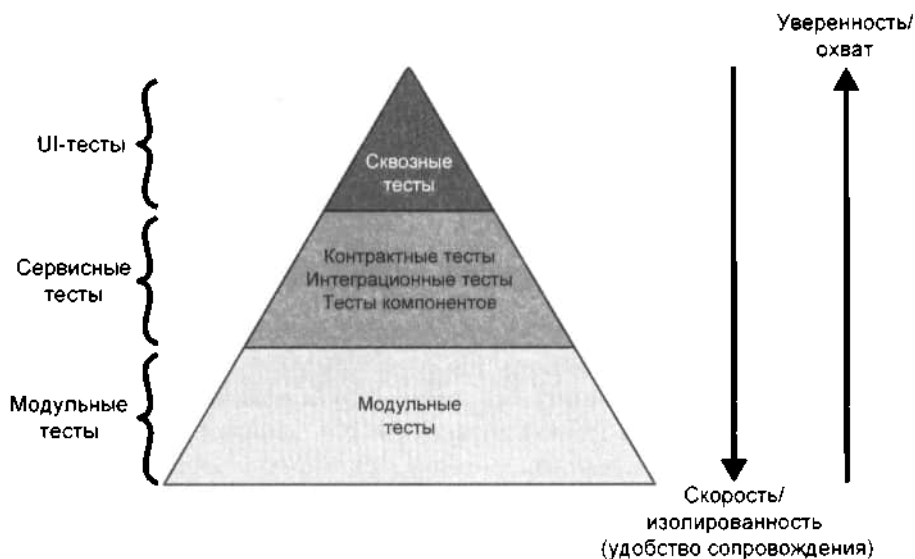


Рис. 2.3. Тестовая пирамида, показывающая желаемое соотношение тестов

Пирамида автоматизации тестирования демонстрирует существующие в отношении доверия, изолированности и масштаба компромиссы. Тестируя небольшие части кодовой базы, вы получаете лучшую изолированность и более быстрые тесты,

⁸ См. <https://oreil.ly/iHOra>.

однако это не дает уверенности в работоспособности всего приложения. Тестирование всего приложения в его экосистеме приводит к обратному эффекту. Тесты при этом обеспечивают большую уверенность в том, что приложение работает, но объем тестирования получается слишком большим, т. к. многие компоненты будут взаимодействовать между собой. Это усложняет сопровождение и снижает скорость. Далее приведены определения каждого из основных элементов тестовой пирамиды:

- ♦ *модульные тесты* находятся в самом низу пирамиды и составляют основу тестирования. Они тестируют небольшие изолированные части кода, чтобы убедиться, что определенный блок работает так, как ожидается⁹. Если ваш тест выходит за границы блока, вы можете использовать *тестовые дублиеры* — объекты, выглядящие как реальные версии внешних сущностей, однако находящиеся под вашим контролем¹⁰. Поскольку модульные тесты составляют основу пирамиды, их должно быть больше, чем любых других типов тестов. Мы рекомендуем использовать практику *разработки через тестирование* (Test-Driven Development, TDD)¹¹. Она предполагает написание тестов до написания логики. Модульные тесты относятся к квадранту 1 и используются для обеспечения качества внутренних компонентов приложения. Модульное тестирование далее рассматриваться не будет, поскольку мы сосредоточимся на тестах, проверяющих API с точки зрения внешнего потребителя, а не внутренние компоненты API;
- ♦ *сервисные тесты* составляют средний уровень пирамиды. Они дают больше уверенности в правильности работы API, чем модульные тесты, хотя и обходятся дороже. Повышенные расходы связаны с тем, что тесты имеют больший объем и меньшую степень изоляции, что влечет за собой увеличение затрат на сопровождение и разработку. Сервисные тесты включают в себя некоторые из следующих случаев: проверка совместной работы нескольких модулей на соответствие поведения ожидаемому, а также устойчивости самого приложения. Таким образом, сервисные тесты попадают в квадранты 1, 2 и 4 квадранта тестирования;
- ♦ *тесты пользовательского интерфейса (UI)* находятся на вершине пирамиды тестирования. В былые времена большинство приложений, создаваемых для сети Интернет, представляли собой стеки LAMP¹², и единственным способом тестирования приложения от фронтенда до бэкенда была проверка сетевого UI. В API существует пользовательский интерфейс — он просто не графический,

⁹ Типичным примером модуля в объектно-ориентированных (ОО) языках служит класс.

¹⁰ Тестовые дублиеры включают в себя заглушки, выглядящие как реальные реализации внешних сущностей, за исключением того, что они возвращают жестко закодированные ответы. Такие имитации выглядят как реальные реализации запрограммированных объектов, но используются для проверки поведения.

¹¹ Книга Кента Бека «Экстремальное программирование. Разработка через тестирование» — исчерпывающий сборник информации для получения дополнительных сведений о TDD (<https://www.piter.com/collection/biblioteka-programmista/product/ekstremalnoe-programmirovaniye-razrabotka-cherez-testirovaniye-2>).

¹² См. <https://oreil.ly/dtgJF>.

поэтому такие тесты теперь будут называться *сквозными*. Они охватывают ту же область, что и запрос, идущий от начальной до конечной точки, но не обязательно подразумевают или предполагают, что трафик исходит из сетевого UI. Сквозные тесты считаются самыми сложными. У них самый большой охват, и выполнение происходит медленно, но при этом они проверяют совместную работу целых модулей, что позволяет получить уверенность в результате работы. Сквозные тесты обычно располагаются в квадрантах 2, 3 и 4. Инструментарий для тестирования улучшается и становится все более совершенным, и теперь внедрять автоматизацию можно во все большее количество тестов квадранта 3.

Один тип тестирования не лучше другого — пирамида тестирования служит руководством по рекомендованным пропорциям каждого типа тестирования. Может возникнуть соблазн проигнорировать пирамиду тестирования и сосредоточиться на сквозном тестировании, поскольку оно дает высокую степень уверенности. Однако это заблуждение. Из-за него возникает ложное чувство уверенности в том, что эти тесты более высокого уровня имеют более высокое качество/ценность, чем модульные. Это заблуждение порождает представление тестирования в виде рожка мороженого, являющегося противоположностью тестовой пирамиды. Чтобы получить развернутую аргументацию по этой теме, ознакомьтесь с постом в блоге Стива Смита «End-to-End Testing considered harmful»¹³. Вы, конечно можете рассмотреть возможность проведения тестов в других пропорциях, хотя это и не рекомендуется. Мартин Фаулер опубликовал обновленную статью о формах тестирования¹⁴, в которой рассказал, почему он считает, что тестирование, основанное на любой форме, кроме пирамиды тестирования, будет неправильным.

Руководство ADR по стратегиям тестирования

Принять обоснованное решение о выборе стратегии тестирования вам поможет руководство ADR, приведенное в табл. 2.1.

Таблица 2.1. Руководство ADR: стратегии тестирования

Решение	Какую стратегию тестирования при создании API следует сделать частью процесса разработки?
Пункты обсуждения	Есть ли у всех сторон, заинтересованных в API, время и возможность регулярно обсуждать, как должен работать API? Если вы не сможете эффективно общаться с заинтересованными сторонами, то в итоге застопорите свой продукт в ожидании принятия решения. Имеются ли у команды навыки и опыт для эффективного использования этих стратегий тестирования? Не все специалисты ранее использовали эти практики, поэтому необходимо взвесить, хватит ли у вас времени на обучение всех желающих. Существуют ли у вас на работе другие рекомендованные к использованию методы? Иногда могут существовать внутренние стратегии построения программного обеспечения, работающие в организации или требующиеся в силу специфики бизнеса

¹³ См. <https://oreil.ly/1v1Yd>.

¹⁴ См. <https://oreil.ly/2DdPE>.

Таблица 2.1 (окончание)

Решение	Какую стратегию тестирования при создании API следует сделать частью процесса разработки?
Рекомендации	Мы рекомендуем использовать тестовые квадранты и пирамиду. Квадрант тестирования очень важен для того, чтобы убедиться, что ваши клиенты получают правильный продукт. Квадрант тестирования в сочетании с пирамидой тестирования поможет вам создать отличный API. Мы понимаем, что не всегда возможно использовать квадрант тестирования в его истинном виде, имея в своем распоряжении человека, готового помочь в проведении тестирования. Однако как минимум следует ориентироваться на тестовую пирамиду, поскольку она собирает в себе автоматизированную часть тестового квадранта. Это так или иначе обеспечит поиск ошибок на ранних стадиях разработки. Как бы то ни было, вам всегда нужен человек, способный помочь сориентироваться в направлении развития продукта

Контрактное тестирование

В *контрактном тестировании* участвуют два субъекта: потребитель и производитель. Потребитель (например, веб-клиент, терминальная оболочка) запрашивает данные из API, а производитель (он же провайдер) отвечает на запросы API, т. е. производит данные, например, в виде RESTful веб-сервиса. *Контракт* — это определение взаимодействия между потребителем и производителем. Если потребитель делает запрос, соответствующий определению запроса в контракте, то производитель возвращает ответ, соответствующий определению ответа в контракте. В случае с API Участник именно он является производителем, а потребителем выступает унаследованная конференц-система. Эта система становится потребителем, поскольку она обращается к API Участник¹⁵. Так зачем же использовать контракты? Что они дают нам?

Почему часто предпочтительнее проводить контрактное тестирование?

В разд. «Определение REST API с помощью OpenAPI» главы 1 вы узнали, что у API должна быть спецификация, и очень важно, чтобы ваши ответы на API соответствовали имеющейся спецификации. Наличие письменного определения этих взаимодействий у производителя гарантирует, что потребители смогут продолжать использовать ваш API, и делает возможным создание тестов. Контракт определяет, как должны выглядеть запрос и ответ. И эти шаблоны могут быть использованы для проверки выполнения контракта производителем (API). Если контрактный тест не пройден, это означает, что производитель больше не выполняет контракт, а значит, возникнут нарушения работы и у потребителей.

¹⁵ Поясним: также возможно, что сервис может быть как производителем, так и потребителем.

Поскольку контракт содержит определение ответа, то можно сгенерировать и сервер-заглушку¹⁶, который может быть использован потребителями для проверки корректности вызова производителя и синтаксического анализа ответа от него. Контрактное тестирование можно проводить локально — для этого не требуется запускать дополнительные сервисы, что делает его частью сервисных тестов. Контракты будут развиваться, а потребители и производители смогут подхватывать эти изменения по мере их появления, что обеспечит им возможность постоянной интеграции с последним контрактом.

Уже много сказано о том, зачем нужно использовать контракты. Кроме того, контрактное тестирование имеет хорошо развитую экосистему. Существуют устоявшиеся методологии, определяющие, каким должен быть контракт, а также фреймворки и тестовые интеграции, позволяющие генерировать контракты и обеспечивающие эффективные способы их распространения. Мы считаем, что контракты — это лучший способ определить взаимодействие между реализуемым сервисом и потребителем. Другие тесты также важны, и их необходимо реализовать, но эти дают наибольшую отдачу.



Важно отметить, что выполненное контрактное тестирование не всегда утверждает, что API соответствует схеме. Система либо совместима со схемой (например, OpenAPI Spec), либо нет, — контракт лишь предполагает определенное взаимодействие между сторонами и содержит примеры. У Мэтта Феллоуза есть отличная статья на эту тему под названием «Schema-based contract testing with JSON schemas and Open API (Part 1)»¹⁷.

Порядок реализации контракта

Как уже говорилось, контракт — это совместное определение порядка взаимодействия производителя и потребителя. В следующем примере показан контракт на запрос GET к конечной точке `/conference/{conference-id}/attendees`. В нем говорится, что в ожидаемом ответе есть свойство `value`, содержащее массив значений об участниках. В этом примере определения контракта видно, что он определяет взаимодействие, используемое для генерации тестов и сервера-заглушки:

```
Contract.make {
  request {
    description('Get a list of all the attendees at a conference')
    method GET()
    url '/conference/1234/attendees'
    headers {
      contentType('application/json')
    }
  }
}
```

¹⁶ Сервер-заглушка — это запускаемый локально сервис, он будет возвращать готовые ответы.

¹⁷ См. <https://oreil.ly/QTbbZ>.

```

response {
  status OK()
  headers {
    contentType('application/json')
  }
  body{
    value: [
      $(
        id: 123456,
        givenName: 'James',
        familyName: 'Gough'
      ),
      $(
        id: 123457,
        givenName: 'Matthew',
        familyName: 'Auburn'
      )
    ]
  }
}
}
}
}

```

На рис. 2.4 показано, как сгенерированные тесты используются потребителем и производителем.

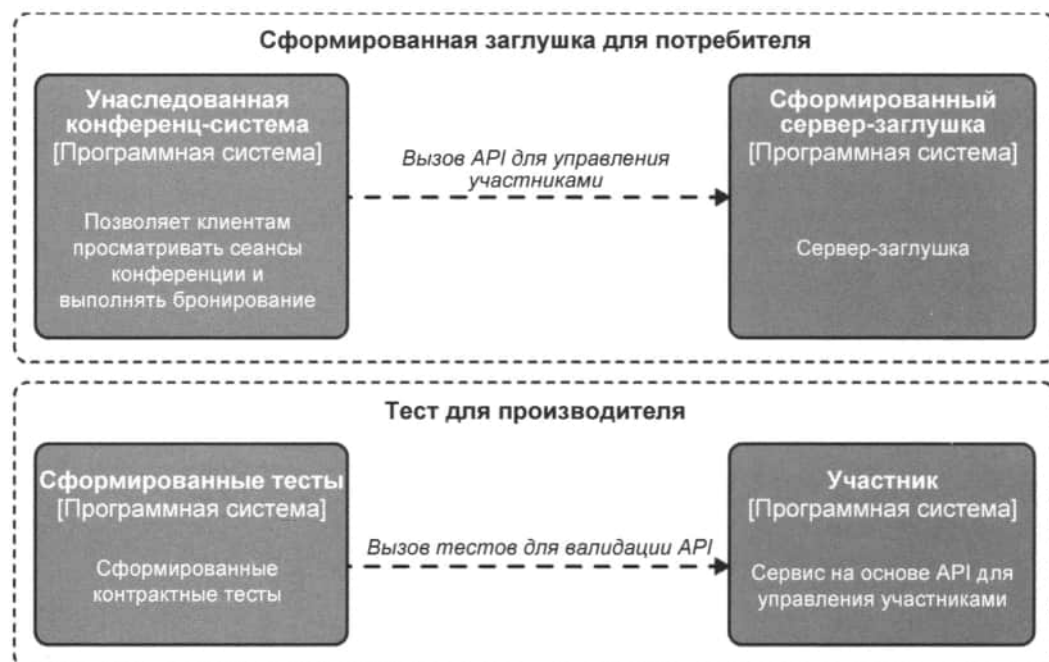


Рис. 2.4. Сформированные сервер-заглушка и тесты из контракта



Заманчиво использовать контракты для тестирования сценариев. Например:

- шаг 1: добавление участника в конференцию;
- шаг 2: получение списка участников конференции и проверка правильности добавления участника.

Фреймворки это поддерживают, но не поощряют. Контракты определяют взаимодействие — если же вы хотите протестировать такое поведение, то используйте тесты компонентов.

Ключевым преимуществом использования контрактов является то, что как только производитель соглашается реализовать контракт, это убирает зависимость построения потребителя и производителя.



Мы использовали сгенерированные серверы-заглушки при запуске демоверсий для заинтересованных сторон. Это было полезно, т. к. производитель всё еще внедрял логику, но уже согласился с контрактами.

У потребителя есть сервер-заглушка для разработки, а у производителя — тесты, позволяющие убедиться, что он создает правильное взаимодействие. Процесс контрактного тестирования экономит время, поскольку при развертывании потребитель и производитель должны будут без проблем интегрироваться.



Сгенерированные тесты необходимо выполнить на работающем API (производителе). При запуске API следует использовать тестовые дублиры для внешних зависимостей. Не стоит тестировать интеграцию с другими сервисами в рамках генерируемых контрактных тестов с потребителем.

Чтобы понять, как происходит согласование контрактов, рассмотрим две основные методологии их создания.

Контракты производителя

Контрактное тестирование производителей — это когда производитель сам определяет свои контракты. Такая практика обычно применяется, когда API используется за пределами вашей непосредственной организации (т. е. внешними третьими лицами). Когда вы разрабатываете API для внешней аудитории, API должен сохранять свою целостность, поскольку интерфейс не может вносить изменения без плана миграции, как было отмечено в *разд. «Версионирование API» главы 1*. Несмотря на то что взаимодействие будет обновляться и совершенствоваться, ни один потребитель, скорее всего, не сможет запросить изменения, затрагивающие весь API, и быстро получить их, поскольку эти изменения должны быть тщательно соркестрованы.

Реальным примером такого API служит Microsoft Graph API. У Microsoft есть тысячи потребителей этого API из компаний по всему миру. Заставлять компании или частных лиц корректировать контракты для Graph API в соответствии с тем, как, по их мнению, должен выглядеть контракт, не представляется возможным. Это не значит, что изменения не должны предлагаться Microsoft — они, безусловно, есть. Однако даже если изменения будут согласованы, они не смогут быть сделаны быстро, поскольку их необходимо тщательно проверить и протестировать.

Если API Участник планируется сделать общедоступным, возникнут те же проблемы. Для этого API важно использовать контракты, чтобы не допустить расхождений во взаимодействиях и обеспечить согласованность возвращаемых данных.

Еще одна причина, по которой следует использовать контракты производителя, заключается в том, что так проще начать работу. Это хороший способ внедрить контракты в ваши API. Наличие контрактов гораздо выгоднее, чем их отсутствие. Однако в тех случаях, когда потребители и производители находятся в одной организации, мы предлагаем использовать методологию контрактов, ориентированных на потребителя.

Контракты, ориентированные на потребителя

Контракты, ориентированные на потребителя (Consumer-Driven Contracts, CDC), по определению реализуются потребителем, определяющим функциональность, которую он хочет видеть в процессе взаимодействия. Потребители представляют производителю контракты или изменения к контракту для получения новой или дополнительной функциональности API. Когда новый/обновленный контракт будет представлен производителю, начнется обсуждение изменений. Его результатом станет принятие или отклонение этих изменений.

CDC — это в значительной степени интерактивный и социальный процесс. Владельцы приложений, являющихся потребителями и производителями, должны находиться в пределах досягаемости (например, в одной и той же организации). Когда потребитель хочет получить новое взаимодействие (например, вызов API) или обновить взаимодействие (например, добавить новое свойство), он отправляет запрос на эту функцию.

Наш пример: использование CDC

В нашем случае это может означать, что из унаследованной конференц-системы в новый сервис API Участник отправляется запрос на извлечение. Затем запрос на новое взаимодействие рассматривается, и происходит обсуждение этой новой функциональности. Обсуждение проводится для того, чтобы убедиться, что это то, что сервис Участник должен и будет выполнять. Например, если для запроса PUT предлагается использовать контракт, то можно провести обсуждение, т. е., возможно, предпочтительнее было бы оформить этот запрос как PATCH.

Именно в этом заключается большая часть ценности контрактов — обсуждение обеими сторонами сути проблемы и использование контракта для утверждения того, что обе стороны принимают и соглашаются именно с этим. После согласования контракта производитель (сервис Участник) принимает контракт как часть проекта и может приступить к его выполнению.

Обзор методологии контрактов

Надеюсь, эти методологии дадут вам представление о том, как использовать контракты в качестве части процесса разработки. Не следует принимать это за истину в последней инстанции, поскольку возможны вариации в отношении конкретных

шагов. Например, один из процессов может потребовать от потребителя при написании контракта создать базовую реализацию кода производителя для выполнения контракта. Другой пример — потребителю следует провести TDD необходимой ему функциональности, а затем создать контракт перед отправкой запроса на извлечение. Конкретный порядок действий может варьироваться в зависимости от команды. Как только вы поймете основные концепции и паттерны CDC, точный используемый процесс станет лишь деталью реализации.

Если вы только начинаете работу по добавлению контрактов, то должны учитывать, что это связано с определенными затратами: временем на подготовку к включению договоров в проект, а также затратами на их написание. Стоит обратить внимание на инструментарий, позволяющий создавать контракты на основе спецификации OpenAPI¹⁸.

Фреймворки для контрактного тестирования

Скорее всего, когда речь идет о фреймворках контрактного тестирования для HTTP, вам следует обратить внимание на Pact¹⁹. Он превратился во фреймворк для контрактного тестирования по умолчанию благодаря экосистеме, созданной вокруг него, и огромному количеству поддерживаемых языков. Существуют и другие фреймворки для контрактного тестирования, и они могут вести себя по-разному. Pact — категоричный, он принуждает к выполнению CDC и специально для этого предназначен. Потребитель пишет тест, который генерирует контракт, имеющий вид промежуточного представления взаимодействия. Именно благодаря этому промежуточному представлению, не зависящему от языка, Pact получил столь широкое применение. У других фреймворков взгляды расходятся. Например, у Spring Cloud Contracts нет четкого мнения по поводу контрактов CDC или контрактов производителей, и в этом случае допустимо применение любого из вариантов. Это возможно, поскольку в Spring Cloud Contracts вы пишете контракты вручную, а не генерируете их. Хотя Spring Cloud Contracts не зависит от языка за счет использования контейнерной версии продукта, для получения максимальной отдачи от него необходимо использовать экосистему Spring и JVM²⁰.

Существуют варианты контрактного тестирования для других протоколов — оно не предназначено исключительно для HTTP-коммуникаций.

Хранение и публикация контрактов API

После ознакомления с принципами работы контрактов и методологиями их включения в процесс разработки следует рассмотреть вопрос о том, где хранятся контракты и как они должны публиковаться.

¹⁸ На момент подготовки книги существовало несколько доступных проектов, но ни один из них не поддерживался активно, поэтому трудно рекомендовать какой-либо из них.

¹⁹ См. <https://pact.io/>.

²⁰ В Pact проделали серьезную работу по сравнению собственного контрактного фреймворка с другими (<https://oreil.ly/h8WrT>).

Существует несколько вариантов хранения и публикации контрактов, и они опять-таки зависят от того, какие настройки доступны вам и вашей организации.

Контракты могут храниться вместе с кодом производителя в системе контроля версий (например, Git).

Они также могут быть опубликованы вместе со сборкой в репозитории артефактов, например в системе Artifactory²¹.

В конечном счете контракты должны быть доступны и производителю, и потребителю. Кроме того, точка хранения должна обеспечивать возможность представления новых контрактов. Производитель должен получить контроль над тем, какие контракты принимаются в проект, вследствие чего он сможет гарантировать, что не будут добавлены дополнительные контракты или внесены нежелательные изменения в существующие. Недостатком такого подхода в большой организации станут трудности при поиске всех API-сервисов, использующих контракты.

Другой вариант — хранить все контракты в централизованном местоположении, чтобы обеспечить видимость других доступных API-взаимодействий. Таким централизованным местом может быть Git-репозиторий. Но при отсутствии правильной организации в таком подходе существует риск попадания контрактов в модуль, который производитель не намерен выполнять.

Еще один вариант хранения контрактов — использовать *брокер*. В контрактной системе Pact имеется продукт, носящий такое название (Broker)²². Его можно использовать в качестве центрального узла для размещения контрактов. Брокер может показать все подтвержденные производителем контракты, поскольку производитель будет публиковать выполненные контракты. Также брокер позволяет видеть, кто использует тот или иной контракт, создавать сетевые диаграммы, выполнять интеграцию с конвейерами CI/CD и предоставлять еще больше ценной информации. Это наиболее полное из имеющихся решений, и если вы работаете с фреймворком, совместимым с Pact Broker, то рекомендуется ориентироваться именно на него.

Руководство ADR: контрактное тестирование

Понять, насколько уместно применение контрактного тестирования в вашем случае, и взвесить все за и против использования контрактов вам помогут рекомендации ADR, приведенные в табл. 2.2.

Таблица 2.2. Руководство ADR: контрактное тестирование

Решение	Следует ли при создании API использовать контрактное тестирование и, если да, то каких: от потребителя или от производителя?
Пункты обсуждения	Определите, готовы ли вы включить контрактное тестирование в состав тестирования API. Хотите ли вы добавить к своему API дополнительный уровень тестирования, о котором разработчики должны будут узнать?

²¹ См. <https://oreil.ly/jXuCS>.

²² См. <https://oreil.ly/PIThd>.

Таблица 2.2 (окончание)

Решение	Следует ли при создании API использовать контрактное тестирование и, если да, то каких: от потребителя или от производителя?
	Если контракты раньше не использовались, то необходимо время, чтобы решить, как их использовать. Должны ли контракты быть централизованными или в проекте? Нужны ли дополнительные инструменты и обучение, чтобы помочь людям с контрактами? Если принять решение об использовании контрактов, то какая методология должна применяться — контракты CDC или производителя? Знаете ли вы, кто будет использовать этот API? Будет ли этот API использоваться только в вашей организации? Есть ли у API потребители, готовые взаимодействовать с вами, чтобы помочь в развитии функциональности?
Рекомендации	Мы рекомендуем использовать контрактное тестирование при создании API. Даже если в процессе обучения разработчику придется впервые решать, как настроить контракты, мы считаем, что это стоящая трата сил и времени. Определенные и протестированные взаимодействия позволяют сэкономить много времени при интеграции сервисов. Если вы открываете свой API для большой внешней аудитории, важно использовать контракты производителей. Опять же, наличие определенных взаимодействий, позволяющих гарантировать, что ваш API не нарушит обратную совместимость, имеет решающее значение. Если вы создаете внутренний API, идеальным вариантом будет работа в направлении CDC, даже если придется начинать с контрактов производителей и переходить к CDC. Если контрактное тестирование не представляется возможным, то для производителя необходимы альтернативные варианты, обеспечивающие соответствие API согласованным взаимодействиям и предоставляющие потребителям возможность тестирования. Это означает, что при тестировании необходимо очень тщательно следить за тем, чтобы ответы и запросы соответствовали ожидаемым, что может быть непросто и отнимает много времени.

Тестирование компонентов API

Тестирование компонентов может использоваться для проверки совместной работы нескольких устройств и должно применяться для проверки поведения — это сервисные тесты в пирамиде тестов, приведенной на рис. 2.3. Примером тестирования компонентов будет отправка запроса к API и проверка ответа. На высоком уровне это требует, чтобы ваше приложение могло прочитать запрос, выполнить аутентификацию и авторизацию, десериализовать полезную нагрузку, выполнить бизнес-логику, сериализовать полезную нагрузку и ответить. Это большое количество тестируемых модулей, и было бы трудно указать, где именно может оказаться ошибка. Отличие этого примера от контрактного теста заключается в том, что вы должны проверить правильность поведения сервиса. Например, если речь идет о создании нового участника, необходимо убедиться, что сервис выполнил вызов (имитированной) базы данных. Вы не просто проверяете форму ответа, как это делают контрактные тесты. Поскольку тесты компонентов проверяют совместную работу нескольких модулей, они (как правило) работают медленнее, чем модульные тесты. Тесты компонентов не должны обращаться к внешним зависимостям. Как и кон-

трактное тестирование, эти тесты не используются для проверки внешних точек интеграции. Запускаемый в этой области тип теста зависит от бизнес-задачи. Однако для API следует стремиться подтвердить такие решения, как:

- ◆ возвращается ли правильный код состояния при выполнении запроса;
- ◆ содержит ли ответ правильные данные;
- ◆ отклоняется ли входящая полезная нагрузка, если в ней передан нулевой или пустой параметр;
- ◆ когда отправляется запрос с принятым в нем типом содержимого XML, будут ли данные возвращены в ожидаемом формате;
- ◆ если запрос поступает от пользователя, не имеющего соответствующих прав, каким будет ответ;
- ◆ что произойдет, если будет возвращен пустой набор данных? Будет ли это код 404 или пустой массив;
- ◆ при создании ресурса указывает ли заголовок местоположения на созданный новый актив?

Из этой подборки тестов видно, как они перетекают в две области тестового квадранта. Сюда входят квадрант 1, где вы подтверждаете, что создаваемый API работает (т. е. выдает результаты), и квадрант 2, где вы проверяете правильность ответов API Участник.

Контрактное тестирование в сравнении с тестированием компонентов

Если контрактное тестирование недоступно, следует использовать тесты компонентов API для проверки соответствия API согласованным взаимодействиям, т. е. спецификации API. Использование тестов API-компонентов для проверки соответствия API взаимодействию нельзя считать идеальным вариантом: вероятность ошибок в них гораздо выше, а писать их очень утомительно. Вы должны сделать контракты своим «золотым источником» согласованных взаимодействий, поскольку генерируемые тесты гарантируют, что форма вашего API будет точной.

Практический пример: тестирование компонентов для проверки поведения

Рассмотрим пример исследования для нашей конечной точки `/conference/{conference-id}/attendees` API Участник. Эта конечная точка возвращает список участников конференции. Для тестирования этого компонента используется макет, представляющий зависимость от внешней базы данных, и, как показано на рис. 2.5, в нашем случае это DAO (Data Access Object)²³. Проверить эту конечную точку можно следующим образом:

²³ Абстрактный интерфейс к какому-либо типу базы данных или механизму хранения.

- ♦ успешные запросы содержат ответ в виде статуса 200 (OK);
- ♦ пользователям, не имеющим соответствующего уровня доступа, будет возвращен статус 403 (Forbidden);
- ♦ если в конференции нет участников, будет возвращен пустой массив.



Рис. 2.5. Тест API-компонента с имитацией DAO

Библиотека или фреймворк для тестирования, оборачивающая клиентский запрос, может быть очень полезной. Здесь фреймворк REST-Assured²⁴ используется для вызова конечной точки API Участник и для проверки следующих тестовых случаев²⁵:

```
@Test
void response_for_attendees_should_be_200() {
    given()
        .header("Authorization", VALID_CREDENTIAL)
    .when()
        .get("/conference/conf-1/attendees")
    .then()
        .statusCode(HttpStatus.OK.value());
}

@Test
void response_for_attendees_should_be_403() {
    given()
        .header("Authorization", INVALID_CREDENTIAL)
    .when()
        .get("/conference/conf-1/attendees")
```

²⁴ См. <https://oreil.ly/aaJ77>.

²⁵ Такие библиотеки тестирования обычно поддерживают предметно-ориентированный язык (Domain Specific Language, DSL) и позволяют легко анализировать ответы от API. RestAssured — один из таких фреймворков для тестирования REST на Java, а пакет httptest поставляется в комплекте с Golang. В зависимости от используемого языка или фреймворка, они должны быть доступны. В противном случае создание небольшой обертки вокруг стандартного клиента может значительно упростить интеграцию ответов при написании тестов.

```

    .then()
      .statusCode(HttpStatus.FORBIDDEN.value());
    ...
  }

```

Выполнение такого рода тестов дает нам уверенность в том, что наш API ведет себя корректно.

Интеграционное тестирование API

Интеграционные тесты в нашем понимании — это тесты, проходящие через границы между разрабатываемым модулем и любыми внешними зависимостями. Интеграционные тесты представляют собой разновидность сервисных тестов и отображены в пирамиде тестов на рис. 2.3.

Проведение интеграционного тестирования необходимо, чтобы убедиться в корректности взаимодействия через границу модуля, — т. е. в том, что ваш сервис может корректно взаимодействовать с другим, внешним по отношению к нему сервисом.

Проверить необходимо следующие моменты:

- ◆ обеспечение корректности взаимодействия. Например, для RESTful-сервиса это может быть указание правильного URL или корректность тела полезной нагрузки;
- ◆ может ли устройство, взаимодействующее с внешним сервисом, обрабатывать возвращаемые ответы?

В нашем случае необходимо убедиться, что унаследованная конференц-система может сделать запрос к новому API Участник и интерпретировать ответ.

Использование серверов-заглушек: почему и как?

Если вы используете контрактные тесты, то для проверки возможности взаимодействия потребителя с производителем можно задействовать сгенерированные серверы-заглушки. Унаследованная конференц-система содержит сгенерированный сервер-заглушку и может использоваться для тестирования, что позволит сохранить локальность тестирования и точность сервера-заглушки. Это предпочтительный вариант тестирования внешней границы.

Однако сгенерированный из контракта сервер-заглушка не всегда доступен, и требуются другие варианты — как, например, в случае тестирования с внешним API, таким как Microsoft Graph API, или внутри организации, когда контракты не используются. Самый простой вариант — это создание сервера-заглушки, имитирующего запросы и ответы взаимодействующего сервиса. Это, безусловно, приемлемый вариант, поскольку на выбранном языке и фреймворке разработчику обычно очень легко создать сервер-заглушку с готовыми ответами, интегрируемыми с тестами.

При развертывании сервера-заглушки вручную необходимо убедиться в том, что заглушка достаточно точная. При этом очень легко допустить ошибки — например, неточно указать URL или ошибиться в именах и значениях свойств ответа. Можете ли вы увидеть ошибки в этом ответе, набранном вручную?²⁶

```
{
  "values": [
    {
      "id": 123456,
      "givenName": "James",
      "familyName": "Gough"
    },
    {
      "id": 123457,
      "givenName": "Matthew",
      "familyName": "Auburn"
    },
    {
      "id": 123456,
      "givenName": "Daniel",
      "familyName": "Bryant"
    }
  ]
}
```

Но возможность ошибок не должна вас останавливать, т. к. это хорошее решение. Один из авторов с большим успехом использовал указанный подход, когда в соответствии с требованиями проекта ему пришлось вручную создавать сервер-заглушку для сервиса входа в систему.

Избежать этих неточностей и обеспечить правильную фиксацию запросов к URL-адресам и ответов на них можно с помощью регистратора. Вы можете задействовать инструмент, записывающий запросы и ответы к конечной точке и генерирующий файлы, которые можно использовать для создания заглушек. На рис. 2.6 показано, как это работает.

Содержимое этих сгенерированных файлов представляет собой сопоставление запросов и ответов, которое можно использовать в тестах для точного их представления, а поскольку они не создаются вручную, то их точность гарантируется в момент создания. Для использования этих сгенерированных файлов запускается сервер-заглушка, способный считывать их содержимое. Когда на сервер-заглушку поступает запрос, он проверяет, соответствует ли этот запрос какому-либо из ожидаемых запросов в файле сопоставлений. Если они совпадают, то возвращается сопоставленный ответ²⁷. Запись обращений к API позволяет получить более точные

²⁶ Дублирующее значение в поле `id` и неправильно написано название поля `familyName` (sic).

²⁷ Инструмент Wiremock (<http://wiremock.org/>) можно использовать как самостоятельный сервис, что делает его независимым от языка. Хотя, поскольку он написан на Java, есть некоторые доступные специфические Java-интеграции. Существует множество других инструментов, обладающих аналогичными возможностями на других языках, например camouflager (<https://oreil.ly/mzL1u>), написанный на TypeScript.

заглушки, чем создание заглушки вручную. Если вы используете записи, то необходимо следить за тем, чтобы они постоянно обновлялись и синхронизировались. Кроме того, если вы делаете записи в эксплуатируемой системе, необходимо следить за тем, чтобы в файлах отображения не сохранялась информация уровня PII.



Рис. 2.6. API Участник в качестве потребителя использует регистратор, фиксирующий запрос/ответ для получения тестовых данных

Руководство ADR: интеграционное тестирование

Интеграционное тестирование очень важно, поэтому, чтобы понять, какие виды интеграционного тестирования вам необходимы, обратимся к руководству ADR, приведенному в табл. 2.3.

Таблица 2.3. Руководство ADR: Интеграционное тестирование

Решение	Следует ли добавлять интеграционное тестирование к тестированию API?
Пункты обсуждения	Если ваш API интегрируется с каким-либо другим сервисом, какой уровень интеграционного тестирования следует использовать? Чувствуете ли вы уверенность в том, что можно просто имитировать ответы и не нужно проводить интеграционные тесты? При создании сервера-заглушки для тестирования можно ли точно составить запрос и ответы или их нужно записывать? Сможете ли вы поддерживать серверы-заглушки в актуальном состоянии и распознавать некорректность взаимодействия? Если ваши заглушки некорректны или устарели, это означает, что можно провести тесты на сервере-заглушке, но при развертывании в эксплуатацию сервис не сможет взаимодействовать с другим API, поскольку он изменился
Рекомендации	Мы рекомендуем использовать сгенерированные серверы-заглушки из контрактных тестов. Однако если такой возможности нет, то следующим оптимальным вариантом будет интеграционное тестирование с использованием записей взаимодействий. Наличие локально запускаемых интеграционных тестов дает уверенность в том, что интеграция будет работать, особенно при ее рефакторинге, — это поможет убедиться в том, что изменения ничего не нарушили

Интеграционное тестирование — действительно полезный инструмент, однако при определении этих взаимодействий возникают проблемы. И основная из них заклю-

чается в том, что они представляют собой моментальные снимки. Причем такие заказные настройки не обновляются с учетом изменений.

Для рассмотренных интеграций мы использовали серверы-заглушки, однако для проверки интеграции можно использовать реальный экземпляр внешнего сервиса.

Контейнеризация тестовых компонентов: Testcontainers

Обычно приложения создаются в виде контейнерных образов, а это означает, что многие приложения, с которыми будет интегрироваться ваш сервис, также доступны в виде контейнерных решений. Эти образы можно запустить на локальной машине в рамках тестирования. Использование локальных контейнеров позволяет не только протестировать взаимодействие с внешними сервисами, но и запустить используемый в эксплуатации образ.

Библиотека Testcontainers²⁸ интегрируется с вашим фреймворком тестирования для оркестровки контейнеров. Testcontainers будет запускать, останавливать и вообще организовывать жизненный цикл используемых в тесте контейнеров.

Практический пример:

применение Testcontainers для верификации интеграций

Рассмотрим два случая использования API Участник. В первом случае сервис API Участник будет поддерживать интерфейсы gRPC и RESTful. Интерфейс gRPC должен быть разработан после RESTful, но есть разработчики, желающие начать тестирование с интерфейса gRPC. Из-за этого принято решение применить для интерфейса gRPC сервер-заглушку, предоставляющую несколько заготовленных ответов, и создано приложение, выполняющее такую задачу. Затем заглушка gRPC упаковывается, контейнеризируется и публикуется, после чего может использоваться разработчиками для тестирования через границу, — т. е. они будут делать реальные вызовы этого контейнерного сервера-заглушки в своих тестах, а он может работать локально на их машине.

Второй вариант использования API Участник заключается в том, что у сервиса API Участник уже есть подключение к внешней базе данных, что и станет интеграцией для тестирования. Для тестирования границ интеграции базы данных можно использовать имитацию БД: базу данных в памяти (например, H2²⁹) или запустить локальную версию БД с помощью Testcontainers. Применение реального экземпляра базы данных в тесте дает много преимуществ, поскольку использование имитаторов способно симитировать неправильное возвращаемое значение или сделать неверное предположение. При использовании локальной БД предполагается, что ее реализация соответствует реальной БД. Применение реального экземпляра зависимости, причем той же версии, что используется в эксплуатируемой системе, озна-

²⁸ См. <https://www.testcontainers.org/>.

²⁹ См. <https://oreil.ly/s7mGq>.

чает надежное тестирование через границу, что гарантирует работоспособность интеграции при переходе к эксплуатации. На рис. 2.7 представлена структура теста для подтверждения успешной интеграции через границу с базой данных.

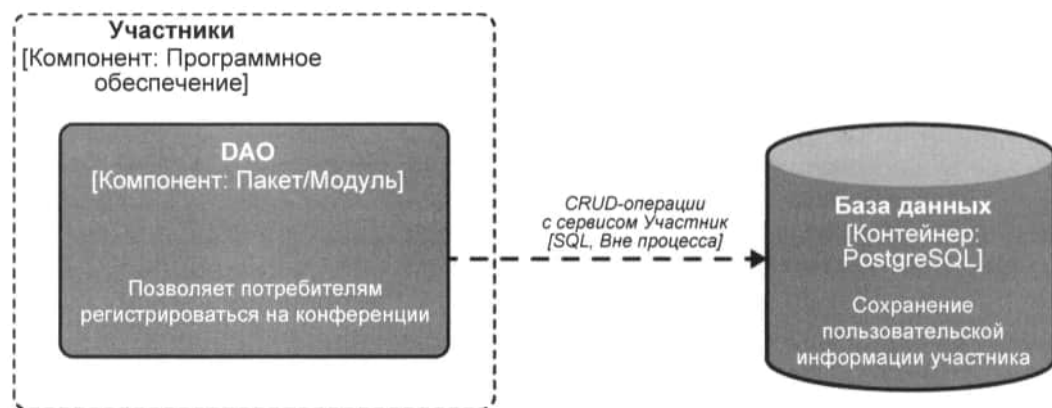


Рис. 2.7. Тестирование DAO Testcontainers

Инструмент Testcontainers достаточно мощный, и его следует учитывать при тестировании границ между любыми внешними сервисами. Среди других распространенных внешних сервисов, выигрывающих от использования Testcontainers, — Kafka, Redis и NGINX. Добавление такого решения увеличит время выполнения тестов, однако интеграционных тестов обычно меньше, и обеспечиваемая ими дополнительная уверенность чаще всего оказывается достойной платой за потраченное время.

При использовании Testcontainers возникает несколько вопросов. Во-первых, считается ли такое тестирование интеграционным или это сквозное тестирование, поскольку тестируется реальный экземпляр другого сервиса? Во-вторых, почему бы просто не задействовать этот подход вместо контрактов?

Использование Testcontainers не делает тесты сквозными, если они остаются в границах интеграции. Мы рекомендуем применять Testcontainers для тестирования интеграций. Обеспечение правильного поведения контейнера не входит в ваши обязанности (при условии, что владелец образа находится вне вашего домена). Например, если выдать запрос на публикацию сообщения брокеру Kafka, нет необходимости затем подписываться на *topic*³⁰, чтобы проверить, что опубликованный элемент был корректным. Остается верить, что Kafka выполняет свою работу и подписчики получают сообщение. Если потребуется проверить это поведение, сделайте его частью сквозных тестов. Именно поэтому имеет значение граница того, что вы тестируете. И становится понятно, почему пример с DAO к базе данных не относится к сквозному тестированию, т. к. проверяется только взаимодействие через границу.

³⁰ Есть такое понятие в Apache Kafka.

Применение Testcontainers и интеграция с реальным сервисом — это реальное преимущество. Оно может принести много пользы при тестировании, однако не заменяет контракты только потому, что вы можете использовать реальную версию сервиса. Работа с реальным экземпляром — это хорошо, однако контракты предоставляют гораздо больше, чем просто сервер-заглушку, — они обеспечивают тестирование, интеграцию и совместную работу.

Сквозное тестирование

Суть *сквозного тестирования* заключается в совместном тестировании сервисов и их зависимостей с целью проверки их работы в соответствии с ожиданиями. Важно убедиться в том, что при поступлении запроса на «входную дверь» (т. е. когда запрос доходит до вашей инфраструктуры) он проходит весь путь и потребитель получает правильный ответ. Такая проверка дает уверенность в том, что все эти системы работают как надо. В нашем случае это тестирование унаследованной конференц-системы, нового сервиса Участник и базы данных вместе.

Автоматизация сквозной валидации

Этот раздел посвящен автоматизированным сквозным тестам. Автоматизация призвана экономить ваше время, поэтому мы опишем виды автоматизированного тестирования, дающие, по нашему мнению, наилучшие результаты. Вам всегда нужно будет проверять совместную работу систем, однако это можно сделать вручную в тестовой среде перед релизом программного обеспечения в эксплуатацию.



Если вы создаете API, ориентированный на внешнюю среду, и его используют несколько сторонних клиентов, не пытайтесь копировать их пользовательский интерфейс и повторять его работу. Это приведет к трате огромного количества времени в попытках воспроизвести что-то из своего домена.

Для сквозного тестирования идеальным вариантом будет наличие реальных версий *ваших* сервисов, работающих и взаимодействующих между собой, однако иногда это сложно обеспечить. Поэтому вполне допустимо исключить некоторые сущности системы, находящиеся вне сферы деятельности вашей организации и предоставляемые внешней стороной. В качестве гипотетического случая можно привести ситуацию, когда сервис Участник требует использования AWS S3³¹. Полагаясь на стороннюю сущность, вы можете столкнуться с такими проблемами, как сетевые сбои или даже недоступность внешнего провайдера. Кроме того, если в тестах не предполагается использовать сущность, то нет необходимости делать ее доступной для тестов. Для сквозного тестирования сервиса Участник необходимо запустить базу данных и сам сервис, но для этого не надо использовать унаследованную конференц-систему, т. к. она будет лишней. Именно поэтому сквозные тесты иногда требуют разграничений. Граница для этого сквозного теста показана на рис. 2.8.

³¹ См. <https://oreil.ly/pzeua>.



Рис. 2.8. Границы сквозного теста

Управление и координацию работы нескольких систем нелегко автоматизировать, а сквозные тесты могут оказаться хрупкими. Однако выполнять сквозное тестирование локально становится проще. Контейнеризация позволяет локально развернуть несколько систем, как говорилось в разд. «Контейнеризация тестовых компонентов: *Testcontainers*». Несмотря на то что этот процесс, как только что сказано, становится проще, все равно следует придерживаться рекомендаций пирамиды тестирования: сквозные тесты не зря находятся на ее вершине.

При написании сквозных тестов следует задействовать реалистичные полезные нагрузки. Мы наблюдали случаи, когда в тестах использовалась небольшая и лаконичная полезная нагрузка, а при выяснении причин поломки API оказывалось, что потребители регулярно отправляют очень объемные полезные нагрузки — больше, чем могут поддерживать буферы. Именно поэтому сквозное тестирование должно соответствовать тому, как потребитель использует ваш API.

Типы сквозных тестов

При написании сквозных тестов следует исходить из наиболее важных требований, приведенных в разд. «Квадрант тестирования».

В квадранте тестирования 3 представлено тестирование сценариев. Такие тесты представляют собой одну из распространенных форм сквозного тестирования. Они предназначены для тестирования типичных пользовательских маршрутов и обеспечивают уверенность в том, что ваш сервис работает правильно. Тест сценария может быть основан как на одном действии, так и на нескольких. Важно, чтобы вы тестировали только основные маршруты пользователей и не тестировали крайние случаи или исключения. В написании тестов вам может помочь подход *разработки через поведение* (Behavior Driven Development, BDD)³². Это хороший способ написать пользовательские истории как часть тестов, ориентированных на бизнес. Вот пример этого подхода для конференц-системы: когда участник регистрируется для

³² См. <https://oreil.ly/qs7nY>.

работы в конференции, в выдаваемой информации о ней количество участников должно увеличиться.

Тесты сценариев и проверка основных пользовательских маршрутов хороши тем, что вам не надо озабочиваться тем, что компонент работает медленнее, чем в эксплуатируемой системе. Опрашивается правильное поведение и ожидаемые результаты. Однако при проведении сквозных тестов производительности необходимо быть более внимательным. Тестирование производительности в квадранте 4 квадранта тестирования должно быть развернуто в среде, аналогичной производственной. Если среда тестирования будет от нее отличаться, вы не получите результатов, отражающих эффективность работы ваших сервисов. И потом, когда вам придется развертывать свои сервисы на этом производственном оборудовании, это может создать проблемы. Необходимо учитывать ситуации, когда принятый подход делает ваши тесты нестабильными и на их переработку потребуется значительное время. Однако это не должно отпугивать вас, поскольку мы уже сталкивались с успешным проведением такого рода комплексного тестирования.

Тесты производительности, написанные в рамках сквозного тестирования, должны быть направлены на поддержку уверенности в том, что вы обслуживаете запросы в пределах целевых SLO. Необходимо также, чтобы эти тесты производительности показали, что вы не внесли никаких неожиданных задержек в работу своих сервисов (например, случайно не добавили блокирующий код). Если важен объем, надо убедиться, что сервис способен справиться с ожидаемой нагрузкой. Для тестирования производительности существует ряд отличных инструментов, таких как Gatling, JMeter, Locust и K6. Даже если ни один из них почему-то не подходит, существуют и другие, причем на разных, возможно, знакомых вам языках. Необходимые показатели производительности должны определяться требованиями бизнеса.

В рамках сквозного тестирования нужно также убедиться в надежности защиты (т. е. в том, что включен протокол TLS и работают соответствующие механизмы аутентификации). В этих тестах не следует отключать безопасность, поскольку это не делает их репрезентативными для пользовательского перемещения по системе или искажает метрики.

Автоматизированное сквозное тестирование сложнее, чем любой другой вид тестирования, поскольку требует ресурсов для создания тестов и их поддержки. Но оно может дать экономию времени по сравнению с проведением сквозного тестирования вручную, обеспечить уверенность в приложении и подтвердить, что сервисы работали с технической точки зрения в соответствии с соглашениями об обслуживании.

Руководство ADR: сквозное тестирование

Важным моментом является понимание того, что именно необходимо включить в процесс тестирования, и целесообразности сквозного тестирования в вашем случае. Руководство ADR, приведенное в табл. 2.4, должно помочь вам принять решение.

Таблица 2.4. Руководство ADR: сквозное тестирование

Решение	Следует ли использовать в рамках тестирования автоматизированные сквозные тесты?
Пункты обсуждения	Определите уровень сложности вашей системы, чтобы обеспечить возможность сквозного тестирования. Имеете ли вы представление о необходимых и приносящих пользу сквозных тестах? Существуют ли какие-либо особые требования или более сложные сквозные тесты, которые необходимо добавить?
Рекомендации	Мы рекомендуем проводить как минимум сквозное тестирование основных пользовательских маршрутов. Это позволит как можно раньше в цикле разработки получить информацию о том, что внесенные изменения могут повлиять на пользователя. В идеале эти сквозные тесты можно выполнять локально, но если такой возможности нет, они должны быть частью конвейера сборки. Сквозное тестирование очень ценно, но его необходимо соизмерять с временными затратами, необходимыми для его проведения. Если автоматическое сквозное тестирование невозможно, то необходимо создать руководство по разрешенным ручным тестам. Такое руководство следует использовать в тестовой среде перед выпуском производственного релиза. Тестирование вручную значительно замедлит выпуск релизов серийных версий и притормозит процесс предоставления их клиентам.

Заключение

В этой главе вы узнали об основных видах тестирования API, в том числе о том, что следует тестировать и когда на это следует выделить время. Основные выводы:

- ♦ придерживайтесь основ тестирования и сделайте модульное тестирование ядром вашего API;
- ♦ контрактное тестирование может помочь вам разработать согласованный API и провести тестирование с другими API;
- ♦ выполните сервисное тестирование компонента и изолируйте интеграцию для проверки входящего и исходящего трафика;
- ♦ используйте сквозные тесты для воспроизведения основных маршрутов пользователей, чтобы убедиться в правильности интеграции API;
- ♦ воспользуйтесь руководством ADR, чтобы понять, стоит ли добавлять в API различные тесты.

Хотя мы предоставили вам множество информации, идей и методов тестирования API, это далеко не полный список доступных инструментов. Мы рекомендуем вам исследовать доступные фреймворки и библиотеки для тестирования, чтобы убедиться в том, что вы принимаете обоснованное решение.

Однако, сколько бы тестов ни проводилось на предварительном этапе, нет ничего лучше, чем увидеть, как приложение на самом деле работает в реальных условиях. Подробнее о тестировании в рабочей среде будет рассказано в главе 5. А следующая глава посвящена предоставлению доступа к API и управлению ими в производственных условиях с помощью API-шлюзов.

Управление трафиком API

В этой части книги рассматривается управление трафиком API. В него входит как трафик, исходящий от конечных пользователей и поступающий (входящий) в систему, так и трафик, исходящий от внутренних сервисов и проходящий через систему (от сервиса к сервису).

- ♦ В *главе 3* — рекомендованной точке начала путешествия — мы рассмотрим использование технологии API-шлюзов для управления входящим трафиком, или трафиком север-юг.
- ♦ В *главе 4* рассказывается об управлении трафиком восток-запад с помощью схемы сервисной сети.

API-шлюзы: управление входящим трафиком

Теперь, когда вы хорошо понимаете, как определить и протестировать API, можно обратить внимание на платформы и инструментальные средства, отвечающие за доставку API потребителям в производстве. API-шлюз стал важнейшей частью любого современного технологического стека, располагаясь на сетевой «периферии» системы и выступая в качестве инструмента управления, который служит посредником между потребителем и набором сервисов бэкенда.

В этой главе вы получите ответы на вопросы: почему, что и где, в отношении API-шлюзов, а также познакомитесь с историей развития API-шлюзов и других периферийных (граничных) технологий. Вы также изучите таксономию API-шлюзов и узнаете, как они вписываются в общую картину системной архитектуры и моделей развертывания, избегая при этом распространенных ошибок.

Опираясь на все эти темы, в завершение главы вы узнаете, как выбрать подходящий API-шлюз, исходя из ваших требований, ограничений и вариантов использования.

Является ли API-шлюз единственным решением?

Нам часто задают вопрос: «Является ли API-шлюз единственным решением для направления пользовательского трафика к системам бэкенда?». Короткий ответ — нет. Но здесь есть еще один нюанс.

Многие программные системы нуждаются в маршрутизации потребительских API-запросов или входящего трафика от внешнего источника к внутреннему приложению бэкенда. В программных системах на базе веб-технологий запросы к API часто поступают от конечного пользователя, взаимодействующего с системой бэкенда через веб-браузер или мобильное приложение. Запросы потребителя могут также исходить от внешней системы (часто сторонней), делающей запросы к API через приложение, развернутое в другом месте в Интернете. Помимо обеспечения механизма маршрутизации трафика от URL к системе бэкенда, от решения, обеспечивающего поступление трафика, обычно также требуется надежность, наблюдаемость и безопасность.

Как вы узнаете из этой главы, API-шлюз — это не единственная технология, способная обеспечить указанные требования. Например, можно использовать простую реализацию прокси-сервера или балансировщика нагрузки. Однако мы считаем, что это наиболее часто используемое решение, особенно в корпоративном контексте, и по мере увеличения числа потребителей и поставщиков оно часто оказывается наиболее масштабируемым, поддерживаемым и безопасным вариантом.

Как показано в табл. 3.1, необходимо сопоставить текущие требования с возможностями каждого решения. Не волнуйтесь, если вы не понимаете всех этих требований, поскольку вы узнаете о них больше, последовательно знакомясь с материалом этой главы.

Таблица 3.1. Сравнение обратных прокси, балансировщиков нагрузки и API-шлюзов

Функциональная возможность	Обратный прокси	Балансировщик нагрузки	API-шлюз
Единый бэкэнд	*	*	*
TLS/SSL	*	*	*
Многочисленные бэкэнды		*	*
Обнаружение сервиса		*	*
Состав API			*
Авторизация			*
Логика повторных попыток			*
Ограничение количества запросов			*
Ведение журнала и трассировка			*
Автоматические выключатели			*

Рекомендация: прокси, балансировщик нагрузки или API-шлюз

В табл. 3.2 приведен ряд рекомендаций ADR. Они помогут вам выбрать оптимальное решение по входу для системы вашей организации или текущего проекта.

Таблица 3.2. Руководство ADR: прокси, балансировщик нагрузки или API-шлюз

Решение	Следует ли использовать прокси, балансировщик нагрузки или API-шлюзы для маршрутизации входящего трафика?
Пункты обсуждения	Нужна ли вам простая маршрутизация, например от единой конечной точки к одному сервису бэкэнда? Есть ли у вас кросс-функциональные требования, требующие более сложных функций, таких как аутентификация, авторизация или ограничение количества запросов? Требуется ли вам функциональность API-менеджмента, например API-ключи/токены или монетизация/возврат средств? Есть ли у вас уже готовое решение или в масштабах организации существует требование о том, что весь трафик должен проходить через определенные компоненты на периферии сети?
Рекомендации	Всегда используйте самое простое решение, соответствующее вашим требованиям, с учетом ближайшего будущего и известных условий. Если у вас есть расширенные кросс-функциональные требования, то лучшим выбором, как правило, будет API-шлюз. Если ваша организация представляет собой предприятие, рекомендуется использовать API-шлюз, поддерживающий функции API-менеджмента (APIM). Всегда проводите в своей организации комплексную проверку существующих мандатов, решений и компонентов

Практический пример: открытие доступа к сервису Участник для потребителей

Поскольку с момента запуска конференц-система пользуется большим спросом, ее владельцы хотели бы предоставить участникам конференции возможность просматривать свои данные через новое мобильное приложение. Это потребует открытия доступа к API сервиса Участник, чтобы мобильное приложение могло запрашивать такие данные. Поскольку сервис Участник содержит персональную информацию (PII), это означает, что API должен быть не только надежным и наблюдаемым, но и безопасным. Можно просто открыть доступ к API с помощью прокси-сервера или балансировщика нагрузки и реализовать все дополнительные требования, используя специфические для языка или фреймворка функциональные возможности. Однако необходимо задать себе вопросы: будет ли это решение масштабироваться, будет ли оно многократно использоваться (потенциально поддерживая дополнительные API с применением различных языков и фреймворков) и были ли эти задачи уже решены в рамках существующих технологий или продуктов? Наш пример предусматривает, что в будущем планируется выпустить дополнительные API, а при их реализации могут быть использованы дополнительные языки и фреймворки. Поэтому имеет смысл внедрить решение на базе API-шлюза.

По мере развития этой главы вы добавите API-шлюз в существующую конференц-систему для открытия доступа к API Участник таким образом, чтобы он отвечал всем упомянутым требованиям. На рис. 3.1 показано, как будет выглядеть архитектура конференц-системы с добавлением API-шлюза.

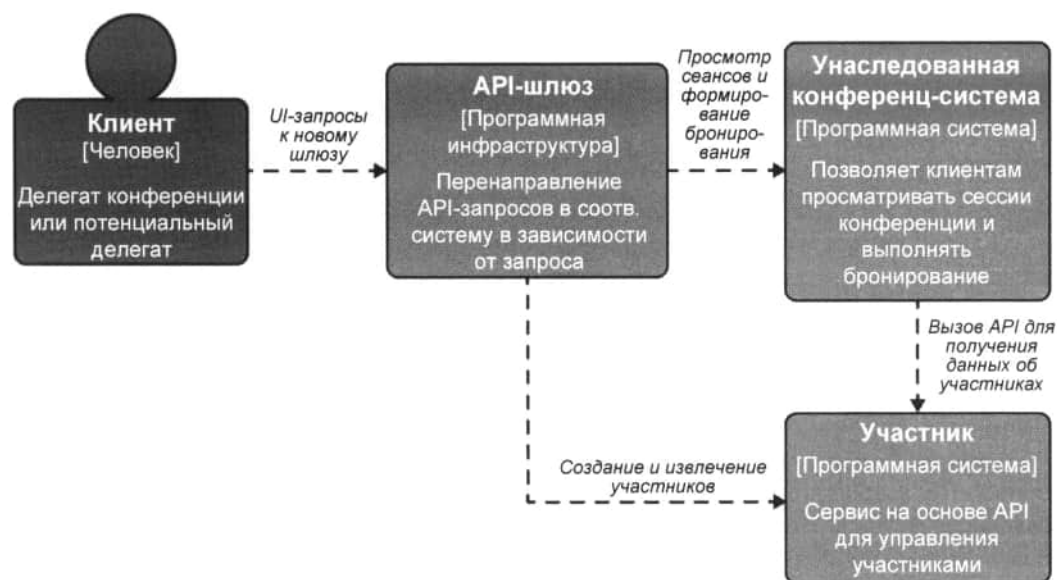


Рис. 3.1. Использование API-шлюза для маршрутизации к сервису Участник, работающему независимо от монолита

Что такое API-шлюз?

В двух словах: API-шлюз — это инструмент управления, расположенный на периферии системы между потребителем и набором бэкенд-сервисов и выступающий в качестве единой точки входа для определенной группы API. Потребителем может быть приложение или устройство конечного пользователя — например, одностороннее веб-приложение, мобильное приложение, другая внутренняя система, а также приложение или система стороннего производителя.

API-шлюз реализуется с помощью двух фундаментальных компонентов высокого уровня: плоскостей управления и данных. Эти компоненты, как правило, могут быть упакованы вместе или развернуты отдельно. В плоскости управления операторы взаимодействуют со шлюзом и определяют маршруты, политики и требуемую телеметрию. Плоскость данных — это место, где происходит вся работа, указанная в плоскости управления, маршрутизируются сетевые пакеты, реализуются политики и выдается телеметрия.

Какие функциональные возможности предоставляет API-шлюз?

На сетевом уровне API-шлюз обычно выступает в роли обратного прокси, принимая все API-запросы от потребителя, вызывая и агрегируя различные бэкенд-сервисы прикладного уровня (и, возможно, внешние сервисы), необходимые для их выполнения, и возвращая соответствующий результат.

Что такое прокси, прямой и обратный?

Прокси-сервер, иногда называемый *прямым прокси*, представляет собой сервер-посредник, перенаправляющий запросы на контент от нескольких клиентов к различным серверам в Интернете. Прямой прокси используется для *защиты клиентов*. Например, на предприятии может быть установлен прокси-сервер, направляющий и фильтрующий трафик сотрудников в публичный Интернет. С другой стороны, *обратный* прокси-сервер обычно располагается за брандмауэром в частной сети и направляет запросы клиентов на соответствующий сервер бэкенда. Обратный прокси призван *защитить серверы*.

API-шлюз обеспечивает такие сквозные требования, как аутентификация пользователей, ограничение количества запросов, тайм-ауты/повторные попытки, а также может предоставлять метрики, журналы и данные трассировки для поддержки реализации наблюдаемости в системе. Многие шлюзы API имеют дополнительные функции, которые позволяют разработчикам управлять жизненным циклом API, помогают разработчикам, использующим API, в регистрации и управлении ими (например, предоставляют портал для соответствующего администрирования учетных записей и контроля доступа), а также обеспечивают корпоративное управление.

Где выполняется развертывание API-шлюза?

API-шлюз обычно развертывается на периферии (границе) системы, но определение «система» в таком случае может быть достаточно гибким. Для стартапов и многих компаний малого и среднего бизнеса (Small-Medium Businesses, SMB) API-шлюз часто развертывается на периферии центра обработки данных или в облаке. В таких ситуациях может использоваться только один API-шлюз (развернутый и работающий в нескольких экземплярах для обеспечения высокой доступности), выступающий в качестве «входной двери» для всего бэкенда. Он будет предоставлять всю пограничную функциональность, обсуждаемую в этой главе, через этот единственный компонент.

На рис. 3.2 показано, как клиенты взаимодействуют с API-шлюзом и системами бэкенда через Интернет.

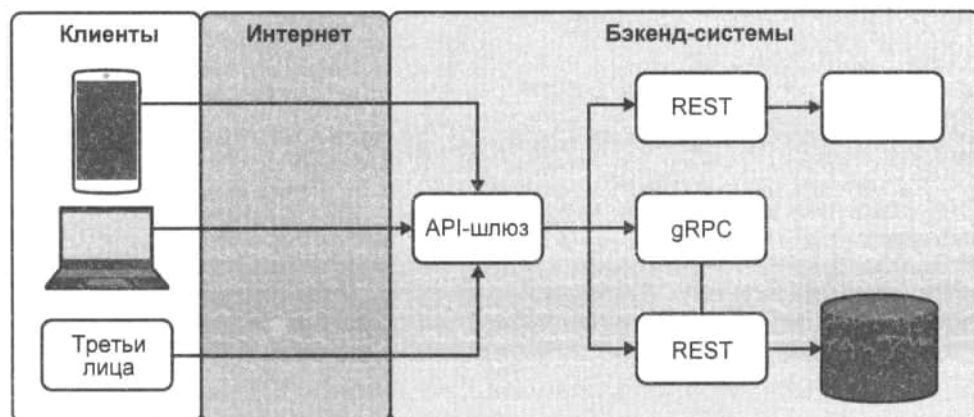


Рис. 3.2. Типичное развертывание API-шлюза для стартапа/SMB

В крупных организациях и предприятиях API-шлюз обычно развертывается в нескольких местах, часто в составе начального пограничного стека на периметре центра обработки данных, а дополнительные шлюзы могут быть развернуты для каждого продукта, направления деятельности или отдела организации. В рассматриваемом контексте эти шлюзы, как правило, представляют собой отдельные реализации и могут иметь различную функциональность в зависимости от географического положения (например, требуемое управление) или возможностей инфраструктуры (например, работа на маломощных периферийных вычислительных ресурсах).

На рис. 3.3 показано, как API-шлюз часто располагается между публичным Интернетом и демилитаризованной зоной (ДМЗ) частной сети.

Далее в этой главе мы увидим, что определение и конкретная функциональность API-шлюза не всегда совпадают в разных реализациях, поэтому приведенные здесь схемы следует рассматривать скорее как концепцию, чем как точную реализацию.

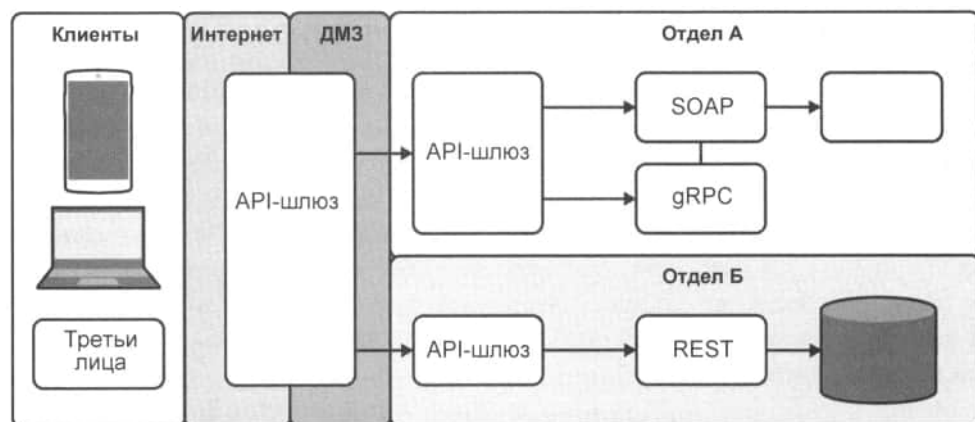


Рис. 3.3. Типичное развёртывание API-шлюза на крупном предприятии

Как API-шлюз интегрируется с другими технологиями на периферии?

Как правило, на периферии системы, основанной на API, развернуто множество компонентов.

Именно здесь происходит первое взаимодействие потребителей и пользователей с бэкендом, и многие сквозные проблемы лучше всего решать как раз на периферии. Поэтому современный стек периферийных технологий, или «периферийный стек», обеспечивает набор функциональных возможностей, отвечающих основным кросс-функциональным требованиям к приложениям на базе API. В некоторых периферийных стеках каждая часть функциональности обеспечивается отдельным развернутым и эксплуатируемым компонентом, а в других функциональность и/или компоненты комбинируются. Более подробно об отдельных требованиях вы узнаете в следующем разделе главы, а пока посмотрите на рис. 3.4, где можно выделить основные слои современного периферийного стека.

Эти слои не следует рассматривать как монолитный компонент. Как правило, они развертываются отдельно и могут принадлежать и эксплуатироваться отдельными командами или сторонними поставщиками услуг. Некоторые API-шлюзы предоставляют всю функциональность в рамках периферийного стека. Другие просто фокусируются на функциональности API-шлюза и API-менеджменте. Также часто в облачных средах поставщик облака предоставляет балансировщик нагрузки, который можно интегрировать с API-шлюзом.

Теперь, когда вы получили представление об ответах на вопросы: что и где, для API-шлюза, давайте рассмотрим, зачем организации использовать API-шлюз.

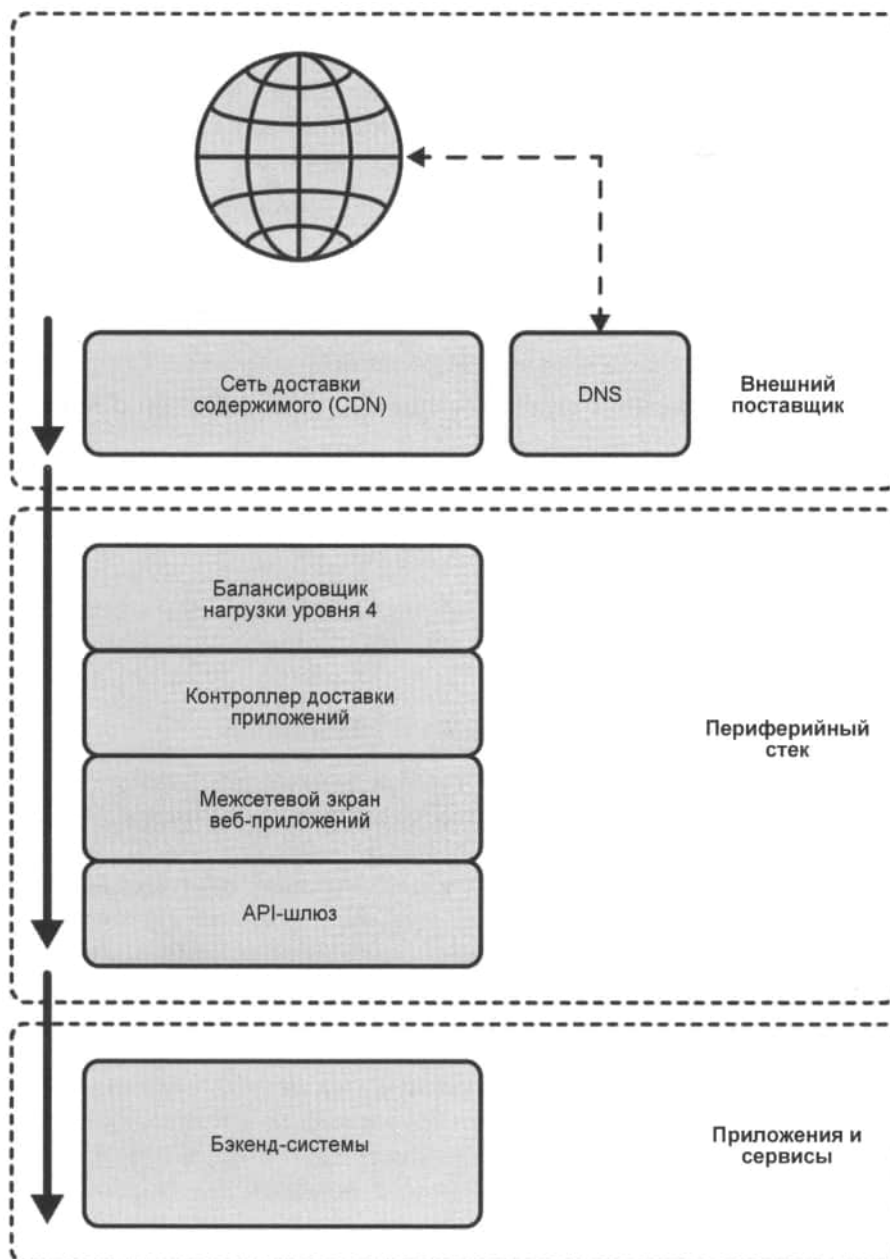


Рис. 3.4. Современный периферийный стек

Зачем нужен API-шлюз?

Значительная часть обязанностей современного архитектора программного обеспечения заключается в том, чтобы задавать трудные вопросы о проектировании и реализации. Иначе обстоит дело с API, управлением трафиком и сопутствующими

технологиями. Необходимо найти баланс между краткосрочным внедрением и долгосрочной возможностью сопровождения. Существует множество сквозных проблем, связанных с API, включающих удобство сопровождения, расширяемость, безопасность, наблюдаемость, управление жизненным циклом продукта и монетизацию. API-шлюз может помочь в решении всех этих задач!

В этом разделе главы представлен обзор основных проблем, решаемых при помощи API-шлюза, таких как:

- ◆ уменьшение связанности за счет использования адаптера/фасада между фрон- тендами и бэкендами;
- ◆ упрощение потребления за счет агрегации/преобразования бэкенд-сервисов;
- ◆ защита API от чрезмерного использования и злоупотреблений с помощью систем обнаружения и устранения угроз;
- ◆ понимание того, как потребляются API (наблюдаемость);
- ◆ управление API как продуктами с помощью функции управления жизненным циклом API;
- ◆ монетизация API с помощью управления учетными записями, биллинга и оплаты.

Снижение связанности: адаптер/фасад между фронтендами и бэкендами

Три фундаментальные концепции, с которыми каждый архитектор программного обеспечения должен познакомиться в самом начале своей карьеры, — это связанность (coupling)¹, связность (cohesion)² и скрытие информации. Им внушают, что системы, спроектированные с учетом низкой связанности и сильной связности, легче понимать, поддерживать и модифицировать. Скрытие информации — это принцип разделения проектных решений в программной системе, которые с наибольшей вероятностью могут измениться. Слабая связанность позволяет легко менять местами различные реализации, что может быть особенно полезно при тестировании систем (например, легче имитировать и заглушать слабосвязанные зависимости). Сильная связность способствует понятности (весь код в модуле или системе поддерживает основную цель), а также надежности и возможности повторного использования. Скрытие информации защищает другие части системы от обширной модификации при изменении проектного решения. По нашему опыту, API часто становятся теми местами в системе, где архитектурная теория встречается с реальностью. API — это в прямом и переносном смысле интерфейс, с помощью которого интегрируются части системы.

API-шлюз может выступать как в качестве единой точки входа, так и в качестве фасада или адаптера и, следовательно, способствовать слабой связанности и связ-

¹ Из всего набора вариантов перевода термин «coupling» обычно во многих изданиях переводится как «связанность». — *Прим. пер.*

² Точно так же термин «cohesion» чаще всего переводится как «связность». — *Прим. пер.*

ности. *Фасад* определяет новый более простой интерфейс для системы, в то время как *адаптер* использует старый интерфейс с целью поддержки совместимости между двумя существующими интерфейсами. Клиенты интегрируются с API, представленным на шлюзе, что при условии соблюдения согласованного контракта позволяет компонентам на бэкенде менять местоположение, архитектуру и реализацию (язык, фреймворк и т. п.) с минимальными последствиями. На рис. 3.5 показано, как API-шлюз может выступать в качестве единой точки входа для клиентских запросов к API и сервисам бэкенда.

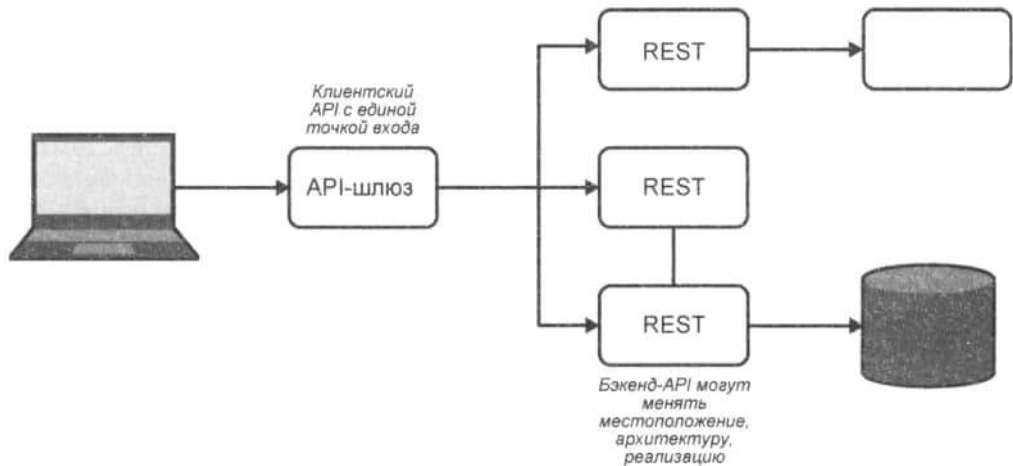


Рис. 3.5. API-шлюз, обеспечивающий фасад между фронтендами и бэкендами

Упрощение использования: агрегирование/преобразование бэкенд-сервисов

Продолжим здесь обсуждение связанности, начатое в предыдущем разделе. Часто бывает так, что предоставляемый фронтенд-системам API, отличается от текущего интерфейса, предоставляемого бэкендом или композицией бэкенд-систем. Например, вы можете объединить API нескольких бэкенд-сервисов, принадлежащих нескольким владельцам, в один API, ориентированный на потребителя, чтобы упростить ментальную модель для фронтенд-инженеров, оптимизировать управление данными или скрыть архитектуру интерфейса бэкенда. Именно по этим причинам часто используется язык GraphQL³. Конечно, реализация такого рода функциональности имеет свои компромиссы и может оказаться слишком простым вариантом, когда у логики внутри API-шлюза возникает сильная связанность с бизнес-логикой бэкенд-сервиса.

³ См. <https://graphql.org/>.

Оркестровка параллельных вызовов API

Популярным подходом к упрощению взаимодействия, реализованным в API-шлюзах, стала оркестровка параллельных обращений к API бэкэнда. Именно здесь шлюз выполняет оркестровку и координирует параллельный вызов нескольких независимых API бэкэнда. Как правило, для экономии времени при сборе результатов для потребителя необходимо вызывать несколько независимых и несвязанных API не последовательно, а параллельно. Обеспечение этого процесса в шлюзе избавляет от необходимости самостоятельно реализовывать такую функциональность в каждом из потребителей. И снова компромисс: бизнес-логика может оказаться распределена по API-шлюзу и бэкэнд-системам. Также необходимо учитывать вопросы, относящиеся к эксплуатационной связанности. Изменение реализации API-шлюза, меняющее порядок вызовов API, может повлиять на ожидаемые результаты, особенно если вызовы бэкэнда не являются идемпотентными.

Кроме того, в корпоративном контексте часто требуется преобразование (перевод) протоколов. Например, у вас может быть несколько «наследственных» систем, предоставляющих только API на основе протокола SOAP, но вы хотите предоставить потребителям только REST-подобные API. API-шлюз может обеспечить такую функциональность агрегирования и преобразования, хотя при этом следует соблюдать осторожность. Чтобы убедиться в том, что преобразование выполнено с должной точностью, необходимо оплатить его разработку, внедрение и тестирование. Кроме того, существуют затраты вычислительных ресурсов на реализацию преобразования, и этот процесс может оказаться дорогостоящим при работе с большим количеством запросов. На рис. 3.6 показано, как API-шлюз может обеспечивать агрегацию вызовов бэкэнд-сервисов и преобразование протоколов.

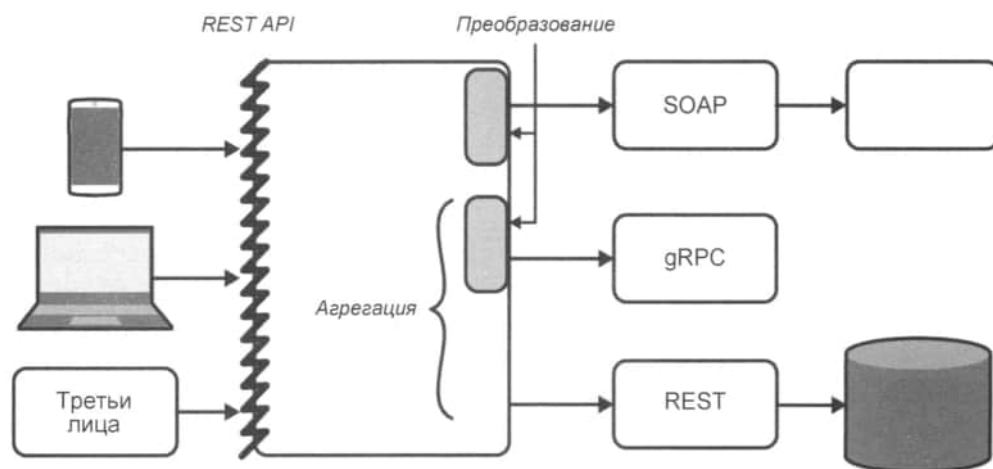


Рис. 3.6. API-шлюз, обеспечивающий агрегацию и преобразование

Защита API от чрезмерного использования и злоупотреблений: обнаружение и устранение угроз

Периферия, или граница системы — это место, где пользователи впервые взаимодействуют с вашими приложениями. Кроме того, часто именно в этой точке к ва-

шим системам впервые пытаются получить доступ злоумышленники и хакеры. Хотя подавляющее большинство корпоративных организаций имеют в своем периферийном стеке несколько уровней безопасности, таких как *сеть доставки содержимого* (Content delivery network, CDN) и *межсетевой экран веб-приложений* (Web Application Firewall, WAF), и даже сетевой периметр и выделенную *демилитаризованную зону* (ДМЗ), для многих небольших организаций API-шлюз может стать первой линией обороны. По этой причине многие API-шлюзы включают в себя функции, ориентированные на обеспечение безопасности, такие как завершение TLS, аутентификация/авторизация, списки разрешенных/запрещенных IP-адресов, WAF (встроенные или с помощью внешней интеграции), ограничение количества запросов и сброс нагрузки, проверка контрактов API. На рис. 3.7 показано, как список разрешенных/запрещенных адресов и ограничение количества запросов могут быть использованы для борьбы со злоупотреблениями в API.

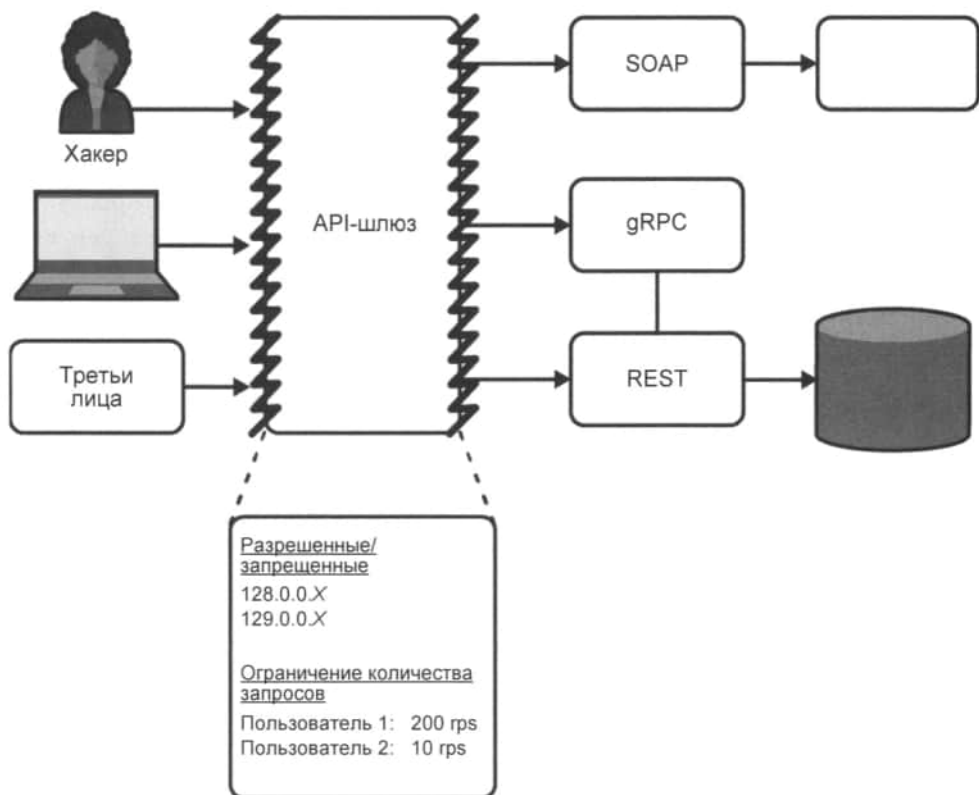


Рис. 3.7. Предотвращение API-шлюзами чрезмерного использования и злоупотреблений

Важной частью этой функциональности является возможность обнаружения случайных или преднамеренных злоупотреблений API, для чего необходимо реализовать комплексную стратегию наблюдаемости.

Понимание того, как используются API: наблюдаемость

Понимание того, как работают системы и приложения, жизненно необходимо для обеспечения достижения бизнес-целей и удовлетворения требований заказчиков⁴. Все чаще цели бизнеса измеряются с помощью *ключевых показателей эффективности* (Key Performance Indicators, KPI), таких как конверсия клиентов, доход в час, запуск потока в секунду и т. д. Инфраструктура и платформы обычно рассматриваются через призму *индикаторов уровня обслуживания* (Service-Level Indicators, SLI)⁵, таких как задержка, ошибки, глубина очереди и т. п.

Поскольку подавляющее большинство (если не все) пользовательских запросов проходит через границу системы, это важный момент для наблюдаемости. И является идеальным местом для сбора основных метрик входящего потока, таких как количество ошибок, пропускная способность и задержка, а также ключевым местом для идентификации и аннотирования запросов (возможно, с метаданными, специфичными для конкретного приложения), проходящих через всю систему дальше по потоку выполнения. Идентификаторы корреляции⁶ обычно внедряются в запрос через API-шлюз и затем могут быть распространены каждым вышестоящим сервисом. Этими идентификаторами можно воспользоваться для корреляции записей журналов и трассировки запросов в различных сервисах и системах.

Хотя получение и сбор данных о наблюдаемости очень важны на уровне системы, необходимо также тщательно продумать, как обрабатывать, анализировать и интерпретировать эти данные, превращая их в практическую информацию, используемую позже для принятия решений. Создание информационных панелей для визуального отображения и управления, а также определение оповещений являются жизненно важными для успешной стратегии наблюдаемости.

Управление API как продуктами: менеджмент жизненного цикла API

Современные API часто проектируются, создаются и работают в качестве продуктов, потребляемых как внутренними системами, так и сторонними организациями, и управлять ими нужно именно так. Многие крупные организации рассматривают API в качестве критически важного и стратегического компонента и, соответственно, разрабатывают стратегию программы API, устанавливают четкие бизнес-цели, ограничения и ресурсы. При наличии стратегии ежедневный тактический подход часто сосредоточен на управлении жизненным циклом API. API-менеджмент (APIM) полного жизненного цикла API охватывает весь период его существования: он начинается на этапе планирования и заканчивается при выводе API из эксплуатации. Многие этапы жизненного цикла в значительной степени зависят от реализации, предоставляемой API-шлюзом. По этим причинам выбор подходящего API-

⁴ Книга Синди Шридхаран под названием «Distributed Systems Observability» (<https://oreil.ly/plmte>) — отличное пособие для изучения темы наблюдаемости.

⁵ См. <https://oreil.ly/S0ktZ>.

⁶ Например, заголовки OpenZipkin b3 (https://oreil.ly/U_Oghv).

шлюза явится критически важным решением, если вы придерживаетесь процесса APIM.

Существует множество определений основных этапов жизненного цикла API, и мы считаем, что команда Axway достигла хорошего баланса, определив три ключевых компонента: создание, управление и потребление, а также 10 основных этапов жизненного цикла API⁷:

- ◆ **Создание.**
Проектирование и создание API.
- ◆ **Тестирование.**
Проверка функциональности, производительности и ожидаемой безопасности.
- ◆ **Публикация.**
Предоставление доступа к API для разработчиков.
- ◆ **Обеспечение безопасности.**
Снижение рисков и проблем безопасности.
- ◆ **Управление.**
Сопровождение и управление API для обеспечения их функциональности, актуальности и соответствия требованиям бизнеса.
- ◆ **Ввод в эксплуатацию.**
Позволяет разработчикам быстро научиться использовать API-интерфейсы. Например, предлагать документацию по OpenAPI или AsyncAPI, предоставлять портал и среду песочницы.
- ◆ **Анализ.**
Обеспечение наблюдаемости и анализ данных мониторинга для понимания особенностей использования и выявления проблем.
- ◆ **Продвижение.**
Реклама API для разработчиков — например, размещение на маркетплейсе API.
- ◆ **Монетизация.**
Обеспечение возможности взимания платы и сбора доходов за использование API. Этот аспект управления жизненным циклом API мы рассмотрим как отдельный этап в следующем разделе.
- ◆ **Вывод из эксплуатации.**
Поддержка вывода из эксплуатации и удаления API, что происходит по разным причинам, включая изменение приоритетов бизнеса, технологические изменения и проблемы безопасности.

На рис. 3.8 показано, как управление жизненным циклом API интегрируется с API-шлюзом и бэкенд-сервисами.

⁷ См. <https://oreil.ly/2F8mV>.

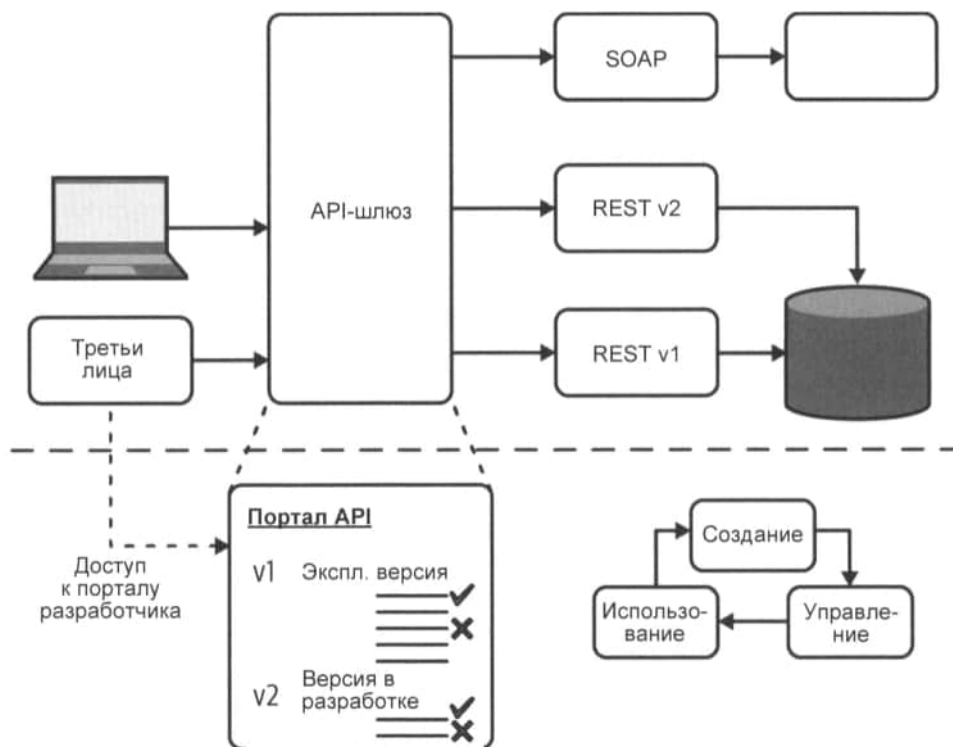


Рис. 3.8. Управление жизненным циклом API-шлюза

Монетизация API: управление учетными записями, биллинг и оплата

Тема биллинговых монетизированных API тесно связана с управлением жизненным циклом API. API, предоставляемые заказчикам, обычно должны быть оформлены в виде продукта и предлагаться через портал разработчика, включающий также возможности управления учетными записями и оплаты. Многие корпоративные API-шлюзы предусматривают монетизацию⁸. Такие платежные порталы часто интегрируются с платежными решениями типа PayPal или Stripe и позволяют настраивать тарифные планы разработчиков, ограничения количества запросов и другие параметры потребления API.

Современная история API-шлюзов

Теперь, когда вы хорошо понимаете ответы на вопросы: что, где и почему, в отношении API-шлюзов, прежде чем перейти к рассмотрению современных технологий шлюзов API, надо оглянуться назад и посмотреть на историю их развития.

⁸ Включая такие примеры, как Apigee Edge (<https://oreil.ly/1xrMi>) и 3Scale (<https://oreil.ly/kVrdp>).

Как говорил Марк Твен, «история не повторяется, но часто рифмуется», и тот, кто проработал в сфере информационных технологий более нескольких лет, наверняка оценит актуальность этой цитаты для общего подхода, наблюдаемого в отрасли. Архитектурные стили и паттерны повторяются в различных «циклах» на протяжении всей истории разработки программного обеспечения, как и операционные подходы. Эти циклы, как правило, сопровождаются развитием подходов, но мы все вместе должны быть осторожны, чтобы не пропустить предлагаемые историей уроки.

Именно поэтому важно понимать исторический контекст API-шлюзов и управления трафиком на периферии систем. Заглядывая в прошлое, можно опереться на прочное основание, понять фундаментальные требования, а также постараться избежать повторения одних и тех же ошибок.

1990-е годы и далее:

аппаратные балансировщики нагрузки

Концепция Всемирной паутины (World Wide Web, WWW) была предложена Тимом Бернерсом-Ли в конце 1980-х годов, но в сознание широкой общественности она вошла только в середине 1990-х, когда первоначальный ажиотаж вылился в бум и крах доткомов конца 1990-х годов. Период Web 1.0 привел к развитию веб-браузеров (Netscape Navigator был выпущен в конце 1994 года), веб-сервера (релиз Apache Web Server состоялся в 1995 году) и аппаратных балансировщиков нагрузки (компания F5 была основана в 1996 году). В Web 1.0 пользователи посещали сайты, делая HTTP-запросы через браузер, а в ответ возвращался весь HTML-документ каждой целевой страницы. Динамические аспекты веб-сайта реализовывались с помощью общего интерфейса шлюза (Common Gateway Interface, CGI) в сочетании со скриптами, написанными на таких языках, как Perl или C. Это было, пожалуй, первое воплощение того, что сегодня мы называем «функция как услуга» (Function as a Service, FaaS).

По мере роста числа пользователей, обращающихся к каждому из сайтов, возрастала нагрузка на базовые веб-серверы. В связи с этим возникло требование к разработке систем, поддерживающих распределение возросшей нагрузки, а также обеспечивающих отказоустойчивость. На периферии дата-центров были развернуты аппаратные балансировщики нагрузки, позволяющие инженерам инфраструктуры, сетевым специалистам и системным администраторам распределять запросы пользователей по нескольким экземплярам веб-серверов. Ранние реализации балансировщиков нагрузки, как правило, поддерживали базовую проверку работоспособности, и если веб-сервер выходил из строя или начинал отвечать с повышенной задержкой, то запросы пользователей могли быть направлены в другое место. Аппаратные балансировщики нагрузки широко используются и сегодня. Технология может совершенствоваться вместе с транзисторной технологией и архитектурой микросхем, но основная функциональность остается прежней.

Начало 2000-х годов и далее: программные балансировщики нагрузки

По мере того как Интернет преодолевал первые трудности, возникшие в результате краха доткомов, спрос на поддержку различных видов деятельности, таких как обмен контентом между пользователями, электронная коммерция и интернет-магазины, сотрудничество и интеграция систем между предприятиями, продолжал расти. В ответ на это веб-архитектуры программного обеспечения стали принимать различные формы. Небольшие организации развивали свои первые наработки при помощи стандарта CGI, а также создавали монолитные приложения на появляющихся языках, дружественных к Интернету, таких как Java и .NET. Крупные предприятия начали внедрять сервис-ориентированную архитектуру (SOA), а связанные с ней спецификации веб-сервисов (WS-* Specifications) получили кратковременную известность.

Требования к высокой доступности и масштабируемости веб-сайтов возрастали, а дороговизна и отсутствие гибкости у ранних аппаратных балансировщиков нагрузки превратились в сдерживающий фактор. Вспомним программные балансировщики нагрузки и прокси-серверы общего назначения, позволявшие реализовать эту функциональность: в 2001 году был выпущен HAProxy, а в 2002 году — NGINX. Целевыми пользователями по-прежнему оставались операционные команды, но системные администраторы, умеющие конфигурировать веб-серверы на базе программного обеспечения, все охотнее брали на себя ответственность за то, что раньше относилось к аппаратной части.



Программные балансировщики нагрузки — по-прежнему популярный вариант выбора

Несмотря на то что с момента своего появления на рынке NGINX и HAProxy претерпели значительные изменения, они по-прежнему широко используются и очень полезны для небольших организаций и простых API-шлюзов (обе системы также предлагают коммерческие варианты, более подходящие для развертывания на предприятиях). Развитие облачных вычислений (и виртуализации) закрепило значимость роли программных балансировщиков нагрузки, и мы рекомендуем ознакомиться с основами этой технологии.

В этот период времени появились и другие технологии, требующие специализированного аппаратного обеспечения. Сети доставки содержимого (CDN), в первую очередь призванные устранить узкие места в производительности сети Интернет, стали все чаще использоваться для разгрузки запросов от исходных веб-серверов. Все большее распространение получают межсетевые экраны веб-приложений (WAF). Сначала они реализовывались на специализированном оборудовании, а затем и с помощью программного обеспечения. Проект ModSecurity с открытым исходным кодом и его интеграция с веб-сервером Apache, пожалуй, послужили толчком к массовому внедрению WAF.

Середина 2000-х: контроллеры доставки приложений (ADC)

В середине 2000-х годов продолжалось все более обширное проникновение Интернета в повседневную жизнь. Появление смартфонов с поддержкой Интернета только ускорило этот процесс: сначала в этой области лидировала компания BlackBerry, а с появлением первого iPhone в 2007 году все вышло на новый уровень. Веб-браузер на базе ПК все еще оставался фактическим способом доступа к сети, а в середине 2000-х годов появилось понятие Web 2.0 наряду с широким распространением в браузерах API XMLHttpRequest и соответствующей техники, получившей название Асинхронный JavaScript и XML (Asynchronous JavaScript and XML, AJAX). В то время эта технология была революционной. Асинхронный характер API означает, что больше не нужно возвращать всю HTML-страницу, выполнять ее синтаксический анализ и полностью обновлять отображение при каждом запросе. Отделив уровень обмена данными от уровня представления, AJAX позволил веб-страницам динамически изменять содержимое без необходимости перезагрузки всей страницы.

Все эти изменения предъявили новые требования к веб-серверам и балансировщикам нагрузки. Эти инструменты должны были не только справиться с возросшей нагрузкой, но и поддерживать более безопасный (SSL) трафик, все более объемные (насыщенные мультимедиа) данные и различные приоритеты запросов. Это привело к появлению *контроллеров доставки приложений* (Application Delivery Controllers, ADC) — термин, введенный уже существующими сетевыми игроками, такими как F5 Networks, Citrix и Cisco. ADC обеспечивали поддержку сжатия, кеширования, мультиплексирования соединений, формирования трафика и разгрузки SSL в сочетании с балансировкой нагрузки. Целевыми пользователями вновь стали инженеры по инфраструктуре, сетевые специалисты и системные администраторы.

К середине 2000-х годов практически все компоненты современного периферийного стека управления трафиком уже были широко распространены в отрасли. Однако внедрение и эксплуатация многих компонентов все чаще осуществлялись отдельными командами самостоятельно. Если разработчик хотел представить новое приложение в рамках крупной организации, это, как правило, означало множество отдельных встреч с поставщиками CDN, командами балансировки нагрузки, командами InfoSec и WAF, а также с командой веб-сервера или сервера приложений. Появились такие движения, как DevOps. Отчасти их породило стремление устранить трения, налагаемые этим разделением структур внутри предприятия. Если у вас все еще большое количество слоев в периферийном стеке и вы уже переходите на облачные технологии или новую платформу, то сейчас самое время задуматься о компромиссах с многочисленными уровнями и специализированными командами.

Начало 2010-х: API-шлюзы первого поколения

Период конца 2000-х — начала 2010-х годов ознаменовался появлением API-экономики и связанных с ней технологий. Такие организации, как Twilio, революцио-

низировали телекоммуникации, а основатель Twilio Джефф Лоусон заявил⁹: «Мы взяли весь беспорядочный и сложный мир телефонии и свели его к пяти вызовам API». Google Ads API позволял создателям контента монетизировать свои сайты, а Stripe давала возможность крупным организациям легко взимать плату за доступ к сервисам. Компания Mashape, основанная в конце 2007 года, была одним из первопроходцев в попытке создания рынка API для разработчиков. Хотя эта концепция не оправдала себя (возможно, она опередила свое время, глядя на развитие решений «no code» / «low code»¹⁰), побочным продуктом бизнес-модели Mashape стало создание Kong API Gateway, построенного на базе OpenResty¹¹ и реализации NGINX с открытым исходным кодом. Среди других внедрений: WSO2 с Cloud Services Gateway, Sonos Systems с Apigee и Red Hat с 3Scale Connect.

Это были первые периферийные технологии, ориентированные не только на платформенные команды и сисадминов, но и на разработчиков. Большое внимание было уделено управлению жизненным циклом разработки программного обеспечения (Software Development Lifecycle, SDLC) API и обеспечению функциональности системной интеграции, такой как конечные точки и соединители протоколов, а также модули трансляции. В силу широты предлагаемых функциональных возможностей подавляющее большинство API-шлюзов первого поколения было реализовано программно. Во многих продуктах появились порталы для разработчиков, позволившие инженерам документировать свои API и обмениваться ими в структурированном виде. Эти порталы также обеспечивали контроль доступа, управление учетными записями пользователей и разработчиков и контроль и аналитику публикаций. Предполагалось, что это позволит легко монетизировать API и управлять «API как продуктом».

В процессе эволюции взаимодействия разработчиков на границе все большее внимание уделялось HTTP-части прикладного уровня (уровень 7) сетевой модели OSI. Предыдущие поколения периферийных технологий часто ориентировались на IP-адреса и порты, работающие в основном на транспортном уровне (уровень 4) модели OSI. Предоставление разработчикам возможности принимать решения о маршрутизации в шлюзе API на основе метаданных HTTP, таких как маршрутизация на основе путей или на основе заголовков, позволило создать более богатую функциональность.

Также наметилась тенденция к созданию более компактных архитектур на основе сервисов. В них использовались некоторые идеи, заложенные в оригинальной SOA, но переработанные с использованием более легких для производительности технологий и протоколов реализации. Организации извлекали из существующих монолитных кодовых баз одноцелевые автономные приложения. Некоторые из этих

⁹ См. <https://oreil.ly/jQsob>.

¹⁰ No-code — метод разработки ПО без кода с помощью визуальных инструментов. Такие платформы работают как конструктор: вам предоставляется библиотека с готовыми шаблонами и блоками, которые можно комбинировать на свое усмотрение. Low-code — тот же метод визуального программирования, только с возможностью дописать код вручную.

¹¹ См. <https://openresty.org/en>.

приложений выступали в роли API-шлюза или предоставляли функциональность, подобную API-шлюзу, — например, маршрутизацию и аутентификацию. В первом поколении API-шлюзов зачастую функциональные и кросс-функциональные задачи, такие как маршрутизация, безопасность и отказоустойчивость, решались как на границе, так и внутри приложений и сервисов.

2015 год и далее: API-шлюзы второго поколения

В середине 2010-х годов появилось новое поколение модульных и сервис-ориентированных архитектур, а к 2015 году в моду вошла концепция микросервисов. Во многом это произошло благодаря тому, что такие «организации-единороги», как Netflix, AWS и Spotify, поделились своим опытом работы с этими архитектурными паттернами. Помимо декомпозиции бэкэнд-систем на более многочисленные и мелкие сервисы, разработчики стали использовать контейнерные технологии на базе Linux LXC. Релиз Docker состоялся в марте 2013 года, а Kubernetes последовал за ним по пятам, выпустив версию 1.0 в июле 2015 года. Смена архитектурного стиля и изменение сред выполнения обусловили новые требования к периферии. В середине 2013 года компания Netflix запустила релиз своего собственного API-шлюза на базе JVM под названием Zuul¹². Он поддерживал обнаружение сервисов для динамических бэкэнд-сервисов, а также позволял внедрять скрипты Groovy во время выполнения программы для динамической модификации поведения. Этот шлюз также объединил в единый периферийный компонент многие общие проблемы, такие как аутентификация, тестирование (канареечные релизы), ограничение количества запросов и снижение нагрузки, а также наблюдаемость. Zuul тогда стал революционным API-шлюзом в пространстве микросервисов, с тех пор он обзавелся второй версией, а на ее основе было создано приложение Spring Cloud Gateway.

По мере распространения Kubernetes и в связи с релизом в 2016 году инженерной командой Lyft прокси-сервера Envoy с открытым исходным кодом на основе этой технологии было создано множество API-шлюзов, в том числе Ambassador Edge Stack (построенный на базе CNCF Emissary-ingress), Contour и Gloo Edge. Это способствовало дальнейшему развитию инноваций в области API-шлюзов — появились зеркальные функции Kong, предлагаемые следующим поколением шлюзов, а также другие шлюзы, такие как Traefik, Tyk и др.

Путаница в облаке:

API-шлюзы, периферийные прокси и Ingress-контроллеры

Как отметил Кристиан Поста, опубликовавший в своем блоге пост под названием «API Gateways Are Going Through an Identity Crisis»¹³, существует некоторая путаница в понимании того, что такое API-шлюз по отношению к прокси-технологиям, применяемым в сфере облачных вычислений. Вообще говоря, в этом контексте API-шлюз позволяет в той или иной форме управлять API, начиная от простой функциональности типа адаптера, работающего на прикладном уровне (уровень 7 модели OSI) и обеспечивающего фундаментальные сквозные

¹² См. <https://oreil.ly/rdN4q>.

¹³ См. <https://oreil.ly/Qkufv>.

задачи, и заканчивая управлением API на протяжении всего жизненного цикла. Периферийные прокси — это прокси-серверы трафика общего назначения или обратные прокси, работающие на сетевом и транспортном уровнях (уровни 3 и 4 модели OSI соответственно). Они обеспечивают решение основных сквозных задач и, как правило, не предлагают специфической для API функциональности. Ingress-контроллеры входа — это специфическая для Kubernetes технология. С ее помощью выполняется контроль поступающего в кластер трафика и того, как этот трафик обрабатывается.

Целевыми пользователями второго поколения API-шлюзов были в основном те же, что и у первого поколения, но с более четким разделением ответственности и большим акцентом на самостоятельное обслуживание разработчиков. При переходе от первого ко второму поколению API-шлюзов усилилась консолидация как функциональных, так и кросс-функциональных требований, реализуемых в шлюзе. Хотя стало общепризнанным, что микросервисы должны строиться на основе идеи Джеймса Льюиса и Мартина Фаулера об «умных конечных точках и глупых каналах»¹⁴, распространение полиглотных стеков привело к появлению «шлюзов микросервисов» (подробнее о них в следующем разделе). Они предлагают сквозную функциональность, не зависящую от языка.

Текущая таксономия API-шлюзов

Как и в случае с терминологией в области разработки программного обеспечения, часто нет точного согласия относительно того, что определяет или классифицирует шлюз API. Существует распространенное соглашение в отношении обеспечиваемой этой технологией функциональности. Однако различные сегменты отрасли предъявляют к API-шлюзу разные требования, а значит, и исповедуют разные взгляды. Это привело к появлению и обсуждению нескольких подтипов API-шлюзов. В этом разделе главы вы изучите появившуюся таксономию API-шлюзов и узнаете о соответствующих вариантах их использования, достоинствах и недостатках.

Традиционные корпоративные шлюзы

Традиционный корпоративный API-шлюз, как правило, предназначен для управления и открытия доступа к API, ориентированным на бизнес. Такой шлюз также часто интегрируется с решением для управления жизненным циклом API, поскольку это является необходимым условием при релизе, эксплуатации и монетизации API в масштабах компании. Большинство шлюзов в этой области могут предлагать версию с открытым исходным кодом, но предпочтение обычно отдается версии с открытым ядром (коммерческой версии шлюза).

Такие шлюзы, как правило, требуют развертывания и эксплуатации зависимых сервисов — например, хранилищ данных. Для поддержки корректной работы шлюза эти внешние зависимости должны выполняться с высокой степенью готовности, что необходимо учитывать в текущих расходах и планах DR/BC.

¹⁴ См. <https://oreil.ly/1.ySYe>.

Микросервисы/микрошлюзы

Основная задача API-шлюза микросервиса, или микрошлюза API — маршрутизация входящего трафика к API и сервисам бэкенда. По сравнению с традиционными корпоративными шлюзами, для управления жизненным циклом API, как правило, предусмотрено не так много функций. Такие шлюзы часто доступны с открытым исходным кодом или предлагаются в виде облегченной версии традиционного корпоративного шлюза.

Как правило, они разворачиваются и эксплуатируются как отдельные компоненты и часто используют базовую платформу (например, Kubernetes) для управления любым внутренним состоянием — таким как данные о жизненном цикле API, подсчеты ограничения количества запросов и управление учетными записями потребителей API. Поскольку шлюзы микросервисов обычно строятся с использованием современных прокси-технологий, таких как Envoy, возможности интеграции с сервисными сетями (особенно построенными с использованием той же прокси-технологии), как правило, хорошие.

Шлюзы сервисных сетей

Входящий API-шлюз или API-шлюз, включенный в состав сервисной сети, как правило, предназначен только для обеспечения основной функциональности — маршрутизации внешнего трафика в сеть. По этой причине в них часто отсутствуют некоторые типичные корпоративные функции, такие как комплексная интеграция с решениями для аутентификации и идентификации, а также интеграция с другими средствами защиты, например, WAF.

Шлюз сервисной сети обычно управляет состоянием с помощью своей собственной внутренней реализации или реализации, предоставляемой платформой (например, Kubernetes). Этот тип шлюза также неявно связан с соответствующей сервисной сетью (и эксплуатационными требованиями), поэтому если вы еще не планируете разворачивать сервисную сеть, то, скорее всего, такой шлюз не подойдет вам в качестве первого выбора API-шлюза.

Сравнение типов API-шлюзов

В табл. 3.3 показаны различия между тремя наиболее распространенными типами API-шлюзов по шести важным критериям.

Таблица 3.3. Сравнение API-шлюзов сервисной сети, корпоративного и микросервисного типов

Пример использования	Традиционный корпоративный API-шлюз	API-шлюз микросервисов	Шлюзы сервисных сетей
Основная цель	Открытие доступа, компоновка и управление внутренними бизнес-API и связанными с ними сервисами	Открытие доступа, компоновка и управление внутренними бизнес-сервисами	Открытие доступа к внутренним сервисам внутри сети

Таблица 3.3 (окончание)

Пример использования	Традиционный корпоративный API-шлюз	API-шлюз микросервисов	Шлюзы сервисных сетей
Функциональные возможности публикации	Команда управления API или команда сервиса регистрирует (обновляет) шлюз через API администратора (в зрелых организациях это достигается с помощью конвейеров поставки)	Команда сервиса регистрирует (обновляет) шлюз с помощью декларативного кода в рамках процесса развертывания	Команда сервиса регистрирует (обновляет) сеть и шлюз с помощью декларативного кода в рамках процесса развертывания
Мониторинг	Ориентирован на администраторов и операторов — например, измеряет количество вызовов API на одного потребителя, сообщает об ошибках (например, внутренних 5XX)	Ориентирован на разработчика — например, на задержку, трафик, ошибки, насыщенность	Ориентирован на платформу — например, на использование, насыщенность, ошибки
Обработка и отладка проблем	Обработка ошибок на уровне 7 (например, пользовательская страница ошибок). Для устранения неполадок запустите шлюз/API с дополнительным журналированием и отладьте проблему в инсценированной среде (staging-среде)	Обработка ошибок на уровне 7 (например, пользовательская страница ошибки, обход отказа или полезная нагрузка). Для отладки проблем настройте более детальный мониторинг и включите теневое и/или канареечное копирование трафика для повторного воспроизведения проблемы	Обработка ошибок на уровне 7 (например, пользовательская страница ошибки или полезная нагрузка). Для поиска и устранения неисправностей настройте более детальный мониторинг и/или используйте «прослушивание» трафика для просмотра и отладки конкретного взаимодействия между сервисами
Тестирование	Эксплуатация нескольких сред для QA, инсценировки и эксплуатации. Автоматизированное интеграционное тестирование и автоматическое развертывание API. Используйте версию API, ориентированную на потребителя, для обеспечения совместимости и стабильности (например, semver)	Позволяет использовать канареечную маршрутизацию и запуск типа dark launching для динамического тестирования. Используйте контрактное тестирование для улучшения управления	Упрощает канареечную маршрутизацию для динамического тестирования
Локальная разработка	Развертывание шлюза локально (с помощью скрипта установки: Vagrant или Docker) и попытка устранить различия в инфраструктуре по сравнению с производством. Использование специфичных для конкретного языка фреймворков, имитации и создания заглушек шлюзов	Развертывание шлюза локально с помощью платформы оркестровки сервисов (например, контейнера или Kubernetes)	Развертывание сервисной сети локально с помощью платформы оркестровки сервисов (например, Kubernetes)
Пользовательский опыт	Веб-интерфейс администрирования, портал разработчика и каталог сервисов	IaC или CLI-управляемые решения с простым порталом разработчика и каталогом сервисов	IaC или CLI-управляемые решения с ограниченным каталогом сервисов

Практический пример: развитие конференц-системы с помощью API-шлюза

В этом разделе главы вы узнаете, как установить и настроить API-шлюз для направления трафика непосредственно к сервису Участник, извлеченному из монолитной конференц-системы. Здесь также будет показано, как можно использовать популярный паттерн «Душител»¹⁵, подробно описанный в одноименном разделе главы 8, для эволюции системы от монолита к архитектуре на основе микросервисов. Этот процесс растягивается во времени и выполняется путем постепенного выделения частей существующей системы в виде независимо развертываемых и запускаемых сервисов. На рис. 3.9 представлена схема архитектуры конференц-системы с добавлением API-шлюза.

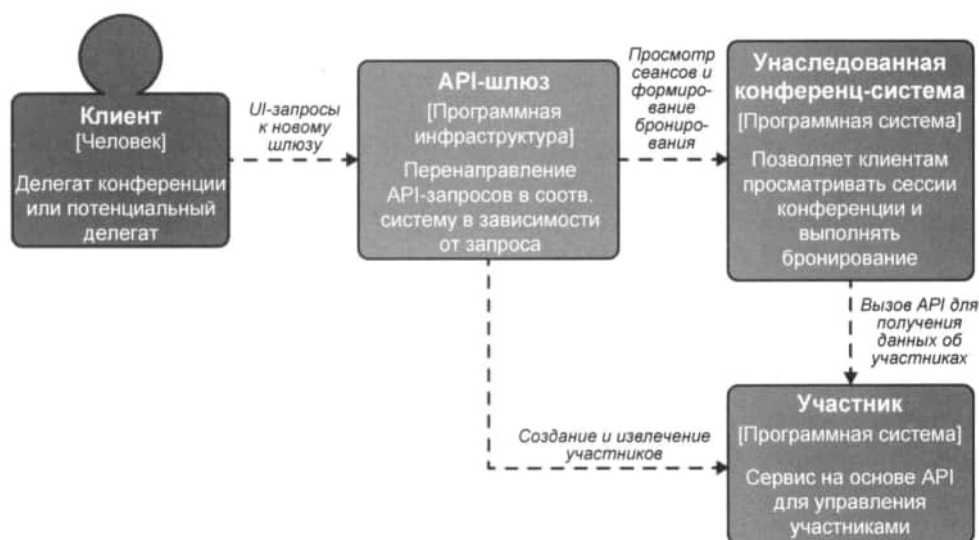


Рис. 3.9. Использование API-шлюза для маршрутизации к новому сервису Участник, работающему независимо от монолита

Многие организации часто начинают такую миграцию с извлечения сервисов, но при этом монолитное приложение выполняет маршрутизацию и другие сквозные задачи для внешних сервисов. Зачастую это самый простой выбор, поскольку монолит уже должен обеспечивать эту функциональность для внутренних функций. Однако это приводит к возникновению сильной связанности между монолитом и сервисами, при этом весь трафик проходит через монолитное приложение, а периодичность конфигурирования определяется частотой развертывания монолита. С точки зрения управления трафиком как повышенная нагрузка на монолитное приложение, так и увеличение «радиуса взрыва» в случае его отказа означают, что эксплуатационные расходы могут быть высокими. А ограничения в обновлении

¹⁵ Мартин Фаулер о приложении StranglerFigApplication (<https://oreil.ly/KU'xU3>).

информации о маршрутизации или межсетевой конфигурации, связанные с медленным выпуском релизов или неудачным развертыванием, могут помешать быстрому проведению итераций. В связи с этим мы не рекомендуем использовать монолит для маршрутизации трафика таким образом, особенно если вы планируете извлекать много сервисов в течение относительно короткого промежутка времени.

При условии, что шлюз развернут для обеспечения высокой доступности и разработчики имеют прямой доступ (самостоятельное обслуживание) для управления маршрутизацией и конфигурацией, извлечение и централизация маршрутизации приложений и сквозных проблем в API-шлюзе обеспечивают безопасность и скорость. Далее мы рассмотрим практический пример развертывания API-шлюза в конференц-системе и использования его для маршрутизации к новому сервису Участник.

Установка Ambassador Edge Stack в Kubernetes

При развертывании конференц-системы в кластере Kubernetes можно легко установить API-шлюз, используя стандартные нативные подходы Kubernetes, такие как применение YAML-конфигурации или использование Helm, а также утилиты командной строки. Например, API-шлюз Ambassador Edge Stack API¹⁶ можно установить с помощью Helm. После развертывания и настройки этого API-шлюза вы можете легко получить сертификат TLS от LetsEncrypt, следуя руководству *Host configuration tutorial*¹⁷.

Если API-шлюз работает и обеспечивает HTTPS-соединение, приложению конференц-системы больше не нужно заботиться о завершении TLS-соединений или о прослушивании нескольких портов. Аналогичным образом аутентификацию и ограничение количества запросов также можно легко настроить без необходимости изменения конфигурации или развертывания приложения.

Настройка сопоставления URL-путей с бэкенд-сервисами

Теперь можно использовать Ambassador Edge Stack Mapping Custom Resource¹⁸ для сопоставления корня вашего домена с сервисом `conferencessystem`, прослушивающим порт 8080 и работающим в пространстве имен `legacy` в кластере Kubernetes. Это сопоставление (Mapping) должно быть знакомо всем, кто настраивал веб-приложение или обратный прокси-сервер на прослушивание запросов пользователей. Метаданные задают имя Mapping, а префикс определяет путь (в нашем случае корень: `/`). Сопоставляемый с целевым сервисом (в формате `service-name.namespace:port`). Вот соответствующий пример:

```
---
```

```
apiVersion: getambassador.io/v3alpha1
kind: Mapping
```

¹⁶ См. <https://oreil.ly/4rakU>.

¹⁷ См. <https://oreil.ly/1tthW>.

¹⁸ См. <https://oreil.ly/1Be0g>.

```

metadata:
  name: legacy-conference
spec:
  hostname: "*"
  prefix: /
  rewrite: /
  service: conferencesystem.legacy:8080

```

Можно добавить еще один Mapping — для направления трафика, отправленного по пути `/attendees`, на новый (nextgen) микросервис участников, извлеченный из монолита. Информация, включаемая в Mapping, должна выглядеть знакомой по предыдущему примеру. Здесь задается параметр `rewrite` — для «перезаписи» соответствующего префикса (`prefix`) пути в метаданных URL перед обращением к целевому сервису Участник. В результате этому сервису будет казаться, что запрос пришел по пути `/`, а часть пути `/attendees` удалена:

```

---
apiVersion: getambassador.io/v3alpha1
kind: Mapping
metadata:
  name: legacy-conference
spec:
  hostname: "*"
  prefix: /attendees
  rewrite: /
  service: attendees.nextgen:8080

```

Этот паттерн создания дополнительных сопоставлений по мере извлечения каждого нового микросервиса из унаследованного приложения может быть продолжен. Соответствующие префиксы могут быть вложенными (например, `/attendees/affiliation`) или записанными с использованием регулярных выражений (например, `/attendees/^[a-z].*`). В итоге унаследованное приложение превращается в небольшую оболочку с горсткой функций, а вся остальная функциональность передается микросервисам, у каждого из которых будет отдельный файл сопоставления.

Настройка сопоставлений с использованием маршрутизации на основе хостов

Большинство API-шлюзов также позволяют выполнять маршрутизацию на основе хоста (например, `host: attendees.conferencesystem.com`). Это может быть полезно, если необходимо создать новый домен или поддомен для размещения новых сервисов. Вот пример использования сопоставления Ambassador Edge Stack Mappings:

```

---
apiVersion: getambassador.io/v3alpha1
kind: Mapping
metadata:
  name: attendees-host

```


spec:

```
hostname: "attendees.conferencesystem.com"
prefix: /
service: attendees.nextgen:8080
```

Многие современные API-шлюзы также поддерживают маршрутизацию на основе путей или строк запросов. Какими бы ни были ваши требования и какими бы ни были ограничения вашей текущей инфраструктуры, у вас должна быть возможность легко прокладывать маршруты как к существующим приложениям, так и к новым сервисам.



Избегайте маршрутизации по полезной нагрузке запроса

Некоторые API-шлюзы позволяют осуществлять маршрутизацию на основе полезной нагрузки или тела запроса, но этого, как правило, следует избегать по двум причинам. Во-первых, при этом в конфигурацию API-шлюза часто просачивается информация, специфичная для конкретного домена (например, полезная нагрузка часто соответствует схеме/контракту, который может измениться в приложении, и тогда шлюз должен быть синхронизирован с ним). Во-вторых, десериализация и синтаксический анализ большой полезной нагрузки с целью извлечения необходимой для маршрутизации информации могут быть вычислительно затратными (и трудоемкими).

Развертывание API-шлюзов: понимание и управление отказами

Независимо от схемы развертывания и количества шлюзов в системе, API-шлюз, как правило, находится на критическом пути многих, если не всех, пользовательских запросов, поступающих в систему. Выход из строя шлюза, установленного на периферии, как правило, приводит к недоступности всей системы. А выход из строя шлюза, расположенного выше по потоку обработки, как правило, приводит к недоступности какой-либо основной подсистемы. Поэтому темы понимания и управления отказами API-шлюза жизненно важны для изучения.

API-шлюз как единая точка отказа

В стандартной веб-системе первой очевидной единой точкой отказа, как правило, становится DNS. Несмотря на то что эта система часто управляется извне, избежать того, что в случае сбоя DNS ваш сайт будет недоступен, невозможно. Следующими единичными точками отказа, как правило, становятся глобальные и региональные балансировщики нагрузки на уровне 4, а также, в зависимости от места развертывания и конфигурации, пограничные компоненты безопасности, такие как межсетевой экран или WAF.

После этих основных периферийных компонентов следующим уровнем, как правило, становится API-шлюз. Чем больше функциональность шлюза, тем выше риск и тем сильнее последствия сбоя. Поскольку API-шлюз часто участвует в релизе программного обеспечения, его конфигурация также постоянно обновляется. Очень важно уметь обнаруживать и устранять проблемы, а также минимизировать риски.



Проверка предположений о единых точках отказа в системе безопасности

В зависимости от продукта, его развертывания и конфигурации некоторые компоненты безопасности могут находиться в состоянии «открытого отказа» — т. е. при отказе компонента трафик будет просто передаваться вышестоящим компонентам или бэкэнду. Для некоторых сценариев, где доступность является наиболее важной целью, это желательно, но для других (например, для финансовых или государственных систем), скорее всего, нет. Обязательно проверьте предположения, содержащиеся в текущей конфигурации системы безопасности.

Обнаружение и принадлежность проблем

На первом этапе обнаружения проблем следует обеспечить сбор и доступ к соответствующим сигналам системы мониторинга — т. е. к данным метрик, журналов и трассировок. У каждой критически важной системы должна быть своя четко определенная команда владельцев, несущая ответственность за любые возникающие проблемы. Команды должны поддерживать цели уровня обслуживания (Service-Level Objective, SLO), которые и могут быть закреплены в соглашениях об уровне обслуживания (Service-Level Agreement, SLA) как для внутренних, так и для внешних заказчиков.



Дополнительные материалы: наблюдаемость, оповещение и SRE

Если вы впервые знакомитесь с концепцией наблюдаемости, то мы рекомендуем вам ознакомиться с методом Брендана Грегга «Утилизация, насыщение и ошибки» (Utilization, Saturation and Errors, USE)¹⁹, методом Тома Уилки «Скорость, ошибки и продолжительность» (Rate, Errors and Duration, RED)²⁰, а также с четырьмя золотыми сигналами мониторинга (Four golden signals of monitoring)²¹ компании Google. Если вы хотите узнать больше об ассоциированных организационных целях и процессах, то настоятельно рекомендуем прочесть книгу «Site Reliability Engineering (SRE). Надежность и безотказность как в Google»²².

Разрешение инцидентов и проблем

Прежде всего, у каждого работающего в вашей системе API-шлюза должен быть владелец, несущий ответственность, если с компонентом что-то пойдет не так. В небольших организациях это могут быть разработчики или команда SRE, также отвечающие за базовые сервисы. В более крупных организациях для этого выделяется специальная инфраструктурная команда. Поскольку API-шлюз находится на критическом пути запросов, часть команды, владеющей им, должна быть на связи при возникновении необходимости в режиме 24/7/365. В этом случае перед оперативной командой будет стоять непростая задача: как можно быстрее устранить

¹⁹ См. <https://oreil.ly/SgSSQ>.

²⁰ См. <https://oreil.ly/yFnDt>.

²¹ См. <https://oreil.ly/tESfc>.

²² См. <https://oreil.ly/rMBW3>.

проблему, но при этом собрать достаточно информации (или найти и поместить в карантин системы и конфигурации), чтобы узнать, что именно пошло не так.

После любого инцидента организация должна стремиться к проведению детального анализа, документированию и распространению всех полученных знаний. Эта информация не только может быть использована для предотвращения повторного возникновения проблемы, но и будет полезной как для инженеров, изучающих систему, так и для внешних команд, работающих с аналогичными технологиями или проблемами²³.

Снижение рисков

Любой компонент, находящийся на критическом пути обработки пользовательских запросов, должен быть настолько высокодоступным, насколько это целесообразно с точки зрения стоимости и сложности эксплуатации. Архитекторы программного обеспечения и технические руководители имеют дело с компромиссами, и этот вид компромиссов один из самых сложных. В мире API-шлюзов высокая доступность обычно начинается с запуска нескольких экземпляров. В случае размещения экземпляров внутри предприятия или в дата-центре это означает эксплуатацию нескольких аппаратных устройств (с резервированием), в идеале расположенных в разных местах. При размещении в облаке это означает разработку и эксплуатацию экземпляра API-шлюза в нескольких зонах доступности (центрах обработки данных) и регионах. Если перед экземплярами API-шлюза развернут глобальный балансировщик нагрузки, то он должен быть настроен соответствующим образом с проверкой работоспособности и процессами обхода отказа, проходящими регулярное тестирование. Это особенно важно, если экземпляры API-шлюза работают в режимах активный/пассивный или лидер/узел.

Необходимо убедиться, что процесс обхода отказа балансировщика нагрузки и API-шлюза отвечает всем вашим требованиям в отношении непрерывности обслуживания. К числу распространенных проблем, возникающих при обходе отказа, относятся:

- ♦ проблемы управления состоянием клиента пользователя — например, некорректная миграция состояния бэкенда, что приводит к отказу сессий типа *Sticky Sessions*;
- ♦ низкая производительность, т. к. клиенты не перенаправляются по географическому принципу (например, европейские пользователи перенаправляются на западное побережье США при наличии центра обработки данных на восточном побережье);
- ♦ непреднамеренный каскадный отказ — например, сбой компонента выборов лидера, приводящий к тупиковой ситуации, в результате которой становятся недоступными все бэкенд-системы.

²³ Если вы новичок в этой области, то сайт *Learning from Incidents* (<https://oreil.ly/4aFqy>) станет для вас отличной отправной точкой.

Общие ошибки при внедрении API-шлюзов

Вы уже убедились, что никакая технология не является панацеей, но, продолжая тему технологических клише, можно утверждать, что когда у вас есть технологический молоток, все имеет тенденцию выглядеть как гвоздь. Такая ситуация может возникнуть при использовании «молотка» в виде API-шлюза, и существует несколько распространенных «подводных камней» при применении API-шлюза, которых всегда следует избегать.

Loopback API-шлюза

Как и все типичные «подводные камни», реализация этого паттерна часто начинается с благих намерений. Наличие в организации всего нескольких сервисов, как правило, не считается основанием для установки сервисной сети. Однако иногда возникает потребность в подмножестве функций сервисной сети и, в частности, в обнаружении сервисов. Простой вариант реализации такой функции заключается в том, чтобы направлять весь трафик через периферийный или API-шлюз, хранящий официальный каталог расположения всех сервисов. На этом этапе схема несколько напоминает сетевую диаграмму «ступица и спица». Проблемы проявляются в двух формах. Во-первых, когда весь трафик между сервисами уходит из сети, а затем возвращается через шлюз, это может вызвать проблемы с производительностью, безопасностью и стоимостью (поставщики облачных решений часто взимают плату за трафик на извлечение данных и между зонами доступности). Во-вторых, эта схема не масштабируется больше нескольких сервисов, поскольку шлюз из-за перегрузки становится узким местом, превращаясь в настоящую единую точку отказа. Этот паттерн также способен добавить сложности в наблюдаемость, т. к. многократные циклы могут затруднить понимание того, что произошло при каждом вызове.

Если посмотреть на текущее состояние конференц-системы с двумя настроенными файлами сопоставления, то можно увидеть корни возникновения этой проблемы. Любой внешний трафик (например, пользовательские запросы) корректно направляется API-шлюзом к целевым сервисам. Однако, как унаследованное приложение узнает о местонахождении сервиса Участник? Часто решение заключается в направлении всех запросов обратно через публично адресуемый шлюз (например, унаследованное приложение обращается к www.conferencesystems.com/attendees). Вместо этого унаследованное приложение должно использовать тот или иной механизм обнаружения внутренних сервисов и держать все внутренние запросы в пределах внутренней сети. Подробнее о том, как использовать сервисную сеть для реализации этой задачи, вы узнаете в следующей главе.

Шлюз API в качестве корпоративной сервисной шины (ESB, Enterprise Service Bus)

Подавляющее большинство API-шлюзов поддерживают расширение своей базовой функциональности путем создания плагинов или модулей. Веб-сервер NGINX под-

держивал модули Lua, чем и воспользовались OpenResty и Kong. Прокси Envoy Proxy первоначально поддерживал расширения на языке C, а теперь и фильтры WebAssembly. А о том, как оригинальная реализация API-шлюза Zuul компании Netflix поддерживала расширение с помощью скриптов Groovy, мы уже рассказывали в разд. «2015 год и далее: API-шлюзы второго поколения». Многие из реализованных в этих плагинах сценариев использования чрезвычайно полезны, например `authn/z`, фильтрация и протоколирование. Однако может возникнуть соблазн поместить в эти плагины бизнес-логику, что поспособствует возникновению сильной связанности шлюза с сервисом или приложением. А это приводит к созданию потенциально хрупкой системы. В ней изменение в одном подключаемом модуле вызывает волнения во всей организации или создает дополнительные трения при релизе, когда целевой сервис и подключаемый модуль должны быть развернуты синхронно.

Черепahi (API-шлюзы) — и нет им конца²⁴

Если один API-шлюз — это уже хорошо, то когда их несколько, должно быть еще лучше, не так ли? Обычно в рамках крупной организации развертывается несколько API-шлюзов, часто в иерархическом порядке или в попытке сегментировать сети или отделы. Намерения, как правило, благие: либо обеспечение инкапсуляции для внутренних направлений деятельности, либо разделение ответственности с каждым шлюзом (например, «это шлюз транспортной безопасности, это шлюз аутентификации, это шлюз протоколирования...»). Распространенная проблема возникает, когда стоимость изменений слишком высока. Например, для релиза простого обновления сервиса приходится координировать работу с большим количеством команд шлюзов, возникают проблемы с пониманием («кому принадлежит функциональность трассировки?») или страдает производительность, поскольку каждый сетевой переход, естественно, связан с определенными затратами.

Выбор API-шлюза

Теперь, когда вы узнали о функциональности, предоставляемой API-шлюзом, об истории этой технологии и о том, как API-шлюз вписывается в общую архитектуру системы, встает вопрос на миллион: как выбрать API-шлюз для включения в свой стек?

Определение требований

Одним из первых шагов при реализации любого нового проекта по созданию программного обеспечения или инфраструктуры должно стать определение соответствующих требований. Это может показаться очевидным, но слишком легко отвлечь-

²⁴ Отсылка к книге «Черепahi — и нет им конца» Джона Грина. — Прим. пер.

ся на блестящие технологии, волшебный маркетинг или хорошую документацию по продажам!

Вы можете обратиться к *разд. «Зачем нужен API-шлюз?»*, чтобы более подробно разобраться с основными требованиями, которые следует учитывать в процессе выбора. Важно задавать вопросы, ориентированные как на текущие болевые точки, так и на вашу будущую дорожную карту.

Создание или покупка?

При выборе API-шлюза часто обсуждается дилемма: создать или покупать. Это не уникальный вопрос для этого компонента программной системы, но функциональность, которую предполагается реализовать с помощью API-шлюза, вызывает у некоторых инженеров мысли о том, что они могут сделать его «лучше», чем предлагают существующие поставщики, или что их организация какая-то «особенная» и выиграет от индивидуальной реализации. В целом мы считаем, что компонент API-шлюза достаточно хорошо отработан, поэтому, как правило, лучше использовать реализацию с открытым исходным кодом или коммерческое решение, а не создавать шлюз с чистого листа. Представление аргументов в пользу создания или покупки ПО с использованием технологии его доставки может занять целую книгу, поэтому в этом разделе мы остановимся лишь на некоторых общих проблемах:

- ◆ недооценка совокупной стоимости владения (Total Cost of Ownership, TCO).

Многие инженеры не учитывают стоимость разработки решения, затраты на его дальнейшее обслуживание и текущие эксплуатационные расходы.

- ◆ непродуманность альтернативных издержек.

Если вы не поставщик облачных технологий или платформ, то маловероятно, что индивидуальный API-шлюз обеспечит вам конкурентное преимущество. Вы можете принести больше пользы своим клиентам, создав некоторые функциональные возможности в пределах своего общего ценового предложения.

- ◆ незнание современных технических решений по продуктам.

Как в области компонентов с открытым исходным кодом, так и в области коммерческих платформ, происходит быстрое развитие, и следить за актуальностью бывает непросто. Это, однако, одна из основных составляющих роли технического лидера.

Руководство ADR: выбор API-шлюза

В табл. 3.4 приведены основные рекомендации руководства ADR по этой теме. Их можно использовать для принятия решения о том, какой API-шлюз следует внедрить в вашей организации или проекте.

Таблица 3.4. Руководство ADR: чек-лист по выбору API-шлюза

Решение	Как подойти к выбору API-шлюза для своей организации?
Пункты обсуждения	Разобрались ли мы со всеми требованиями, связанными с выбором API-шлюза, и расставили ли приоритеты? Определили ли мы текущие технологические решения, развернутые в этой области в организации? Знаем ли мы все ограничения, касающиеся команд и организации? Изучили ли мы нашу будущую дорожную карту в связи с этим решением? Честно ли мы подсчитали затраты для вариантов создания или покупки? Изучили ли мы текущий технологический ландшафт и знаем ли обо всех доступных решениях? Провели ли мы консультации со всеми заинтересованными сторонами и проинформировали ли их при анализе и принятии решений?
Рекомендации	Особое внимание уделяется требованиям к снижению связанности между API и системой, упрощению потребления, защите API от чрезмерного использования и злоупотреблений, пониманию того, как API используются, управлению API как продуктами и монетизации API. Необходимо задать следующие вопросы. Существует ли уже используемый API-шлюз? Был ли собран набор технологий для обеспечения схожей функциональности (например, аппаратный балансировщик нагрузки в сочетании с монолитным приложением, выполняющим аутентификацию и маршрутизацию на уровне приложений)? Сколько компонентов в настоящее время составляет периферийный стек (например, WAF, LB, периферийный кеш и т. д.)? Ориентируйтесь на уровень технологической подготовки команды, наличие людей для работы над проектом API-шлюза, доступные ресурсы, бюджет и т. п. Важно определить все планируемые изменения, новые возможности и текущие цели, способные повлиять на управление трафиком, и другие функциональные возможности, предоставляемые API-шлюзом. Рассчитайте совокупную стоимость владения (Total Cost of Ownership, TCO) для всех существующих реализаций API-шлюзов и потенциальных будущих решений. Ознакомьтесь с известными аналитическими материалами, отчетами о тенденциях и обзорами продуктов, чтобы понять все имеющиеся на сегодняшний день решения. Выбор и развертывание API-шлюза затронет многие команды и отдельных сотрудников. Обязательно проконсультируйтесь с разработчиками, QA²⁵, архитектурным советом, командой платформы, InfoSec²⁶ и пр.

Заключение

В этой главе вы разобрались с тем, что такое API-шлюз, а также рассмотрели исторический контекст, приведший к эволюции возможностей, предоставляемых в настоящее время этим важнейшим компонентом любого стека веб-программ:

- ♦ вы узнали, что API-шлюз — очень полезный инструмент для миграции и развития систем, и получили практические навыки использования API-шлюза для

²⁵ QA (Quality Assurance, обеспечение качества). Специалисты QA тестируют программное обеспечение и помогают снизить количество ошибок на этапе разработки. — Прим. пер.

²⁶ InfoSec — распространенное сокращение для понятия «информационная безопасность». Это набор процедур и инструментов, которые обеспечивают всестороннюю защиту конфиденциальной корпоративной информации от неправильного использования, несанкционированного доступа, искажения или уничтожения. — Прим. пер.

маршрутизации к извлеченному из сценария использования конференц-системы сервису Участник;

- ◆ изучили существующую таксономию API-шлюзов и модели их развертывания, что дало вам повод задуматься о том, как управлять потенциальными едиными точками отказа в архитектуре, где весь пользовательский трафик направляется через периферийный шлюз;
- ◆ познакоившись с концепцией управления трафиком на границе систем (входящим), вы узнали о межсервисном взаимодействии и о том, как избежать распространенных ошибок, — таких как развертывание API-шлюза в качестве менее функциональной сервисной шины предприятия (Enterprise Service Bus, ESB);
- ◆ сочетание всех этих знаний позволит вам понять ключевые моменты, ограничения и требования, необходимые для эффективного выбора API-шлюза для ваших текущих задач;
- ◆ как и в случае с большинством решений, принимаемых архитектором программного обеспечения или техническим руководителем, на все поднятые в этой главе вопросы нет однозначного правильного ответа, но часто можно найти достаточно много плохих решений, и их следует избегать.

Теперь, когда вы изучили функциональность API-шлюзов для управления входящим трафиком север-юг и соответствующие API, в следующей главе мы рассмотрим роль сервисных сетей для управления трафиком восток-запад — от сервиса к сервису.

Сервисные сети: управление трафиком между сервисами

В предыдущей главе вы познакомились с тем, как с помощью API-шлюза надежно, наблюдаемо и безопасно предоставлять доступ к API-интерфейсам и управлять связанным с ними входящим трафиком от конечных пользователей и других внешних систем. Здесь вы узнаете об управлении трафиком для внутренних API — т. е. об обмене данными между сервисами с аналогичными целями.

На фундаментальном уровне реализации сервисных сетей обеспечивают функциональность для маршрутизации, наблюдения и защиты трафика при межсервисном взаимодействии. Стоит отметить, что даже такой выбор технологии не будет однозначным — как и во всех архитектурных решениях, здесь есть свои компромиссы. Но для архитектора бесплатный сыр бывает только в мышеловке!

В этой главе мы продолжим рассмотрение нашего тематического примера, извлекая функциональность обработки сессий из унаследованной конференц-системы в новый внутренний сервис под названием Сессия (Session). В процессе работы вы узнаете о проблемах взаимодействия, возникающих при создании или извлечении новых сервисов и API, которые развертываются и работают параллельно с существующей монолитной конференц-системой. Все приемы работы с API и управления трафиком, рассмотренные в предыдущей главе, будут применимы и здесь, поэтому естественным желанием может стать использование API-шлюза для предоставления доступа к новому сервису Сессия. Однако, учитывая имеющиеся требования, это, скорее всего, приведет к возникновению неоптимального решения. Именно здесь альтернативным подходом может стать паттерн сервисной сети и связанные с ним технологии.

Сервисная сеть — это единственное решение?

Практически каждое веб-приложение должно выполнять вызовы от сервиса к сервису, даже если это просто монолитное приложение, взаимодействующее с базой данных.

В связи с этим уже давно существуют решения для управления этим видом связи.

Наиболее распространенным подходом является использование библиотеки для конкретного языка — например, библиотеки набора инструментов разработки программного обеспечения (Software Development Kit, SDK) или драйвера базы данных. Эти библиотеки отображают вызовы приложений для обслуживания API-запросов, а также управляют соответствующим трафиком, как правило, с помощью

протоколов HTTP или TCP/IP. С внедрением в проектирование современных приложений сервис-ориентированных архитектур расширилось проблемное пространство межсервисных вызовов. Очень часто бывает, что для удовлетворения запроса пользователя сервис должен вызвать API другого сервиса. Помимо обеспечения механизма маршрутизации трафика, обычно требуется также соблюдать требования надежности, наблюдаемости и безопасности.

Как вы узнаете из этой главы, решения на основе библиотек и сервисных сетей часто могут удовлетворить ваши требования к межсервисной связи. Мы наблюдаем быстрое внедрение сервисных сетей, особенно в корпоративном контексте, и по мере увеличения числа потребителей и провайдеров они часто оказываются наиболее масштабируемым, обслуживаемым и безопасным вариантом. Соответственно, в этой главе мы сосредоточимся в основном на паттерне сервисной сети.

Рекомендация: стоит ли внедрять технологию сервисной сети?

В табл. 4.1 приведен ряд рекомендаций ADR, способных помочь вам принять решение о целесообразности внедрения технологии сервисной сети в вашей организации.

Таблица 4.1. Руководство ADR: руководство по сервисной сети или по библиотекам

Решение	Следует ли использовать сервисную сеть или библиотеку для маршрутизации трафика сервисов?
Пункты обсуждения	Используется ли в организации один язык программирования? Требуется ли вам только простая маршрутизация между сервисами для REST-или RPC-подобного взаимодействия? Есть ли у вас кросс-функциональные требования, предполагающие наличие более сложных функций, таких как аутентификация, авторизация или ограничение количества запросов? Есть ли у вас уже готовое решение, или же в масштабах организации существует требование того, что весь трафик должен проходить через определенные компоненты сети?
Рекомендации	Если ваша организация требует использования одного языка программирования или фреймворка, то, как правило, вы можете воспользоваться библиотеками или характерными для этого языка механизмами взаимодействия между сервисами. Всегда используйте самое простое решение, соответствующее вашим требованиям, с учетом ближайшего будущего и известных условий. Если у вас есть расширенные кросс-функциональные требования, особенно к сервисам, использующим разные языки программирования или технологические стеки, лучшим выбором может стать сервисная сеть. Всегда проводите в своей организации комплексную проверку существующих мандатов, решений и компонентов

Практический пример: извлечение функциональности сессий в сервис

На этом этапе развития нашего примера конференц-системы вы сосредоточитесь на запросах владельцев конференций о поддержке новой ключевой функции: про-

смотр и управление сеансами участников конференций с помощью мобильного приложения.

Это серьезное изменение, требующее создания ADR. В табл. 4.2 приведен пример ADR, который мог быть предложен инженерной группой, владеющей конференц-системой.

Таблица 4.2. ADR501 Выделение сессий из унаследованной конференц-системы

Статус	Предложено
Контекст	Владельцы конференц-системы попросили внести еще одну новую функцию в существующую систему. Маркетинговая команда считает, что вовлеченность участников конференции повысится, если они смогут просматривать подробную информацию о сессиях конференции и обозначать свою заинтересованность в них через мобильное приложение. Маркетинговая команда также хочет иметь возможность видеть, сколько слушателей заинтересованы в каждой сессии
Решение	Мы сделаем эволюционный шаг к выделению компонента Сессия в отдельный сервис. Это позволит выполнять разработку с использованием подхода API-First для сервиса Сессия и вызывать API из унаследованного сервиса конференций. Это также позволит сервису Участник напрямую вызывать API сервиса Сессия для предоставления информации о сессии мобильному приложению
Последствия	Унаследованное приложение будет вызывать новый сервис Сессия при обработке всех запросов, связанных с сессиями, как для существующей, так и для новой функциональности. Когда пользователь захочет просмотреть, добавить или удалить интересующие его сессии на конференции, сервис Участник должен вызвать сервис Сессия. Когда администратор конференции захочет увидеть, кто участвует в каждой сессии, сервис Сессия должен будет вызвать сервис Участник, чтобы это определить. Сервис сессий может стать единой точкой отказа в архитектуре, и нам, возможно, потребуются принять меры по снижению потенциальных последствий работы одного экземпляра сервиса Сессия. Поскольку во время проведения конференции резко возрастает объем просмотра и управления сессиями участниками, нам также необходимо будет учитывать большие скачки трафика и потенциальную перегрузку или ухудшение работы одного или нескольких экземпляров сервиса Сессия

Модель C4, демонстрирующая предлагаемое изменение архитектуры, приведена на рис. 4.1.

Обратите внимание, что, несмотря на то, что новый сервис Сессия не обязательно должен быть доступен извне, вы можете легко выполнить требования к маршрутизации и надежности, указанные в приведенном ADR, предоставив доступ к этому сервису через API-шлюз и настроив как унаследованную систему, так и сервис Участник на вызов этого нового сервиса через внешний адрес шлюза. Однако такой подход будет примером антипаттерна «Loopback API-шлюза» (о нем шла речь в разд. «Общие ошибки при внедрении API-шлюзов» главы 3) и может привести к тому, что трафик, предназначенный для внутренних нужд, покинет вашу сеть, что скажется на производительности, безопасности и стоимости (для поставщика облачных вычислений). Теперь давайте рассмотрим, как сервисная сеть может помочь вам удовлетворить новые требования, избежав при этом возникновения этого антипаттерна.



Рис. 4.1. Модель C4, показывающая извлечение сервиса Сессия из конференц-системы

Что такое сервисная сеть?

По сути, *сервисная сеть* — это модель управления всеми связями от сервиса к сервису (или от приложения к приложению) в распределенной программной системе. Модели сервисных сетей и API-шлюзов во многом пересекаются, но основные различия между ними состоят в двух моментах. Во-первых, сервисные сети оптимизированы для обработки трафика между сервисами (трафика типа восток-запад) внутри кластера или центра обработки данных. Во-вторых, исходя из этого, в качестве инициатора коммуникации обычно выступает (в некоторой степени) известный внутренний сервис, а не устройство пользователя или система, работающая вне ваших приложений.



Сервисная сеть — это не ячеистая сеть

Сервисную сеть не следует путать с *ячеистой сетью*¹, представляющей собой сетевую топологию более низкого уровня. Ячеистая топология получает все большее распространение в контексте Интернета вещей (Internet of Things, IoT), а также применяется для реализации инфраструктуры мобильной связи в удаленных или сложных сценариях (например, при ликвидации последствий стихийных бедствий). Реализации сервисной сети строятся на основе существующих сетевых протоколов и топологий.

Паттерн сервисных сетей ориентирован на обеспечение управления трафиком (маршрутизации), отказоустойчивости, наблюдаемости и безопасности при взаимо-

¹ См. <https://oreil.ly/BbWJm>.

действию между сервисами. Не беспокойтесь, если вы ничего не слышали о сервисных сетях, поскольку только в 2016 году команда Buoyant ввела такой термин для объяснения функциональности своей технологии Linkerd². Это, в сочетании с появлением других родственных технологий (типа Istio, поддерживаемой Google), привело к быстрому распространению термина «сервисная сеть» в областях облачных вычислений, DevOps и архитектуры.

Подобно API-шлюзу, сервисная сеть реализуется с помощью двух фундаментальных компонентов высокого уровня: *плоскости управления* и *плоскости данных*. В сервисной сети эти компоненты всегда развертываются отдельно. В плоскости управления операторы взаимодействуют с сервисной сетью и определяют маршруты, политики и необходимую телеметрию. Плоскость данных — это место, где происходит вся работа, указанная в плоскости управления: маршрутизируются сетевые пакеты, реализуются политики и передается телеметрия.

Если взять в качестве примера конфигурирование трафика между сервисами в кластере Kubernetes, то человек-оператор сначала определит маршрутизацию и политику с помощью инструмента Custom Resource configuration. Например, в нашем примере укажет, что сервис Участник может вызывать сервис Сессия, а затем «применит» это действие к кластеру с помощью инструмента командной строки (например, `kubectl`) или конвейера непрерывной доставки. В качестве плоскости управления здесь выступает работающее в кластере Kubernetes приложение-контроллер сервисной сети. Оно анализирует эту конфигурацию и дает указания плоскости данных — обычно это серия *sidecar*-прокси, работающих рядом с каждым из сервисов Участник и Сессия.

Сервисные сети *sidecar* и прокси

В контексте сервисной сети часто можно встретить взаимозаменяемые термины «*sidecar*» и «прокси» (*proxy*). Однако с технической точки зрения это неверно, поскольку *sidecar* — это паттерн общего назначения, обычно реализуемый с помощью прокси внутри сервисной сети. Поэтому любое использование термина «*sidecar*» должно включать и постфикс «прокси» (например, «*sidecar*-прокси»). Паттерн *sidecar* заимствован из принципа использования мотоциклетной коляски и заключается в разделении функциональных возможностей приложения или сервиса на ряд отдельных процессов, выполняемых в рамках одной сети и одного пространства имен процессов. В архитектуре программного обеспечения побочное приложение присоединяется к родительскому приложению и расширяет/дополняет его функциональные возможности в режиме слабой связанности. Такие паттерны позволяют добавить в приложение ряд возможностей без использования библиотек конкретного языка или других методов. Более подробно о развитии этого паттерна в рамках реализации сервисной сети вы узнаете из разд. «Эволюция сервисных сетей».

Весь трафик между сервисами в кластере Kubernetes направляется через *sidecar*-прокси, как правило, прозрачно (без распознавания базовыми приложениями наличия прокси), что позволяет маршрутизировать, наблюдать и защищать этот трафик

² Проект Linkerd возник на базе технологии Finagle компании Twitter. Эта технология была создана для обеспечения коммуникационной структуры для разработчиков, создающих распределенные приложения Twitter. В настоящее время Linkerd превратился в законченный проект фонда Cloud Native Computing Foundation (CNCF).

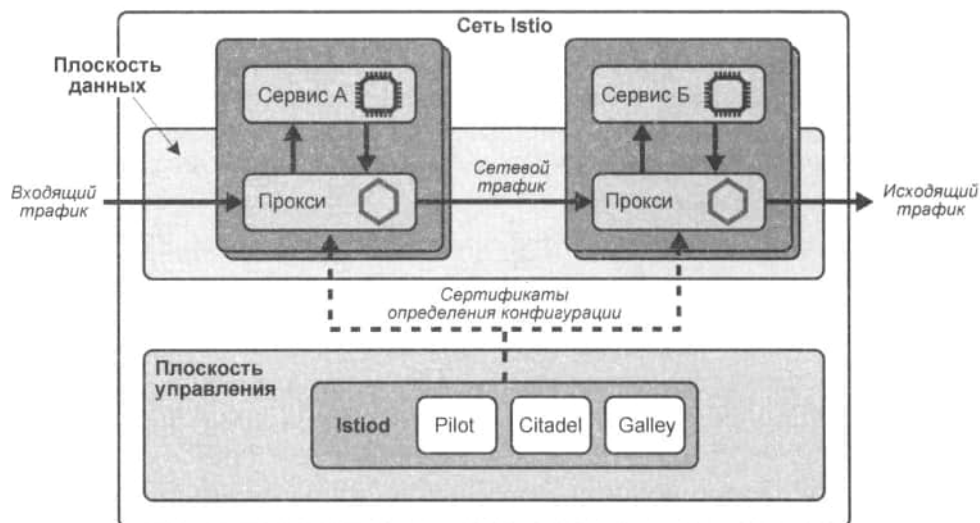


Рис. 4.2. Топология сервисов, плоскости управления и плоскости данных сервисной сети (на примере Istio)

в соответствии с требованиями. Пример топологии плоскости управления и плоскости данных сервисной сети показан на рис. 4.2.

Какие функциональные возможности предоставляет сервисная сеть?

На сетевом уровне прокси-сервер сервисной сети действует как полноценный прокси-сервер, принимая весь входящий трафик от других сервисов, а также инициируя все исходящие запросы к другим сервисам. Сюда входят все вызовы API, а также другие запросы и ответы. В отличие от API-шлюза, сопоставление плоскости данных сервисной сети с сервисом обычно взаимно однозначно. Это означает, что прокси-сервер сервисной сети не агрегирует вызовы нескольких сервисов. Сервисная сеть обеспечивает сквозную функциональность, такую как верификация пользователей, ограничение количества запросов, тайм-ауты/откаты, а также может предоставлять метрики, журналы и данные трассировки для поддержки реализации наблюдаемости в системе. Это необходимая нам для развития примера функциональность — мы извлекаем сервис Сессия и вызываем его как из унаследованной конференц-системы, так и из сервиса Участник.

Сервисные сети используют полные прокси-серверы для перехвата всего трафика сервисов

Как правило, все прокси-серверы сервисной сети работают как «полные прокси», поскольку им необходимо наблюдать и управлять всем трафиком, проходящим через сеть. В отличие от половинного прокси, полный прокси обрабатывает весь обмен данными между клиентом и сервером. Принципиальное различие заключается в том, что полный прокси-сервер поддерживает два разных сетевых стека: один на стороне клиента и один на стороне сервера — и полностью проксирует обе стороны. Благодаря тому, что прокси находится в центре всех

коммуникаций, он может манипулировать, отбрасывать, наблюдать и делать все необходимое с трафиком с обеих сторон и в обоих направлениях. Такая мощь и гибкость, конечно, стала компромиссом, поскольку полноценный прокси требует больше ресурсов и потенциально вносит больше накладных расходов/задержек в обмен данными.

Хотя по сравнению с API-шлюзом сервисные сети встречаются реже, некоторые из них предоставляют дополнительные возможности, позволяющие разработчикам управлять жизненным циклом API. Например, привязанный каталог сервисов может помочь в регистрации и управлении разработчикам, использующим сервисные API, а портал разработчика обеспечит администрирование учетных записей и контроль доступа. Некоторые сервисные сети также поддерживают аудит политик и менеджмент трафика для удовлетворения требований управления предприятием.

Где разворачивается сервисная сеть?

Сервисная сеть разворачивается во внутренней сети или кластере. Управление крупными системами или сетями обычно осуществляется путем разворачивания нескольких экземпляров сервисных сетей. Часто каждая из них охватывает сегмент сети или бизнес-домен. Пример сетевой топологии сервисной сети показан на рис. 4.3.

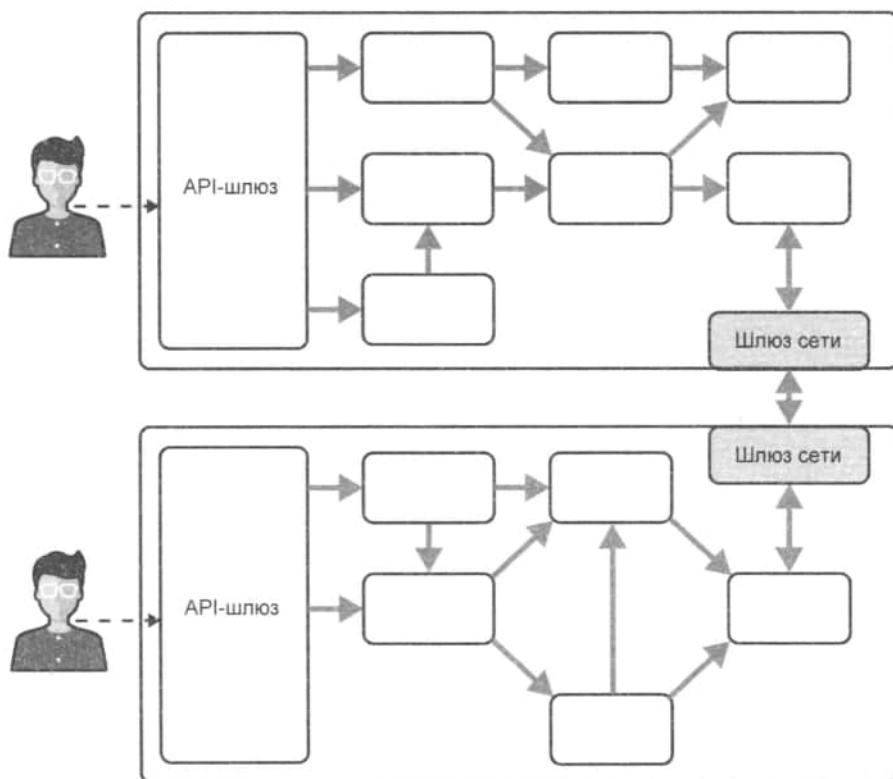


Рис. 4.3. Типичная топология сервисной сети, развернутая на двух кластерах (сплошными стрелками показан трафик сервисной сети)

Развертывается ли сервисная сеть на периферии?

Несмотря на то что сервисная сеть развернута в рамках кластера, она может открывать конечные точки в демилитаризованной зоне сети (ДМЗ) либо внешним системам, либо дополнительным сетям или кластерам. Для этого часто используется прокси-сервер, получивший название «шлюза сети», «оконечного шлюза» или «транзитного шлюза». Внешние шлюзы такого типа, как правило, не обеспечивают обычно присущего внешним API-шлюзам уровня функциональности. Существуют определенные споры по поводу того, к какому типу относится управление трафиком с помощью этих шлюзов сервисной сети: север-юг или восток-запад. И это может повлиять на требования, необходимые политики безопасности и т. д.

Как сервисная сеть интегрируется с другими сетевыми технологиями?

В современном сетевом стеке может быть много уровней, особенно при работе с облачными технологиями, где виртуализация и песочница существуют на нескольких уровнях. Сервисная сеть должна работать в гармонии с другими сетевыми уровнями, но разработчики и операторы также должны быть осведомлены о возможных взаимодействиях и конфликтах. На рис. 4.4 показано взаимодействие физической (и виртуализованной) сетевой инфраструктуры, типичного сетевого стека и сервисной сети.

Например, при развертывании приложений в кластере Kubernetes сервис может находить и обращаться к другому сервису в том же кластере через предписанное имя.

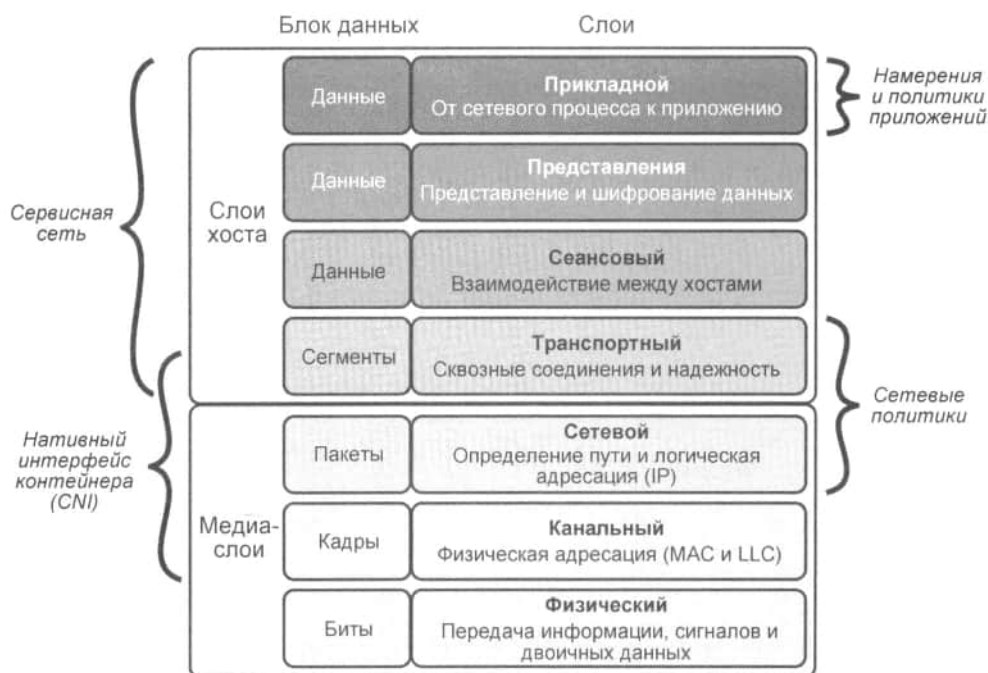


Рис. 4.4. Модель OSI показывает, что сервисная сеть работает между уровнями 3 и 7

сопоставленное с IP-адресом. Основные политики безопасности управления трафиком могут быть реализованы с помощью сетевых политик *NetworkPolicies*, контролирующих трафик на уровне IP-адресов и портов (уровень OSI 3 или 4), а дополнительные средства контроля политик часто предоставляются плагином *Container Networking Interface (CNI)* кластера³.

Сервисные сети могут переопределять стандартное разрешение и маршрутизацию CNI между сервисами и IP-адресами, а также предоставлять дополнительную функциональность. Это включает в себя прозрачную маршрутизацию между кластерами, обеспечение безопасности уровней 3/4 и 7 (например, идентификацию и авторизацию пользователей), балансировку нагрузки уровня 7 (что полезно при использовании мультиплексированного протокола *keepalive* — например, *gRPC* или *HTTP/2*), а также наблюдаемость на уровне сервисов и всего сетевого стека.

Зачем нужна сервисная сеть?

По аналогии с решением вопроса о целесообразности внедрения API-шлюза в существующую архитектуру, определение причин внедрения в нее сервисной сети представляет собой многогранную тему. Необходимо найти баланс между краткосрочными выгодами и затратами на внедрение и долгосрочными требованиями к удобству сопровождения. Существует множество сквозных проблем, связанных с API. Они могут быть характерны для каждого или всех ваших внутренних сервисов, включая управление жизненным циклом продукта (последовательные релизы новых версий сервиса), надежность, поддержку многоязычных коммуникаций, наблюдаемость, безопасность, удобство сопровождения и расширяемость. Сервисная сеть может помочь в решении всех этих вопросов.

В этом разделе главы приведен обзор основных проблем, решаемых при помощи сервисной сети, таких как:

- ♦ обеспечение тонкого контроля маршрутизации сервисов, надежности и управления трафиком;
- ♦ улучшение наблюдаемости межсервисных вызовов;
- ♦ обеспечение безопасности, включая шифрование на транспортном уровне, аутентификацию и авторизацию;
- ♦ поддержка требований к кросс-функциональной коммуникации на различных языках;
- ♦ раздельное управление входящим трафиком и трафиком между сервисами.

³ Более подробно о концепциях сетевого взаимодействия Kubernetes можно узнать из официальной документации: *Service* (<https://oreil.ly/LBZS3>), *NetworkPolicies* (<https://oreil.ly/KVVU1>) и *Container Networking Interface, CNI* (<https://oreil.ly/wNkuw>).

Тонкий контроль маршрутизации, надежности и управления трафиком

Маршрутизация трафика в распределенной системе на базе микросервисов может оказаться более сложной задачей, чем это представляется на первый взгляд. Как правило, в среде разворачивается несколько экземпляров сервиса с целью повышения производительности (балансировка нагрузки между сервисами) и надежности (обеспечение резервирования). Кроме того, многие современные инфраструктурные платформы построены на «товарном оборудовании» (commodity hardware). Оно представляет собой эфемерные вычислительные ресурсы, способные в один момент выключиться, перезагрузиться или исчезнуть, а это означает, что местоположение сервиса может меняться изо дня в день (или от минуты к минуте!).

Разумеется, можно использовать технологии маршрутизации и связанные с ними методы — о них было рассказано в *главе 3*. Проблема заключается в том, что количество внутренних сервисов и API обычно намного больше, чем количество внешних API, доступ к которым открывается приложениями, а темпы изменения внутренних систем и соответствующих им API и функциональности зачастую намного выше. Соответственно если развернуть API-шлюз перед каждым внутренним сервисом, то эксплуатационные расходы резко возрастут как с точки зрения требуемых вычислительных ресурсов, так и с точки зрения затрат на обслуживающий персонал.

Прозрачная маршрутизация и нормализация имен сервисов

По сути, маршрутизация — это процесс выбора пути для трафика в сети или между несколькими сетями. В веб-приложениях маршрутизация на сетевом уровне обычно осуществляется в рамках стека TCP/IP и соответствующей сетевой инфраструктуры (на уровне 3/4 модели OSI). Это означает, что требуется указать только IP-адрес и порт как целевого, так и исходного соединения. До появления облака, а зачастую и в локальных центрах обработки данных IP-адреса внутренних сервисов часто фиксированы и хорошо известны. Несмотря на то что DNS широко задействуется для сопоставления доменных имен с IP-адресами, до сих пор многие приложения и сервисы используют жестко заданные IP-адреса. Это означает, что любое изменение местоположения сервиса требует повторного развертывания всех вызывающих его сервисов.

Переход на облачные технологии и связанный с этим процессом эфемерный характер нашей инфраструктуры ведет к регулярному изменению IP-адресов вычислительных экземпляров и соответствующих им сервисов. Это, в свою очередь, означает, что при жестком задании IP-адресов и адресов портов их придется часто менять. С ростом популярности архитектур на базе микросервисов боль от повторного развертывания возрастала в зависимости от количества сервисов в приложении. Первые микросервисы создавали решения для преодоления этой проблемы путем внедрения внешних каталогов или реестров «обнаружения сервисов», содержащих динамическое сопоставление имен сервисов с IP-адресами и портами⁴.

⁴ Платформа Airbnb SmartStack (<https://oreil.ly/mJDVW>) была одной из первых реализаций внешнего обнаружения сервисов микросервисов.

Сервисные сети могут обрабатывать этот динамический поиск имени сервиса и его местоположения вне сервиса, а также прозрачно, без необходимости модификации кода, выполнять повторные развертывания или перезапуск. Еще одним преимуществом сервисной сети является возможность нормализации именования в различных средах с использованием «осведомленности о среде» в сочетании с конфигурацией, хранящейся вне приложения. Например, сервисная сеть, развернутая в среде эксплуатации, распознает, что она запущена в этой среде. Затем сервисная сеть прозрачно сопоставляет имя сервиса (`sessions-service`) на уровне кода с местоположением, специфичным для среды (`AWS-us-east-1a/prod/sessions/v2`), путем поиска местоположения в реестре сервисов (который может быть интегрирован в сеть или запускаться извне). Тот же код, развернутый в промежуточной среде с соответствующим образом настроенной сервисной сеткой, будет маршрутизировать `sessions-service` к адресу `internalstaging-server-a/stage/sessions/v3`.

Надежность

Эфемерный характер современных вычислительных и кластерных сред порождает проблемы, связанные не только с изменением местоположения, но и с надежностью. Например, каждый сервис должен корректно решать вопросы взаимодействия с другим сервисом. Чуть позже вы узнаете о «Восьми ошибках распределенных вычислений», но в рассматриваемом контексте следует обратить внимание на такие проблемы, как прерывание связи с сервисом, временная недоступность сервиса или его медленный отклик. Эти проблемы могут быть решены в коде с помощью хорошо известных моделей надежности, таких как повторные попытки, тайм-ауты, автоматические выключатели, переборки и резервные копии. Книга Майкла Нейгарда под названием «Release It! Проектирование и дизайн ПО для тех, кому не всё равно»⁵, выходящая уже во втором издании, представляет собой всеобъемлющее руководство по их исследованию и внедрению. Однако далее, в разд. «Поддержка кросс-функционального взаимодействия на разных языках», вы узнаете, что попытка реализовать эту функциональность в коде обычно приводит к несогласованному поведению, особенно на разных языках и платформах.

Поскольку сервисная сеть участвует в инициировании и управлении каждым межсервисным взаимодействием, она становится идеальным местом для последовательной реализации этих моделей надежности, обеспечивающих отказоустойчивость и плавное вырождение. В зависимости от реализации, сервисная сеть может также обнаруживать проблемы и обмениваться этой информацией по всей сети, позволяя каждому сервису внутри сети принимать соответствующие решения о маршрутизации трафика. Например, если задержка ответа сервиса увеличивается, все вызывающие его сервисы могут получить инструкции по инициации своих резервных действий.

Для нашего примера сервисная сеть позволит определить, как обрабатывать любые отказы при взаимодействии с новым сервисом под названием Сессия. Представьте себе несколько тысяч участников мероприятия, только что прослушавших основ-

⁵ См. <https://tinyurl.com/yurw2bmh>.

ной доклад утренней конференции и желающих узнать свое расписание на весь день. Такой резкий скачок трафика для сервиса Сессия может привести к ухудшению его работы. Для большинства случаев использования необходимо определить соответствующие тайм-ауты и повторные попытки, но можно также определить действие автоматического выключателя, запускающее поведение приложения. Например, если вызов API от сервиса Участник к сервису Сессия для получения расписания ежедневных сессий участника неоднократно терпит неудачу, можно запустить автоматический выключатель в сервисной сети, который быстро отменит все эти вызовы (чтобы дать сервису возможность восстановиться). Скорее всего, в мобильном приложении вы справитесь с этим отказом, выполнив откат к отображению всего расписания сессий конференции, а не своего персонального расписания.

Продвинутая маршрутизация трафика: придание формы, контроль, разделение и зеркалирование

Со времен бума доткомов конца 90-х годов прошлого века потребительские веб-приложения обслуживали всё большее количество пользователей и трафика. Пользователи также стали более требовательными как к производительности, так и к предлагаемым функциям. Соответственно, всё большее значение приобретает необходимость управления трафиком с учетом требований безопасности, производительности и расширения функциональных возможностей. В *разд. «Как API-шлюз интегрируется с другими технологиями на периферии?» главы 3* вы узнали, что на границе сети появились специализированные устройства для удовлетворения этих требований, но такая инфраструктура не подходила для развертывания перед каждым внутренним сервисом. Далее в этом разделе будет рассказано о требованиях в отношении формирования и контроля внутреннего трафика, ставших типичными для приложений на базе микросервисов.

Придание формы трафику

Придание формы трафику — это метод управления пропускной способностью, при котором часть или весь сетевой трафик задерживается для того, чтобы соответствовать желаемому профилю трафика. Этот метод используется для оптимизации или гарантии производительности, снижения задержек или увеличения полезной полосы пропускания для одних видов трафика за счет задержки других. Наиболее распространенным считается придание формы трафику на основе приложений, когда с помощью инструментов для создания цифровых отпечатков сначала определяют интересующие приложения, а затем на них накладываются политики формирования. В случае трафика восток-запад сервисная сеть может генерировать или отслеживать цифровые отпечатки, такие как идентификатор сервиса или другой прокси для этого сервиса, или заголовок запроса, содержащий соответствующие метаданные, — например, исходит ли запрос от пользователя конференц-приложения на бесплатном или на платном тарифе.

Контроль трафика

Контроль трафика — это процесс мониторинга сетевого трафика на предмет его соответствия политике трафика или договору и принятия мер по обеспечению соблюдения этого договора. Трафик, нарушающий ту или иную политику, может быть немедленно отброшен, помечен как несоответствующий требованиям или оставлен как есть, в зависимости от административной политики. Этот метод полезен для предотвращения атаки типа «отказ в обслуживании» (Denial of Service, DoS) на неработающий внутренний сервис или для предотвращения чрезмерного насыщения трафиком критически важного или хрупкого внутреннего ресурса (например, хранилища данных). До появления облачных технологий и сервисных сетей управление трафиком во внутренних сетях обычно реализовывалось только в масштабах предприятия с помощью специализированных аппаратных или программных средств, таких как сервисная шина предприятия (Enterprise Service Bus, ESB). Облачные вычисления и программно-определяемые сети (Software-Defined Network, SDN) упростили внедрение методов контроля трафика за счет использования групп безопасности (Security Group, SG) и списков контроля сетевого доступа (Network Access Control List, NACL).

При управлении связью типа восток-запад сервисы, находящиеся на границе сети или кластера, могут знать о контракте на трафик и применять внутреннее формирование трафика, чтобы их выходной сигнал находился в рамках контракта. Например, в сервисе Участник может быть реализован внутренний ограничитель количества запросов, предотвращающий чрезмерное количество обращений к API сервиса Сессия в течение определенного периода времени.

Сервисная сеть позволяет осуществлять гранулярное управление формированием, разделением и зеркалированием трафика, что дает возможность постепенно переводить или мигрировать трафик с одной версии целевого сервиса на другую. В разд. «Стратегии релизов» главы 5 мы рассмотрим, как этот подход может быть реализован для разделения сборки и выпуска в рамках стратегий релизов на основе трафика.

Обеспечение прозрачной наблюдаемости

При эксплуатации любой распределенной системы — например, приложения на базе микросервисов, возможность наблюдения как за работой конечного пользователя, так и за произвольными внутренними компонентами, жизненно необходима для выявления неисправностей и преодоления соответствующих проблем. Исторически внедрение общесистемного мониторинга требовало интеграции в приложения агентов или библиотек среды выполнения с высокой связанностью, что вызывало необходимость развертывания всех приложений при первоначальном внедрении и всех последующих обновлениях.

Сервисная сеть может обеспечить некоторые из необходимых наблюдаемых параметров — в частности, метрики приложений (уровень 7) и сети (уровень 4), причем сделать это прозрачно. Соответствующее обновление компонентов сбора телеметрии или самой сервисной сети не должно требовать повторного развертывания всех

приложений. Конечно, существуют ограничения на наблюдаемость, накладываемые сервисной сетью, поэтому следует также оснастить свои сервисы с помощью метрик, специфичных для используемого языка, и библиотек, выдающих записи для журналирования. Например, в нашем примере сервисная сеть будет предоставлять метрики по количеству, задержке и частоте ошибок вызовов API сервиса Сессия, а вы также можете принять решение регистрировать специфичные для бизнеса метрики и ключевые показатели эффективности (KPI) вызовов API.

Обеспечение безопасности: транспортная безопасность, аутентификация и авторизация

Так же, как и наблюдаемость, безопасность межсервисных взаимодействий исторически реализовывалась с помощью библиотек, специфичных для конкретного языка. Эти подходы с сильной связанностью имеют те же недостатки и нюансы. Например, реализация шифрования на транспортном уровне во внутренней сети — достаточно распространенное требование, однако различные языковые библиотеки по-разному обрабатывают управление сертификатами, что увеличивает операционную нагрузку, связанную с развертыванием и ротацией сертификатов. Управление идентификацией сервисов (машин) и пользователей (людей) для аутентификации и авторизации на разных языках также затруднено. Кроме того, часто удавалось случайно (или намеренно) обойти любую реализацию безопасности, не включив необходимые библиотеки.

Поскольку плоскость данных сервисной сети находится на пути любого трафика внутри системы, обеспечить требуемый профиль безопасности относительно просто. Так, плоскость данных сервисной сети может управлять идентификаторами сервисов (например, с помощью фреймворка SPIFFE⁶) и криптографическими сертификатами, включением mTLS, аутентификацией и авторизацией на уровне сервисов. Это позволяет легко реализовать mTLS в рамках нашего примера без необходимости модификации кода.



mTLS (Mutual TLS) и TLS (Transport Layer Security) — это протоколы шифрования, которые обеспечивают безопасность передачи данных в рамках сети Интернет.

mTLS является расширением протокола TLS и предоставляет дополнительные слои безопасности. Основное различие между этими протоколами заключается в том, что mTLS обеспечивает двустороннюю аутентификацию — т. е. и клиент, и сервер проходят процесс проверки подлинности.

Поддержка кросс-функционального взаимодействия на разных языках

При создании или извлечении сервисов из приложения, основанного на микросервисах, и переходе от внутрипроцессного к внепроцессному взаимодействию необходимо задуматься об изменениях в маршрутизации, надежности, наблюдаемости и

⁶ См. <https://spiffe.io/>.

безопасности. Необходимая для этого функциональность может быть реализована в коде приложения — например, в виде библиотеки. Однако если в приложении или системе используется несколько языков программирования, а многоязычный подход весьма характерен для систем на базе микросервисов, то это означает, что придется реализовывать каждую библиотеку для каждого используемого языка. Поскольку сервисная сеть обычно реализуется по схеме *sidecar*, где все соединения сервисов направляются через сетевой прокси, внешний по отношению к сервису, но работающий в том же сетевом пространстве имен, необходимая функциональность может быть реализована в прокси один раз и повторно использована во всех сервисах. Это можно назвать «внедрением инфраструктурных зависимостей». В нашем примере это позволило бы нам переписать сервис Участник на другом языке (возможно, для удовлетворения новых требований к производительности) и по-прежнему рассчитывать на то, что кросс-функциональные аспекты взаимодействия между сервисами будут обрабатываться последовательно.

Разделение управления входящим и межсервисным трафиком

Напомним, что в *разд. «Практический пример: эволюционный этап»* введения мы вкратце познакомились с ключевыми понятиями трафика север-юг и восток-запад. Попросту говоря, трафик север-юг — это трафик, проникающий в вашу систему из внешней точки. Трафик восток-запад — это внутренний транзит от системы к системе или от сервиса к сервису. В приведенных определениях легко запутаться, если углубиться в определение понятия «ваши системы». Например, распространяется ли это определение на системы, разработанные и эксплуатируемые только вашей командой, вашим отделом, вашей организацией, доверенными третьими лицами и т. д.

Некоторые участники API-пространства, в том числе и Марко Палладино из компании Kong, утверждают⁷, что использование понятий север-юг и восток-запад в значительной степени неактуально и стало скорее отголоском предыдущего поколения компьютерных сетей, когда границы между системами были более четкими. Более подробно мы рассмотрим эту позицию в *главе 9*, поскольку она касается идеи API как продукта (включая управление жизненным циклом API) и связности сервисов 7-го и 4-го уровней (из модели сетей OSI). Различия между основными свойствами и характеристиками входящего и межсервисного трафика приведены в табл. 4.3.

Таблица 4.3. Различия между свойствами входящего и межсервисного трафика

	Входящий (с/ю)	Межсервисный (в/з)
Источники трафика	Внешние (пользовательские, сторонние, Интернет)	Внутренние (в пределах доверительной границы)
Назначение трафика	Публичный или бизнес-ориентированный API, веб-сайт	API сервиса или домена

⁷ См. <https://oreil.ly/Wv1yi>.

Таблица 4.3 (окончание)

	Входящий (с/ю)	Межсервисный (в/з)
Аутентификация	Ориентирована на «пользователя» (субъект реального мира)	Ориентирована на «сервис» (машинный объект) и «пользователя» (субъект реального мира)
Авторизация	«Пользовательские» роли или уровень возможностей	«Сервисные» идентификаторы или сегмент сети, ориентированный на пользователя, и «пользовательские» роли или уровень возможностей
TLS	Односторонние, часто принудительные (например, обновление протокола)	Взаимный, может быть сделан обязательным (строгий mTLS)
Первичные реализации	API-шлюз, обратный прокси-сервер	Сервисная сеть, библиотеки приложений
Основной владелец	Шлюзовая, сетевая, операционная команда	Команда платформы, кластера, операционной системы
Пользователи в организации	Архитекторы, API-менеджеры, разработчики	Разработчики

Как можно здесь видеть, свойства и соответствующие требования к управлению этими двумя типами трафика зачастую совершенно различны. Например, обработка внешнего трафика конечных пользователей, предназначенного для API продукта, имеет принципиально иные требования по сравнению с обработкой внутреннего трафика между сервисами, предназначенного для API внутреннего бизнес-процесса, домена или компонента. На практике это означает, что плоскости управления для API-шлюза и сервисной сети должны обладать различными возможностями для поддержки конфигурации соответствующих плоскостей данных. Например, в нашем примере команде разработчиков сервиса Сессия может потребоваться указать, что сервис можно вызывать только унаследованным конференц-приложением и сервисом Участник. В то время как команда разработчиков сервиса Участник обычно не указывает, какие внешние системы могут или не могут вызывать публичный API, — за это отвечает соответствующая команда разработчиков шлюзов или сетей.

Это различие между управлением входящими и межсервисными API-вызовами можно лучше понять, если сравнить развитие и использование технологии API-шлюзов, рассмотренное в разд. «Современная история API-шлюзов» главы 3, с описанным в следующем разделе развитием технологии сервисной сети.

Эволюция сервисных сетей

Хотя термин «сервисная сеть» появился в 2016 году, некоторые из первых «компаний-единогоров», таких как Twitter, Netflix, Google и Amazon, создавали и использовали соответствующие технологии в своих внутренних платформах с конца 2000-х — начала 2010-х годов. Например, компания Twitter создала свой RPC-фреймворк Finagle на базе Scala, ставший открытым ПО в 2011 году. В 2012 году компания Netflix создала и запустила релиз своих «OSS» библиотек коллективного

пользования для микросервисов на базе Java, включая Ribbon, Eureka и Hystrix⁸. Позже команда Netflix выпустила Prana sidecar, позволяющий сервисам, организованным не на базе JVM, использовать преимущества этих библиотек. Создание библиотек Finagle и внедрение sidecar в конечном счете привело к появлению Linkerd — возможно, первой сервисной сети на базе sidecar, а также к появлению начального проекта в CNCF, когда эта основа была сформирована. Компания Google быстро последовала этому примеру, предоставив релиз сервисной сети Istio. Она была построена на базе проекта Envoy Proxy, созданного командой Lyft Engineering.

Похоже, что отрасль проходит полный круг, и возможности сервисной сети возвращаются в разделяемые библиотеки, как это происходит с gRPC, или добавляются в ядро ОС. Эту эволюцию можно проследить на рис. 4.5. Хотя разработка и использование многих из этих ранних компонентов и платформ в настоящее время устарели, полезно провести краткий экскурс в их эволюцию. Это позволяет выявить ряд проблем и ограничений использования паттерна сервисной сети. Некоторые из них сохраняются до сих пор.

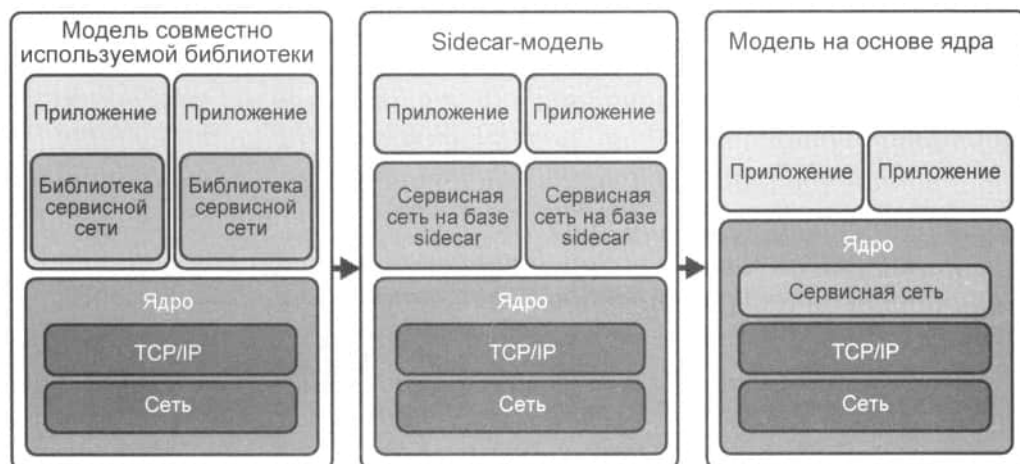


Рис. 4.5. Развитие технологии сервисных сетей

Ранняя история и мотивация

В 90-х годах прошлого века Питер Дойч и его коллеги из компании Sun Microsystems составили список «Восемь заблуждений о распределенных вычислениях»⁹, где перечислили допущения, которые обычно делают инженеры при работе с распределенными системами. Они также отметили, что хотя приведенные в списке пред-

⁸ Следует отметить, что фреймворк Finagle RPC (<https://oreil.ly/wHf0Q>) и библиотеки Netflix OSS (<https://oreil.ly/BPvAv>) уже устарели и не рекомендуются для использования в современных эксплуатируемых системах.

⁹ См. <https://oreil.ly/hdvuC>.

положения могли быть верны в более примитивных сетевых архитектурах или теоретических моделях, в современных сетях они не соответствуют действительности:

1. Сеть надежна.
2. Задержка равна нулю.
3. Пропускная способность бесконечна.
4. Сеть безопасна.
5. Топология не изменяется.
6. Есть один администратор.
7. Транспортные издержки равны нулю.
8. Сеть однородна.

Питер и его команда утверждали, что все эти заблуждения «в конечном счете оказываются ложными и приводят к большим неприятностям и болезненному опыту обучения». Инженеры не могут просто игнорировать эти вопросы, они должны открыто их решать.



Игнорируйте ошибки распределенных вычислений на свой страх и риск!

Поскольку документ «Восемь заблуждений о распределенных вычислениях» был разработан в 1990-е годы, есть соблазн считать их компьютерным пережитком. Однако это будет ошибкой! Как и многие другие вечные законы и закономерности вычислительной техники, выведенные в 70–80-х годах прошлого века, эти заблуждения остаются неизменными даже при изменении технологий. Работая в качестве архитектора, вы должны постоянно напоминать своим командам, что многие сетевые проблемы, описанные в этих заблуждениях, актуальны и сегодня, и проектировать системы нужно соответствующим образом!

С появлением в 2010-х годах распределенных систем и микросервисных архитектур многие новаторы в этой области, такие как Джеймс Льюис, Сэм Ньюмен и Фил Кальсадо, осознали важность создания систем, признающих и компенсирующих эти недостатки в дополнение к функциональности, предоставляемой стандартными сетевыми стеками. Опираясь на первоначальный набор «Необходимых условий для микросервисов» Мартина Фаулера, Фил создал «Необходимые условия для микросервисов Кальсадо» и включил в них «стандартизированный RPC» в качестве ключевой предпосылки, воплощающей многие практические уроки, которые он извлек из ошибок распределенных вычислений. В своем блоге в конце 2017 года Фил заявил: «хотя стек TCP/IP и общая сетевая модель, разработанная много десятилетий назад, по-прежнему являются мощным инструментом, позволяющим компьютерам общаться друг с другом, более сложные архитектуры [на базе микросервисов] предъявляют еще один уровень требований, которые снова должны выполняться инженерами, работающими в таких архитектурах»¹⁰.

¹⁰ Более подробную информацию по этой теме можно получить в статье «Microservice Prerequisites» Фаулера (<https://oreil.ly/GIVvp>), статье «Calçado's Microservices Prerequisites» (<https://oreil.ly/d4IEh>) и в блоге Фила «Pattern: Service Mesh» (<https://oreil.ly/h45Te>).

Шаблоны реализации

Хотя сегодня наиболее распространенные реализации сервисных сетей используют модель развертывания *sidecar* на основе прокси, так было не всегда. И в будущем это может оказаться не так. В этом разделе главы вы узнаете, как развивались шаблоны реализации сервисных сетей до настоящего времени, и поймете, что может ждать нас в будущем.

Библиотеки

Хотя многие технические руководители осознавали необходимость создания нового уровня сетевой функциональности в системах на базе микросервисов, они понимали, что внедрение этих технологий будет нетривиальным. Они также признавали, что потребуется многократное повторение усилий как внутри организации, так и между организациями. Это привело к появлению ориентированных на микросервисы сетевых фреймворков и общих библиотек. Их можно создать один раз и использовать повторно — сначала в рамках организации, а затем сделать их открытым ПО для широкого применения.

В своей упомянутой ранее записи в блоге Фил Кальсадо отметил, что даже такие основные сетевые функции, как обнаружение сервисов и автоматическое отключение, сложно реализовать корректно. Это привело к созданию крупных, сложных библиотек — Finagle компании Twitter и OSS-стек Netflix. Они стали очень популярны в качестве средств, позволяющих избежать переписывания одной и той же логики в каждом сервисе, а также как проекты, дающие возможность сосредоточить общие усилия на обеспечении корректности. Некоторые небольшие организации сами брали на себя бремя написания необходимых сетевых библиотек и инструментов, но стоимость таких работ обычно была высока, особенно в долгосрочной перспективе. Иногда эти затраты были явными и четко выраженными — например, затраты на инженеров, выделенных в группы, занимающиеся созданием инструментов сборки. Но чаще всего реальные расходы трудно оценить в полной мере, поскольку они проявляются в виде времени, затрачиваемого новыми разработчиками на освоение принятых решений, ресурсов, необходимых для оперативного обслуживания, или других форм отвлечения времени и сил от работы над продуктами, ориентированными на клиента.

Фил также заметил, что использование библиотек, раскрывающих функциональность через языковые привязки или SDK, ограничивает набор инструментов, сред выполнения и языков, доступных для работы с микросервисами. Библиотеки для микросервисов часто пишутся для конкретной платформы, будь то язык программирования или среда выполнения — например, JVM. Если вы используете отличные от поддерживаемой библиотекой платформы, то, скорее всего, вам придется переносить код на новую платформу самостоятельно, причем ваши затраты будут расти в зависимости от количества языков.



Библиотеки сервисных сетей и плата за многоязычные решения

Многие организации придерживаются многоязычного подхода к программированию приложений и задействуют различные языки, выбирая наиболее подходящий для выполнения тех или иных требований. Например, использование Java — для дол-

временных бизнес-сервисов, Go — для инфраструктурных сервисов, а Python — для работы в сфере науки о данных. Если вы используете библиотечный подход к реализации сервисной сети, то должны понимать, что вам придется создавать, поддерживать и обновлять все свои библиотеки синхронно, чтобы избежать проблем с совместимостью или не столкнуться с неоптимальными возможностями для разработчиков на некоторых языках. Также можно обнаружить тонкие различия между реализациями на разных языковых платформах или ошибки, влияющие только на конкретную среду выполнения.

Sidcar

В начале 2010-х годов многие инженеры стали использовать подход многоязычного программирования, и нередко в одной организации были развернуты сервисы, написанные на разных языках. Желание написать или поддерживать одну библиотеку, управляющую всеми необходимыми сетевыми абстракциями, привело к созданию библиотек, запускаемых вне сервиса в виде отдельных процессов. Так родился микросервис под названием *sidcar*. В 2013 году компания Airbnb написала о «Synapse and Nerve» — своей открытой *sidcar*-реализации системы обнаружения сервисов. Год спустя компания Netflix представила Prana — *sidcar*, предоставляющий HTTP-интерфейс для интеграции не-JVM-приложений с остальной экосистемой Netflix OSS для обнаружения сервисов, их автоматического отключения и т. п. Основная концепция заключалась в том, что сервис не подключался напрямую к своим зависимостям, расположенным ниже по потоку выполнения, а весь трафик проходил через Prana *sidcar*, прозрачно добавлявший необходимые сетевые абстракции и возможности.

С ростом использования стиля архитектуры микросервисов появилась новая волна прокси-серверов, достаточно гибких для адаптации к различным компонентам инфраструктуры и требованиям к коммуникациям. Первой широко известной системой в этой области стала Linkerd, созданная компанией Buoyant и основанная на ее инженерном опыте работы над платформой микросервисов Twitter. Вскоре после этого команда инженеров компании Lyft анонсировала Envoy Proxy, работающий по схожему принципу и быстро принятый на вооружение компанией Google в ее сервисной сети Istio. При использовании паттерна *sidcar* у каждого из ваших сервисов будет свой прокси-процесс, работающий отдельно от приложения. Как правило, такое *sidcar*-решение использует общее пространство имен процессов, файлов и сетей, при этом обеспечиваются определенные гарантии безопасности (например, что любое взаимодействие с «локальной» сетью изолировано от внешней сети). Учитывая, что сервисы взаимодействуют друг с другом только через *sidcar*-прокси, в итоге мы получаем развертывание, похожее на схему, показанную на рис. 4.6.

Как отмечают Фил Кальсадо и Уильям Морган из компании Buoyant, самый мощный аспект такой интеграции *sidcar*-прокси заключается в том, что она позволяет отказаться от восприятия прокси как изолированных компонентов и признать образуемую ими сеть как нечто ценное само по себе.

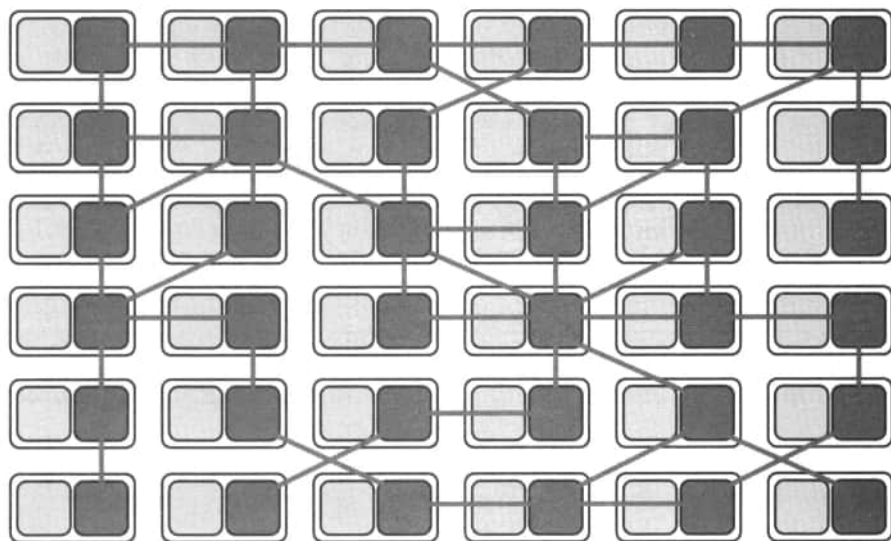


Рис. 4.6. Прокси-серверы сервисной сети, образующие сетевую абстракцию более высокого уровня

В середине 2010-х годов организации начали переводить развертывание своих микросервисов на более сложные среды выполнения — такие как Apache Mesos (с Marathon), Docker Swarm и Kubernetes, и стали применять инструменты, предоставляемые этими платформами, для реализации сервисной сети. Это привело к отказу от использования набора независимых прокси-серверов, работающих изолированно, как это было в случае с «Synapse and Nerve», и переходу к работе с централизованной плоскостью управления. Если посмотреть на эту схему развертывания сверху вниз, то можно увидеть, что трафик сервисов по-прежнему идет от прокси к прокси напрямую, но плоскость управления знает о каждом экземпляре прокси и может влиять на него. При этом плоскость управления позволяет прокси-серверам реализовать такие функции, как управление доступом и сбор метрик, которые требуют взаимодействия и координации между сервисами, как показано на рис. 4.7.

Подход на основе *sidecar* стал в настоящее время наиболее распространенным и, вероятно, хорошо подходит для нашей конференц-системы. Основные затраты на развертывание сервисной сети на основе *sidecar* связаны с первоначальной установкой и текущим эксплуатационным обслуживанием, а также с ресурсами, необходимыми для работы всех *sidecar*-прокси. Поскольку наши потребности в масштабируемости в настоящее время скромны, нам не требуется больших вычислительных мощностей для работы этих прокси.



Стоимость запуска *sidecar*-прокси в масштабной системе

Многие из популярных сегодня решений для создания сервисной сети требуют добавления и запуска контейнера *sidecar*-прокси, такого как Envoy, Linkerd-proxy или NGINX, для каждого сервиса или приложения, работающего в кластере. Даже в относительно небольшой среде, состоящей, скажем, из 20 сервисов, каждый из которых работает с пятью подами¹¹, распределенными по трем узлам, у вас будет

¹¹ Под (pod) — наименьшая функциональная единица для развертывания в объектной модели Kubernetes.

100 работающих прокси-контейнеров. Какой бы маленькой и эффективной ни была реализация прокси, дублирование прокси будет сказываться на ресурсах.

В зависимости от конфигурации сервисной сети объем памяти, используемой каждым прокси, может увеличиваться пропорционально количеству сервисов, с которыми он должен иметь возможность взаимодействовать. Пранай Сингхал¹² написал статью о своем опыте настройки Istio для снижения потребления с примерно 1 Гбайт на прокси до гораздо более разумных 60–70 Мбайт на каждый из них. Однако даже в небольшой воображаемой среде со 100 прокси-серверами на трех узлах эта оптимизированная конфигурация все равно требует около 2 Гбайт на узел.

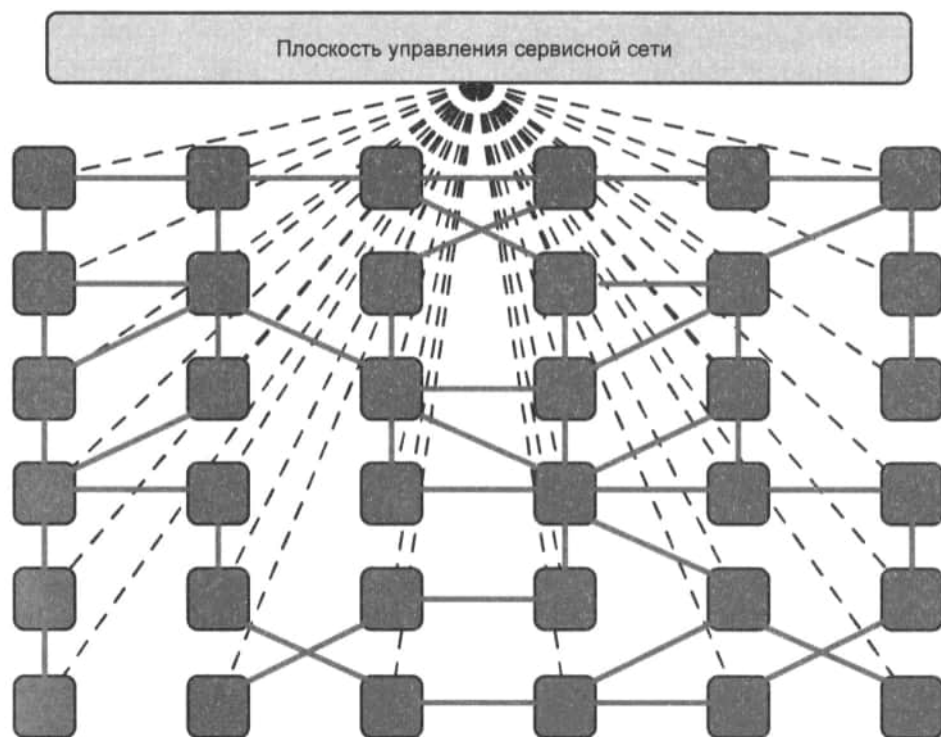


Рис. 4.7. Управление и координация плоскости данных сервисной сети

Библиотеки gRPC без прокси

В начале 2021 года Google Cloud начала продвигать Proxyless gPRC (gPRC без прокси)¹³, где сетевые абстракции снова перенесены в библиотеку, специфичную для конкретного языка (хотя и поддерживаемую Google и большим сообществом OSS). Эти библиотеки gRPC включаются в состав каждого сервиса и выступают в качестве плоскости данных в сервисной сети. Для координации работы библиотек требуется доступ к внешней плоскости управления — например, к сервису Google Traffic

¹² См. <https://oreil.ly/khfYY>.

¹³ См. <https://oreil.ly/ZzbYw>.

Director (распределитель трафика). Он использует xDS API с открытым исходным кодом для конфигурирования библиотек gRPC непосредственно в приложениях¹⁴. Эти gRPC-приложения выступают в роли xDS-клиентов, подключаясь к глобальной плоскости управления Traffic Director, обеспечивающей глобальную маршрутизацию, балансировку нагрузки и региональное восстановление после сбоев в случае использования сервисной сети и балансировки нагрузки. Traffic Director поддерживает даже «гибридный» режим работы с развертываниями, включающими как прокси-сервисы на основе sidecar, так и беспрокси-сервисы (рис. 4.8).

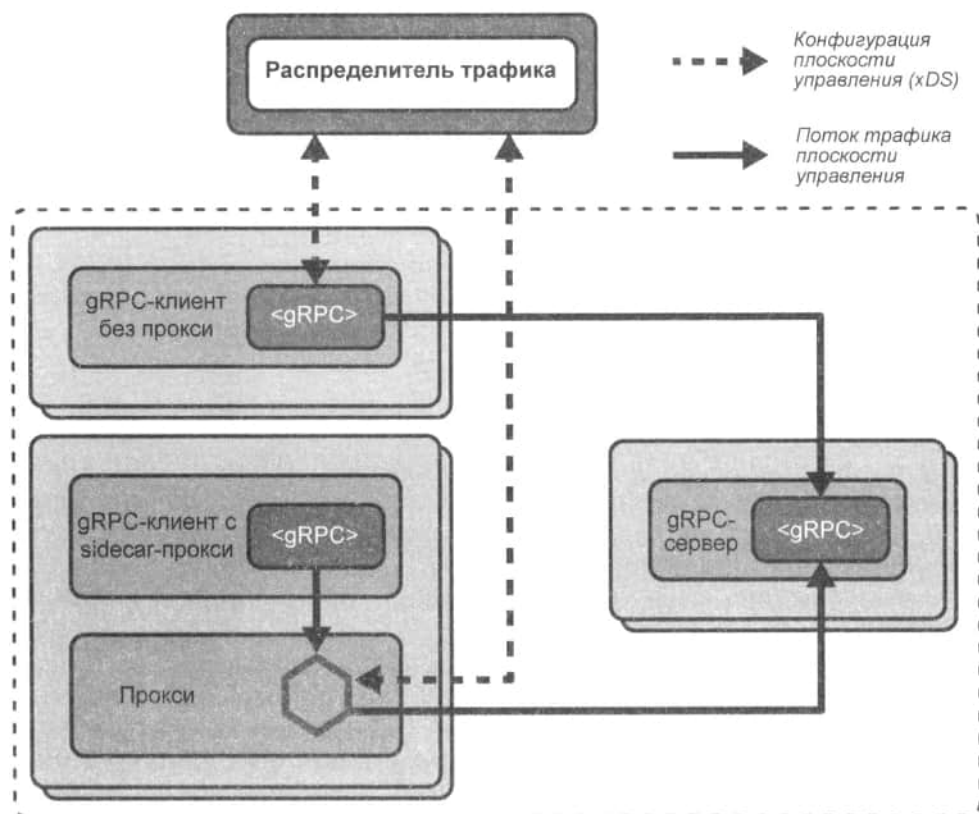


Рис. 4.8. Пример сетевой схемы сервисов, использующих sidecar и связь без прокси

Поскольку в нашей конференц-системе, помимо gRPC API, используются REST API, то в настоящее время такой выбор реализации сервисной сети для нас исключен. Если использование REST API внутри компании будет отменено, или библиотеки gRPC будут усовершенствованы для поддержки взаимодействия, не основанного на gRPC, отказ от использования этого подхода может быть пересмотрен.

¹⁴ Узнать больше о Traffic Director (<https://oreil.ly/vao6J>) и протоколе xDS, основанном на Envoy Proxy (<https://oreil.ly/rOo5t>), вы сможете на соответствующих сайтах документации.

Будущее сервисных сетей — без прокси?

Как гласит уже упомянутое ранее нами популярное клише, хотя история не повторяется, она часто рифмуется. Многие преимущества и ограничения подхода без прокси аналогичны возникающим при использовании библиотек, специфичных для какого-либо языка. В качестве примеров того, как может быть полезно развертывание сервиса без прокси, команда Google Cloud называет следующие сценарии использования:

- эффективность использования ресурсов в крупномасштабной сервисной сети — экономия ресурсов за счет отказа от запуска дополнительных *sidecar*-процессов;
- высокопроизводительные приложения *gRPC* — сокращение количества сетевых переходов и задержек;
- сервисная сеть для сред, в которых невозможно развернуть *sidecar*-прокси, — например, где нельзя выполнить второй процесс, или *sidecar* не может манипулировать необходимым сетевым стеком;
- переход от сервисной сети с прокси к сети без прокси-серверов.

Без *sidecar*: реализации ядра операционной системы (eBPF)

Другая появляющаяся альтернативная реализация сервисной сети основана на переносе необходимых сетевых абстракций в ядро операционной системы (ОС). Это стало возможным благодаря появлению и широкому распространению *eBPF*¹⁵ — технологии ядра, позволяющей запускать пользовательские программы в среде песочницы внутри ядра¹⁶. Программы *eBPF* запускаются в ответ на события на уровне ОС, которых могут быть тысячи. К таким событиям относятся вход и выход из любой функции в пространстве ядра или пользователя, «точки трассировки» и «точки зондирования», а также — что очень важно для сервисной сети — поступление сетевых пакетов. Поскольку на каждом узле имеется только одно ядро, все контейнеры и процессы, выполняющиеся на узле, используют одно и то же ядро. Если добавить программу *eBPF* к событию в ядре, то она будет срабатывать независимо от того, какой процесс вызвал это событие — запущенный в контейнере приложений или непосредственно на хосте. Это позволит исключить возможные попытки случайного или иного варианта обхода сервисной сети.

Проект *Cilium*, созданный на основе *eBPF*, предоставляет возможности для защиты и наблюдения за сетевыми соединениями между контейнерными рабочими нагрузками. *Cilium* привносит эту модель «без *sidecar*» в мир сервисных сетей. Использование *Cilium* позволяет сократить время ожидания между вызовами сервисов, т. е. некоторые функциональные возможности могут предоставляться ядром без необходимости совершать сетевой переход к *sidecar*-прокси¹⁷. Помимо традиционной *sidecar*-модели, *Cilium* поддерживает работу плоскости данных сервисной сети с использованием одного экземпляра *Envoy Proxy* на узел, что позволяет снизить потребление ресурсов. На рис. 4.9 показано взаимодействие двух сервисов с помощью *Cilium* и одного *Envoy Proxy* на узел.

¹⁵ См. <https://ebpf.io/>.

¹⁶ *eBPF*, enhanced Berkeley Packet Filter, берклийский улучшенный фильтр пакетов.

¹⁷ Подробнее об этом можно узнать в статье «How eBPF will solve Service Mesh — Goodbye Sidecars» (<https://oreil.ly/2uxSR>).

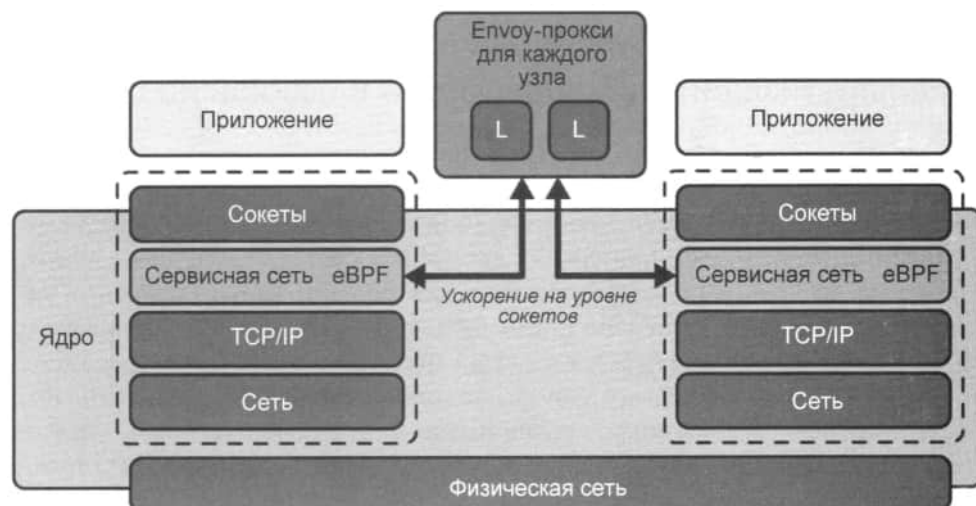


Рис. 4.9. Использование Cilium, eBPF и одного Envoy Proxy на узел для реализации функциональности сервисной сети

Таксономия сервисных сетей

В табл. 4.4 приведены различия между тремя вариантами реализации сервисной сети, рассмотренными в предыдущем разделе.

Таблица 4.4. Сравнение сервисных сетей на основе библиотек, прокси и ядра ОС

Пример использования	На основе библиотек (и вариант «без прокси»)	Sidecar, на основе прокси	На основе ядра ОС
Поддержка языков/платформ	Одноязычные библиотеки, не зависящие от платформы	Независимость от языка, широкая поддержка платформ	Независимость от языка, поддержка на уровне ОС
Механизм выполнения	Упаковывается и запускается внутри приложения	Запуск параллельного приложения в отдельном процессе	Выполняется как часть ядра ОС, с полным доступом к пространству пользователя и ядра
Модернизация компонентов сервисной сети	Требуется перестройка и повторное развертывание всего приложения	Требуется повторное развертывание sidecar-компонентов (часто может быть с нулевым временем простоя)	Требуется обновление/патч программы ядра
Наблюдаемость	Полная информация о приложениях и трафике с возможностью легкого распространения контекста	Только понимание трафика, для распространения контекста требуется языковая поддержка или оболочка	Только понимание трафика, для распространения контекста требуется языковая поддержка или оболочка
Модель угроз безопасности	Библиотечный код выполняется как часть приложения	Побочные модули обычно совместно используют пространство имен процессов и сетей с приложением	Приложение взаимодействует с ОС напрямую через системные вызовы

Практический пример: использование сервисной сети для маршрутизации, наблюдаемости и безопасности

В этом разделе главы мы рассмотрим несколько конкретных примеров использования сервисной сети для реализации общих требований к маршрутизации, наблюдаемости и безопасному сегментированию (с помощью авторизации) трафика между сервисами. Во всех этих примерах будет использоваться Kubernetes, поскольку это наиболее распространенная платформа для развертывания сервисных сетей. Однако представленные концепции применимы ко всем платформам и инфраструктурам, для которых поддерживается каждая сервисная сеть. Хотя мы рекомендуем выбирать и использовать только одну реализацию сервисной сети в технологическом стеке вашего приложения, конфигурация конференц-системы — исключительно в учебных целях — будет продемонстрирована с использованием трех различных сервисных сетей.

Маршрутизация с помощью Istio

Istio можно установить в кластер Kubernetes с помощью инструмента `istioctl`¹⁸. Основным условием использования Istio будет включение автоматического внедрения `sidecar`-прокси во все сервисы, работающие в кластере. Это можно сделать следующим образом:

```
$ kubectl label namespace default istio-injection=enabled
```

При настроенном автоматическом внедрении двумя основными пользовательскими ресурсами в вашей предстоящей работе будут `VirtualServices` и `DestinationRules`¹⁹. `VirtualService` задает набор правил маршрутизации трафика, применяемых при обращении к хосту, — например: `http://sessions`. `DestinationRule` устанавливает политики, применяемые к трафику, предназначенному для сервиса, после выполнения маршрутизации. Эти правила определяют конфигурацию балансировки нагрузки, размер пула соединений из `sidecar`-прокси, а также параметры обнаружения отклонений для выявления и исключения нездоровых хостов из пула балансировки нагрузки.

Например, чтобы обеспечить маршрутизацию к сервисам Сессия и Участник в рамках примера можно создать следующие сервисы `VirtualServices`:

```
---
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: sessions
```

¹⁸ См. <https://oreil.ly/8NyoV>.

¹⁹ Вы можете узнать больше о `VirtualServices` (<https://oreil.ly/L01fC>) и `DestinationRules` (<https://oreil.ly/0069e>) из документов Istio.

```
spec:
  hosts:
  - sessions
  http:
  - route:
    - destination:
        host: sessions
        subset: v1
---
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: attendees
spec:
  hosts:
  - attendees
  http:
  - route:
    - destination:
        host: attendees
        subset: v1
```

Также могут быть созданы следующие правила `DestinationRules`. Обратите внимание, что в `DestinationRule` для участников указаны две версии сервиса: это основа для включения канареечной маршрутизации для новой версии сервиса v2:

```
---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: sessions
spec:
  host: sessions
  subsets:
  - name: v1
    labels:
      version: v1
---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: attendees
spec:
  host: attendees
  subsets:
  - name: v1
    labels:
      version: v1
```

```
- name: v2
  labels:
    version: v2
```

Установив Istio и предварительно настроив VirtualServices и DestinationRules, можно приступать к маршрутизации трафика и вызовов API между сервисами Участник и Сессия. Начать работу действительно очень просто, хотя настройка и поддержка Istio в эксплуатируемой среде могут быть более сложными. Istio будет выполнять маршрутизацию, а также генерировать телеметрию, связанную с каждым соединением.

А теперь давайте узнаем больше о наблюдаемости на примере сервисной сети Linkerd.

Наблюдение за трафиком с помощью Linkerd

Установить Linkerd в кластер Kubernetes можно, следуя инструкциям раздела *Getting Started*²⁰. Функции телеметрии и мониторинга Linkerd включаются автоматически, не требуя внесения каких-либо изменений в конфигурацию стандартной установки. К таким функциональным возможностям наблюдаемости относятся:

- ◆ запись основных («золотых») метрик (объем запросов, коэффициент успешности и распределение задержек) для трафика HTTP, HTTP/2 и gRPC;
- ◆ запись метрик уровня TCP (входящие/исходящие байты и т. д.) для другого TCP-трафика;
- ◆ отчетность по метрикам для каждого сервиса, для пары абонент/клиент или для каждого маршрута/пути (при использовании профилей сервисов);
- ◆ генерация топологических графов, отображающих взаимосвязь между сервисами во время выполнения программы;
- ◆ отбор образцов по запросу в режиме реального времени.

Эти данные можно использовать несколькими способами:

- ◆ через Linkerd CLI — например, с помощью команд `linkerd viz stat` и `linkerd viz routes`;
- ◆ с помощью информационной панели Linkerd и готовых панелей Grafana;
- ◆ непосредственно из встроенного в Linkerd экземпляра Prometheus.

Чтобы получить доступ к возможностям наблюдаемости Linkerd, достаточно установить расширение `viz` и открыть информационную панель в локальном браузере:

```
linkerd viz install | kubectl apply -f -
linkerd viz dashboard
```

Это обеспечивает доступ к графам сервисов, отображающим поток трафика. На рис. 4.10 показана информационная панель, демонстрирующая трафик, проходя-

²⁰ См. <https://oreil.ly/dfnZ8>.

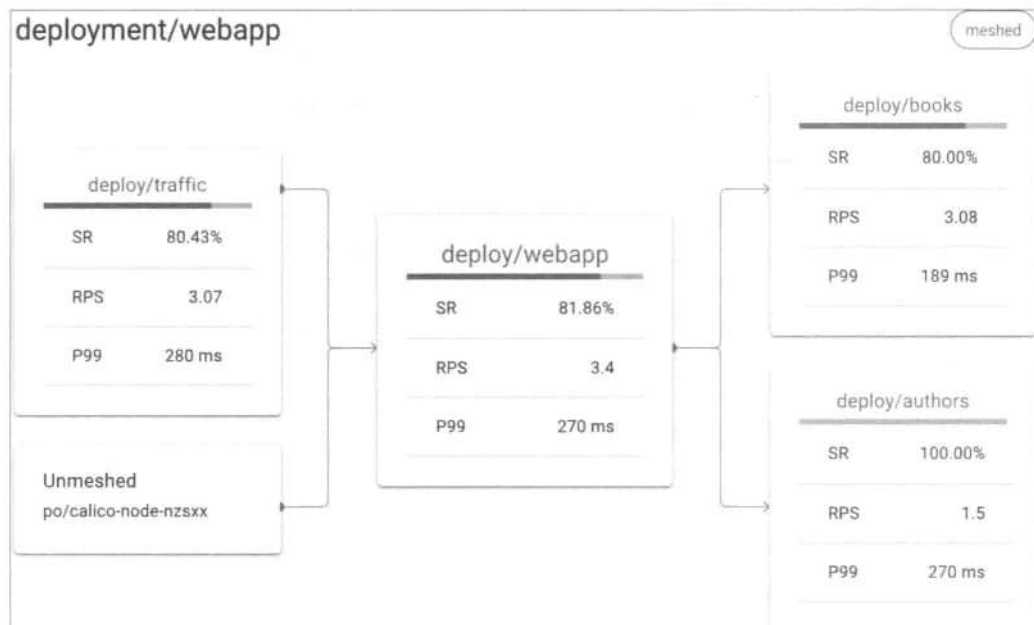


Рис. 4.10. Использование Linkerd viz для наблюдения за потоком трафика между сервисами



Рис. 4.11. Просмотр информационных панелей Linkerd viz Grafana

щий через сеть от веб-приложения (webapp) к сервисам книг (books) и авторов (authors).

Вы также можете просмотреть основные показатели трафика с помощью готовых панелей Grafana (рис. 4.11).

Использование сервисной сети для обеспечения наблюдаемости в приложениях полезно как при разработке, так и при эксплуатации. Хотя в среде эксплуатации всегда следует автоматизировать обнаружение некорректного трафика между сервисами, можно также использовать этот инструмент наблюдения за сервисной сетью для выявления случаев некорректного вызова внутренних API или сервисов.

А теперь мы рассмотрим использование политики для определения того, какие именно сервисы могут взаимодействовать друг с другом в сервисной сети, задействовав инструмент Consul компании HashiCorp.

Сегментация сети с помощью Consul

Установить и настроить Consul как сервисную сеть в кластере Kubernetes можно, выполнив указания руководства «Getting Started with Consul Service Mesh for Kubernetes»²¹. До появления микросервисов авторизация межсервисного взаимодействия осуществлялась в основном с помощью правил межсетевого экрана и таблиц маршрутизации. Consul упрощает управление межсервисной авторизацией с помощью *намерений*, позволяющих определять разрешения на межсервисное взаимодействие по имени сервиса.

Намерения управляют тем, какие сервисы могут взаимодействовать друг с другом, и обеспечиваются *sidecar*-прокси при входящих соединениях. Идентичность входящего сервиса проверяется по его TLS-сертификату клиента, и Consul предоставляет каждому сервису идентификатор, закодированный в виде сертификата TLS. Этот сертификат используется для установления и приема соединений с другими сервисами²². Затем *sidecar*-прокси проверяет, существует ли намерение, разрешающее входящему сервису взаимодействовать с целевым сервисом. Если входящий сервис не авторизован, соединение будет прервано. Намерение состоит из четырех частей:

- ◆ Исходный сервис (Source service).

Указывается сервис, инициирующий обмен данными. Это может быть полное имя сервиса или символ '*' для обозначения всех сервисов.

- ◆ Сервис назначения (Destination service).

Указывается сервис, принимающий сообщение. Им будет элемент выше по потоку (сервис), заданный в определении сервиса. Это может быть полное имя сервиса или символ '*' для обозначения всех сервисов.

²¹ См. <https://oreil.ly/RbMcy>

²² Идентификация кодируется в сертификате TLS в соответствии с документом SPIFFE X.509 Identity Document, что позволяет сервисам Connect устанавливать и принимать соединения с другими SPIFFE-совместимыми системами.

◆ Разрешение (Permission).

Определяет, разрешена ли связь между источником и пунктом назначения. Этот параметр может быть установлен в разрешающее и запрещающее значения.

◆ Описание (Description).

Необязательное поле метаданных, позволяющее связать описание с намерением.

Сейчас мы создадим первое намерение. Оно изменит политику «разрешить всё», в которой разрешен весь трафик, если он не запрещен в определенных правилах, на политику «запретить всё», в которой запрещен весь трафик и разрешены только определенные соединения:

```
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceIntentions
metadata:
  name: deny-all
spec:
  destination:
    name: '*'
  sources:
    - name: '*'
      action: deny
```

Если в поле назначения (*destination*) указать подстановочный символ (*), то это намерение предотвратит все межсервисные взаимодействия. Определив политику по умолчанию как «запретить всё», можно разрешить трафик между унаследованным сервисом конференц-системы, сервисами Участник и Сессия, определив *ServiceIntentions* CRD для каждого требуемого взаимодействия сервисов. Например:

```
---
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceIntentions
metadata:
  name: legacy-app-to-attendee
spec:
  destination:
    name: attendee
  sources:
    - name: legacy-conf-app
      action: allow
---
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceIntentions
metadata:
  name: legacy-app-to-sessions
spec:
  destination:
    name: sessions
```

```
sources:
  - name: legacy-conf-app
    action: allow
---
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceIntentions
metadata:
  name: attendee-to-sessions
spec:
  destination:
    name: sessions
  sources:
    - name: attendee
      action: allow
---
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceIntentions
metadata:
  name: sessions-to-attendee
spec:
  destination:
    name: attendee
  sources:
    - name: sessions
      action: allow
```

Применение этой конфигурации к кластеру Kubernetes позволит осуществлять эти и только эти взаимодействия между сервисами в соответствии с требованиями. Любое другое взаимодействие будет предотвращено, а вызов или запрос API — отменен.

В дополнение к намерениям Consul популярностью пользуется проект Open Policy Agent (OPA), позволяющий реализовать аналогичную функциональность в рамках сервисной сети. Пример использования OPA для настройки межсервисной политики в Istio можно найти в документации «OPA Tutorial documentation»²³.

Теперь, когда вы изучили пример конфигурации, применяемой по мере развития конференц-системы, давайте перейдем к запуску и управлению самой реализацией сервисной сети.

Развертывание сервисной сети: понимание и управление отказами

Независимо от схемы развертывания и количества экземпляров, работающих в системе или в сети, сервисная сеть, как правило, находится на критическом пути многих, если не всех, пользовательских запросов, проходящих через вашу систему.

²³ См. <https://oreil.ly/I4HsD>.

Выход из строя одного из экземпляров сервисной сети в кластере или сети обычно приводит к недоступности всей системы в радиусе действия этой сети. Поэтому темы понимания и управления отказами жизненно важны для изучения.

Сервисная сеть как единая точка отказа

Сервисная сеть часто находится на горячем пути всего трафика, что может стать проблемой с точки зрения надежности и отказоустойчивости. Очевидно, что чем больше базовая функциональность, предоставляемая сервисной сети, тем выше риск и тем серьезнее последствия сбоя. Поскольку сервисная сеть часто используется для организации релизов сервисов приложения, ее конфигурация также постоянно обновляется. Очень важно уметь при этом обнаруживать и устранять проблемы, а также минимизировать риски. Многие моменты, отмеченные в разд. «API-шлюз как единая точка отказа» главы 3, можно применить к пониманию и управлению отказами сервисных сетей.

Общие проблемы внедрения сервисной сети

Поскольку технологии сервисных сетей являются более новыми по сравнению с технологиями API-шлюзов, некоторые из общих проблем их реализации еще только предстоит выявить и распространить в сообществе разработчиков. Однако существует основной набор нежелательных антипаттернов.

Сервисная сеть в качестве ESB

С появлением плагинов для сервисных сетей или фильтров трафика, а также вспомогательных технологий, таких как Web Assembly (Wasm), все более соблазнительным становится представление о сервисных сетях как о функциональности, подобной ESB, такой как преобразование и трансляция полезной нагрузки. По всем уже описанным в этой книге причинам мы настоятельно не рекомендуем добавлять в сервисную сеть бизнес-функциональность или связывать слишком много «умных» функций с платформой или инфраструктурой.

Сервисная сеть в качестве шлюза

Поскольку многие реализации сервисной сети в той или иной форме обеспечивают входной шлюз, мы сталкивались с тем, что организации, желающие внедрить API-шлюз, вместо этого предпочитали развернуть сервисную сеть и использовать только ее функции шлюза. Мотивация вполне логична, поскольку инженеры организации понимают, что в скором времени им потребуется внедрить подобную сервисной сети функциональность, но самой большой проблемой для них является управление входящим трафиком. Однако функциональность большинства шлюзов сервисных сетей не столь богата по сравнению с полноценными API-шлюзами. Кроме того, вы, скорее всего, столкнетесь с расходами на установку и эксплуатацию сервисной сети, не получив при этом никаких преимуществ.

Слишком много сетевых уровней

Мы видели, как некоторые организации предоставляют богатый набор сетевых абстракций и функций, удовлетворяющих современные требования к межсервисному взаимодействию, но команды разработчиков либо не знают об этом, либо по каким-то причинам отказываются внедрять эти решения. По мере того как команды разработчиков пытаются реализовать сервисную сеть поверх существующих сетевых технологий, возникают дополнительные проблемы, такие как несовместимость (например, существующие сетевые технологии удаляют заголовки), увеличение задержки (из-за нескольких прокси-хопов) или реализация функциональности несколько раз в сетевом стеке (например, автоматическое отключение происходит как в сервисной сети, так и в сетевом стеке нижнего уровня). Поэтому мы всегда рекомендуем всем участвующим командам координировать свои действия и взаимодействовать с помощью решений на основе сервисной сети.

Выбор сервисной сети

Теперь, когда вы узнали о функциональности сервисной сети, развитии паттерна и технологий, а также о том, как сервисная сеть вписывается в общую архитектуру системы, возникает ключевой вопрос: как выбрать сервисную сеть для включения в стек технологий вашего приложения?

Определение требований

Как уже говорилось при обсуждении выбора API-шлюза, одним из важнейших этапов любого нового инфраструктурного проекта является определение соответствующих требований. Это может показаться очевидным, но я уверен, что вы можете вспомнить время, когда вас отвлекали блестящие технологии, волшебный маркетинг или хорошая документация по продажам!

Вы можете вернуться к *разд. «Зачем нужна сервисная сеть?»* этой главы, чтобы более подробно рассмотреть основные требования, учитываемые в процессе выбора сервисной сети. Важно задавать вопросы, ориентированные как на текущие болевые точки, так и на вашу будущую дорожную карту.

Создание или покупка?

Если сравнить эти рассуждения с решением о создании или покупке API-шлюза, то в случае сервисных сетей применение тех же аргументов менее вероятно, особенно в организациях, имеющих старые или унаследованные системы. Отчасти это можно объяснить тем, что сервисные сети являются относительно новой категорией технологий. По нашему опыту, в большинстве распределенных систем (например, больших, чем стек LAMP) частичные реализации сервисной сети будут разбросаны по всей организации: например, в одних отделах используются библиотеки для конкретного языка, в других — ESB, а в третьих — простые API-шлюзы или простейшие прокси для управления внутренним трафиком.

В целом, если вы решили использовать модель сервисной сети, мы считаем, что, как правило, лучше принять в качестве стандарта реализацию с открытым исходным кодом или коммерческое решение, а не создавать свое собственное. Представление аргументов в пользу создания или покупки ПО с использованием технологии его доставки может занять целую книгу, поэтому в этом разделе мы остановимся лишь на некоторых общих проблемах:

◆ **недооценка совокупной стоимости владения (Total Cost of Ownership, TCO).**

Многие инженеры не учитывают стоимость разработки решения, затраты на его дальнейшее обслуживание и текущие эксплуатационные расходы.

◆ **непродуманность альтернативных издержек.**

Если вы не поставщик облачных технологий или платформ, то маловероятно, что индивидуально разработанная сервисная сеть обеспечит вам конкурентное преимущество. Вы можете принести больше пользы своим клиентам, создав некоторые функциональные возможности в пределах своего общего ценового предложения.

◆ **эксплуатационные расходы.**

Непонимание того, что внедрение и эксплуатационные расходы связаны с поддержкой нескольких различных реализаций, решающих одни и те же задачи.

◆ **незнание существующих технических решений.**

Как в области компонентов с открытым исходным кодом, так и в области коммерческих платформ происходит быстрое развитие, и следить за актуальностью бывает непросто. Однако оставаться в курсе событий и быть информированным — одна из основных задач технического лидера.

Руководство ADR: выбор сервисной сети

В табл. 4.5 выделены ключевые решения руководства ADR по этой теме. Вы и ваша команда должны учитывать их при принятии решения о реализации паттерна сервисной сети и при выборе соответствующих технологий.

Таблица 4.5. Руководство ADR: чек-лист по выбору сервисной сети

Решение	Как подойти к выбору сервисной сети для организации?
Пункты обсуждения	Определили ли мы все требования, связанные с выбором сервисной сети, и расставили ли приоритеты? Определили ли мы текущие технологические решения, развернутые в этой области в организации? Знаем ли мы все ограничения, касающиеся команд и организации? Изучили ли мы нашу будущую дорожную карту в связи с этим решением? Честно ли мы подсчитали затраты для вариантов создания или покупки? Изучили ли мы текущий технологический ландшафт и знаем ли обо всех доступных решениях? Провели ли мы консультации со всеми заинтересованными сторонами и проинформировали ли их при анализе и принятии решений?

Таблица 4. (окончание)

Решение	Как подойти к выбору сервисной сети для организации?
Рекомендации	Особое внимание уделяется требованиям к уменьшению внутренней связанности между API и системой, упрощению потребления, защите API от чрезмерного использования и злоупотреблений, пониманию того, как API потребляются, и управлению API как продуктами. Необходимо задать следующие вопросы. Существует ли уже используемая сервисная сеть? Была ли собрана коллекция технологий для обеспечения схожей функциональности? Например, разработчики создали библиотеки связи между сервисами, или команда платформы/SRE развернула sidecar-прокси? Ориентируйтесь на уровень технологических навыков в вашей команде, наличие людей для работы над проектом сервисной сети, доступные ресурсы и бюджет и т. д. Важно определить все планируемые изменения, новые возможности и текущие цели, способные повлиять на управление внутренним трафиком и другие функциональные возможности, предоставляемые сервисной сетью. Рассчитайте совокупную стоимость владения (TCO) для всех существующих реализаций, подобных сервисной сети, и потенциальных будущих решений. Ознакомьтесь с известными аналитическими материалами, отчетами о тенденциях и обзорами продуктов, чтобы понять все имеющиеся на сегодняшний день решения. Выбор и развертывание сервисной сети затронут многие команды и отдельных сотрудников. Обязательно проконсультируйтесь с разработчиками, QA, архитектурным советом, командой платформы, InfoSec и пр.

Заключение

В этой главе вы узнали, что такое сервисная сеть, а также рассмотрели функциональные возможности, преимущества и проблемы, получаемые от использования этого паттерна и связанных с ним технологий:

- ◆ по сути, «сервисная сеть» — это паттерн для управления всеми межсервисными взаимодействиями в распределенной программной системе;
- ◆ на сетевом уровне прокси сервисной сети действует как полноценный прокси-сервер, принимая весь входящий трафик от других сервисов, а также иницилируя все исходящие запросы к другим сервисам;
- ◆ сервисная сеть развертывается во внутренней сети или кластере. Управление крупными системами или сетями обычно осуществляется путем развертывания нескольких экземпляров сервисных сетей. Часто каждая из них охватывает сегмент сети или бизнес-домен;
- ◆ сервисная сеть может предоставлять доступ к конечным точкам в пределах сетевой демилитаризованной зоны (ДМЗ) либо для внешних систем, либо для дополнительных сетей или кластеров, но это часто реализуется с помощью «входящего», «завершающего» или «транзитного» шлюза;
- ◆ существует множество сквозных проблем, связанных с API. Они могут быть характерны для каждого или всех ваших внутренних сервисов, в том числе: управление жизненным циклом продукта (последовательные релизы новых версий сервиса), надежность, поддержка многоязычных коммуникаций, наблюдае-

мость, безопасность, удобство сопровождения и расширяемость. Сервисная сеть может помочь в решении всех этих вопросов:

- ♦ сервисная сеть может быть реализована с помощью библиотек, специфичных для используемого языка, `sidecar`-прокси, фреймворков без прокси-сервера (`gRPC`) или технологий на базе ядра, таких как `eBPF`;
- ♦ наиболее уязвимым компонентом сервисной сети, как правило, становится плоскость управления. Она должна быть защищена, контролируема и работать как сервис высокой доступности;
- ♦ антипаттерны использования сервисной сети включают: сервисную сеть в качестве ESB, сервисную сеть в качестве шлюза и использование слишком большого количества сетевых уровней;
- ♦ решение о внедрении сервисной сети и выбор технологии для этого относятся к решениям типа 1 — стратегическим. Необходимо провести исследования, анализ требований и соответствующее проектирование;
- ♦ если вы решили использовать модель сервисной сети, то мы считаем, что обычно лучше принять в качестве стандарта реализацию с открытым исходным кодом или коммерческое решение, а не создавать свое собственное.

Независимо от решения о внедрении сервисной сети, важно продумать как внешние, так и внутренние операции и безопасность API. Этим вопросам посвящена следующая часть книги.

ЧАСТЬ III

Эксплуатация и безопасность API

В этой части книги обсуждаются проблемы эксплуатации и обеспечения безопасности системы, основанной на API:

- ♦ *глава 5* посвящена развертыванию и релизам API с использованием жизненного цикла API. Мы также рассмотрим вопросы наблюдаемости и то, как субъективные (opinionated) платформы могут помочь уменьшить количество проблем с распределенными архитектурами;
- ♦ *глава 6* рассказывает о моделировании угроз для API и о том, как мог бы вести себя человек, пытающийся совершить злонамеренные действия против ваших API;
- ♦ в *главе 7* рассматривается использование аутентификации и авторизации для защиты API.

Развертывание и релизы API

В этой главе мы начнем рассматривать вопросы перехода от проектирования, сборки и тестирования к работе в целевой среде.

Во *введении* вашему вниманию был представлен пример конференц-системы, построенной на основе однопользовательского интерфейса и приложения на стороне сервера. Развертывание обновления сервера или пользовательского интерфейса, скорее всего, повлечет за собой некоторый простой в работе системы вследствие того, что действия по ее развертыванию и релизу, вполне вероятно, тесно связаны и, возможно, неразделимы. Может также потребоваться дополнительное время на откат изменений в случае возникновения проблем с развертыванием. Опыт рассмотрения здесь некоторых вариантов использования унаследованной конференц-системы даст нам понимание того, что более слабая связанность между пользовательским интерфейсом и серверными компонентами реально облегчает развертывание и релиз.

Внедрение управления трафиком дает возможность разделить развертывание и релиз. Мы разберемся с этим вопросом более подробно, рассматривая примеры вариантов конференц-системы, доступные для развертывания изменений, и учитывая при этом, как версионность API влияет на возможности моделирования релизов в конференц-системе.

Одним из ключевых моментов при внедрении является понимание успешности изменения. Архитектуры API по своей природе носят несвязанный характер, и для успешного релиза важно обеспечить наличие необходимых метрик, журналов и трассировок. Мы рассмотрим типы метрик, которые следует учитывать, и то, как они помогают в выпуске релизов, управлении инцидентами и устранении неполадок.

В завершение мы узнаем, как конечная согласованность влияет на изменения, и где могут возникнуть ошибки на уровне приложений. Внедрение дополнительных уровней инфраструктуры, таких как прокси, требует принятия решений по кешированию и распространению заголовков. Мы рассмотрим эти моменты, а также причины, влияющие на выбор субъективной платформы.

Отделение развертывания от релиза

Чтобы извлечь максимальную пользу из этой главы, важно понимать разницу между развертыванием и релизом. *Развертывание* подразумевает перенос функциональной возможности в эксплуатируемую систему, имеющую работающий про-

цесс. Тем не менее хотя развертывание и было выполнено, новая функция пока еще не активна и не выполняется при взаимодействии с эксплуатируемой системой. Релиз же предполагает управляемую активацию новой функции, что позволяет контролировать риски, связанные с ее внедрением. Существуют различные способы разделения развертывания и релиза, и мы рассмотрим их в ближайшее время. Документ *Technology Radar* от консалтинговой компании *Thoughtworks* содержит прекрасное объяснение разницы между развертыванием и релизом:

Внедрение непрерывной доставки по-прежнему остается сложной задачей для многих организаций, и по-прежнему важно освещать такие полезные приемы, как отделение развертывания от релиза. Мы рекомендуем строго использовать термин «развертывание», когда речь идет о развертывании изменений в компонентах приложения или инфраструктуры. Термин «релиз» следует применять в тех случаях, когда изменения в функциональных возможностях предоставляются конечным пользователям и оказывают влияние на бизнес. Используя такие приемы, как переключатели функций (feature toggle) и темные запуски (dark launches), мы можем чаще внедрять изменения в эксплуатируемые системы без релиза функций. Более частое развертывание снижает риск, связанный с изменениями, а заинтересованные стороны сохраняют контроль над сроками предоставления функций конечным пользователям.

Technology Radar om Thoughtworks, 2016 г.

Одним из преимуществ перехода к архитектуре на основе API является то, что отсутствие в ней связанности позволяет командам быстро выполнять релизы изменений. Для использования этого преимущества важно учитывать, что механизмы обеспечения связанности между системами при нем остаются низкими, и риск выпуска релизов, приводящих к отказам, минимизируется.



Мы видели команды, перешедшие на архитектуру на основе API, но не выполнившие разделение развертывания и релиза. Это может работать для сильно связанных сервисов, но быстро приводит к повышению нагрузки и простоям многочисленных сервисов, если релизы приходится согласовывать со многими командами. Далее в этой главе мы рассмотрим, как можно использовать версиюность и жизненный цикл API для предотвращения проблем такого рода.

Практический пример: установка флагов функций

Для эффективного рассмотрения вопроса о том, как разделить развертывание и релиз, мы обратимся к примеру унаследованной конференц-системы. Полезно начать именно с нее, поскольку это позволит нам красиво смоделировать эволюционную архитектуру, дающую возможность контролировать скорость изменения новой инфраструктуры, представленной к настоящему моменту в книге. На рис. 5.1 показано, как унаследованная система учета посетителей и база данных будут соседствовать с модернизированным сервисом на основе API. Используя флаги функций (feature flag), контроллер теперь может на уровне кода принимать решение о выполнении запроса к внутреннему или внешнему API-сервису.

Флаги функций обычно размещаются в хранилище конфигурации вне работающего приложения и позволяют развертывать код с отключенной функцией. Когда команда (или владелец продукта) готова включить функцию, они могут переключить флаг, что приведет к выполнению другой ветви кода. Гранулярность может быть

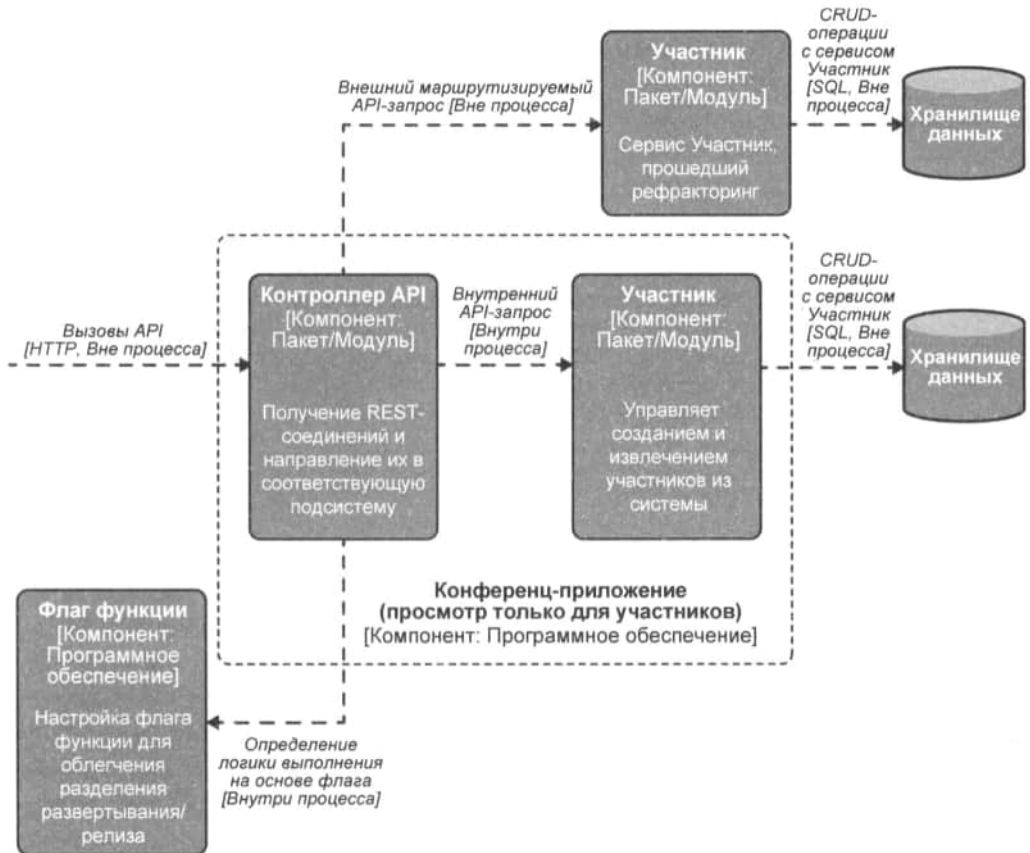


Рис. 5.1. Схема контейнера конференц-приложения для участников с применением флагов функций

обеспечена как на уровне каждого пользователя, так и быть более грубой — например, представлять собой просто глобальное включение определенной возможности. Далее приведен пример псевдокода из популярного Java-инструмента для создания флагов функций LaunchDarkly — в нем данные пользователя находятся в современном хранилище:

```
LDUser user = new LDUser("jim@masteringapi.com");
boolean newAttendeesService =
    launchDarklyClient.boolVariation("user.enabled.modern", user, false);
if (newAttendeesService) {
    // Извлечение участника из современного хранилища
}
else {
    // Извлечение участника из унаследованного хранилища
}
```

Такой подход позволяет перевести небольшую партию пользователей на новую систему и убедиться, что для них функциональность продолжает работать. Если в процессе миграции что-то пойдет не так, переключатель можно просто переки-

нуть обратно, а если всё идет штатно, развертывание продолжается до достижения 100%-ной миграции. Как и в случае с другими сервисами, пересекающимися с приложением, сбой в работе сервиса флагов функций при неправильном управлении может привести к катастрофе, став потенциальной единой точкой отказа. Уменьшить этот эффект можно, используя последнее известное значение в кеше или предоставляя значения по умолчанию.



Флаги функций помогают облегчить разделение развертывания и релиза кода. Но необходимо очищать флаги функций и всегда создавать их с уникальными именами. После завершения миграции код флага функций должен быть полностью удален. В качестве примера неправильной установки флагов функций можно привести ситуацию в компании Knight Capital¹, где неудачное развертывание из-за повторного использования флага функций обошлось в тысячи долларов в секунду, принеся в итоге убытки в 460 млн долларов.

Управление трафиком

Одним из преимуществ перехода на архитектуру на основе API стала возможность быстрой итерации и развертывания новых изменений в сервисе Участник. Для модернизированной части архитектуры у нас также создана концепция трафика и маршрутизации. Это позволяет управлять трафиком в двух местах: на входе в API-шлюз или в рамках конструкций, определенных в сервисной сети для формирования трафика.

Для Kubernetes и систем на основе сервисной сети развертывание выглядит примерно следующим образом:

1. Создается запрос на внесение в приложение необходимых изменений и, как только он будет одобрен и произойдет слияние, автоматически запускается сборка для развертывания.
2. Конвейер сборки создает новый образ с помощью Docker или Open Container Initiative.
3. Новый образ вносится в реестр контейнеров.
4. Производится запуск нового развертывания образа в целевой среде.

По умолчанию Kubernetes заменит работающее развертывание на новое. Позже в этой главе мы рассмотрим методы поэтапного релиза нового пода для активного разделения развертывания и релиза. На этом работа по развертыванию будет завершена, после чего следует другой набор инструкций по конфигурированию релиза работающей системы. Конфигурация также может быть разделена на несколько этапов, и этот механизм можно использовать для настройки различных стратегий релизов. Но прежде чем перейти к рассмотрению вариантов структурирования релиза для управления трафиком, стоит разобраться с типами существующих в системе API релизов.

¹ См. <https://oreil.ly/WBnEI>.

Практический пример: моделирование релизов в конференц-системе

В разд. «Семантическое версионирование» главы 1 мы рассмотрели идею различных стратегий версионирования, связанных с API. При организации релизов может оказаться полезным объединить идеи, изложенные в подходе семантического версионирования, с жизненным циклом API.

Жизненный цикл API

Пространство API быстро развивается, но одно из наиболее четких представлений о жизненном цикле версий содержится в ныне уже архивированных стандартах PayPal API². Подход к моделированию жизненного цикла представлен в табл. 5.1.

Таблица 5.1. Жизненный цикл API (адаптировано из стандартов PayPal API)

Планирование	С технологической точки зрения предоставление доступа к API достаточно просто, однако когда API открыт и запущен в эксплуатацию, у вас появляется множество потребителей API, которыми необходимо управлять. Этап планирования заключается в оповещении о том, что вы создаете API, и сборе первоначальных отзывов о дизайне и форме API от потребителей. Это позволяет обсудить API и область его применения, а также учесть все ранние проектные решения
Бета-версия	Релиз версии нашего API для пользователей, дающей возможность начать интеграцию. Однако, как правило, это делается с целью получения обратной связи и улучшения API. На этом этапе производитель оставляет за собой право нарушить совместимость, поскольку это не версионный API. Этап бета-версии дает возможность быстро получить обратную связь от потребителей по поводу проекта API до принятия решения о его структуре. Цикл обратной связи и изменений позволяет производителю избежать большого количества основных версий в начале жизненного цикла API
Запуск	Теперь API версионирован и запущен в производство. Любые изменения с этого момента будут представлять собой версионные изменения. Должна существовать только одна запущенная версия API, помеченная самой последней комбинацией мажорных и минорных версий. При релизе новой версии текущий API переходит в разряд устаревших
Устаревание	Когда API устаревает, он все еще доступен для использования, но значительные новые разработки не должны вестись на его основе При релизе минорной версии нового API предыдущая будет считаться устаревшей только на короткое время, до завершения проверки нового API в среде эксплуатации. После успешной валидации новой версии минорная версия переходит в разряд выведенных из эксплуатации, поскольку новая версия обратно совместима и может работать с теми же функциями, что и предыдущая версия API Когда происходит релиз мажорной версии API, предыдущая версия становится устаревшей. Скорее всего, это будет происходить в течение нескольких недель или месяцев, поскольку потребителям должна быть предоставлена возможность перейти на новую версию. Вероятно, будет организована связь с потребителями, составлено руководство по миграции, будут отслеживаться метрики и использование устаревшего API
Вывод из эксплуатации	API выведен из эксплуатации и более не доступен

² См. <https://oreil.ly/QMCKz>.

Жизненный цикл помогает потребителю полностью понять, чего ожидать от изменения API. При использовании семантического версионирования в сочетании с жизненным циклом API потребителю необходимо знать только основную версию API. Минорные и патч-версии будут получены без необходимости обновления на стороне потребителя и не нарушат совместимость.

Рассмотрев жизненный цикл API и порядок управления трафиком, мы приступаем к изучению типов изменений и поиску наиболее подходящего способа выполнения релизов новых версий API.

Сопоставление стратегий релизов с жизненным циклом

Мажорные изменения наиболее значимы для потребителей API. Но чтобы использовать новую версию своего программного обеспечения, взаимодействующего с API, потребители должны его активно обновлять. Согласно определению жизненного цикла это означает, что мы должны одновременно запускать и новую, и устаревшую версию API в течение значительного отрезка времени, чтобы дать потребителям возможность обновиться и перейти на новую версию. Это позволяет им сделать однозначный выбор в отношении времени обновления. Один из способов сделать это — добавить версию в URL:

```
GET /v1/attendees
```

Добавление версии практично и легко различимо для потребителя. Однако это не будет частью ресурса, и в некоторых группах не считается соответствующим RESTful-формату. Альтернативным подходом может служить наличие заголовка, описывающего основную версию. Заголовок будет влиять на маршрутизацию на входе в кластер:

```
GET /attendees
Version: v1
```

Минорные изменения свободны от ограничений, накладываемых мажорными изменениями. Для таких изменений можно развернуть новую минорную версию API, не принимая производственный трафик, а затем использовать стратегию релизов для внедрения новой версии. Это не потребует от потребителя каких-либо изменений в коде. Патч-изменения происходят по аналогичной схеме, поскольку они совершенно не меняют форму спецификации API. Для обеспечения соответствующей прозрачности релизов стоит подумать о введении дополнительных средств контроля в процесс сборки, чтобы исключить случайное внесение разрушающих изменений.

В разд. «Спецификация и версионирование OpenAPI» главы 1 мы рассмотрели использование `openapi-diff` для выделения изменений между спецификациями. В случае, если спецификация не является обратно совместимой, сборка должна завершиться неудачей и предотвратить проникновение критического изменения в архитектуру без осознанной корректировки на месте. Большинство релизов API будут представлять собой незначительные изменения или исправления, где слабая связанность становится основной задачей для производителя и потребителя.

Если потребитель и производитель сильно связаны друг с другом, принадлежат одной команде и всегда работают совместно, то версионирование и жизненный цикл API не будут иметь решающего значения. В этой ситуации важно рассмотреть стратегию релиза, позволяющую выпускать оба компонента вместе, с контролем трафика на входе. Как правило, для этого сценария хорошо подходят сине-зеленые модели. Их мы рассмотрим позже, в разд. «Стратегии релизов».

Руководство ADR: отделение релиза от развертывания с помощью управления трафиком и флагов функций

Руководство ADR, приведенное в табл. 5.2, полезно при рассмотрении вопроса о том, как создать ADR, чтобы отделить релиз от развертывания.

Таблица 5.2. Руководство ADR: отделение релиза от развертывания с помощью управления трафиком и флагов функций

Решение	Как отделить релиз от развертывания?
Пункты обсуждения	Возможно ли разделить развертывание и релиз в существующих системах, работающих в настоящее время? Какова степень связанности между потребителем и производителем в системе? Есть ли у вас конвейер сборки, дающий возможность реализовать требования слабой связанности управляемых трафиком API, обеспечив проверку совместимости?
Рекомендации	Начните с работы по разделению развертывания и релиза существующего программного обеспечения. Это позволит реализовать эволюционную архитектуру и упростить существующую систему. Флаги функций — хороший способ создания такого разделения. Если компания ранее не использовала флаги функций, обязательно ознакомьтесь с рекомендуемой практикой и избегайте потенциальных «подводных камней». Без тщательной проработки флаги функций могут превратиться в единую точку отказа. Проанализируйте тип связанности между API в архитектуре и определите правильную стратегию релизов для данной ситуации

В следующем разделе мы рассмотрим различные типы доступных стратегий релизов.

Стратегии релизов

Когда вы адекватно разделили развертывание и релиз, можно рассмотреть механизмы управления постепенным внедрением новых функциональных возможностей. Важно выбрать стратегию релизов, позволяющую снизить риск для эксплуатируемой системы. Снижение риска достигается путем проведения теста или эксперимента с небольшой долей трафика и проверки результата. При успешном результате выполняется релиз всех триггеров трафика. Некоторые стратегии подходят для определенных сценариев лучше, чем другие, и в разной степени требуют дополнительных сервисов и инфраструктуры. Рассмотрим несколько вариантов, популярных в инфраструктуре на базе API.

Канареечный релиз

При канареечном релизе³ представляется новая версия программного обеспечения и на нее направляется небольшой процент трафика. Суть организации канареечного релиза показана на рис. 5.2. Так, на этапе До у нас имеются шлюз, унаследованный конференц-сервис v1.0 и сервис Участник версии 1.0. При этом существует концепция разделения трафика между унаследованным сервисом конференций и сервисом Участник, различающаяся в зависимости от целевой платформы. На этапе развертывания появляется новая версия 1.1 сервиса Участник, и в момент релиза мы можем начать направлять часть трафика на сервис Участник версии 1.1.

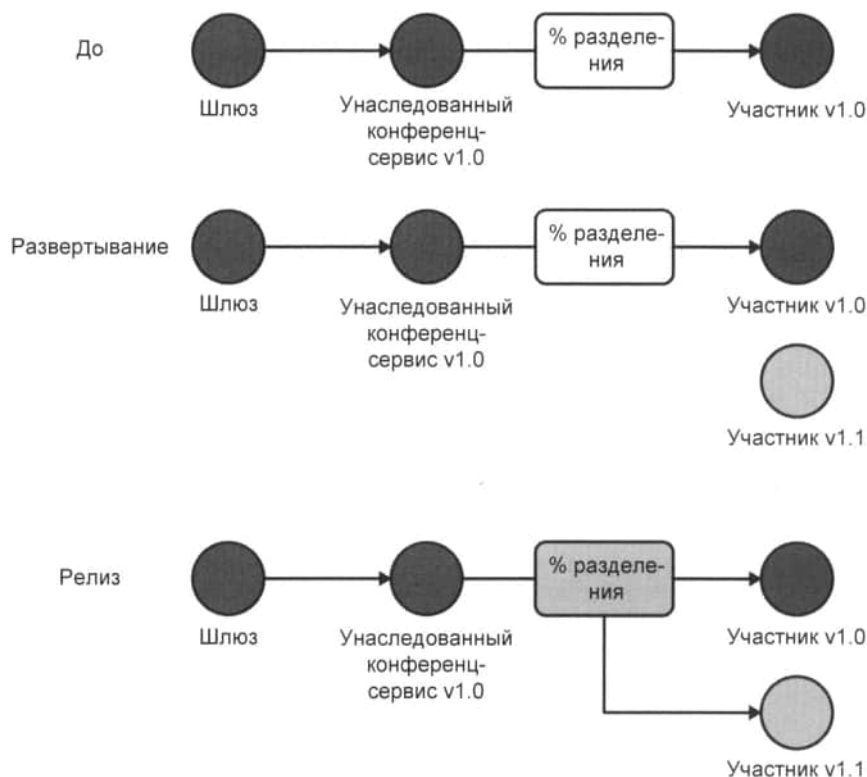


Рис. 5.2. Развертывание сервиса Участник с использованием канареечного подхода

В Kubernetes разделение трафика может быть достигнуто путем внедрения нового пода для сервиса (эта идея будет рассмотрена далее — в разд. «Практический пример: выполнение внедрения при помощи Argo Rollouts»). Однако при этом довольно сложно получить небольшой процент разделения — т. е. если вы хотите иметь показатель, равный 1%, вам нужно запустить 99 подов первой версии и один под версии 2. Для большинства ситуаций это нецелесообразно.

³ Назван в честь канареек, которых первыми спускали в угольные шахты, чтобы определить наличие опасных газов.

В сервисных сетях и API-шлюзах перераспределение трафика позволяет постепенно переводить или выполнять миграцию трафика с одной версии целевого сервиса на другую. Например, когда новая версия сервиса (v1.1) развернута наряду с первоначальной (v1.0), перераспределение трафика позволяет проводить канареечное тестирование или канареечный релиз нового сервиса, сначала направляя на версию 1.1 лишь небольшой процент пользовательского трафика, например 1%, а затем со временем переводя на новый сервис весь трафик. Это позволяет отслеживать работу нового сервиса и искать технические проблемы, такие как увеличение задержки или частоты ошибок, а также выявлять желаемый эффект для бизнеса — например, увеличение ключевых показателей эффективности, таких как коэффициент конверсии клиентов или средняя стоимость оформления покупки. Перераспределение трафика дает возможность проводить A/B или мультивариантные тесты, используя разделение трафика, предназначенного для целевого сервиса, между несколькими его версиями. Например, можно разделить трафик 50/50 между версиями 1.0 и 1.1 целевого сервиса и посмотреть, какая из них работает лучше в течение определенного периода времени.

Поскольку сервисная сеть участвует во всех коммуникациях между сервисами, вы можете реализовать эти методы выполнения релизов и экспериментов для любого сервиса в своем приложении. Например, можно сделать релиз новой версии сервиса Сессия, реализующей внутреннее кеширование расписания сессий конференции для участников. Вы сможете отслеживать оба варианта KPI для бизнеса — например, частоту просмотра и взаимодействия участника с расписанием сеансов, а также операционные индикаторы уровня обслуживания (Service-Level Indicators, SLI) — например, снижение загрузки процессора в сервисе.



Отделение развертывания от релиза: канареечный подход в любой ситуации

С ростом популярности прогрессивной доставки⁴, а также с развитием требований в рамках предшествующей ей непрерывной поставки, разделение развертывания и релиза сервиса (и соответствующего API) приобрело характер весьма важной технологии. Возможность канареечного релиза сервисов или проведения A/B-тестов может дать конкурентное преимущество вашему бизнесу как в плане снижения рисков неудачного релиза, так и в смысле более эффективного понимания требований клиентов (подробнее об этом рассказано в главе 9).

Там, где это целесообразно, канареечный релиз представляется отличным вариантом, поскольку процент трафика, подвергающегося воздействию канареечного варианта ПО, строго контролируется. Некоторая сложность заключается в том, что в системе должен быть организован хороший мониторинг, позволяющий быстро выявить проблему и при необходимости выполнить откат (который может быть автоматизирован). Преимущество же канареечного подхода состоит в том, что создается только один новый экземпляр, в то время как в таких стратегиях, как синее-зеленое развертывание, требуется полный второй стек сервисов. Это позволяет снизить затраты и операционные сложности, связанные с параллельной работой двух сред.

⁴ См. <https://oreil.ly/9qVQc>.

Зеркалирование трафика

Помимо использования *разделения* трафика для проведения экспериментов, можно также использовать *зеркалирование* трафика для его копирования или дублирования и отправки на дополнительную точку или ряд точек. При зеркалировании трафика результаты дублирования запросов обычно не возвращаются вызывающему сервису или конечному пользователю. Вместо этого ответы просто оцениваются на предмет корректности — например, сравниваются результаты, выданные существующим и прошедшим рефракторинг сервисом. Или просматриваются метрики эксплуатационных свойств при обработке запроса новой версией сервиса — например, задержка ответа или требуемый процессор. Использование зеркалирования трафика позволяет осуществлять «темный» релиз сервисов, когда пользователь остается в неведении относительно нового релиза, но вы можете вести внутреннее наблюдение для достижения необходимого эффекта. Основное отличие зеркалирования трафика от канареечного релиза заключается в том, что на этапе релиз/эксперимент зеркалирование позволяет дублировать запрос к сервису Участник v1.1. На рис. 5.3 этапы **До** и **Развертывание** идентичны канареечному релизу, поэтому «темное» развертывание часто называют *специализированным канареечным подходом*.

Реализация зеркалирования трафика на периферии систем с годами становится все более популярной, а сервисная сеть позволяет эффективно и последовательно реа-

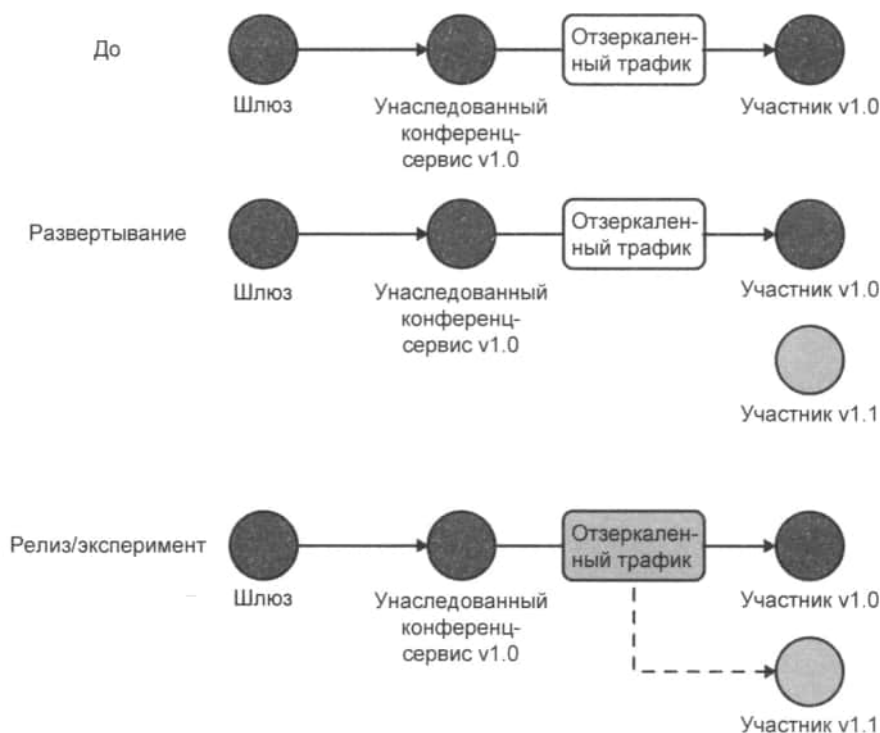


Рис. 5.3. Развертывание сервиса Участник с использованием зеркалирования трафика

лизовать этот подход во всех внутренних сервисах. При рассмотрении примера с релизом новой версии сервиса Участник, содержащей реализацию внутреннего кеширования, «темный» запуск этого сервиса позволит вам оценить операционную производительность релиза, но не его влияние на бизнес-составляющую.

Сине-зеленое развертывание

Сине-зеленое развертывание обычно реализуется в точке архитектуры, где используется маршрутизатор, шлюз или балансировщик нагрузки, за которым располагается полноценная «синяя» среда и «зеленая» среда. Синяя среда представляет собой текущую работающую среду, а зеленая — следующую версию стека. Зеленая среда перед переключением на реальный трафик проверяется, и при ее запуске трафик с синей перенаправляется на нее. Синяя среда при этом становится как бы «выключенной», но если обнаруживается проблема, система имеет возможность быстро откатиться назад.

На рис. 5.4 унаследованный сервис конференций и сервис Участник имеют версию 1.0 и представляют собой нашу синюю модель. В процессе развертывания необходимо запустить унаследованный сервис конференций v1.1 и сервис Участ-

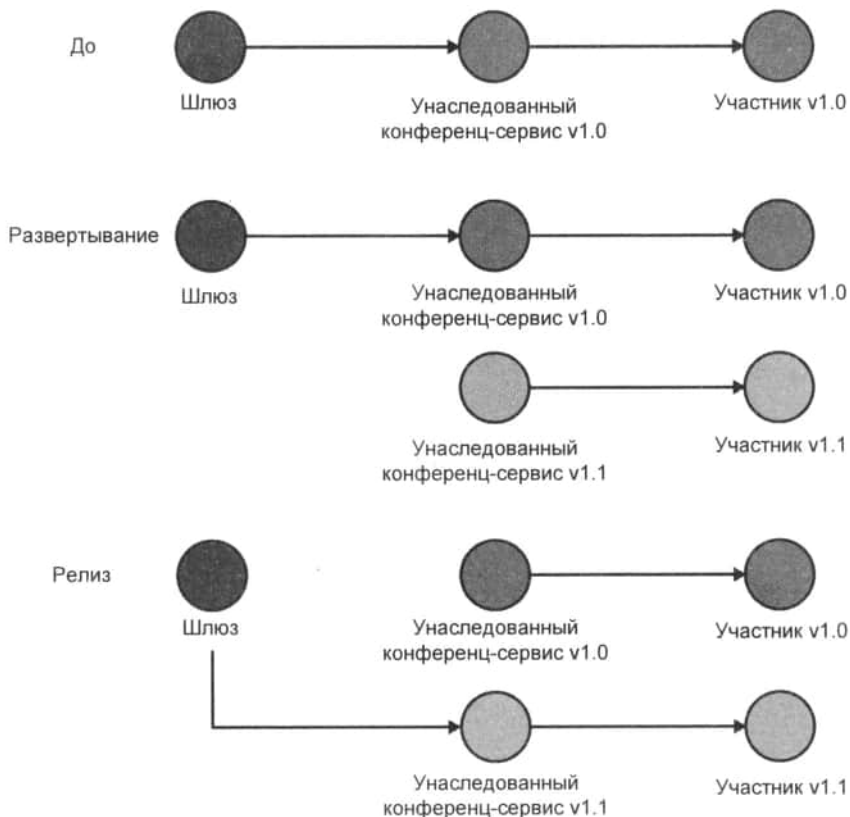


Рис. 5.4. Развертывание сервиса Участник с использованием сине-зеленого подхода

ник v1.1 одновременно, создав «зеленую» среду. На этапе релиза конфигурация обновляется, чтобы шлюз указывал на зеленую среду.

Сине-зеленый подход хорошо работает благодаря своей простоте, и для сервисов с сильной связанностью стал одним из лучших вариантов развертывания. Кроме того, проще управлять постоянными сервисами, хотя в случае отката все равно нужно быть осторожным. Учитывать надо и то, что для холодного запуска среды, параллельной с текущей активной средой, требуется вдвое больше ресурсов.

Практический пример: развертывание с помощью Argo Rollouts

Рассмотренные стратегии развертывания приносят немалую пользу, но само их внедрение — это задача, которой не хотелось бы управлять вручную. Именно в этом случае имеет смысл воспользоваться инструментом Argo Rollouts⁵, способным практически помочь при решении некоторых из обсуждаемых проблем. Используя Argo, можно определить Rollout CRD (Custom Resource Definition, пользовательское определение ресурса), представляющее собой стратегию развертывания новой версии v1.2 вашего API Участник на основе канареечного подхода. CRD позволяет Argo расширить API Kubernetes, чтобы он мог поддерживать поведение при развертывании. Вообще, CRD — популярный паттерн в Kubernetes. Он дает пользователю возможность взаимодействовать с одним API, расширяя его для поддержки различных функциональных возможностей.

Rollout CRD представляет собой комбинацию стандартного Kubernetes Deployment CRD со стратегией внедрения новых функций. В приведенной далее конфигурации YAML мы запускаем пять подов API Участник и определяем канареечный подход к внедрению новой функциональной возможности. При запуске развертывания 20% подов будут заменены на новую версию. Элемент {} в строке `pause:` предписывает Argo ожидать подтверждения от пользователя, прежде чем продолжить работу:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: attendees
spec:
  replicas: 5
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {}
        - setWeight: 40
        - pause: {duration: 10}
        - setWeight: 60
```

⁵ См. <https://oreil.ly/ljj6y>.

```

- pause: {duration: 10}
- setWeight: 80
- pause: {duration: 10}
revisionHistoryLimit: 2
selector:
  matchLabels:
    app: attendees-api
template:
  metadata:
    labels:
      app: attendees-api
spec:
  containers:
    - name: attendees
      image: jpgough/attendees:v1

```

После установки Argo на наш кластер и применения описанной конфигурации на кластере будут запущены пять подов сервиса Участник в версии 1. Очень приятная особенность Argo заключается в том, что его информационная панель помогает наглядно представить текущий статус внедрения. На рис. 5.5 показана начальная точка развертывания с пятью запущенными подами сервиса Участник v1.

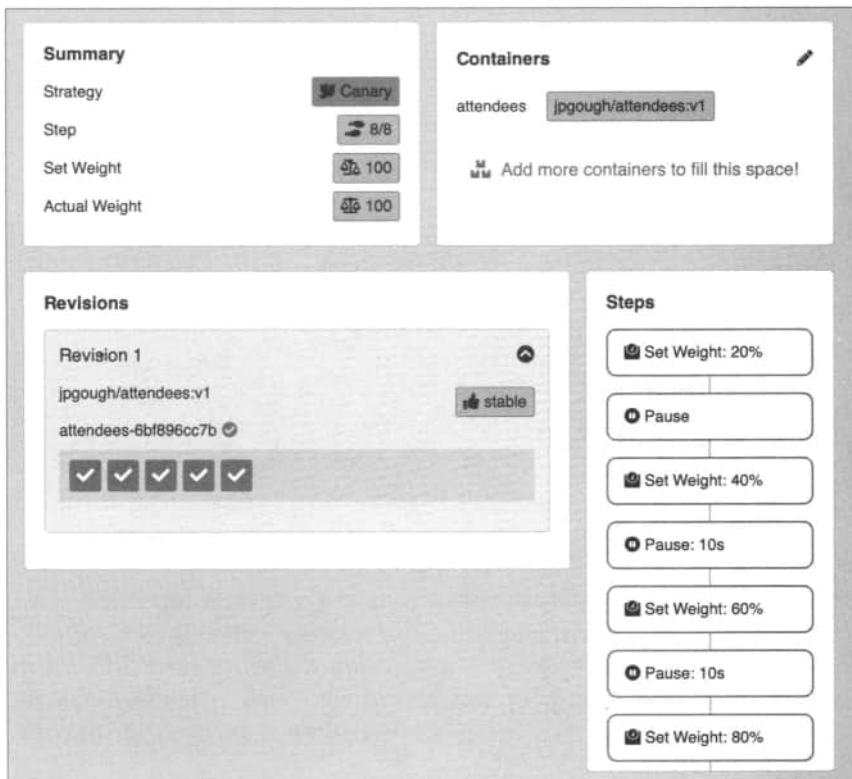


Рис. 5.5. Информационная панель Argo: начальная точка развертывания с пятью запущенными подами сервиса Участник v1

Выполнение следующей команды вводит в платформу канареечную версию 1.2:

```
kubectl argo rollouts set image attendees attendees=jpgough/attendees:v1.2
```

Информационная панель, приведенная на рис. 5.6, показывает, что релиз находится на первом этапе стратегии, а вес канареечной версии для участников версии 1.2 составляет 20%. Как можно видеть, сейчас развертывание остановлено на этапе **Pause**, ожидая запуска продвижения вручную для продолжения развертывания либо из пользовательского интерфейса, либо из командной строки. Кроме того, при возникновении проблем есть возможность быстро откатиться от канареечного релиза.

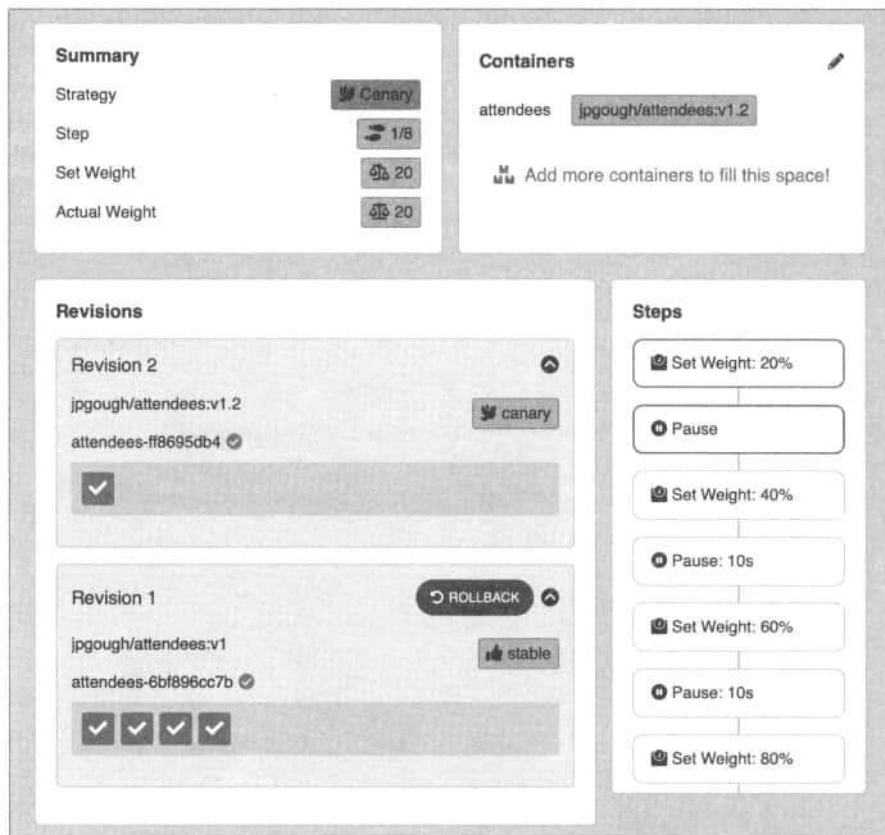


Рис. 5.6. Информационная панель Argo. канареечный релиз

В этом простом примере мы рассмотрели только уровень Kubernetes, однако существует возможность полной интеграции с функциями сервисной сети для управления развертыванием. Также возможна интеграция с входными шлюзами, такими как NGINX и Ambassador, для координации управления трафиком при релизе. Такие инструменты, как Argo, делают развертывание и релизы на основе трафика весьма привлекательными.

В дополнение к выполняемым вручную действиям по продвижению, описанным в приведенном примере, можно также управлять продвижением на основе анализа

метрик. Далее приведен пример шаблона `AnalysisTemplate`, использующего метрики `prometheus` для наблюдения за степенью успешности канареечного развертывания. Этот этап анализа может быть представлен в определении `Rollout`, что позволит внедрить проект, если будут выполнены критерии успешности процесса:

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
spec:
  args:
    - name: service-name
    - name: prometheus-port
      value: 9090
  metrics:
    - name: success-rate
      successCondition: result[0] >= 0.95
      provider:
        prometheus:
          address: "http://prometheus.example.com:({args.prometheus-port})"
```

Коэффициент успешности — достаточно простая метрика, однако бывают ситуации, когда отказ API свидетельствует не о неисправности инфраструктуры, а о запросе клиента. Давайте рассмотрим некоторые ключевые принципы, важные с точки зрения API. Их можно использовать как для управления предприятием, так и для разработки стратегии внедрения.

Мониторинг успешного выполнения и выявление отказов

Вернемся к примеру унаследованной конференц-системы, описанной во *введении*, и рассмотрим, как исследовать проблему в одном приложении. У одного сервиса есть один файл журнала для отслеживания запросов и их обработки приложением. И есть только одно приложение, по которому можно судить об общем состоянии работоспособности процесса на сервере. Выделение нескольких сервисов — например, сервиса Участник, приводит к увеличению операционной сложности. Большое количество *переходов* между сервисами создает вероятность возникновения отказов, и вручную определить, что именно вышло из строя, вскоре становится сложно.

Три столпа наблюдаемости

Архитектуры на основе API не связаны и без соответствующей поддержки сложны для рассмотрения и устранения неисправностей. Наблюдаемость обеспечивает прозрачность системы, позволяя в любой момент времени получить полное представление о происходящем. Наблюдаемость лучше всего описывается тремя основными

принципами. Вот операционный минимум, необходимый для рассуждений о распределенной архитектуре:

- ♦ *метрики* — это измерения, проводимые через регулярные промежутки времени и представляющие собой важный элемент общего состояния платформы. Они могут находиться на разных уровнях платформы, а свобода структуры означает, что платформа может сама определять, какие метрики важно фиксировать. Например, в Java-платформе можно фиксировать загрузку процессора, текущий размер кучи, время паузы сборки мусора (и т. д.);
- ♦ *журналы (логи)* содержат подробную информацию об обработке данных конкретным компонентом, и зачастую качество журналов тесно связано с создающим их компонентом приложения или инфраструктуры. Формат журнала существенно влияет на удобство поиска и обработки содержащихся в нем данных, а структурированный журнал способствует более эффективному поиску и извлечению нужных данных. В распределенных системах одних только журналов обычно недостаточно, и их, как правило, лучше исследовать с добавлением контекста, обеспечиваемого трассировками и метриками;
- ♦ *трассировка* очень важна при переходе к распределенной архитектуре, поскольку позволяет отслеживать прохождение каждого запроса через все компоненты взаимодействия архитектуры. Например, если запрос не выполняется, трассировка позволит быстро найти в архитектуре давший сбой конкретный компонент. Трассировка осуществляется путем добавления уникального заголовка как можно ближе к месту отправления запроса — этот заголовок распространяется во всех последующих обработках этого запроса. Если контекст запроса переходит к другому типу инфраструктуры (например, к очереди), то уникальный заголовок будет записан в «конверт» сообщения.

Более подробно эта тема раскрыта в книге Синди Шридхаран «Distributed Systems Observability»⁶.



Внедрения трех указанных основных принципов в рамках всей платформы недостаточно. В разд. «Считывание сигналов» мы рассмотрим, как использовать три столпа наблюдаемости для управления инфраструктурой, связанной с платформой API.

Изучение трех столпов наблюдаемости лучше всего начать с проекта OpenTelemetry⁷. Проект предлагает открытый стандарт Cloud Native Computing Foundation (CNCF), предотвращающий блокировку производителей и обеспечивающий максимально возможную совместимость. Несмотря на то что стандарты метрики и трассировки уже созданы и стабильны, журналирование — несколько более сложная проблема (из-за огромного количества различных возможных поставщиков данных), но и она решается в рамках проекта OpenTelemetry.

⁶ См. <https://oreil.ly/ZVDL0>.

⁷ См. <https://oreil.ly/fJPPd>.

Важные метрики для API

Понимание того, какие метрики важны для платформы API, является ключевым решением. Оно поможет обнаружить сбои на ранней стадии и, возможно, даже предотвратить их. Вы можете измерять и собирать широкий спектр различных метрик, но некоторые из них будут зависеть и от вашей платформы. Метрики метода RED (Rate, Error, Duration — скорость, ошибка, продолжительность) часто упоминаются в качестве одного из подходов к оценке архитектуры сервисов на основе трафика. Отчасти привлекательность этих метрик заключается в их способности дать хорошее представление о том, что происходит в конкретный момент времени: скорость показывает, сколько запросов в секунду обрабатывает сервис (его пропускную способность), мы также узнаем, какие ошибки возвращаются и какова продолжительность (или задержка) каждого запроса. В мире SRE (Site Reliability Engineering, проектирование надежности объекта) эти метрики помогают вывести «четыре золотых сигнала»⁸: задержка, трафик, ошибки и насыщение (Latency, Traffic, Errors, Saturation).

Возможно, один из самых больших недостатков методов RED и «золотые сигналы» заключается в том, что легко применить правила и упустить более широкий контекст (или понимание) системы. Например, все ли ошибки, возникающие при работе с API, вызваны тем или иным сервисом в цепочке запросов? Для API очень важен контекст ошибки — например, ошибка диапазона 5XX важна, т. к. указывает на сбой, вызванный компонентом инфраструктуры или сервисом. Ошибка диапазона 4XX не относится к проблемам сервиса и скорее будет находиться в сфере ответственности клиента, но можно ли просто игнорировать этот код ошибки? Серия ошибок 403 Forbidden может свидетельствовать о том, что злоумышленник пытается получить доступ к данным, на которые не имеет права. Это один из примеров того, почему контекст имеет решающее значение, а время, потраченное на выяснение того, какие показатели важны, выводит рассуждения об API за рамки RED-метрик.

Важные метрики должны быть привязаны к оповещениям, чтобы обеспечить оперативное решение проблем (или потенциальных проблем). При настройке оповещений необходимо соблюдать осторожность, чтобы избежать ложных срабатываний. Например, если предупреждение генерируется при низкой активности или ее отсутствии, то оно может срабатывать в праздничные или выходные дни. Помочь в этом может только планирование такого оповещения на основные рабочие часы, а также привязка его к текущему количеству входов на сайт.

В нашем исследовании конференц-системы необходимо фиксировать следующие важные метрики:

- ◆ количество запросов в минуту от участников;
- ◆ средняя задержка ответов, входящая в цель уровня обслуживания (Service-Level Objective, SLO). Если задержка начинает значительно отклоняться от нормы, это может быть первым признаком проблемы;

⁸ См. <https://oreil.ly/iv1bJ>.

- ♦ поступление кода 401 из системы CFP может свидетельствовать о компрометации поставщика или краже токена;
- ♦ измерение доступности и времени безотказной работы сервиса Участник;
- ♦ использование памяти и процессора приложениями;
- ♦ общее количество участников в системе.

Считывание сигналов

До сих пор мы обсуждали наблюдаемость и ее важность, а также назначение каждого компонента. Рассмотрели также некоторые ключевые метрики для API, но при этом предупредили, что одной их реализации без контекста недостаточно. Мы уже упоминали о сборе метрики работающего приложения — такой как время сбора мусора. Увеличение времени, затрачиваемого на сбор мусора, может быть ранним симптомом того, что ваше приложение приближается к сбою. Сборщики мусора обычно затрачивают время на приостановку работы приложений, что, в свою очередь, приводит к задержке запросов и может повлиять на общую задержку системы. Заметить это на ранней стадии — то же самое, как если бы двигатель автомобиля издавал странный звук: он все еще работает, но что-то в нем не так.

Для считывания сигналов очень полезно установить ожидаемый или базовый уровень, а затем проводить измерения в этом диапазоне и предупреждать о выходе за его пределы, что поможет выявить проблему. Следующим шагом должно стать наблюдение за фактическим влиянием метрики задержки API — эквивалент того, что теперь в автомобиле загорается индикатор ошибки. Чем раньше вы сможете распознать сигналы о потенциальной проблеме, тем меньше вероятность того, что она будет иметь последствия для клиента. Если пренебречь обеими этими метриками, то выполнение приложения в конце концов прекратится и команда будет вынуждена искать способы его восстановления.

Понимание работы программного обеспечения и его связи с ключевыми метриками помогает создать зрелую операционную платформу с ранним выявлением и надежным решением проблем. В распределенных архитектурах отказы неизбежны, и в случае сбоя трассировка станет первым шагом к поиску первопричины. Нам приходилось сталкиваться с тем, что на поиск причины проблемы без таких инструментов, как трассировка, уходили часы, а разработчики, превратившись в детективов, просматривали журналы, пытались найти подсказки для выявления виновника. Еще одним ключевым моментом является скорость реагирования команды на различные события. Если впервые откажет весь трафик API, это вызовет стресс (и, возможно, повлияет на бизнес).

Прикладные решения для эффективных релизов программного обеспечения

Распределенные архитектуры создают при релизе программного обеспечения те или иные проблемы и требуют изменений на уровне приложений. В этом разделе

мы рассмотрим некоторые проблемы, возникающие при релизе в распределенной архитектуре, и способы их решения.

Кеширование ответов

Кеширование ответов может стать настоящей проблемой, когда речь идет о компонентах приложений, в частности о шлюзах и прокси-серверах. Рассмотрим следующий сценарий. Мы пробуем выполнить канареечный релиз сервиса Участник, и, похоже, все идет хорошо, поэтому мы переходим к развертыванию всех новых сервисов. Однако сервис, выполняющий вызов `GET /attendees`, использовал прокси, который теперь скачет, выдавая везде коды группы 500. Оказалось, что кешированный результат маскировал тот факт, что наше новое программное обеспечение было сломано.

Чтобы избежать кеширования результатов, важно установить на клиенте, выполняющем GET-запрос, заголовок: `Cache-Control: no-cache, no-store`. В конце концов кеш закончится, и мы достигнем согласованного состояния.

Распространение заголовков на уровне приложений

Любые API-сервисы, завершающие API-запрос и создающие запрос к другому сервису, должны копировать заголовки из завершенного запроса в новый. Например, для обеспечения соблюдения распределенной трассировки в запрос необходимо добавить любые заголовки, связанные с трассировкой или наблюдаемостью.

Для заголовков аутентификации и авторизации важно иметь мнение о том, что можно безопасно пересылать вниз по потоку обработки. Например, пересылка заголовка аутентификации может привести к тому, что сервис сможет выдать себя за другой или пользователя, что вызовет проблемы. Однако bearer-токен OAuth2 можно смело отправлять вниз по потоку обработки (при условии, что транспортировка безопасна).

Ведение журнала для помощи в отладке

В распределенной архитектуре все идет наперекосяк! Часто возможность видеть, что запрос дошел до сервиса, действительно ценна, особенно если не учитывать кеширование. Журналы целесообразно рассматривать в двух различных видах: регистрирующие и диагностические. *Регистрирующий* журнал позволяет фиксировать важные операции/события в системе и используется ограниченно. Примером события регистрации может служить получение нового сообщения для обработки и результат этого события. *Диагностика* в большей степени связана со сбоями в обработке и любыми неожиданными ошибками, не относящимися к зарегистрированным в журнале событиям. В структурированный журнал можно добавить поле, обозначающее тип журнала, что позволит быстро получить доступ либо только к регистрационным записям, либо к полной диагностике.

Рассмотрение субъективной платформы

Часто нет осознанного решения о том, какой подход использовать при принятии решений, рассмотренных в этом разделе, что может привести к повторному выполнению работы или к непоследовательным подходам. Один из вариантов решения этой проблемы — создание команды платформы и разработка *субъективной платформы* (opinionated-платформы). Субъективная платформа будет принимать ключевые решения по урегулированию проблем в рамках технической платформы, что избавит каждого разработчика от необходимости реализовывать одни и те же функции платформы.

Для того чтобы субъективные платформы успешно выполняли работу, им необходимо расширить путь к среде эксплуатации с учетом DevOps и других ключевых факторов, необходимых для работы на платформе. Это часто называют «мощной дорогой» или «золотым путем» к среде эксплуатации. Создание платформы, которую захотят использовать команды разработчиков и которая упростит решение бизнес-задач, будет иметь гораздо больше шансов на принятие. Важно помнить, что формирование субъективного подхода создает ограничения, поэтому нужен компромисс между свободой разработчика и приложениями, работающими так, как ожидается в организации.

Руководство ADR: субъективные платформы

Выбор в пользу создания субъективной платформы, наиболее успешен, когда разработчики платформы вовлечены в процесс проектирования. В руководстве, приведенном в табл. 5.3, отмечено, какие моменты необходимо учитывать и как важно привлекать разработчиков для создания успешной opinionated-платформы.

Таблица 5.3. Руководство ADR: субъективные платформы

Решение	Следует ли вам использовать субъективную платформу для развертывания и выпуска релизов?
Пункты обсуждения	Какие в организации используются языки для разработки программного обеспечения? Можно ли сосредоточиться на нескольких, чтобы работать в рамках opinionated-платформы? Можно ли в организации предоставить разработчикам возможность стать клиентами opinionated-платформы и использовать ее в качестве внутреннего продукта? Каковы ограничения или функциональные возможности, способные добавить преимуществ при внедрении платформы? Например, должны ли мониторинг и наблюдаемость быть функциями, предоставляемыми разработчикам «из коробки»? Как обновляются рекомендации по платформе и как вы помогаете предоставлять изменения командам, уже использующим платформу?
Рекомендации	Рассматривать разработчиков как потребителей продукта платформы и создать механизм, поддерживающий внесение разработчиками своего вклада. Ключевые особенности должны быть максимально прозрачными для разработчиков (например, конфигурирование библиотеки для внедрения открытой телеметрии). Новые приложения всегда получают новейшие функциональные возможности стека. Однако как сделать так, чтобы существующие пользователи платформы могли легко получить доступ к новейшим функциям?

Заключение

Эта глава представляет собой введение в развертывание и распространение программного обеспечения в архитектуре API:

- ♦ ценной отправной точкой служит понимание важности разделения развертывания и релиза. В существующих приложениях одним из подходов к конфигурированию и включению новых функций на уровне кода будет установка флагов функций;
- ♦ управление трафиком дает новую возможность использовать маршрутизацию трафика для моделирования релизов;
- ♦ мажорные, минорные и патч-релизы помогают разделить стиль вариантов релизов. Приложения, имеющие сильно связанный API, могут использовать другую стратегию;
- ♦ мы рассмотрели стратегии релизов и ситуации, в которых они применяются, а также увидели, как инструменты типа Argo могут помочь в эффективном внедрении;
- ♦ мониторинг и метрики служат важным показателем успеха API-платформы. Вы узнали, почему некоторые метрики могут быть случайными ошибками и наводить на мысль о проблеме там, где ее нет. Мы поняли, что такое наблюдаемость, и почему применение этой технологии имеет решающее значение для успешной работы платформы API;
- ♦ наконец, мы рассмотрели прикладные решения для поддержки эффективного внедрения и то, что владельцы платформ должны учитывать, стремясь к единообразию на предприятии.

Эффективное развертывание и релиз API-интерфейсов — важнейшие условия успешной архитектуры на основе API. Однако важно продумать угрозы безопасности, встречающиеся в работе API-систем, и выбрать способы эффективного снижения рисков. Этим вопросам посвящена *глава 6*.

Операционная безопасность: моделирование угроз для API

К этому моменту мы рассмотрели полный жизненный цикл API, приняв во внимание их проектирование и тестирование, варианты развертывания и стратегии релизов. Может показаться, что API Участник уже готов к взаимодействию с внешними системами. API быстро создаются, их сложно спроектировать с учетом будущей совместимости, но еще сложнее их защитить. Дело в том, что разработчики и архитекторы сосредоточены на обеспечении функциональности, а о безопасности зачастую задумываются лишь ближе к концу проекта.

В этой главе вы увидите, почему безопасность важна, и что отсутствие надлежащей защиты может повредить вашей репутации и дорого обойтись. Вы научитесь исследовать архитектуру системы на предмет поиска слабых мест в системе безопасности и определять вероятные угрозы, возникающие в производственной среде. Конечно, вы не сможете выявить все угрозы — злоумышленники коварны, и картина угроз постоянно меняется, — но критически важным навыком для архитекторов стало умение «сдвинуть влево» проектирование и реализацию проблем безопасности, как для себя, так и для более широких групп разработчиков¹. Чем раньше вы начнете рассматривать вопросы безопасности в рамках жизненного цикла разработки программного обеспечения (т. е. чем дальше «влево»), тем легче и экономичнее вы сможете адаптироваться к меняющемуся ландшафту угроз. Это поможет вам принимать обоснованные решения при проектировании систем безопасности API. В разд. «Обеспечение безопасности: транспортная безопасность, аутентификация и авторизация» главы 4 мы уже обсуждали, как можно защитить коммуникации внутри плоскости управления или системы с помощью mTLS. Однако при появлении «внешних» систем, находящихся вне зоны действия плоскости управления, требуется новый подход.

Практический пример: применение OWASP к API участников

Мы начнем свой путь к проектированию защищенных систем с введения в моделирование угроз. Затем мы рассмотрим, как выполнить упражнение по созданию модели угроз, используя в качестве примера сервис Участник и API (рис. 6.1).

¹ В идеале безопасность необходимо начинать создавать «слева», чтобы заложить ее в основу.



Рис. 6.1. API Участник, который будет использоваться при моделировании угроз

Основным компонентом моделирования угроз является поиск потенциальных слабых мест в системе безопасности, поэтому стоит изучить список OWASP API Security Top 10². Его можно использовать как в качестве источника вдохновения при поиске проблем, так и в качестве источника мер по их снижению при попытке устранить найденные угрозы. К концу главы вы поймете, что такое моделирование угроз и как его можно применить в собственных проектах³.

Открытый проект по безопасности веб-приложений (Open Web Application Security Project, OWASP) — некоммерческий фонд, занимающийся вопросами повышения безопасности программного обеспечения. Наиболее известным проектом OWASP стал рейтинг OWASP Top 10, представляющий собой список наиболее критичных рисков безопасности, встречающихся при работе веб-приложений. В 2019 году в OWASP подготовили новый список из 10 угроз под названием API Security Top 10. Список составлен на основе работы экспертов по безопасности, изучавших случаи нарушения безопасности и данные программы «вознаграждение за ошибку» (Bug bounty), специалисты по тестированию на проникновение также высказали свое мнение о том, что должно входить в эту десятку. Список представляет собой далеко не полный перечень всех возможных на вашем пути угроз. Тем не менее их следует иметь в виду, рассматривая возможные способы эксплуатации вашего API. Списки

² См. <https://oreil.ly/k9MSt>.

³ Исчерпывающую информацию по этому вопросу и описание способов моделирования угроз можно найти в книге «Threat Modeling» (<https://oreil.ly/0JCpa>) Изара Тарандаша и Мэтью Дж. Коулса (O'Reilly).

OWASP по мере развития угроз периодически обновляются, поэтому важно следить за изменениями и дополнениями в упомянутой десятке.

Риск при отсутствии защиты внешних API

Несмотря на то что тема безопасности в последнее время стала более привлекательной и оказалась в центре внимания⁴, она с трудом выходит на тот уровень популярности, достигнутый такими технологиями, как машинное обучение, большие данные и квантовые вычисления. Для большинства специалистов в области программного обеспечения безопасность не всегда стоит на первом месте. Разработчики сосредоточены на программировании бизнес-решений, команды SRE обеспечивают работоспособность оборудования, а владельцы продуктов заняты планированием новых ценных функций. Обеспечение безопасности часто откладывается на потом, а если вам повезло и у вас есть команда специалистов по безопасности, то эта задача может быть делегирована им. Воспринимаемая заказчиком ценность разрабатываемой системы (обычно) заключается не в применяемых средствах защиты, а в сервисе, предоставляемом этой системой.

Нарушения безопасности могут иметь катастрофические последствия — как правило, они влекут за собой значительные риски для репутации организации. В финансовом отношении последствия невероятны: «средняя стоимость кибервзлома для публичных компаний составила 116 млн долл.»⁵, а средняя стоимость утечки данных для организаций в 2021 году составила 4,24 млн долл.⁶, что на 10% больше, чем в предыдущем году.

Далее приведено несколько примеров нарушений безопасности, повлекших за собой огромные финансовые и социальные потери:

- ♦ утечка данных о 419 млн пользователей⁷;
- ♦ угроза утечки данных затронула 143 млн американцев⁸;
- ♦ в результате взлома системы безопасности была раскрыта личная информация 47 млн пользователей⁹;
- ♦ урегулирование проблемы утечки данных обошлось в 17,5 млн долл.¹⁰;
- ♦ похищено 106 млн записей клиентов и наложен штраф в размере 80 млн долл.¹¹;

⁴ Такие личности, как Эдвард Сноуден, и такие телевизионные шоу, как «Мистер Робот», способствовали повышению уровня осведомленности широкой общественности в вопросах безопасности.

⁵ См. <https://oreil.ly/W70CP>.

⁶ См. <https://oreil.ly/FpH3D>.

⁷ См. <https://oreil.ly/iCZOOb>.

⁸ См. <https://oreil.ly/sK321>.

⁹ См. <https://oreil.ly/DREQ0>.

¹⁰ См. <https://oreil.ly/RfiQh>.

¹¹ См. <https://oreil.ly/8fR4j>.

- ♦ выплачен штраф в размере 16,4 млн фунтов стерлингов за ошибки при отражении кибератак¹².

Последние два примера интересны тем, что регуляторы стали выписывать штрафы за нарушение правил регулирования или неадекватное реагирование. Важно посмотреть на свою операционную среду, чтобы понять, какие требования существуют для управления данными клиента. Пользователи справедливо полагают, что для защиты их конфиденциальности и данных принимаются соответствующие меры. И если это не так, то ответственность несет ваша организация. Одним из таких нормативных требований служит Общий регламент по защите данных (General Data Protection Regulation, GDPR). Он предоставляет гражданам более широкий контроль над их личной информацией. Несоблюдение этих требований может повлечь за собой серьезные финансовые санкции. В настоящее время список самых крупных штрафов, выписанных за нарушение GDPR, включает в себя штраф компании Amazon на сумму 636 млн фунтов стерлингов¹³ и штраф WhatsApp на сумму 225 млн евро¹⁴.



Подотчетность организации выходит за рамки API и систем, разработанных ею самой. Продукты поставщиков и программное обеспечение с открытым исходным кодом могут создать реальные проблемы, если ими тщательно не управлять. Убедитесь, что продукты поставщика отвечают тем же требованиям, что и ваши собственные стандарты разработки программного обеспечения. Программное обеспечение с открытым исходным кодом может содержать широкий спектр уязвимостей, которые способны вызвать далеко идущие последствия. Очень важно, чтобы организация отслеживала общеизвестные уязвимости из базы CVE (Common Vulnerabilities and Exposures) и имела возможность восстанавливать пострадавшее программное обеспечение.

Моделирование угроз 101

Моделирование угроз — это «техника, которую можно использовать для определения угроз, атак, уязвимостей и контрмер, способных повлиять на ваше приложение»¹⁵. Если прибегнуть к реальной аналогии, то при проведении моделирования угроз для своего дома или квартиры вы определите такие моменты, как точки проникновения (двери, окна), и то, давали ли вы ключ от входной двери соседу. Такой подход выгоден, поскольку снизить риски безопасности можно только после четкого определения угроз. Он также помогает определить приоритетность мер по повышению безопасности и избежать бессмысленных усилий или возникновения *театра безопасности*¹⁶. Продолжая пример с домом, можно сказать, что не стоит

¹² См. <https://oreil.ly/LnEuX>.

¹³ См. <https://oreil.ly/RcWyY>.

¹⁴ См. <https://oreil.ly/TuYnM>.

¹⁵ См. <https://oreil.ly/ahFgn>.

¹⁶ Театр безопасности — явление в обществе, когда целью проводимых мероприятий является повышение безопасности, но по факту не делается ничего для ее достижения (<https://oreil.ly/peFDx>).

тратить большие деньги на усиленную стальную входную дверь, оставив при этом ключ под половиком или в цветочном горшке прямо на улице.

Процесс моделирования угроз необходимо интегрировать в весь жизненный цикл разработки программного обеспечения. В идеале он выполняется в начале проекта и постоянно пересматривается по мере развития системы и архитектуры. Хорошая новость заключается в том, что существует ряд четко разработанных методик моделирования угроз. В этой книге мы будем опираться на методологию STRIDE, разработанную Праеритом Гаргом и Лореном Конфелдером из компании Microsoft. Ее основы подробно описаны далее в этой главе.

Моделирование угроз программных систем исторически осуществлялось с помощью *диаграмм потоков данных* (Data Flow Diagram, DFD)¹⁷. DFD отражают динамические (потоки данных) аспекты системы, в то время как модели C4 в первую очередь отражают статические (структурные) аспекты системы. Диаграммы DFD просты для понимания и ориентированы на данные, что позволяет легко увидеть, как данные проходят через систему. Основные компоненты DFD следующие:

◆ **Внешние сущности.**

Приложения или сервисы, которые не являются частью вашей системы. В нашем случае это будет мобильное приложение.

◆ **Процессы.**

Приложение или задача, находящаяся в нашем домене, — например, шлюз API.

◆ **Хранилища данных.**

Место, где хранятся данные. Для рассматриваемого примера это будет база данных.

◆ **Потоки данных.**

Соединение, представляющее поток данных, — например, от мобильного приложения к шлюзу API.

◆ **Границы.**

Привилегированная или доверительная граница для отображения изменения уровня доверия. В качестве границы для нашего примера можно взять границу Интернета между мобильным приложением и шлюзом API.

В рамках моделирования угроз мы создали диаграмму DFD, представленную далее на рис. 6.2.

Мыслите как злоумышленник

Архитекторы и разработчики порой неохотно рассматривают вопросы безопасности, считая, что это дело команды соответствующих специалистов. Однако кто, как не люди, проектирующие и создающие ключевые структурные компоненты про-

¹⁷ Для получения полной информации о DFD посетите вводную страницу OWASP DFD (<https://oreil.ly/0VlaM>).

граммной системы, могут выявить и понять ее потенциальные слабые места? После чего архитекторы и специалисты по безопасности могут сотрудничать в решении этих проблем и совместно исследовать различные варианты атаки. Интересный подход заключается в том, что для проведения моделирования угроз не нужно быть экспертом в области безопасности, а надо лишь мыслить как злоумышленник или нечестный актер.

Мыслить как злоумышленник зачастую проще, чем кажется, — ведь вы делаете это постоянно (просто задавая себе вопрос: что бы сделал злоумышленник?!). Например, когда вы вечером ставите машину на стоянку, что вы делаете с ключами от нее? Оставляете их в машине? Скорее всего, нет, если она стоит на улице, хотя можно и оставить, если машина в гараже. Вы можете оставить ключи за входной дверью, однако кто-то может просунуть через щель почтового ящика какой-либо крючок, чтобы достать их, или, если речь идет об автомобиле с бесключевым доступом, воспользоваться усилителем сигнала. Поэтому вы, наверняка заберете ключи в комнату. А с появлением электронных систем запираания и противоугонных устройств положите их в клетку Фарадея¹⁸. В отмеченных случаях вы рассматриваете ситуацию, оцениваете угрозу и взвешиваете риск. Теперь необходимо применить этот подход к проектированию программных систем с помощью существующих четко определенных методологий.

Как построить модель угрозы?

Как и во многих других методологиях проектирования и разработки программного обеспечения, в моделировании угроз существуют четко определенные цели, подходы и методы, совершенствуемые архитекторами и инженерами на протяжении многих лет. Общий подход к моделированию угроз выполняется следующим образом:

1. Определите цели — составьте список целей бизнеса и безопасности. Они должны быть простыми (например, не допускайте несанкционированного доступа).
2. Осуществите сбор нужной информации — создайте обобщенный проект системы и убедитесь, что у вас есть нужная информация. Чтобы понять, как работают и взаимодействуют системы проекта, привлечите к обсуждению нужных специалистов.
3. Выполните декомпозицию системы — разбейте обобщенный проект уровня на части, чтобы можно было приступить к моделированию угроз. Для этого может потребоваться несколько моделей и диаграмм.
4. Выявляйте угрозы — осуществляйте систематический поиск угроз для ваших систем.
5. Оцените риск угроз — расставьте приоритеты, сфокусировавшись на наиболее вероятных угрозах, а затем определите меры по снижению риска этих вероятных угроз.

¹⁸ См. <https://oreil.ly/AAdSK>.

6. Выполните проверку — спросите себя и свою команду, были ли проведенные изменения успешными. Следует ли провести еще один цикл моделирования?

Теперь рассмотрим эти этапы более подробно, используя в качестве примера систему, для которой необходимо выполнить упражнение по моделированию угроз.

Этап 1: определение целей

Первым этапом моделирования угроз будет определение целей, служащих драйвером такого моделирования. При определении целей для собственных систем следует сосредоточиться на том, какие цели в области безопасности вы пытаетесь достичь. Эти цели должны быть актуальны для всей организации, а не только для вашей команды и команды InfoSec. Задачи безопасности часто определяются бизнес-целями — например, недопущением утечки данных во избежание судебного преследования или обеспечением соответствия нормативным требованиям, таким как GDPR. Если данные о целях получены только из вашего ближайшего окружения, то вы не получите полного представления о наиболее важных проблемах, стоящих перед всей организацией. Ваша задача для сервиса Участник — подготовить API к внешнему использованию третьими сторонами, обеспечив защиту от угроз из списка OWASP Top 10.

Этап 2: сбор нужной информации

Определив цели, на втором этапе моделирования угроз необходимо получить информацию о том, как работает система. Для этого надо привлекать экспертов по каждой области системы и связанным с ней кодовым базам или продуктам. Такая информация даст вам понимание того, как все работает, и исключит появление каких-либо скрытых предположений. Для API Участник это потребует привлечения членов команды, работающих со всеми компонентами: мобильными приложениями, шлюзом, базами данных и сервисом Участник.

Этап 3: декомпозиция системы

Третий этап процесса моделирования угроз заключается в создании диаграммы потоков данных системы, отображающей взаимодействие ее компонентов с потоком данных, для чего используется собранная совместными усилиями на этапе 2 информация. Создание диаграмм может занять много времени, поэтому мы рекомендуем обратиться к специализированным средствам моделирования угроз. Для построения диаграммы потоков данных, показанной на рис. 6.2, мы воспользовались программой Microsoft Threat Modeling Tool¹⁹, хотя есть и другие инструменты²⁰.

¹⁹ Полноценной локализованной версии этого ПО нет, поэтому на рис. 6.2 показан условный перевод позиций диаграммы. — *Прим. пер.*

²⁰ Получить инструмент Microsoft Threat Modeling Tool можно здесь: <https://oreil.ly/ahFgn>, а другие варианты подобных инструментов — в репозитории OWASP Threat Dragon на GitHub (<https://oreil.ly/NXF8U>).

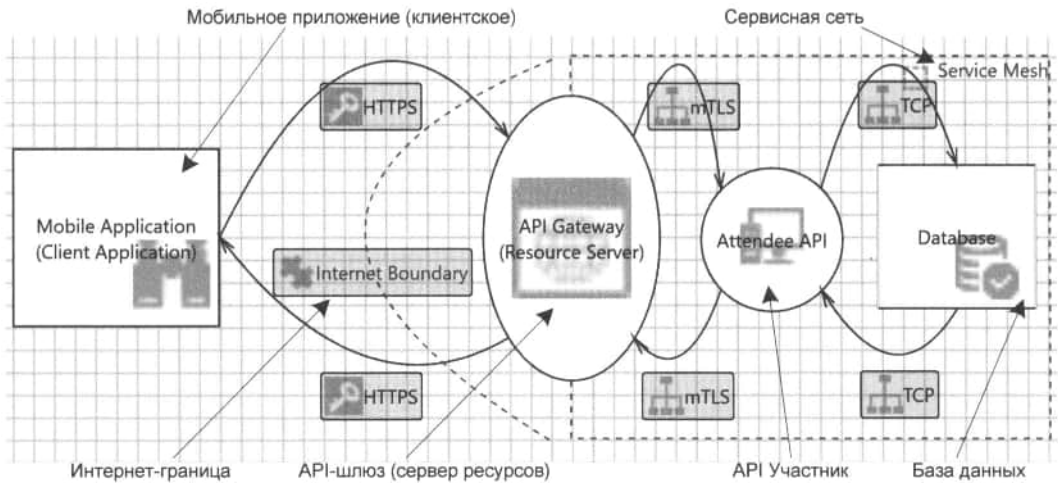


Рис. 6.2. Диаграмма потоков данных

Этап 4: выявление угроз и включение их в модель STRIDE

На четвертом этапе моделирования угроз рассматриваются собственно угрозы для системы. Изучая диаграмму потока данных, важно помнить о целях моделирования угроз, иначе может возникнуть соблазн отклониться от темы.

Преимущество использования специализированного инструмента Microsoft Threat Modeling Tool заключается в том, что он может провести для вас автоматизированный анализ по методологии STRIDE. Сформированный список, возможно, не будет полным, но его можно использовать в качестве отправной точки. Список сгенерированных угроз для нашей системы API Участник представлен на рис. 6.3. В нашем случае инструмент обнаружил 27 потенциальных угроз.

Threat List		
Title	Category	Interaction
Replay Attacks	Tampering	MTLS
Collision Attacks	Tampering	MTLS
Weak Authentication Scheme	Information Disclosure	MTLS
Elevation Using Impersonation	Elevation Of Privilege	MTLS
Potential Data Repudiation by API Gateway (Resource Server)	Repudiation	HTTPS
Potential Process Crash or Stop for API Gateway (Resource Server)	Denial Of Service	HTTPS
Data Flow HTTPS Is Potentially Interrupted	Denial Of Service	HTTPS
API Gateway (Resource Server) May be Subject to Elevation of Privi	Elevation Of Privilege	HTTPS
Elevation by Changing the Execution Flow in API Gateway (Resourc	Elevation Of Privilege	HTTPS

Рис. 6.3. Анализ угроз на основе диаграммы потоков данных, выполненный с помощью программы Microsoft Threat Modeling Tool

Акроним STRIDE означает²¹:

◆ **Spoofing (Спуфинг).**

Нарушение аутентификационной информации пользователя. В этом случае хакер получает персональные данные пользователя или что-то, что позволяет ему воспроизвести процедуру аутентификации. Угрозы спуфинга связаны с тем, что хитрый хакер может выдать себя за действительного пользователя системы или ресурса, получить доступ к системе и тем самым нарушить ее безопасность.

◆ **Tampering (Подмена).**

Модификация системных или пользовательских данных с обнаружением или без обнаружения. Неавторизованное изменение хранимой или перемещаемой информации, форматирование жесткого диска, внедрение злоумышленником необнаруживаемого сетевого пакета в коммуникацию, внесение необнаруживаемого изменения в конфиденциальный файл — все это угрозы, связанные с подменой.

◆ **Repudiation (Отрицание).**

Недоверенный пользователь²², выполняющий незаконную операцию, которую невозможно отследить. Угрозы надежности связаны с пользователями (злоумышленными или иными), которые могут отрицать правонарушения, не имея возможности доказать обратное.

◆ **Information disclosure (Разглашение сведений).**

Компрометация частной или критически важной для бизнеса информации пользователя. Угрозы разглашения информации открывают доступ к ней лицам, не имеющим на это прав. Возможность пользователя прочитать файл, доступ к которому ему не был предоставлен, а также возможность злоумышленника прочесть данные, находящиеся в процессе передачи между двумя компьютерами, представляют собой угрозы разглашения информации. Отметим, что такая угроза отличается от угрозы спуфинга тем, что здесь злоумышленник получает доступ к информации напрямую, а не путем подмены легитимного пользователя.

◆ **Denial of Service (Отказ в обслуживании).**

Временная недоступность или невозможность использования системы, например, вследствие атаки, способной заставить перезагрузить или перезапустить компьютер пользователя. Когда злоумышленник может временно сделать недоступными или непригодными для использования системные ресурсы (вычислительное время, память и т. п.), мы имеем дело с угрозой отказа в обслуживании. Необходимо защищаться от определенных типов DoS-атак для повышения доступности и надежности системы. Однако от некоторых типов DoS-атак очень

²¹ Приведенные здесь определения взяты из статьи «The threats to our Products» (<https://oreil.ly/a7NHe>), опубликованной в 1999 году Лореном Конфелдером и Праеритом I аргом, создателями STRIDE.

²² Недоверенный пользователь — любой пользователь, не указанный в списке доверенных.

трудно защититься, поэтому как минимум необходимо выявить и рационально обосновать такие угрозы²³.

♦ Elevation of privilege (Расширение полномочий).

Непривилегированный пользователь получает расширенный доступ и, таким образом, обретает достаточно возможностей для полной компрометации или разрушения всей системы. Более опасным аспектом подобной угрозы является компрометация системы необнаруживаемыми способами. В таких ситуациях пользователь может воспользоваться полномочиями без ведома системных администраторов. К угрозам расширения полномочий относятся ситуации, когда злоумышленнику предоставляется больше прав, чем положено, что полностью нарушает безопасность всей системы и наносит ей серьезный ущерб. В этом случае злоумышленник фактически проникает через все средства защиты системы, становится частью самой доверенной системы и может делать всё, что угодно.

Определения STRIDE можно использовать при оценке системы в каждой точке архитектуры на предмет наличия угроз. Впрочем, существуют и другие доступные методики моделирования угроз²⁴.

Посмотрев на диаграмму потоков данных, приведенную на рис. 6.2, можно увидеть границу, существующую между клиентским приложением и API-шлюзом. API-шлюз часто располагается на периферии сети, а также может быть обращен к Интернету. Об этом говорилось в *разд. «Где выполняется развертывание API-шлюза?» главы 3*. Мы рассмотрим далее ряд различных угроз, связанных с API-шлюзом, и узнаем, как с его помощью защитить свою систему от многих распространенных уязвимостей API. Если защитить систему на границе, то риски часто могут быть снижены во всей системе, но это не всегда так. Подробнее о переходе от зональной архитектуры, когда трафик внутри периметра безопасности обрабатывается иначе, чем снаружи, к модели нулевого доверия, когда трафик постоянно проходит повторную аутентификацию, рассказывается в *разд. «От зональной архитектуры к принципу нулевого доверия» главы 9*.

Цели обеспечения безопасности в нашем примере вполне конкретны: API Участник должен быть подготовлен к внешнему использованию, и для достижения этой цели мы убедимся, что каждый процесс снижает вероятность возникновения проблем из списка OWASP API Security Top 10. Поскольку это прямая задача, DFD можно задействовать для сопоставления потоков данных с проблемами и уязвимостями, описанными на сайте OWASP. Однако обычно целью моделирования угроз может стать что-то вроде «Предотвратить утечку ПИ-данных в соответствии с GDPR» или «Обеспечить 99,9%-ную доступность API для выполнения контрактных обязательств». Вторая цель может показаться не связанной с безопасностью, однако вы

²³ Хотя в этом определении речь идет об отдельных машинах, суть того, что такое атака типа «отказ в обслуживании», остается неизменной и сегодня. Речь идет о выводе ресурсов из строя.

²⁴ Две дополнительные методологии — это P.A.S.T.A (<https://oreil.ly/OYYQq>) и Trike (<https://oreil.ly/TQayg>).

должны помнить о DoS-атаках, поскольку невыполнение контрактных обязательств, даже при такой атаке, может привести к финансовым штрафам.

Теперь рассмотрим нашу систему и применим методику STRIDE. Чтобы выделить проблемы из списка OWASP API Security Top 10, угрозы мы сгруппируем по соответствующему значению STRIDE. Это делается для того, чтобы продемонстрировать применение STRIDE и OWASP API Security Top 10 совместно с мерами по смягчению последствий. При выявлении угроз в собственной архитектуре рекомендуется применять STRIDE к каждому процессу и соединению — это называется *STRIDE для каждого элемента*.

Spoofting (Спуфинг)

Спуфинг — это ситуация, когда человек или программа может маскироваться под другого человека или программу. Чтобы снизить вероятность спуфинга, необходимо выполнять аутентификацию всех запросов и убеждаться в их правомочности. В рейтинге OWASP API Top 10 одной из проблем безопасности указано *Нарушение аутентификации пользователя* (Broken User Authentication²⁵). Эта проблема, безусловно, относится к категории спуфинга, поэтому необходимо убедиться, что поток аутентификации не нарушен. Подробнее об этом можно узнать в разд. «Аутентификация» главы 7, где приведена соответствующая информация и рассмотрен практический пример.

Tampering (Подмена)

Следующей проблемой в методологии STRIDE считается «подмена». Наша цель заключается в том, что пользователи или клиенты не должны иметь возможности непреднамеренно изменять систему, приложение или данные. Например, у злоумышленника не должно быть возможности модифицировать сервис Участник, перенаправив трафик, предназначенный для этого сервиса, во внешнее местоположение или неправомерно обновив пользовательские данные участников. Существуют два основных способа вмешательства: внедрение полезной нагрузки и массовое переназначение.

Внедрение полезной нагрузки

Внедрение полезной нагрузки происходит, когда злоумышленник пытается внедрить вредоносную полезную нагрузку в запрос к API или приложению. Заметим, что в перечне OWASP Security Top 10 речь идет не только об общеизвестном внедрении SQL-кода, но и о внедрении данных в любой пользовательский ввод. В нашем примере предотвратить атаки внедрения можно на ранних этапах цепочки обработки запросов, используя API-шлюз для проверки соответствия выполняемого запроса определенному контракту или схеме. Любой запрос, не соответствующий условиям контракта, может быть отклонен, а соответствующий поток трафика прерван.

²⁵ См. <https://oreil.ly/59Tgo>.

Этот подход описан в разд. «Практическое применение спецификаций OpenAPI» главы 1. Спецификации OpenAPI все чаще используются для проверки HTTP-запросов.

Следует отметить, что хотя проверка ввода на API-шлюзе — это очень полезный процесс, его наличие не означает, что можно отказаться от дальнейшей проверки и санации ввода в бэкенд-сервисах. Доверяй, но проверяй!

Примером процесса проверки ввода на API-шлюзе для сервиса Участник может служить получение POST-запроса со следующим образом полезной нагрузки для создания пользователя:

```
POST /attendees
{
  "name": "Danny B",
  "age": 35,
  "profile": "Hack; DROP ALL TABLES; --"
}
```

Спецификация OpenAPI для API Участник определяет, что поле имени `name` может принимать только буквы, поле `age` — целые положительные числа, а поле `profile` — буквы, цифры и специальные символы в определенном значении (поскольку пользователь должен написать немного о себе). API-шлюз, в этом случае выполняющий проверку ввода, проверит полезную нагрузку и пропустит ее только в случае успешной проверки. Но даже если проверка ввода проходит успешно, API Участник все равно должен санировать вводимые данные, чтобы предотвратить атаку. Сервис Участник при взаимодействии с базой данных будет использовать подготовленные операторы. Важно построить несколько линий защиты на случай, если одна из них выйдет из строя.

Массовое переназначение

Изменяемые свойства, привязанные к сущностям базы данных, уязвимы для неправомерного изменения, которое может выполняться с помощью уязвимости, известной как *массовое переназначение*²⁶. Это важный момент, и его необходимо учитывать, особенно, если в вашем базовом приложении используется шаблон Active Record²⁷ или какая-либо форма автоматической сериализации/десериализации баз данных сущностей, что часто обеспечивается фреймворками объектно-реляционного отображения (Object-Relational Mapping, ORM).

Рассмотрим гипотетический случай для нашего API Участник. Представьте, что существует свойство `devices`, возвращаемое при запросе данных участника. Это свойство представляет собой список устройств, используемых участником для подключения к API, доступный лишь для чтения извне, и должно обновляться только кодом приложения участника.

²⁶См. <https://oreil.ly/5Q6aC>.

²⁷ Шаблон Active Record — это практика предоставления доступа к объекту данных и его функциям, которые в большей или меньшей степени отображаются на базовую модель базы данных.

Злоумышленник делает GET-запрос данных участника (/attendees/123456) и получает следующий ответ:

```
{
  "name": "Danny B",
  "age": 35,
  "devices": [
    "iPhone",
    "Firefox"
  ]
}
```

Затем он отправляет PUT-запрос к API Участник для обновления атрибута age, а также злонамеренно пытается обновить список devices:

```
PUT /attendees/123456
{
  "name": "Danny B",
  "age": 36,
  "devices": [
    "vulnerableDevice"
  ]
}
```

Любые данные в списке devices должны быть проигнорированы при сохранении сущности в базу данных. Массовое переназначение типично для случаев, когда входные данные клиента привязываются к внутренним объектам без того, чтобы задуматься о последствиях, что часто становится следствием предоставления доступа к API базы данных как к веб-интерфейсу. В главе 1 обсуждались проблемы, связанные с раскрытием базовой модели данных с точки зрения удобства использования, что дает дополнительные причины не делать этого.

Эта уязвимость, как правило, не может быть устранена на уровне API-шлюза, а должна быть защищена в самой реализации API.

Repudiation (Отрицание)

Согласно STRIDE атака с отрицанием происходит, когда приложение или система не использует средства контроля для надлежащего отслеживания и регистрации действий пользователей, что позволяет осуществлять злонамеренные манипуляции или подделывать идентификацию новых действий. Для многих запросов к API важно понимать детали запроса, полезную нагрузку и генерируемый ответ (и соответствующие внутренние действия). В некоторых случаях, связанных с соблюдением нормативных требований, может потребоваться произвольная проверка того, что было в обмене. Если запрос может быть опровергнут — т. е. нет доказательств действий злоумышленника, — злоумышленник может отвергнуть или не согласиться с тем, что он пытался выполнить какое-либо злонамеренное действие. Именно поэтому в методологию STRIDE включены угрозы с отрицанием (буква «R» в акрониме STRIDE).

Чтобы выявить запросы, проходящие через вашу систему, и понять, что происходит, необходимо добавить журналирование и мониторинг. *Неэффективные журналирование и мониторинг*²⁸ относится к уязвимостям в перечне OWASP API Top 10. Поскольку все запросы пользователей проходят через API-шлюз, это очевидная централизованная точка для мониторинга трафика и регистрации запросов и ответов. Многие API-шлюзы предоставляют такую функциональность «из коробки», однако необходимо понимать, как хранить, искать и извлекать эту информацию, особенно с течением времени. Как и в случае с любыми возможностями аварийного восстановления и обеспечения непрерывности бизнеса (Disaster Recovery and Business Continuity, DR/BC), журналирование и мониторинг должны регулярно проверяться, чтобы убедиться, что вы фиксируете то, что ожидается.

Information disclosure (Разглашение сведений)

Разглашение сведений — это буква «I» в акрониме STRIDE, и это предостережение направлено на то, чтобы не раскрывать информацию, которая должна использоваться только внутри компании или храниться в секрете. Два распространенных антипаттерна в этой категории угроз: чрезмерное раскрытие данных и некорректное управление активами.

Чрезмерное раскрытие данных

Описание проблемы *чрезмерного раскрытия данных* (Excessive data exposure²⁹) из перечня OWASP API Top 10 ориентировано на недопущение неправомерного раскрытия данных. В качестве гипотетического сценария представим, что в сервисе Участник хранится информация ПИ — например, номер паспорта. При разработке API важно предотвратить неправомерное раскрытие этих данных. Слишком легко сделать наивные предположения о том, как будет вызываться API, особенно по мере развития системы с течением времени. API, изначально предназначенные лишь для внутреннего использования, а также предыдущий API, доступный только для доверенных клиентских приложений, могут быть открыты (с благими намерениями) для публичного использования.

Если API вызывается через веб-приложение, то с помощью инструментов разработчика, входящих в состав современных браузеров, можно легко изучить запросы, ответы и соответствующую полезную нагрузку. Например, любой запрос информации о пользователе, сделанный к API Участник, может случайно вернуть паспортные данные:

```
{
  "values": [
    {
      "id": "0",
      "name": "Danny B",
      "age": 65,
```

²⁸ См. <https://oreil.ly/Gmzj6>.

²⁹ См. <https://oreil.ly/SpHdH>.

```

    "email", "danny.b@masteringapis.com",
    "passport": "Abc12408NJUILM"
  },
  {
    "id": "1",
    "name": "Jimmy G",
    "age": 93,
    "email": "jimmy.g@masteringapis.com",
    "passport": "ZYX123ASJJ0072M"
  }
]

```

В API-шлюзе можно выполнить проверку ответа. Однако те, кто создает API, обязаны знать, к каким данным открывают доступ, и должны следить за тем, чтобы не раскрывать конфиденциальные, закрытые данные. Любая реализация в API-шлюзе должна рассматриваться как крайняя мера верификации (или как часть подхода под названием «belt and braces»³⁰ — с удвоенной безопасностью по части верификации). Также необходимо исключить утечку конфиденциальных данных обратно к вызывающим клиентам — например, версии используемого веб-сервера или трассировку стека приложения, сгенерированную в результате сбоя.

Некорректное управление активами

Некорректное управление активами (Improper assets management³¹), как правило, происходит по мере развития систем, когда организация теряет представление о том, к каким API (и каким версиям) доступ был открыт или какие API были предназначены только для внутреннего использования. В качестве гипотетического примера с API Участник можно привести несколько версий API, развернутых в производстве, причем ранняя версия API по умолчанию раскрывает все свойства участников. По мере развития модели данных добавляется несколько закрытых полей, содержащих данные PII, а новые версии сервиса Участник удаляют эту информацию при запросе к API. Но даже если старая версия сервиса Участник функционирует не полностью, ее все равно можно использовать для извлечения дополнительной информации, содержащейся в модели данных.

Так, например, для сервиса Участник может возникнуть ситуация, когда конечная точка `/beta/attendees` находится в открытом доступе. Это условие в ранней версии использовалось для тестирования, а потом версия была забыта. И при отсутствии надлежащего управления активами с открытым доступом такая ситуация остается незамеченной, но злоумышленник может попытаться вызвать эту конечную точку. Если весь трафик API управляется через шлюз, то в нем должна присутствовать регистрация, позволяющая знать, какие потоки существуют. Можно также иссле-

³⁰ Дословно фраза *belt and braces* переводится как «ремень и подтяжки». Сама по себе эта идиома означает высокую степень готовности к любым неожиданностям. — *Прим. пер.*

³¹ См. <https://oreil.ly/ZViZw>.

довать запросы и искать аномалии, когда запросы направляются к неожиданным конечным точкам.

Для решения этой проблемы можно использовать платформу управления API или портал разработчика для каталогизации и отслеживания всех API, развернутых в производственной системе. Многие решения по API-менеджменту включают эту функциональность в стандартную комплектацию, поскольку она рассматривается как важнейший компонент управления жизненным циклом API.

Denial of Service (Отказ в обслуживании)

В рамках методологии STRIDE буква «D» относится к отказу в обслуживании (DoS). DoS-атака — это попытка перегрузить систему или любой из ее защитных механизмов в злонамеренных целях. Например, перегруженный брандмауэр может по умолчанию разрешить весь трафик, что позволит злоумышленнику совершать ранее заблокированные вредоносные вызовы. Или же злоумышленник просто хочет перекрыть доступ к критически важному сервису — например, сайту для голосования. Перегрузка системы трафиком приведет к тому, что ни один правомерный запрос не пройдет и ни один пользователь не сможет проголосовать. В перечне OWASP API Top 10 содержится подробное описание опасности DoS-атаки³².

API Участник должен удовлетворять вашим требованиям к масштабируемости, но при этом не перегружаться трафиком. Для этого можно использовать методы ограничения количества запросов и сброса нагрузки.

Со злонамеренными DoS-атаками или распределенными DoS-атаками (Distributed DoS, DDoS) лучше всего справляются специализированные поставщики услуг, программного или аппаратного обеспечения. Например, многие провайдеры *сетей доставки содержимого* (Content Delivery Network, CDN) по умолчанию включают защиту от DoS-атак, а большинство публичных поставщиков предлагают аналогичную услугу — ее можно подключить к публичным доменным именам и IP-адресам.



Отказ в обслуживании может произойти случайно — например, это может быть «дружественный огонь DoS», вызванный собственными системами. В процессе развития систем нередко случайно возникают циклические зависимости, и при благоприятствующих им условиях это может привести к тому, что внутренние сервисы станут вызывать API друг друга в бесконечном цикле. Именно поэтому внедрение ограничения количества запросов и мониторинга ошибок при внутренних вызовах API может оказать неоценимую помощь!

Ограничение количества запросов и сброс нагрузки

Ограничение количества запросов (Rate limiting), как следует из названия, ограничивает количество разрешенных запросов к вашему API за определенный период времени³³. Использование ограничения количества запросов обычно относится

³² См. <https://oreil.ly/ccNCH>.

³³ Один из авторов книги, Дэниэл, написал серию статей об ограничении количества запросов и его применении для API-шлюзов. Первая статья из этой серии доступна в Интернете: «Part 1: Rate Limiting: A Useful Tool with Distributed Systems» (<https://oreil.ly/2rzHx>).

к отклонению трафика на основе свойств отдельных запросов (слишком много запросов от того или иного пользователя, клиентского приложения или местоположения). Под сбросом нагрузки (Load shedding) понимается отклонение запросов в зависимости от общего состояния системы (база данных на пределе возможностей, нет больше рабочих потоков). По умолчанию во многих приложениях, веб-серверах и API-шлюзах не реализовано ограничение количества запросов или сброс нагрузки, и соответствующие режимы отказа могут быть не определены. Выполнение нагрузочного тестирования может дать представление о пределах, точках разрыва и видимом поведении.



Важно понять, какая у вашего API-шлюза и других периферийных средств защиты предусмотрена политика: типа «fail open» (открытая) или «fail closed» (закрытая). Политика fail open будет разрешать доступ к вашим сервисам даже при наличии условий отказа. В качестве гипотетического примера можно привести ситуацию, когда в службе скорой медицинской помощи важнее предоставить информацию об истории болезни пациента, чем аутентифицировать запрос. Политика fail close — это политика, при которой соединения в условиях отказа будут блокироваться. Не существует единственно правильной реализации, и принятый вариант должен удовлетворять вашим требованиям. Например, для большинства финансовых API по умолчанию требуется закрытая политика отказа, в то время как публичная метеорологическая служба может быть реализована на основе открытой политики отказа.

В нашем примере наиболее подходящим местом для реализации ограничения количества запросов будет API-шлюз. Чтобы выполнить ограничение на количество запросов, обычно требуется определить инициатора каждого запроса (или набора запросов в совокупности). В качестве примера свойств можно привести IP-адрес, геолокацию или идентификатор клиента, отправляемый самим клиентом. Вы можете не устанавливать ограничения на входящее свойство и вместо этого рассматривать все запросы как равные.

После того как свойство запроса выбрано (не выбрано или выбрано как-то иначе), необходимо применить стратегию для выполнения ограничения. Наиболее распространенные примеры:

◆ **Фиксированные окна.**

Фиксированное ограничение в пределах периода — например, 2400 запросов в день.

◆ **Скользящие окна.**

Ограничение в пределах последнего периода — например, 100 запросов в течение последнего часа.

◆ **Корзина токенов (Token bucket).**

Допускается заданное количество общих запросов (корзина токенов), и при каждом запросе берется один токен. Корзина периодически пополняется.

◆ **Дырявое ведро (Leaky bucket).**

Хотя, как и в случае с корзиной токенов, скорость обработки запросов фиксирована, это и есть «утечка» из ведра.



Рис. 6.4. Пример ограничения количества запросов с помощью API-шлюза

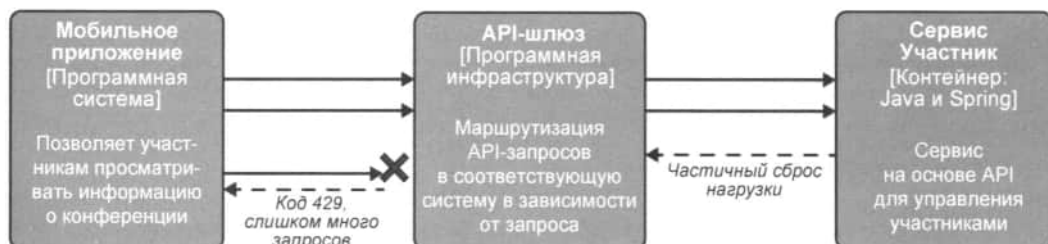


Рис. 6.5. Пример распределения нагрузки с помощью API-шлюза

На рис. 6.4 показано применение ограничения количества запросов, а на рис. 6.5 — пример сброса нагрузки.

Elevation of privilege (Расширение полномочий)

Последняя буква «Е» в акрониме STRIDE указывает на *расширение полномочий*. Такое происходит, когда пользователь или приложение находит способ выполнить задачу, выходящую за рамки допустимого с учетом текущего контекста безопасности, — например, пользователь может выполнять задачи, предназначенные только для администратора. Вот два пункта перечня OWASP Security Top 10, имеющие отношение к этой проблеме:

- ◆ нарушенная авторизация на уровне объектов³⁴;
- ◆ нарушенная авторизация на функциональном уровне³⁵.

Оба они направлены на обеспечение авторизации и гарантируют, что запросы к вашему API имеют право на выполнение операции. Подробнее это рассматривается в разд. «Обеспечение выполнения авторизации» главы 7.

Неправильная конфигурация безопасности

Неверное конфигурирование системы безопасности не ограничивается одной из описанных категорий STRIDE, поскольку неправильная конфигурация может иметь место в самых разных местах. Например, при разглашении сведений, когда неверно

³⁴ См. <https://oreil.ly/26Dpz>.

³⁵ См. <https://oreil.ly/gPaqN>.

назначено разрешение, или при отказе в обслуживании и в политике ограничения количества запросов, когда некорректно установлена открытая политика отказа. Решение проблемы неправильной конфигурации системы безопасности направлено на обеспечение того, чтобы имеющиеся средства защиты были правильно сконфигурированы. И это еще один элемент, требующий внимания при оценке каждой части STRIDE на предмет наличия угроз. Известно, что неправильно настроенная система безопасности — это хуже, чем ее полное отсутствие, поскольку пользователи ведут себя совершенно иначе, если считают, что их действия и данные не защищены. Скорее всего, вы обычно будете пользоваться такими функциями безопасности, как, например, Transport Layer Security (TLS), или другими, возможно, специфичными для API или настроек, — например, списком разрешенных IP³⁶.

В нашем примере API-шлюз является ключевым местом, где неправильная конфигурация системы безопасности может привести к катастрофическим последствиям. Его настройке следует уделить особое внимание, поскольку API-шлюз выполняет роль «входной двери».

Терминирование TLS-соединений

Протокол TLS гарантирует, что получаемый вами трафик не был перехвачен и изменен. Кроме того, сертификаты TLS предоставляют информацию о владельце домена, поэтому вы можете быть уверены в том, с кем установлена связь. Поскольку API-шлюз работает со всем входящим трафиком, в нем можно включить TLS. Наличие централизованного места для управления внешними TLS-сертификатами для входящих запросов также удобно — по сравнению с отсутствием шлюза, когда сертификаты TLS должны быть добавлены на каждый веб-сервер, прокси-сервер и приложение, обрабатывающее трафик запросов, что сложнее в управлении и более вероятно подвержено ошибкам. Важно использовать современный протокол и надежное шифрование, и на момент подготовки книги рекомендовалось использовать TLS 1.2 или более позднюю версию из-за известных проблем с более ранними версиями этого протокола³⁷.

Совместное использование ресурсов разными источниками

Совместное использование ресурсов разными источниками (Cross-Origin Request Sharing, CORS) — это механизм на основе HTTP-заголовков. Он позволяет серверу указывать любые источники (домен, схему или порт), отличные от его собственного, с которых браузер должен разрешить загрузку ресурсов. Поддержка CORS — основное требование любого современного веб-браузера, и в целях безопасности браузеры ограничивают кросс-оригинальные HTTP-запросы, инициированные из скриптов. Работа CORS заключается в том, что веб-браузер делает «предваритель-

³⁶ Списки разрешенных IP-адресов в буквальном смысле слова представляют собой списки IP-адресов, которым разрешено подключаться к вашей системе. Если подключаемый IP-адрес не входит в этот список, то запрос отклоняется.

³⁷ Большинство коммерческих API-шлюзов по умолчанию разрешают использовать только текущие версии TLS, поэтому, если в случае необходимости вам придется включить более слабые версии, имейте в виду наличие в них известных уязвимостей.

ные» запросы, чтобы убедиться, что ему разрешено выполнить нужный вызов. Вы можете узнать, так ли это, заглянув в «Инструменты разработчика» (Developer Tools) браузера. В разделе «Сетевые вызовы» (Network Calls) этих инструментов обычно отображаются запросы HTTP Options — как правило, это CORS-запросы³⁸.

Усиление директив безопасности

Запрос к конечной точке API может содержать произвольную полезную нагрузку, включая заголовки и данные. Хотя все подлинные запросы будут соответствовать ожидаемому контракту, злоумышленник может добавить неизвестные, неверные или искаженные заголовки и данные, пытаясь получить доступ или иным образом скомпрометировать вашу систему. Необходимо принять меры по снижению этого показателя. В нашем примере, например, необходимо подумать о внедрении разрешающего списка HTTP-заголовков в API-шлюзе и удалении всех недопустимых HTTP-заголовков. Злоумышленник может передать в API Участник дополнительные HTTP-заголовки — например: X-Assert-Role=Admin или X-Impersonate=Admin. При этом он надеется, что их никто не удалит и они будут использоваться внутри системы, что предоставит ему некоторые дополнительные полномочия.

Этап 5: Оценка рисков угроз

Когда вы проводите собственное моделирование угроз и в итоге получаете их список, важно понять приоритетность их устранения. Именно этому посвящен пятый этап процесса моделирования угроз. Для оценки угроз можно использовать качественный расчет риска, известный под аконимом DREAD. Как и STRIDE, DREAD был разработан в компании Microsoft. Эта методология дает возможность добавлять к угрозам значения риска. Хотя DREAD больше не используется компанией Microsoft, он по-прежнему применяется многими компаниями и пропагандируется как полезный способ определения степени риска угрозы. У DREAD простая система оценок, основанная на следующих понятиях:

- ◆ **Damage (Повреждения).**
Насколько серьезной может быть атака?
- ◆ **Reproducibility (Воспроизводимость)**
Можно ли легко воспроизвести атаку?
- ◆ **Exploitability (Возможность использования)**
Насколько легко провести успешную атаку?
- ◆ **Affected Users (Затронутые пользователи).**
Сколько пользователей будет затронуто?
- ◆ **Discoverability (Возможность обнаружения)**
Какова вероятность обнаружения этой угрозы?

³⁸ Полное объяснение механизма CORS можно найти в статье по адресу <https://oreil.ly/k70rh> на сайте Mozilla.

Каждая угроза оценивается по этим категориям DREAD, где каждая из них получает оценки от 1 до 10 баллов. Значение риска, присваиваемое угрозе, равно

$$(\text{Повреждения} + \text{Воспроизводимость} + \text{Возможность использования} + \text{Затронутые пользователи} + \text{Возможность обнаружения}) / 5.$$

Рассмотрим для примера угрозу, показанную на рис. 6.6. Она представляет собой DDoS-атаку на API-шлюз, не содержащий ограничения на количество запросов.

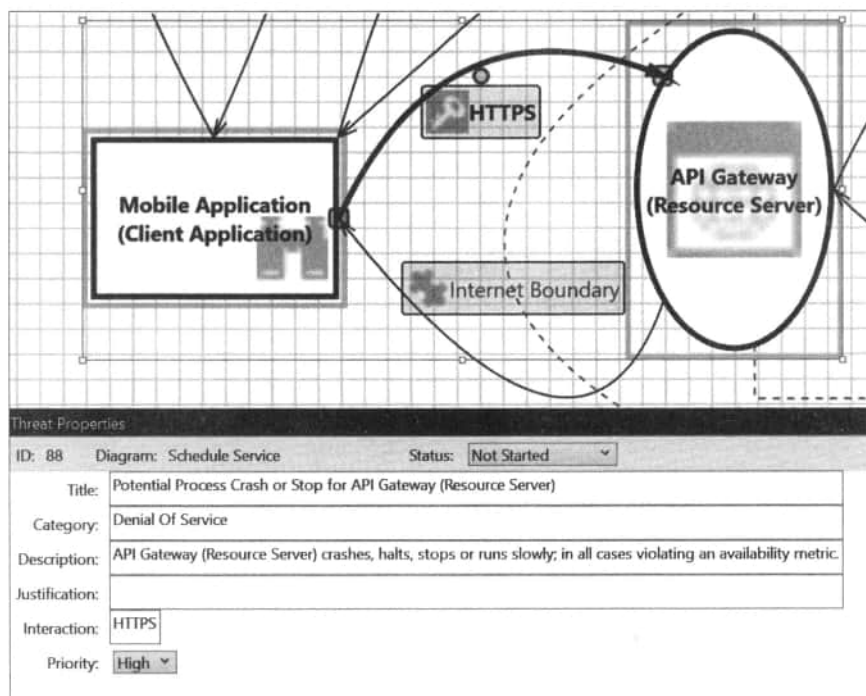


Рис. 6.6. Диаграмма потока данных для угрозы спуфинга TCP

Рейтинг этой угрозы будет следующим:

♦ **Damage (Повреждения): 8**

Ограничение на количество запросов отсутствует. Это серьезный повод для беспокойства, поскольку позволяет любому человеку отправлять сколько угодно запросов к API-шлюзу и потенциально перегружать его, делая непригодным для использования.

♦ **Reproducibility (Воспроизводимость): 8**

Многочисленное обращение к API-шлюзу с большим количеством ежесекундных запросов приведет к деградации и в конечном счете к прекращению работы шлюза.

♦ **Exploitability (Возможность использования): 5**

Злоумышленник может находиться вне нашей сети и пытаться провести DoS-атаку. Сначала API-шлюз проверяет аутентификацию и авторизацию для выпол-

нения запроса. Это означает, что запрос должен исходить от одного из наших правомочных и известных клиентских приложений, интегрированных с системой.

◆ **Affected Users (Затронутые пользователи): 10**

Это может иметь катастрофические последствия, поскольку если шлюз будет недоступен, то это отразится на всех наших пользователях.

◆ **Discoverability (Возможность обнаружения): 10**

Эту слабость легко обнаружить любому, кто захочет воспользоваться лазейкой и нанести ущерб нашей системе.

Суммарный балл составляет $(8 + 8 + 5 + 10 + 10) / 5 = 8,2$.

Следует отметить, что значения, присваиваемые риску, носят субъективный характер. Чтобы получить более согласованную оценку, для каждой категории следует определить, какой смысл несут значения: например, если затронуты все пользователи, то оценка равна 10, если затронуты все внутренние или все внешние пользователи, то оценка равна 7, если затронута половина группы, то оценка равна 3, а если никто не затронут, то оценка равна 0.

Для практического исследования все выявленные угрозы собираются, оцениваются, а затем расставляются по приоритетам. В нашем случае наиболее приоритетным пунктом будет отсутствие DDoS-защиты для API-шлюза. Как было отмечено ранее, решение этой проблемы заключается в том, чтобы реализовать ограничение на количество запросов и снизить нагрузку для API-шлюза.

Другие инструменты оценки рисков

Существуют и другие способы оценки угроз — один из них DREAD-D (произносится как Dread минус D). При расчете риска DREAD один из пунктов, обозначенных буквой «D», — это «Возможность обнаружения» (Discoverability), что в некоторых случаях может означать *безопасность через неясность* (Security Through Obscurity)³⁹, а это считается ужасным способом защиты любых данных. Соответственно этот элемент отбрасывается, и способ получает название DREAD-D. Другим доступным инструментом стал стандарт под названием «Общая система оценки уязвимостей» (Common Vulnerability Scoring System, CVSS)⁴⁰. Его можно применять для измерения серьезности (т. е. размера повреждения) эксплуатируемой уязвимости. CVSS используется NIST для оценки уязвимостей их базы CVE, поэтому если вы когда-нибудь обратитесь к CVE, то сможете найти CVSS. Например, применение CVSS можно заметить, посмотрев на уязвимости Log4J CVE⁴¹ и NIST CVSS⁴².

Этап 6: Валидация

Шестой и заключительный этап процесса моделирования угроз — подтверждение того, что цели безопасности достигнуты, и постановка вопроса о необходимости

³⁹ См. <https://oreil.ly/rGpCD>.

⁴⁰ См. <https://oreil.ly/YJmOE>.

⁴¹ См. <https://oreil.ly/d4EMu>.

⁴² См. <https://oreil.ly/XIcbd>.

еще одного пересмотра. В рамках моделирования угроз вы должны были оценить все обнаруженные и идентифицированные угрозы и предпринять действия по их снижению. Также необходимо убедиться в том, что цели безопасности, поставленные в начале моделирования угроз, достигнуты. Моделирование угроз должно представлять собой рекурсивный процесс, в ходе которого при каждом запуске выявляются ранее неизвестные проблемы. Также следует периодически и непрерывно проводить процесс моделирования угроз, особенно при добавлении новой функциональности в систему, а также в связи с постоянным развитием внешней среды угроз.

Моделирование угроз — это очень трудоемкий навык, и для его освоения требуется время. Однако, как и в случае с любым другим навыком, чем чаще он используется и интегрируется в привычный рабочий процесс, тем легче его освоить.

Заключение

В этой главе вы узнали, как проводить моделирование угроз как на основе тематического исследования, так и как применять его к вашим собственным системам и API.

В этой главе мы на нашем тематическом примере рассмотрели, как проводится моделирование угроз, и узнали, как применить его к собственным системам и API:

- ◆ неспособность обеспечить безопасность API влечет за собой серьезные финансовые санкции и ущерб репутации;
- ◆ моделирование угроз для системы на базе API обычно начинается с создания диаграммы потоков данных (DFD). Для быстрого анализа и выявления потенциальных угроз можно использовать автоматизированные средства;
- ◆ для проведения моделирования угроз не обязательно быть экспертом в области безопасности, ключевым навыком послужит подход «мыслить как злоумышленник»;
- ◆ процесс моделирования угроз включает в себя: определение целей, сбор необходимой информации, декомпозицию системы, выявление угроз, оценку риска этих угроз, а также проверку результатов и действий;
- ◆ OWASP API Security Top 10 — это отличный ресурс, позволяющий понять, каких угроз следует ожидать;
- ◆ методология STRIDE направляет ваши действия на борьбу с угрозами спуфинга, подмены, отрицания, разглашения сведений, отказа в обслуживании и расширения полномочий;
- ◆ методология DREAD может быть использована для расчета качественной метрики риска, что поможет определить приоритетность угроз, требующих первоочередного устранения;
- ◆ API-шлюз в рамках системы на основе API часто может обеспечить высокоуровневое снижение выявленных рисков. Однако по мере роста распределен-

ности систем необходимо всегда учитывать индивидуальные реализации сервисов и межсервисное взаимодействие.

Вы познакомились с различными существующими угрозами и способами их нейтрализации. Однако при возврате данных потребителям API необходимо убедиться в подлинности этих потребителей. А потребитель API может выполнять только те действия, на которые у него есть права. Чтобы понять, как определить, кто выступает в роли вызывающего абонента и что он может делать, в следующей главе мы более подробно рассмотрим аутентификацию и авторизацию.

Аутентификация и авторизация API

Из предыдущей главы вы узнали, как моделировать угрозы для систем на базе API, а также о десятке лучших стандартов безопасности API-систем по версии OWASP API Security Top 10. А PI Участник готов к приему трафика из внешнего мира, однако как именно идентифицировать потребителя API? В этой главе мы рассмотрим аутентификацию и авторизацию для API. *Аутентификация* говорит нам о том, кем является вызывающий абонент, а *авторизация* — о том, что ему разрешено делать.

Начнем мы с подробного рассказа о том, что такое аутентификация и авторизация для API, и продолжим разговором о важности защиты API и потенциальных ограничениях при использовании ключей и токенов API. OAuth2 — это система авторизации на основе токенов, представленная в 2012 году и быстро ставшая отраслевым стандартом для защиты API и определения действий, которые приложение может выполнять с API. Значительная часть этой главы посвящена OAuth2 и описанию подходов к обеспечению безопасности, предлагаемых как для конечных пользователей, так и для системных взаимодействий. Потребителям API иногда необходимо знать данные пользователя, от имени которого они действуют, и чтобы показать, как этого можно добиться, мы представим систему OpenID Connect (OIDC)¹.

Различные подходы к обеспечению безопасности мы проиллюстрируем на примере подготовки API Участник для внешнего использования системой CFP (Call for Papers), которая позволяет клиентам представлять доклады для потенциальных выступлений.

Аутентификация

Аутентификация — это процесс проверки подлинности. Для случая пользователя наиболее традиционным считается предоставление им своих учетных данных в виде имени пользователя и пароля, хотя в настоящее время в стандартный процесс входа в систему всё чаще включается *многофакторная аутентификация* (Multi-Factor Authentication, MFA)², которая обеспечивает более высокий уровень уверенности в том, что пользователь является тем, за кого себя выдает. Для аутентификации между машинами учетные данные могут быть представлены в виде ключей или

¹ См. далее разд. «Введение в OIDC».

² См. <https://oreil.ly/4WQkd>.

сертификатов. Проводя проверку подлинности предъявленных учетных данных, мы определяем, кто пытается взаимодействовать с нашими системами.

Разберемся с этим в контексте нашего сервиса Участник. API Участник содержит персональные данные (PII) — такие как имя пользователя и адрес его электронной почты. И пользователь ожидает, что эти данные будут защищены. Для защиты этой информации первым шагом при попытке входа в систему следует подвергнуть сомнению личность участника и провести идентификацию вызывающего API. Подтверждение подлинности участника называется *аутентификацией*. После аутентификации вызывающего абонента API Участник устанавливает, к чему ему разрешен доступ и что он может получить, — этот тип проверки прав представляет собой *авторизацию*.

На рис. 7.1 показаны взаимодействия пользователей с API Участник. Одно из взаимодействий заключается в том, что мобильное приложение подключается через API-шлюз и запрашивает вход в API Участник. Другое взаимодействие происходит аналогичным образом с системой CFP, однако эта система принадлежит третьей стороне. Рассмотрим имеющиеся у нас варианты аутентификации конечных пользователей (пользователя мобильного приложения и докладчика системы CFP) и взаимодействия нашей системы с системой CFP.

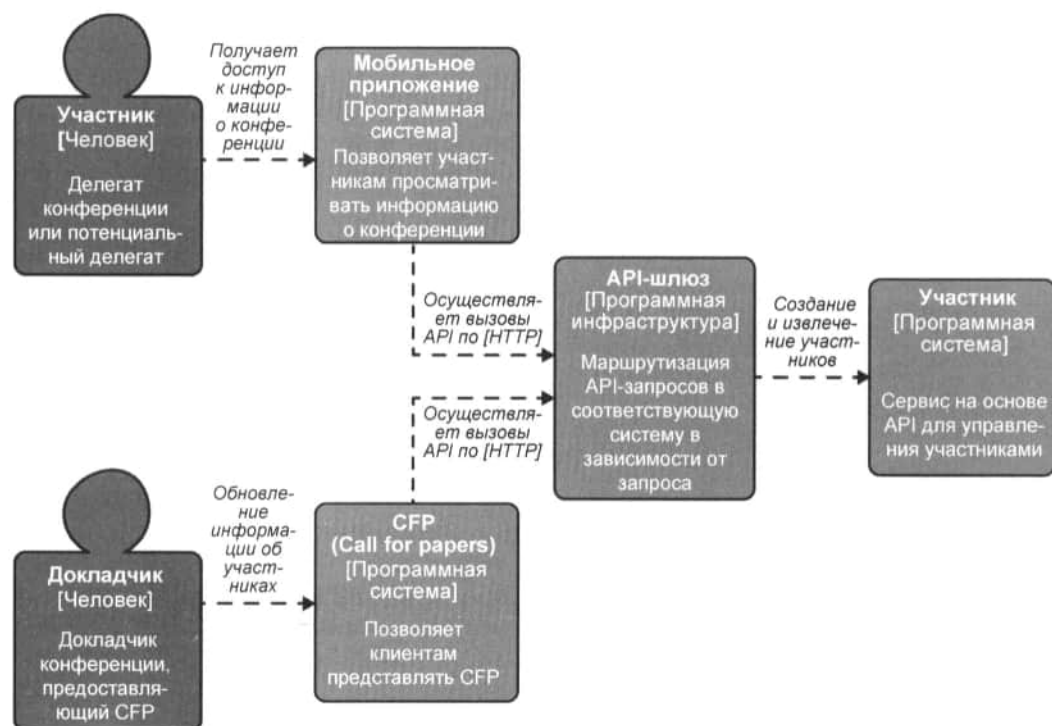


Рис. 7.1. Схема обеспечения безопасности в нашем примере

Аутентификация конечных пользователей на основе токенов

Мобильное приложение действует от имени участника, получает и отображает информацию о нем. При аутентификации на основе *токенов* пользователь вводит свое имя пользователя и пароль, и они обмениваются на токен. Выдаваемый токен зависит от реализации, но в простейшем случае это может быть непрозрачная строка. Токен передается в REST-запросе как часть *bearer-заголовка аутентификации на предъявителя* (Authentication Bearer)³. Токены — это конфиденциальная информация, поэтому важно, чтобы REST-запрос был отправлен по протоколу HTTPS для защиты передаваемой информации. После получения токена в составе запроса происходит его изучение и проверка на валидность. На рис. 7.2 показан типичный процесс поиска токена, когда он хранится в базе данных.

Токен должен иметь ограниченный срок действия — например, в течение часа, и по истечении его срока действия пользователю необходимо будет получить новый токен. Преимущество токенов заключается в том, что долго хранящиеся учетные данные, такие как пароли, не передаются по сети при каждом запросе на доступ к ресурсам.

На первый взгляд, использование токенов может показаться идеальным, однако основным недостатком этого подхода стала необходимость ввода пользователем имени и пароля в приложении, обращающемся к API для получения данных. Кроме того, когда токен помещается в хранилище, поиск токена для проверки его валидности каждый раз может вызывать проблемы с производительностью, и это необходимо устранить. Предпочтительнее использовать обладающий целостностью токен, который может быть проверен внутри процесса.



Можно получить доступ к API, используя базовую (HTTP Basic) аутентификацию, однако, если стороннее приложение запрашивает доступ к API от вашего имени, это означает передачу вашего имени пользователя и пароля⁴. Мы рекомендуем запретить использование подхода HTTP Basic для доступа к вашим API.

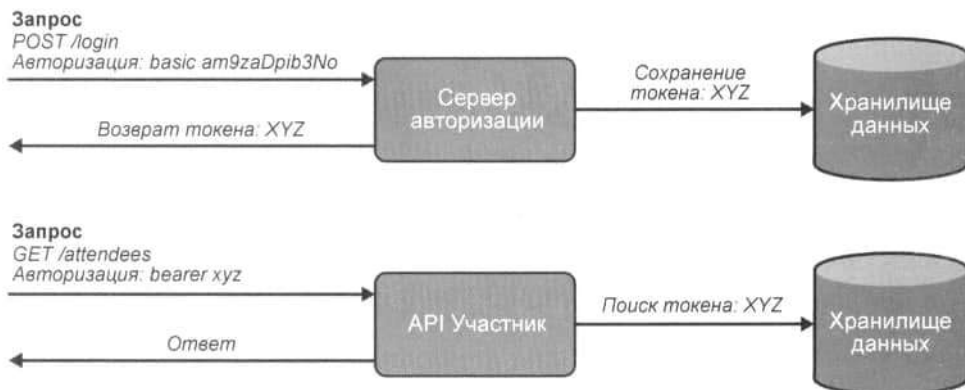


Рис. 7.2. Процесс проверки поиска токена на стороне сервера

³ См. <https://oreil.ly/cU6Al>.

⁴ Если вы не знаете, что такое HTTP Basic, обратитесь к спецификации RFC7617 (<https://oreil.ly/NAKYU>).

Аутентификация между системами

В некоторых ситуациях конечный пользователь не участвует во взаимодействии, и требуется обеспечить связь между несколькими системами. Одним из вариантов может служить использование API-ключа, не соответствующего какому-либо определенному стандарту. При этом API-ключ должен быть безопасным, т. е. генерироваться с помощью криптографически защищенного генератора случайных чисел и иметь не поддающуюся определению длину. Если такой ключ угадывается (он короткий и детерминированный), то это создает уязвимость для взлома клиента. Обычно API-ключи представляют собой строки длиной 32 символа (256 битов). Чтобы получить доступ к API с помощью API-ключа, достаточно добавить ключ в *заголовок запроса*⁵ и отправить его на конечную точку. API-ключ ассоциируется с приложением или проектом, что позволяет идентифицировать запрашивающего⁶. Использование API-ключа очень похоже на использование пароля. На рис. 7.3 показан пример использования API-ключа в составе запроса.

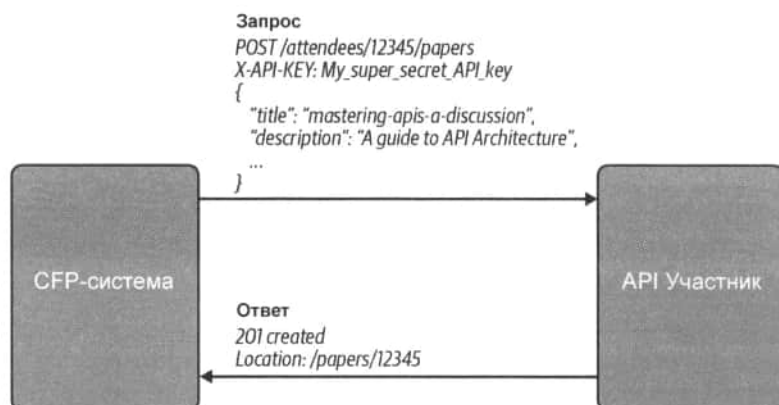


Рис. 7.3. Внешняя CFP-система вызывает API Участник с помощью API-ключа

Почему не следует смешивать ключи и пользователей?

Рассмотрим сценарий, в котором докладчик использует CFP-систему, принадлежащую третьей стороне, и эта CFP-система запрашивает обновление адреса электронной почты, связанного с данными этого пользователя. То, что система CFP использует API-ключ и может быть идентифицирована, не означает, что эта сторонняя система должна иметь возможность подтверждать, кем является конечный пользователь, или от чьего имени он действует. В таком случае доверие ко всей системе оказалось бы в руках третьей стороны. Конечно, можно было бы передовать в CFP-систему имя пользователя и пароль (с использованием HTTP Basic) вместе

⁵ Заголовок запроса — это пользовательский заголовок (например, X-API-KEY: My_super_secret_API_Key) или заголовок авторизации.

⁶ На сайте Google есть хорошая статья на эту тему (<https://oreil.ly/fraOb>).

с API-ключом, чтобы сервис Участник мог выполнить аутентификацию пользователя. Однако, как мы уже предупреждали, это означает, что пользователь должен передать системе CFP свое имя пользователя и пароль для сервиса Участник, что нежелательно. Идеальным вариантом была бы ситуация, когда CFP-система может вызывать сервис Участник, но любой запрос, выполняемый системой от имени пользователя, не требовал бы обмена учетными данными и выполнялся с особого разрешения пользователя. Решением этой проблемы, по сути, станет использование системы OAuth2.

OAuth2

OAuth2 — это система авторизации на основе токенов, существующая с 2012 года. Он заменила систему OAuth⁷, которая все еще существует, но ее использование ограничено. OAuth2 позволяет пользователю дать согласие на то, чтобы стороннее приложение получило доступ к его данным от его имени. Согласие, предоставляемое пользователем, служит авторизацией — он разрешает или запрещает доступ. OAuth2 избавляет пользователя от необходимости передавать свои учетные данные третьей стороне, что дает ему возможность защитить свои данные. Это делает систему OAuth2 привлекательной, поскольку она поддерживает решение задач, рассмотренных в предыдущем разделе.

Для дальнейшего изучения важно понять, какие роли существуют в спецификации OAuth2 (приведенные здесь определения взяты непосредственно из спецификации OAuth2⁸):

◆ Владелец ресурса.

Сущность, способная предоставить доступ к защищаемому ресурсу. Когда в роли владельца ресурса выступает человек, его называют *конечным пользователем*.

◆ Сервер авторизации.

Сервер выдает клиенту access-токены (токены доступа) после успешной аутентификации владельца ресурса и получения авторизации. Большинство поставщиков идентификационных данных, такие как Google или Auth0, будут выступать серверами авторизации OAuth2.

◆ Клиент.

Приложение, выполняющее запросы к защищенным ресурсам от имени владельца ресурса и с его разрешения.

◆ Сервер ресурсов.

Сервер, хранящий защищенные ресурсы. Он способен принимать и отвечать на запросы защищенных ресурсов с использованием access-токенов.

⁷ См. <https://oreil.ly/hQllg>.

⁸ См. <https://oreil.ly/1916b>.

Роль сервера авторизации при взаимодействии с API

У сервера авторизации две конечные точки:

- ◆ конечная точка авторизации используется в тех случаях, когда *владельцу ресурса* необходимо разрешить доступ к защищаемым ресурсам;
- ◆ конечная точка токена используется *клиентом* для получения access-токена.

Если бы сервис Участник вызывался непосредственно клиентом, то он был бы *сервером ресурсов*, поскольку на нем размещаются защищенные ресурсы. Однако сервер ресурсов не обязательно должен быть отдельным приложением — он может представлять собой целую систему. Одна из популярных схем — использование API-шлюза в качестве сервера ресурсов, как показано на рис. 7.4. Два клиента — мобильное приложение и система CFP — обращаются к сервису Участник через API-шлюз. За API-шлюзом может находиться несколько сервисов, но для клиента API-шлюз все равно будет выступать в роли сервера ресурсов, поскольку на нем размещаются защищенные ресурсы. В рассматриваемом случае владельцами ресурсов будут участники, использующие мобильное приложение, и докладчики, использующие систему CFP.



Рис. 7.4. API-шлюз в качестве сервера ресурсов

Веб-токены JSON (JWT)

Веб-токены в нотации JavaScript (JavaScript Object Notation, JSON) — это стандартизированный формат токена RFC (Request for Comments)⁹, ставший де-факто стандартным токеном для OAuth2. Веб-токен JSON (JSON Web Token), также известный как JWT (произносится «джот»), состоит из *утверждений*, имеющих связанные с ними значения. JWT структурированы и закодированы с использованием

⁹ См. <https://oreil.ly/9AAY1>.

стандартов, обеспечивающих неизменяемость токена и возможность его дополнительного шифрования. Они особенно полезны при передаче информации в «ограниченных по пространству средах, таких как заголовки авторизации HTTP»¹⁰.

Вот пример JWT:

```
{
  "iss": "http://mastering-api/",
  "sub": "18f913b1-7a9d-47e6-a062-5381d1e21ffa",
  "aud": "Attendee-Service",
  "exp": 1618146900,
  "nbf": 1618144200,
  "iat": 1618144200,
  "jti": "4d13ba71-54e4-4583-9458-562cbf0ba4e4"
}
```

В этом примере содержатся утверждения `iss`, `sub`, `aud`, `exp`, `nbf`, `iat` и `jti`, и все они зарезервированы для JWT RFC. Зарезервированные утверждения имеют особое значение. Они не являются обязательными в токене, однако предлагают отправную точку для получения минимального объема информации. Раскроем приведенные в примере токена сокращения утверждений и покажем, как они обычно используются:

◆ `iss` (issuer — эмитент).

Элемент выпустивший токен. Обычно это провайдер идентификации (например, Google или Auth0).

◆ `sub` (subject — субъект).

Уникальный идентификатор для подтверждения участника JWT. В случае мобильного приложения, действующего от имени пользователя, это будет собственно участник (например, Мэтью Оберн)¹¹. Если это соединение между серверами, то им может быть приложение (например, CFP-система). Значение `subject` не имеет никакого формата, и если вы определяете, каким должен быть `subject`, необходимо решить, должен ли он быть уникальным в пределах вашей системы или универсально уникальным (например, с использованием стандарта UUID¹²).

◆ `aud` (audience — аудитория).

Для кого предназначен этот токен.

◆ `exp` (expiration time — срок действия).

Когда истекает срок действия токена (в нашем случае через 45 минут после выпуска).

¹⁰ См. <https://oreil.ly/hArh0>.

¹¹ Электронную почту или имя пользователя вряд ли стоит считать удачным выбором, поскольку пользователи со временем могут их изменить. Наличие согласованного идентификатора упрощает управление.

¹² См. <https://oreil.ly/NmTRn>.

- ◆ `nbf` (not before — не ранее).

Токен нельзя использовать раньше этого времени (в нашем случае — того же времени, что и эмиссия).

- ◆ `iat` (issued at — выпущен в).

Время выпуска токена.

- ◆ `jti` (JWT ID).

Уникальный идентификатор для JWT.



Токены могут содержать дополнительную информацию — например, предпочтительное имя, электронную почту пользователя, сведения о стороне, выпустившей токен, и о том, какое приложение запросило токен. При использовании API с высоким уровнем безопасности обычно методом аутентификации на сервере авторизации становится утверждение, которое может быть использовано для проверки того, применял ли владелец ресурса MFA для аутентификации.

Кодирование и верификация веб-токенов JSON

Существуют два популярных механизма кодирования JWT, получивших свой собственный формат:

- ◆ JSON Web Signatures (JWS)¹³ обеспечивает целостность JWT. Содержимое токена видно любому, кто его получит. Однако утверждения подписываются цифровой подписью. Такой механизм гарантирует, что если содержимое токена будет изменено, токен немедленно станет недействительным;
- ◆ JSON Web Encryption (JWE)¹⁴ обеспечивает целостность, но при этом шифруется. Это означает, что содержимое токена не может быть исследовано.



Как правило, когда используется JWT, это означает применение формата JWS, а когда зашифрованный JWT — применение JWE.

Наиболее часто задействуется механизм JWS, и цифровая подпись осуществляется с помощью закрытого ключа. Открытый ключ свободно передается любой стороне, которой необходимо проверить целостность токена, и используется получателем токена для подтверждения того, что токен был подписан конкретной стороной-эмитентом.



Если вы используете JWT с JWS, то не следует вставлять в значения утверждений конфиденциальные данные. JWS обеспечивает целостность утверждений, однако любой, получивший JWT, может прочитать утверждения. Чтобы гарантировать, что JWT невозможно прочитать, используйте формат JWE.

JWT — отличный вариант формата токенов. API-сервисы получают JWT, проверяют его, выполняя верификацию подписи, и не нуждаются в поиске токена в базе

¹³ См. <https://oreil.ly/SOfsT>.

¹⁴ См. <https://oreil.ly/ZH1fR>.

данных. Поскольку access-токен будет получен от сервера авторизации, находящегося (скорее всего) под вашим контролем, вы можете добавить в JWT всю ожидаемую/требуемую информацию.

В полученном JWT необходимо проверить несколько его частей. Во-первых, проверяется подпись — на предмет того, что она выдана предполагаемой стороной и не была изменена или подделана. Затем должны быть проверены другие утверждения в токене — например, что срок действия токена не истек (утверждение `exp`) или что токен не используется до того, как это разрешено (утверждение `nbf`). Все выпускаемые токены должны быть недолговечными, поскольку долговечные токены представляют опасность в случае их утраты или кражи. Что касается долговечных утверждений, то в руководстве по цифровым технологиям Национального института стандартов и технологий (National Institute of Standards and Technology, NIST)¹⁵ говорится следующее:

Утверждения с длительным сроком существования подвержены большому риску быть украденными или воспроизведенными. Короткое время жизни утверждений снижает этот риск.

Не существует официального стандарта, определяющего, в течение какого времени должен быть действителен коротко- или долгоживущий токен. Типичное предполагаемое время существования короткоживущего токена составляет от одной до 60 минут, а долгоживущего — от одного года до десяти лет. Рекомендуется максимально сократить время существования токенов.

Использование JWT в качестве access-токена имеет много положительных сторон. Теперь рассмотрим их использование в рамках системы OAuth2.

Терминология и механизмы предоставления доступа OAuth2

Протокол OAuth2 разработан как расширяемый. Официальная спецификация OAuth2, выпущенная в 2012 году, предусматривала четыре вида доступа, но с тех пор были утверждены дополнительные виды доступа и модификации для расширения его использования. Это стало возможным благодаря тому, что OAuth2 представляет собой *абстрактный протокол*¹⁶ (рис. 7.5):

- ◆ Клиент запрашивает авторизацию у владельца ресурса.
- ◆ Владелец ресурса предоставляет или отказывает клиенту в доступе к своим ресурсам.
- ◆ Клиент запрашивает у сервера авторизации access-токен к предоставленным ему полномочиям.
- ◆ Сервер авторизации выдает access-токен, если клиент был авторизован владельцем ресурса.

¹⁵ См. <https://www.nist.gov/>.

¹⁶ См. <https://oreil.ly/BSIdZ>.

- ♦ Клиент делает запрос на ресурс к ресурсному серверу — в нашем случае к API. Запрос содержит access-токен как часть запроса.
- ♦ Сервер ресурсов вернет ресурс, если access-токен действителен.

Этот абстрактный протокол, показывающий, как должно работать предоставление доступа OAuth2, подчеркивает, что владелец ресурса имеет контроль над своими собственными ресурсами. Клиент запрашивает авторизацию у владельца ресурса — т. е. «могу ли я (приложение) получить доступ к вашим ресурсам от вашего имени?». Способ выдачи авторизации не имеет значения. Важно, чтобы владелец ресурса имел возможность предоставлять или запрещать доступ. При запросе ресурса с сервера ресурсов (т. е. при обращении к API) не имеет значения, каким образом клиент получил access-токен. Если запрос содержит действительный access-токен, сервер ресурсов выдает требуемый ресурс. Каждый этап изолирован и не требует информации о предыдущем этапе. Именно поэтому существуют различные варианты предоставления доступа для разных сценариев, поскольку у них есть собственные реализации, обеспечивающие безопасность этих шагов для рассматриваемой среды.

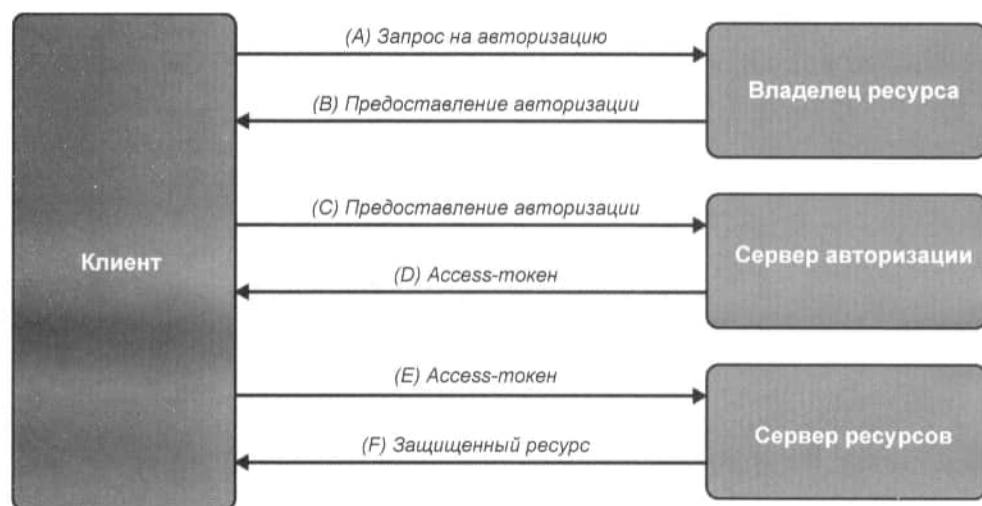


Рис. 7.5. Потoki абстрактного протокола

Руководство ADR: стоит ли рассматривать возможность использования OAuth2?

Важно, чтобы вы понимали причины, по которым следует перейти на использование OAuth2, и были уверены в том, что это станет для вас правильным выбором. Чтобы помочь в принятии этого решения, воспользуйтесь предлагаемым руководством ADR (см. табл. 7.1). Оно поможет вам определить, что подходит именно вам и какие обсуждения вам следует провести.

Таблица 7.1. Руководство ADR: нужно ли использовать OAuth2?

Решение	Следует ли использовать OAuth2 или существует другой стандарт аутентификации и авторизации, более предпочтительный для вашей операционной среды?
Пункты обсуждения	Когда вы начинаете работать с API, у вас появляется возможность решать или влиять на механизмы обеспечения их безопасности: • изучите текущие требования к безопасности и возможные изменения. Например, используются ли API только в плоскости управления или они также задействуются вне плоскости управления (возможно, с третьими сторонами)? • какую модель безопасности предполагается поддерживать? Требовали ли внешние интегрируемые сущности использовать определенную модель безопасности? • необходимо ли поддерживать несколько моделей аутентификации и авторизации? Это важно, если вы хотите перейти от существующей модели аутентификации к другой
Рекомендации	Использование OAuth2 обеспечит максимальную совместимость с другими пользователями API. Это промышленный стандарт с документацией и клиентскими библиотеками, облегчающими интеграцию. OAuth2 поддерживает работу как с конечными пользователями, так и с системами

Authorization Code Grant: предоставление доступа по коду авторизации

Authorization Code Grant (Auth Code Grant)¹⁷ — это реализация предоставления доступа OAuth2, абстрактного протокола, показанного на рис. 7.5. Это наиболее известный вариант предоставления доступа, и, скорее всего, вы пользовались им, даже не подозревая об этом¹⁸. Типичным вариантом предоставления доступа по коду авторизации с использованием Authorization Code Grant можно считать поддерживаемый сервером веб-сайт, который не является общедоступным для Интернета (т. е. может защищать секрет). Клиентское приложение, способное защитить секрет, называется *конфиденциальным клиентом*. На рис. 7.6 более подробно показано, как работает такое предоставление доступа:

1. Клиентское приложение направляет веб-браузер (на схеме Пользовательский агент — это веб-браузер) на сервер авторизации. Перенаправление на сервер авторизации будет содержать идентификацию клиента (ID клиента), а также информацию о том, какой вариант предоставления доступа используется (в нашем случае доступ Authorization Code Grant известен как *code*).
2. Сервер авторизации просит владельца ресурса (конечного пользователя) идентифицировать себя. Серверу авторизации необходимо знать, кто является владельцем ресурса, поэтому владелец ресурса должен пройти аутентификацию на сервере авторизации. Затем сервер авторизации может получить согласие от

¹⁷ См. <https://oreil.ly/CqMhB>.

¹⁸ Типичный сценарий: вы используете LinkedIn, и эта платформа запрашивает доступ к вашим контактам Gmail. LinkedIn перенаправляет вас на сайт Google, и вы входите в свою учетную запись Google. Затем появляется сообщение: «LinkedIn хочет получить доступ к вашим почтовым контактам». После вашего согласия LinkedIn получает доступ к вашей электронной почте.

владельца ресурса, если он предоставит авторизацию клиентскому приложению. (Этапы А и В предоставления доступа Authorization Code Grant — это все, что касается запроса авторизации. В абстрактном протоколе на рис. 7.5 они показаны как один этап А.)

3. Если авторизация подтверждена, то через Пользовательский агент клиентскому приложению передается код авторизации. (Этот этап совпадает с этапом В абстрактного протокола, где показано возвращение разрешения на авторизацию.)
4. Затем клиент запрашивает у сервера авторизации access-токен, представляя код авторизации. Сервер авторизации не может просто принять код авторизации от кого угодно. Клиентское приложение должно аутентифицироваться на сервере авторизации с помощью секрета, известного серверу авторизации и клиентскому приложению. (В абстрактном протоколе это этап С. Здесь грант на авторизацию отправляется на сервер авторизации для обмена.)
5. Если клиентское приложение успешно проходит аутентификацию и предъявляет действительный код авторизации, ему выдается access-токен. (В абстрактном протоколе это этап D, на котором выдается access-токен.)

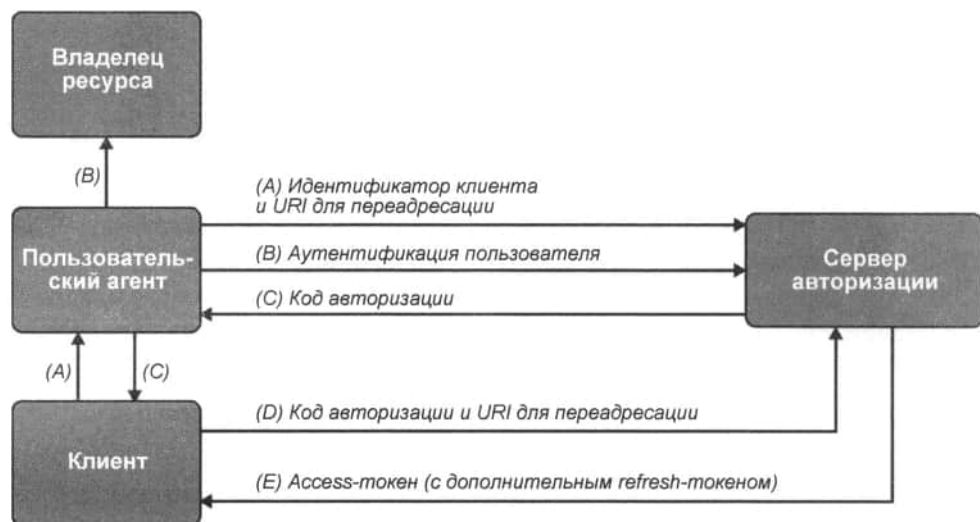


Рис. 7.6. Authorization Code Grant (Предоставление доступа по коду авторизации)

Это решение работает очень хорошо и долго было моделью по умолчанию для веб-приложений. Однако мир веб-сайтов эволюционировал, и теперь существуют одностраничные приложения (Single Page Application, SPA). SPA-сайты создаются на JavaScript и выполняются в браузере пользователя, что означает полную доступность исходного кода для пользователя. Отсюда также следует, что SPA-клиент OAuth2 не может защитить секрет и является *публичным клиентом*, поэтому использование с ним Auth Code Grant в его нынешнем виде невозможно.

Authorization Code Grant (+ PKCE)

В этом случае можно воспользоваться вариантом доступа Authorization Code Grant + PKCE¹⁹, позволяющим расширить возможности OAuth2 на SPA-приложения. Ключ PKCE (Proof Key for Code Exchange, ключ подтверждения для обмена кодами) здесь используется для защиты от атак перехвата. В рамках доступа Auth Code Grant + PKCE необходимы два дополнительных параметра: один — для запроса авторизации (`code_challenge`) и другой — для запроса access-токена (`code_verifier`). Параметр `code_verifier` — это криптографически случайная строка, сгенерированная клиентом, а параметр `code_challenge` — это хешированное значение `code_verifier`. Когда клиентское приложение инициирует запрос к серверу авторизации, оно посылает значение `code_challenge`, а когда запрашивается access-токен, код авторизации представляется вместе с `code_verifier`. Сервер авторизации может хешировать `code_verifier` — чтобы проверить его соответствие значению `code_challenge`, использованному для инициирования запроса токена. Такое расширение делает доступ более безопасным, т. к. параметр `code_verifier` должен быть только у оригинального клиента, что предотвращает атаки, при которых код авторизации может быть перехвачен и подменен на access-токен. На рис. 7.7 мы можем увидеть этот вариант доступа в действии:

1. Выполняется запрос на авторизацию, и параметр `code_verifier` отправляется на сервер авторизации. На схеме `t(code_verifier)` представляет собой преобразование `code_verifier` в `code_challenge`, а `t_m` — метод преобразования (как было показано ранее — это хеш).
2. Как и в случае с Authorization Code Grant, возвращается код авторизации.

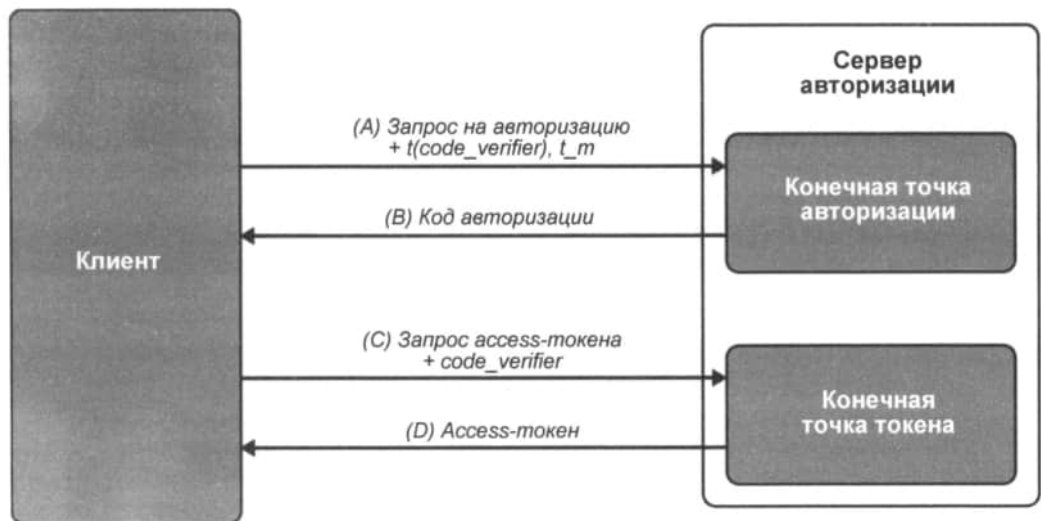


Рис. 7.7. Authorization Code Grant + PKCE

¹⁹ См. <https://oreil.ly/Kb0r2>.

3. Клиент запрашивает access-токен, отправляя запрос на авторизацию, состоящий из кода авторизации и параметра `code_verifier`. Секрет клиента не передается, т. к. это публичный клиент.
4. Затем access-токен выдается клиентскому приложению.

Возможно, глядя на эти шаги, вы зададитесь вопросом, как это соотносится с Authorization Code Grant без PKCE. На первый взгляд схема, представленная на рис. 7.7, отличается от той, что показана на рис. 7.6, но единственное реальное различие заключается в первом этапе — А. На рис. 7.7 этап А содержит запрос на авторизацию (как на этапе А в абстрактном протоколе), и процесс будет протекать так же, как и на этапах А и В в схеме Authorization Code Grant.



Для публичных клиентов должен использоваться PKCE. Однако и для конфиденциальных клиентов можно применить PKCE — в качестве дополнительной защиты.

Authorization Code Grant и его расширение PKCE будут работать в наиболее распространенных сценариях для ваших публичных и конфиденциальных клиентов при наличии в вашем случае конечного пользователя.

Практический пример: доступ к API Участник с помощью Authorization Code Grant

Доступа к API Участник добиваются два клиентских приложения. Оба они собираются воспользоваться Authorization Code Grant для доступа к API Участник от имени пользователей (владельцев ресурсов). Внешняя система CFP относится к конфи-

Authorization Code Grant

Authorization code grant + PKCE



Рис. 7.8. Применение Authorization Code Grant в нашем практическом примере

денциальным клиентам. Такой клиент может хранить секрет, а это означает возможность использования Authorization Code Grant. Мобильное приложение представляется публичным клиентом и не способно хранить секрет, поэтому необходимо задействовать Authorization Code Grant + PKCE. Действия по запросу access-токена внешней CFP-системой и мобильным приложением и его использованием для доступа к сервису Участник показаны на рис. 7.8. Здесь также видно, что использование PKCE не меняет этапы более высокого уровня и путь пользователя.

Refresh-токены

Хорошей практикой считается выпуск токенов с коротким сроком действия, однако необходимость повторного ввода пользователем имени и пароля сводит на нет многие их преимущества. А вот у refresh-токенов (токенов обновления) — длительный срок существования, и они используются клиентом для запроса дополнительных access-токенов по истечении срока действия предыдущего токена. При этом refresh-токены запрашиваются как часть запроса авторизации — т. е. конечный пользователь не участвует в запросе дополнительных access-токенов. В соответствии с передовыми методами²⁰ обеспечения безопасности при обнаружении дважды использованного refresh-токена происходит немедленное аннулирование активного refresh-токена. Refresh-токены содержат дополнительные учетные данные и долго хранятся, поэтому важно обеспечить их сохранность и исключить утечку. Если в какой-то момент клиенту необходимо отказать в доступе, в том числе если владелец ресурса не хочет, чтобы клиент имел дальнейший доступ к его ресурсам, refresh-токен может быть отозван. И когда в следующий раз клиентское приложение запросит новый access-токен (короткоживущий), оно будет остановлено. Это означает, что может возникнуть окно, когда у клиента есть действительный access-токен, но доступа он уже иметь не должен. Именно поэтому важно создавать короткоживущие токены.

Client Credentials Grant: предоставление доступа к учетным данным клиента

Клиент для Client Credentials Grant²¹ — это конфиденциальный клиент, ведь ему необходимо хранить секрет. Поскольку речь идет о взаимодействии между машинами, соединение устанавливается заранее, и доступ (то, что клиент имеет право делать) должен быть заранее оговорен.

Процесс получения клиентом access-токена²² очень прост (рис. 7.9):

1. Клиентское приложение аутентифицируется на сервере авторизации и запрашивает access-токен. Клиент также идентифицирует используемый грант, обозначенный как `client_credentials`.

²⁰ См. <https://oreil.ly/veXgR>.

²¹ См. <https://oreil.ly/6GA8r>.

²² Если хотите подробнее рассмотреть вопрос о повышении уровня безопасности клиентского приложения, получающего access-токены, обратитесь к документу RFC8705 (<https://oreil.ly/kOGTQ>). В этой спецификации для получения access-токенов вместо строк секретов используется Mutual TLS.

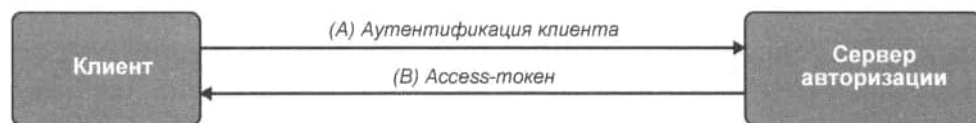


Рис. 7.9. Client Credentials Grant: предоставление доступа к учетным данным клиента

2. При успешной аутентификации клиентского приложения сервер авторизации возвращает access-токен.

Никаких дополнительных действий не требуется, т. к. нет владельца ресурса, предоставляющего разрешение. Клиент действует от своего имени, поэтому от него требуется только идентифицировать себя.

Практический пример: доступ к API Участник из CFP-системы при помощи Client Credentials Grant

Каждые три месяца внешняя CFP-система формирует отчет, показывающий, сколько участников представили свои доклады и стали докладчиками. Этот отчет формируется не от имени участника, а выполняется по требованию внешней CFP-системы. Клиент (внешняя CFP-система) регистрируется на сервере авторизации²³. В сервисе Участник клиент добавляется в список клиентов, имеющих доступ к сервису, и настраивается так, чтобы иметь возможность читать информацию об участниках и запрашивать, какие пользователи представили доклады, — это и есть предварительный доступ. Когда клиент хочет получить доступ к API Участник, он запрашивает access-токен у сервера авторизации и затем использует его при обращении к API.

Итак, вы узнали, как использовать OAuth2 для взаимодействия между машинами, но что делать, если ваш случай до сих пор не рассматривался?



Refresh-токены не используются совместно с Client Credentials Grant — вместо этого клиент запрашивает новый access-токен.

Дополнительные варианты предоставления доступа OAuth2

Существует намного больше вариантов предоставления доступа (грантов) OAuth2, чем два только что рассмотренных. Другие имеющиеся стандартизированные варианты предоставления доступа приведены далее, но мы не станем рассматривать их подробно:

- ◆ Device Authorization Grant используется для устройств, имеющих ограниченный ввод или не имеющих браузера. Это делает его полезным для IoT-устройств — таких как «умный» холодильник или проекты Raspberry Pi;

²³ Нет ничего страшного в том, что клиент зарегистрирован для нескольких типов предоставления доступа, — субъект вызывающего абонента будет различаться в зависимости от используемого гранта. Как мы здесь видим, в учетных данных клиента субъектом является сам клиент, делающий запрос, а не владелец ресурса.

- ◆ Implicit Grant широко применялся для SPA, но в настоящее время он заменен на Authorization Code Grant + PKCE;
- ◆ Resource Owner Password Credentials Grant ранее действовал как отступление от HTTP Basic для начала работы клиентских приложений с OAuth2. Сейчас он не рекомендуется к использованию.

Руководство ADR: какой из грантов OAuth2 стоит поддерживать

Как мы увидели, существует множество вариантов предоставления доступа OAuth2. Важно выбрать подходящий именно для вашего случая вариант. Или вариант, который вы хотите поддержать. Руководство ADR, приведенное в табл. 7.2, содержит пункты обсуждения и соображения. Их стоит обдумать, прежде чем выбрать свой вариант предоставления доступа.

Таблица 7.2. Руководство ADR: какой из грантов OAuth2 стоит поддерживать

Решение	Какие гранты OAuth2 следует поддерживать?
Пункты обсуждения	Определите, какие типы клиентов будут взаимодействовать с вашими API: • Требуется ли вам поддержка IoT-устройств и варианта предоставления доступа Device Authorization Grant? • Есть ли у вас старые клиенты, представленные в виде SPA и поддерживающие только Implicit Grant? • Следует ли категорически запретить использование гранта Resource Owner Password Credential Grant? Если у вас уже есть модель безопасности для аутентификации и авторизации, стоит ли вам переходить на OAuth2? • Какой вариант предоставления доступа лучше всего отражает вашу модель взаимодействия? • Смогут ли клиенты мигрировать на этот вариант? Если они находятся под вашим контролем или у вас небольшое количество сторонних элементов, то начать их привлечение к миграции будет значительно проще. • Должны ли все новые клиенты использовать новый OAuth2 Grant?
Рекомендации	Мы рекомендуем использовать OAuth2 и задействовать только необходимые варианты предоставления доступа, а при необходимости подключать дополнительные. Если у вас есть действующая модель безопасности и много вносящих плату клиентов, то заставить их перейти на использование OAuth2 может оказаться нецелесообразным. Однако, возможно, вам придется перестроить свою архитектуру безопасности на использование OAuth2, чтобы сделать ее более стандартизированной, поскольку это также может быть запросом со стороны третьих сторон. В таком случае им не нужно будет создавать индивидуальные взаимодействия для вашей модели безопасности. Зачастую внедрение OAuth2 в API-систему проще всего начать с предоставления доступа к учетным данным клиента (Client Credentials Grant)

Области видимости OAuth2

Области видимости являются важным механизмом в OAuth2 и эффективно используются для ограничения доступа клиента, действующего от имени пользователя. При первой аутентификации конечный пользователь получает экран согласия, со-

державший описание, для чего клиент запрашивает доступ. Например, «Приложение хотело бы читать напоминания о встречах в вашем календаре» и «Приложение хотело бы бронировать встречи в вашем календаре». Конечный пользователь контролирует ситуацию и может ограничить действия, доступные для выполнения клиентом от его имени.

Практический пример:

применение областей видимости OAuth2 к API Участник

Рассмотрим практический пример, показывающий области моделирования участников с помощью некоторых конечных точек. При этом представим, что в унаследованной конференц-системе также есть две открытые конечные точки:

♦ API Участник:

- GET - /attendees — получение списка участников;
- GET - /attendees/{attendee_id} — получение сведений об участнике;
- POST - /attendees — регистрация нового участника;
- PUT - /attendees/{attendee_id} — обновление информации об участниках.

♦ Унаследованный API конференц-системы:

- GET - /conferences — получение списка конференций;
- POST - /conferences — создание новой конференции.

Внешнее CFP-приложение должно иметь доступ только к API Участник, поэтому вам, как владельцу ресурса, не нужно, чтобы внешнее CFP-приложение имело доступ к информации о конференции. Должно существовать разделение, при котором внешняя CFP-система может быть авторизована только для API Участник.

Для этого создаются две области видимости: Участник и Конференция. Они будут представлены в виде: HTTP-метод — конечная точка — область видимости:

♦ API Участник:

- GET - /attendees — Участник;
- GET - /attendees/{attendee_id} — Участник;
- POST - /attendees — Участник;
- PUT - /attendees/{attendee_id} — Участник.

♦ Унаследованный API конференц-системы:

- GET - /conferences — Конференция;
- POST - /conferences — Конференция.

Таким образом достигается разделение конференций и участников, однако можно пойти еще дальше и разграничить операции чтения (AttendeeRead) и записи (AttendeeAccount):

◆ API Участник:

- GET - /attendees — AttendeeRead;
- GET - /attendees/{attendee_id} — AttendeeRead;
- POST - /attendees — AttendeeAccount;
- PUT - /attendees/{attendee_id} — AttendeeAccount.

У областей видимости нет определенного стандарта, однако они обычно используются в качестве грубого разделения в API. Области видимости должны иметь смысл для конечного пользователя, поскольку он должен дать согласие на их использование. После того как владелец ресурса предоставит полномочия ресурсу, эта информация должна быть использована сервером ресурсов для обеспечения выполнения этих полномочий. При использовании access-токенов в формате JWT к ним обычно добавляется утверждение. Например, "scope": "AttendeeRead AttendeeAccount"²⁴. Здесь будет представлен список всех авторизованных диапазонов. Области видимости не обязательны для OAuth2, хотя они очень полезны и их следует учитывать при грубой авторизации.

Обеспечение выполнения авторизации

Необходимо обеспечить выполнение авторизации, т. к. она является основой безопасности API. Две наиболее распространенные проблемы авторизации безопасности, перечисленные в списке OWASP API Security Top 10, — это *Нарушенная авторизация на уровне объекта* (Broken Object Level Authorization, BOLA)²⁵ и *Нарушенная авторизация на уровне функции* (Broken Function Level Authorization)²⁶. BOLA — это ситуация, когда пользователь может запросить информацию об объекте, к которому он не должен иметь доступа, что часто осуществляется путем подделки идентификатора ресурса. Нарушенная авторизация на уровне функции — это когда пользователь может выполнять действия, на которые он не уполномочен. Например, выступать в качестве обычного пользователя на конечной точке, предназначенной только для администрирования.

Обычно авторизация основывается на тех или иных правах. Для этого, как правило, используется *Управление доступом на основе ролей* (Role Based Access Control, RBAC)²⁷. Хотя точный выбор полномочий — это мелкая деталь, некий контроль доступа должен существовать, и важно, чтобы каждая конечная точка проходила проверку на авторизацию перед выполнением запроса.

Рассматривая авторизацию с помощью OAuth2, следует помнить, что области видимости используются для определения того, что владелец ресурса заявил отно-

²⁴ Это может быть массив, разделенный запятыми или, как в рассматриваемом случае, пробелами.

²⁵ См. https://owasp.org/OWASP_ASP/33333333.

²⁶ См. https://owasp.org/OWASP_ASP/V08ul.

²⁷ См. https://owasp.org/OWASP_ASP/U3ahp.

сительно диапазона доступных для клиента действий. Это не означает, что клиент должен иметь доступ ко всем данным конечного пользователя. Для сервиса Участник возможны различные действия — например, права администратора на управление участниками или права только на просмотр участников. Участник может получить право только на чтение описания профиля участников, однако клиент может запросить разрешение на чтение и управление информацией об участниках. Пользователь может предоставить клиенту доступ для выполнения этих задач от его имени, однако сам пользователь может не иметь такого доступа. Такое дублирование уровней авторизации представлено на рис. 7.10.



Рис. 7.10. Диаграмма Венна для авторизации

API-шлюз может быть полезен для обеспечения авторизации и отклонения запросов, если клиенту не выделена область видимости для доступа к API.

Вкратце об OIDC

OAuth2 предоставляет клиенту механизм доступа к API с использованием аутентификации и авторизации. Общее требование здесь — знание клиентом личности владельца ресурса. Рассмотрим систему внешней CFP-системы. Ей потребуется хранить данные о докладчике, однако варианты предоставления доступа OAuth2 не дают возможности получить идентификационные данные конечного пользователя.

В этом и состоит задача протокола OpenID Connect (OIDC)²⁸ — он обеспечивает уровень идентификации. Этот уровень организуется на основе OAuth2, поскольку

²⁸ См. <https://oreil.ly/umHmK>.

сервер авторизации OAuth2 реализует дополнительную функциональность. Необходимая функциональность превращает сервер авторизации OAuth2 в провайдера OpenID. Теперь клиент может запросить информацию о пользователе, используя специальную область видимости, называемую `openid`. Эта область запрашивается вместе с любыми областями, необходимыми для любых access-токенов. Использование области видимости `openid` позволяет предоставить клиенту ID-токен, представляющий собой JWT, содержащий утверждения о пользователе.

ID-токен, возвращаемый при использовании только области `openid`, содержит очень ограниченное количество информации о пользователе. Единственное утверждение, идентифицирующее здесь пользователя, — это утверждение субъекта, представляющее собой уникальный идентификатор пользователя и никогда не подлежащее изменению (обычно это UUID). Наличия только уникального идентификатора пользователя обычно недостаточно для клиента. Поэтому в OIDC определены дополнительные диапазоны видимости. Их можно добавить к запросу для получения информации в ID-токене:

```
profile
  name, family_name, given_name, middle_name, nickname, preferred_username,
  rofile, picture, website, gender, birthdate, zoneinfo, locale, and updated_at
email
  email and email_verified
address
  address
phone
  phone_number and phone_number_verified
```

В итоге можно получить очень насыщенный ID-токен, содержащий большое количество информации о пользователе. Эти области используются в контексте ID-токенов — вы не увидите этих областей в вашем access-токене, как показано в разд. «Области видимости OAuth2».

В спецификации OIDC шаги по получению ID-токена называются «потокami». Таким образом, в OIDC заявлены три потока: *поток кода авторизации* (Authorization Code Flow), *неявный поток* (Implicit Flow) и *гибридный поток* (Hybrid Flow). Рекомендуется использовать Authorization Code Flow по тем же причинам, что и Authorization Code Grant (+ PKCE) — он более безопасен.

Многие считают, что OAuth2 и OIDC — это одно и то же, и ссылаются на то, что OIDC используется для доступа к API. На самом деле это не так: OAuth2 и OIDC — это две разные сущности. OIDC выполняет свою роль, обеспечивая идентификацию пользователей для клиентов. Однако он не предоставляет доступ к API. Если вам требуется механизм OIDC, то вы должны быть уверены, что используемый провайдер идентификации поддерживает его. Не пытайтесь построить свой собственный уровень идентификации.



Никогда не заменяйте ID-токены access-токенами. Это очень опасная практика, т. к. ID-токены имеют иное предназначение. Они представляют собой существующие продолжительное время токены и предназначены для предоставления клиенту информации о пользователе. Они не предназначены для доступа к ресурсам.

Протокол SAML 2.0

В корпоративных средах принято использовать стандарт SAML 2.0, его часто называют просто SAML. SAML (Security Assertion Markup Language, язык разметки декларации безопасности) — это открытый стандарт, обеспечивающий передачу утверждений. Он часто применяется для однократной регистрации, при этом передаваемые утверждения представляют собой идентификаторы пользователей. SAML популярен в корпоративном мире, поскольку с его помощью сотрудники могут осуществлять вход во внешние приложения. Но SAML не предназначен для использования API в неподготовленном виде. Однако существует расширение OAuth2: Security Assertion Markup Language (SAML) 2.0 Profile for OAuth 2.0 Client Authentication and Authorization Grants²⁹. Это расширение позволяет клиенту запрашивать access-токен с помощью SAML, предполагая, что сервер авторизации реализовал такую функциональность. Вы должны знать об этом, если вам требуется использовать SAML в рамках перехода на OAuth2.

Заключение

В этой главе мы рассмотрели важность обеспечения безопасности API и надежных отраслевых стандартов для достижения этой цели:

- ◆ аутентификация устанавливает личность владельца ресурса. В API им будет либо конечный пользователь, либо приложение, осуществляющее межсистемное взаимодействие;
- ◆ OAuth2 де-факто стал стандартом для защиты API и повсеместно использует JWT в качестве части bearer-заголовка. Токены JWT часто кодируются и подписываются, чтобы обеспечить их защиту от несанкционированного доступа;
- ◆ различные варианты предоставления доступа OAuth2 предназначены для разных сценариев использования. Наиболее распространенными стали Authorization Code Grant + PKCE и Client Credentials Grant;
- ◆ refresh-токены помогают избавить конечного пользователя от постоянных запросов ввести имя пользователя и пароль;
- ◆ области видимости OAuth2 помогают обеспечить грубую авторизацию и позволяют конечному пользователю настраивать доступ клиента;
- ◆ OIDC используется в тех случаях, когда клиенту требуется информация о конечном пользователе. Механизм OIDC предоставляет основную информацию об аутентифицированном пользователе и может опционально предоставлять дополнительные сведения.

Ну что ж, теперь вы должны понимать, как идентифицировать пользователя API и как защитить свои собственные API. Однако это еще не конец пути, поскольку большинство программных архитектур не стоят на месте. Об эволюционной архитектуре с использованием API вы узнаете в следующей главе.

²⁹ См. <https://oreil.ly/AVVHY>.

Эволюционная архитектура с применением API

В этой части книги рассматриваются способы развития архитектуры системы или ряда систем с использованием API. Речь идет о переходе существующих унаследованных приложений к сервис-ориентированным архитектурам на базе API, а также об использовании инфраструктуры API для эволюции или перепланировки системы с целью ее эффективного развертывания в облачной среде.

- ♦ *глава 8* посвящена перепроектированию монолитных приложений на архитектуру, управляемую API;
- ♦ в *главе 9* вы узнаете о том, как использовать инфраструктуру API для перехода от существующих систем к облачным платформам;
- ♦ *глава 10* содержит краткое изложение основных моментов, изученных вами в ходе работы над книгой. В ней также представлены способы, с помощью которых можно продолжить работу над практическим примером и расширить свои знания об архитектуре API.

Перепроектирование приложений на архитектуру, управляемую API

Теперь, когда вы получили четкое представление об операциях API и их безопасности, мы рассмотрим, как можно использовать API для развития и расширения существующих приложений. В книге «Эволюционная архитектура. Поддержка непрерывных изменений»¹, авторы показывают, как эволюционная архитектура поддерживает управляемые, постепенные изменения по многим параметрам. Независимо от того, захотите ли вы принять эволюционную архитектуру, как она определена в этой книге, реальность такова, что почти каждая успешная система должна со временем эволюционировать, чтобы соответствовать новым требованиям пользователей или реагировать на изменения в окружающей среде. Редко какое предприятие или организация не вносит изменения в свою продукцию в зависимости от отзывов клиентов или меняющихся условий рынка. В равной степени нередки случаи, когда на долго работающую систему не влияют изменения в инфраструктуре (например, выход из строя и устаревание аппаратного обеспечения), базовых прикладных фреймворках или в сторонних сервисах.

API — это естественные интерфейсы, абстракции и (инкапсулированные) точки входа в систему и внутри нее, и как таковые они могут играть важную роль в поддержке эволюционной архитектуры. В этой главе вы узнаете о том, зачем нужны изменения, как их проектировать и где применять полезные паттерны.

Однако вы неосознанно уже применяли многие из навыков, рассмотренных в этой главе, на протяжении всего исследования конференц-системы. Мы рекомендуем вам подумать об эволюции примера по ходу чтения этой главы, а окончательное состояние архитектуры конференц-системы рассматривается в разд. «Практический пример: оглянитесь на пройденный путь» главы 10.

Зачем использовать API для развития системы?

Безопасное изменение программного обеспечения может оказаться достаточно трудным процессом. Эта проблема еще больше усугубляется, если программное обеспечение отвечает одной из трех характеристик: большое количество пользователей, внутренняя сложность в архитектуре или тесная интеграция с рядом других систем. И вот что важно — наличие этих характеристик практически неизбежно в программной системе, хорошо принятой, удовлетворяющей потребности пользователей и ставшей ключевой частью рабочего процесса организации. Большинство

¹ См. <https://habr.com/ru/companies/piter/articles/428211/>.

«унаследованных» систем также развивались несколько нестандартно, и многие временные обходные пути, быстрые исправления и сокращения стали неотъемлемой частью системного дизайна.

API могут помочь вам, как архитектору, в развитии системы. API может быть границей модуля или компонента, и это делает API естественной точкой опоры при попытке обеспечить высокую целостность системы и ее слабую связанность.

Создание полезных абстракций: повышение связности

Под *связностью* понимается степень принадлежности элементов системы друг другу. Реализация API и систем с высокой степенью связности обеспечивает более легкую эволюцию как поставщика, так и потребителя API. Являясь поставщиком, вы можете изменять *внутреннее* устройство своего сервиса — например, менять алгоритмы, выполнять рефакторинг кода для повышения производительности или изменять хранилища данных, и при этом вам нужно только не изменить *внешний* интерфейс таким образом, чтобы нарушить обратную совместимость. Как потребитель, вы можете более уверенно модифицировать и масштабировать свой сервис, имея четкие и понятные точки интеграции с существующим API.

Проектирование API с сильной связностью невозможно без критического осмысления создаваемых абстракций. Все мы можем понять и оценить различия в абстракциях при управлении различными транспортными средствами. В автомобиле вы обычно взаимодействуете с приборной панелью, педалями и рулевым колесом. Панель управления космическим кораблем содержит гораздо большее количество различных мониторов, приборных датчиков, рукояток и кнопок. Органы управления космического корабля обладают тесной связностью с поставленной задачей, но уровень контроля и сложности, предлагаемый в этой области, превышает уровень задач, решаемых при проектировании автомобиля. Надеемся, вы видите здесь аналогию с проектированием API. У вас может возникнуть соблазн разработать эквивалент панели управления космического корабля — особенно в связи с задачей создания «неустаревающего» API, — в то время как в действительности ваш базовый бизнес-сервис не сложнее управления автомобилем.



Стремитесь к API с высокой степенью связности

Архитекторам легче понимать, строить мысленные модели и рассуждать об API, обладающих высокой степенью связности. Такие API не нарушают правило наименьшего удивления². API с сильной связностью также могут стать точками фокусировки изменений в системе. Например, серия взаимосвязанных изменений может потребовать модификации только одного API, в отличие от модификации целого ряда API, необходимой при модификации системы с низкой связностью. Всегда стоит стремиться к созданию и развитию систем с сильной связностью.

В качестве контрпримера высокой связности в рамках нашего практического примера представьте, что вы создали API «вспомогательных функций», предостав-

² Правило наименьшего удивления (в эргономике) гласит, что если назначение элемента или сочетания не ясно, то его поведение должно быть для пользователя наиболее ожидаемым.

ляющий набор удобных функций, применяемых во всех сущностях конференц-системы. Это может легко привести к тому, что изменение кода одного API (например, API Участник), потребует обновления и другого API. Если только вы не были автором API или у вас очень хорошая документация и тесты, то можно легко упустить этот момент и оставить систему с непоследовательным или несовместимым поведением.

Существует множество типов связности!

Хотя о связности часто говорят так, будто ее можно оценить однозначно, существует несколько типов этого понятия, и архитекторы должны о них знать. Так, системы могут быть связаны следующими различными способами:

- функциональная связность;
- последовательная³ связность;
- коммуникационная связность;
- процедурная связность;
- временная связность;
- логическая связность.
- случайная связность.

В книге «Software Architect's Handbook»⁴ Джозефа Инджено представлен более полный обзор этого понятия для читателей, желающих узнать больше.

Конечно, связность — это лишь одно из свойств, к которому следует стремиться при проектировании и эволюции систем. Давайте теперь рассмотрим близкого родственника связности в программном обеспечении — *связанность* (также известную как зацепление и сопряжение).

Уточнение границ домена: продвижение слабой связанности

Система со слабой связанностью обладает двумя свойствами. Во-первых, ее компоненты слабо связаны (имеют разрушаемые взаимоотношения) друг с другом. Это означает, что изменения в одном компоненте не влияют на функциональность или производительность другого компонента. И, во-вторых, каждый из компонентов системы практически не знает определений других отдельных компонентов. Компоненты в системе со слабой связанностью могут быть заменены альтернативными реализациями, представляющими собой те же сервисы, причем менее привязанными к одной платформе, языку, операционной системе или среде сборки.

Проектирование или рефакторинг в направлении слабосвязанного API позволит поставщикам и потребителям более эффективно развивать свои системы. Поставщик API со слабой связанностью обеспечит максимальное внедрение вашего сервиса в организации — как с точки зрения простоты интеграции, так и в плане под-

³ Здесь просится сказать «последовательная», но в источниках пишут именно так. — Прим. пер.

⁴ См. <https://oreil.ly/hkS1F>.

держки легкости изменений. А для потребителей API со слабой связанностью обеспечит более легкую замену компонентов (возможно, даже во время выполнения), более легкое тестирование и снижение затрат на управление зависимостями.



Слабая связанность облегчает процесс имитации и виртуализации при тестировании!

API, разработанный с учетом принципа слабой связанности, как правило, гораздо легче имитировать или виртуализировать при проведении интеграционного и сквозного тестирования. Он также позволяет легко менять реализацию поставщика. При тестировании у потребителя реализация API поставщика может быть легко заменена простой заглушкой или виртуальным сервисом, возвращающим требуемый ответ.

Для сравнения: симитировать или заглушить API с сильной связанностью зачастую не представляется возможным. Вместо этого можно запустить API поставщика в качестве части тестового набора или попытаться использовать облегченную (менее функциональную) или встроенную версию сервиса.

Практический пример: установление границ домена сервиса Участник

Представим, что в примере конференц-системы у сервиса Участник имеется сильная связанность с базовым хранилищем данных и он предоставляет данные в формате его базовой схемы. Если вы, выступая в роли сервиса поставщика, хотите поменять хранилище данных на другое, у вас будут два варианта. Вы можете внедрить новую систему адаптации любых создаваемых или получаемых данных между старым и новым форматом, что, скорее всего, потребует сложного и подверженного ошибкам кода перевода. Или же вы можете модифицировать внешний API и заставить всех своих потребителей принять его. При этом не стоит недооценивать трудности, возникающие при работе с широко распространенным сервисом!

Сила скрытия информации

Когда вы проектируете API таким образом, чтобы у него была слабая связанность и сильная связность, вы получаете преимущество, связанное с *принципом скрытия информации*. Его суть заключается в разделении решений по внедрению, которые с наибольшей вероятностью могут измениться. Если все сделать правильно, то можно защитить другие части системы от обширной модификации при изменении проектного решения.

Защита заключается в предоставлении стабильного интерфейса, отгораживающего остальную часть системы от лежащей в ее основе (изменяемой) реализации. Скрытие информации применительно к API — это возможность предотвратить доступность для потребителей определенных подходов поставщика. Этого можно достичь, используя только конечные точки API, ориентированные на бизнес или домен, и не допуская утечки внутренних абстракций, модели или схемы данных, специфичных для конкретной реализации.

Варианты архитектуры конечного состояния

По мере развития и перепроектирования монолитных приложений и API необходимо иметь четкое представление о том, что должна уметь делать система в результате вносимых изменений. Иначе печально известная сцена из «Алисы в Стране чудес»⁵ станет слишком правдивой:

- Будьте добры, вы не подскажете мне дорогу отсюда?
- Это зависит главным образом от того, куда ты хочешь попасть, — сказал Кот.
- Мне не так уж важно, куда... — начала Алиса.
- Тогда неважно, какой дорогой идти, — сказал Кот.
- ...я просто хочу попасть КУДА-НИБУДЬ, — добавила в качестве объяснения Алиса.

Более подробно о подходе к определению общих целей развития систем вы узнаете в следующем разделе этой главы, а пока давайте рассмотрим возможные варианты архитектуры и то, как они влияют на проектирование API.

Монолит

За последние несколько лет монолитный архитектурный стиль приобрел дурную славу. Правда, в основном потому, что слово «монолит» стало синонимом «большого комка грязи»⁶. В действительности монолит — это просто программная система, состоящая из одного блока и работающая как однопроцессное, самодостаточное приложение. В монолитной архитектуре нет ничего принципиально плохого. Для многих систем, особенно для концептуальных приложений, или систем, создаваемых в процессе поиска рыночного соответствия базового бизнес-продукта, этот архитектурный стиль в начале проекта позволяет продвигаться в рабочем плане быстрее всего. Это объясняется тем, что его легко понять и модифицировать, поскольку есть только одна сущность, на которую нужно смотреть, рассуждать и работать над ней.

Сложность реализации API в монолитном приложении заключается в том, что можно случайно создать конструкцию с сильной связанностью, что станет очевидным только при внесении изменений в будущем. Следование лучшим практикам — таким как использование *предметно-ориентированного проектирования* (Domain-Driven Design, DDD), и, возможно, применение гексагональной архитектуры принесет свои плоды в дальнейшем.

Сервис-ориентированная архитектура (SOA)

Сервис-ориентированная архитектура (Service-Oriented Architecture, SOA) — это стиль разработки программного обеспечения, при котором услуги предоставляются

⁵ Льюис Карролл, «Алиса в Стране чудес», перевод Н. Демуровой. — Прим. пер.

⁶ Более подробную информацию об истории создания «большого комка грязи» можно найти в Википедии (<https://oreil.ly/T0z8J>) и здесь: InfoQ (<https://oreil.ly/IPDpX>).

другим компонентам с помощью приложений или служб, которые обмениваются данными по сети. Первое применение SOA, часто называемое «классическим SOA», также заработало неважную репутацию. В основном это было связано с использованием в ранних SOA тяжеловесных технологий, таких как SOAP, WSDL и XML, а также ориентированного на поставщика промежуточного программного обеспечения, такого как ESB, и очереди сообщений. Основное внимание уделялось сети с использованием «умных каналов» для связи, а бизнес-логика включалась в промежуточное ПО.

Эволюция приложений в направлении SOA может быть полезной, но при этом следует избегать использования фреймворков или промежуточного программного обеспечения поставщиков, способствующих возникновению сильной связанности или слабой связанности. Например, всегда следует избегать добавления бизнес-логики в API-шлюз или корпоративную сервисную шину (Enterprise Service Bus, ESB). Одна из самых больших проблем при проектировании систем на базе SOA заключается в том, чтобы «правильно» определить размер и принадлежность сервисов — т. е. найти хороший баланс между согласованностью API, четкой принадлежностью кода к команде в организации, а также стоимостью проектирования и выполнения большого количества сервисов.

Микросервисы

Микросервисы — это новейшая реализация SOA. В ней программное обеспечение состоит из небольших независимых сервисов, взаимодействующих по четко определенным API. Существует несколько отличий подхода, ориентированного на микросервисы, от классической SOA — например, использование «умных конечных точек и глупых каналов»⁷ и отказ от применения ресурсоемкого промежуточного ПО, которое может оказаться сильно связанным с вашими сервисами. О микросервисах написано много книг⁸, и мы предоставим вам далее ссылки на некоторые из них, если вы хотите глубже погрузиться в эту тему. А сейчас надо только отметить, что основные принципы развития микросервисов включают создание управляемых API сервисов со слабой связанностью и сильной связностью.

Как и в случае с классической SOA, одна из самых больших проблем при проектировании API с использованием архитектуры микросервисов — это правильное определение границ (и связанности) API и базовых сервисов. Использование таких методов, как составление карты контекстов и штурм событий (event storming) из мира DDD, перед созданием или эволюцией в сторону микросервисов часто способно ощутимо вознаградить ваши будущие усилия. В идеале микросервисные API должны использовать не занимающие много ресурсов технологии, способствующие слабой связанности. Сюда входят описанные в этой книге технологии, такие как REST, gRPC, а также легкие технологии, основанные на событиях или сообщениях — например, AMQP, STOMP или WebSockets.

⁷ См. <https://oreil.ly/cTt3j>.

⁸ Книга «Создание микросервисов» Сэма Ньюмена (<https://www.piter.com/product/sozdanie-mikroservisov-2-e-izdanie>) — на наш взгляд, лучшая из них.

Функции

Хотя первоначальные обещания, что функции станут следующим шагом в эволюции микросервисов, не оправдались, такая архитектура получила широкое распространение в различных организациях. Этот архитектурный стиль может быть полезен, если у вас есть система с высокой событийной зависимостью — например, торговая система, реагирующая на новости и события на рынке, или система обработки изображений с конвейером стандартизированных преобразований и формируемых отчетов.

Наибольшие трудности при проектировании функционально-ориентированных систем и соответствующих API обычно связаны с обеспечением правильной связанности. Слишком легко разработать настолько упрощенные функции или сервисы, что для получения какой-либо ценности для бизнеса их приходится объединять. Эти сервисы и их API, как правило, оказываются сильно связанными между собой. Баланс между возможностью повторного использования и удобством сопровождения может оказаться непростым, поэтому не следует выбирать этот архитектурный стиль, не понимая, что вам и вашей команде может потребоваться некоторое время для адаптации к нему.

Управление эволюционным процессом

Эволюция системы должна быть сознательно управляемой деятельностью. Рассмотрим, на что необходимо обращать внимание при внесении изменений в API.

Определите свои цели

Прежде чем приступить к эволюции системы, необходимо четко представлять себе мотивацию изменений. Цели должны быть каталогизированы и четко доведены до сведения команды и сотрудников организации. Выявление неверных предположений и целей на ранних этапах процесса изменений требует меньших затрат, чем на этапе начала программирования. Цели в основном делятся на две категории: функциональные и кросс-функциональные.

Функциональные эволюционные цели — это запросы на изменение характеристик или функциональности. Как правило, они определяются конечными пользователями или заинтересованными сторонами бизнеса. Возможно, потребуется рефакторинг, но цели такого рода направлены на написание большего количества кода или интеграцию большего количества систем.

Кросс-функциональные цели, называемые также нефункциональными, сосредоточены на «возможностях» — таких как удобство сопровождения, масштабируемость и надежность. Например, изменения, связанные с удобством сопровождения, часто бывают обусловлены желанием технического руководства сократить время, затрачиваемое инженерами на понимание, исправление или изменение системы. Изменения масштабируемости обычно определяются заинтересованными сторонами бизнеса, прогнозирующими рост использования системы или увеличение спроса на

нее. Работа над надежностью как правило сосредоточена на попытках уменьшить количество и степень влияния отказов в системе. Цели такого рода обычно направлены на рефакторинг существующих систем или внедрение новых компонентов платформы или инфраструктуры.

Создание кросс-функциональных требований вполне логично, но как определить четкие цели желаемых изменений, и как узнать, удалось ли нам это сделать? Именно здесь могут помочь функции приспособленности (fitness functions).

Использование функций приспособленности

Стремление к архитектуре, не подлежащей быстрому устареванию, требует принятия активных решений, предотвращающих деградацию с течением времени. Определение функций приспособленности — это механизм, обеспечивающий постоянный опрос архитектуры системы и артефактов кода, входящих в ее состав. Рассматривайте функцию как своего рода модульный (интеграционный) тест архитектуры, оценивающий «возможности» архитектуры в количественном измерении. Функция приспособленности включается в конвейер сборки для обеспечения постоянной уверенности в достижении целей системы. В блоге Thoughtworks⁹, говоря о функциях приспособленности, предлагается несколько категорий, на которые рекомендуется обратить внимание (табл. 8.1).

Таблица 8.1. Категории функций приспособленности

Качество кода	Скорее всего, эта категория функций приспособленности уже в той или иной степени имеется у многих команд. Выполнение тестов позволяет оценить качество кода перед релизом в производство. Стоит также обратить внимание на дополнительные показатели — например, на минимизацию циклической сложности
Отказоустойчивость	Первоначальный тест на отказоустойчивость заключается в развертывании системы в предпроектной среде, прогона в нее выборочного (или синтетического) трафика и наблюдении за тем, чтобы частота ошибок была ниже определенного порога. API-шлюз или сервисная сеть часто могут использоваться для внесения сбоев в систему и облегчения тестирования отказоустойчивости и доступности к определенным сценариям
Наблюдаемость	Обеспечение соответствия сервисов (и отсутствие регресса) и публикация типов метрик, требуемых платформой наблюдаемости, является критически важным. В разд. «Важные метрики для API» главы 5 показано, какой набор метрик для API следует публиковать, — их можно было бы измерять и обеспечивать с помощью постоянной функции приспособленности
Производительность	Тесты производительности часто остаются на втором плане, однако, если вы можете установить целевые значения задержки и пропускной способности, их можно измерить в конвейере сборки. Возможно, одной из самых сложных частей этой задачи станет получение данных, подобных производственным, для того чтобы необходимые тесты производительности были значимыми. Более подробно мы рассмотрим этот вопрос далее — в разд. «Проблемы с производительностью»
Соответствие нормативным требованиям	Этот раздел очень специфичен для конкретного предприятия (организации) с точки зрения оценки того, что будет критичным для мониторинга. Он может включать требования к аудиту или данным, считающимся ключевыми для дальнейшего предоставления доказательств эффективной и ожидаемой работы бизнеса

⁹ См. <https://oreil.ly/hI9eB>.

Таблица 8.1 (окончание)

Безопасность	Безопасность состоит из множества различных аспектов, и мы рассмотрели некоторые из них в главах 6 и 7. Одной из возможных функций приспособленности может быть анализ зависимостей библиотек в проекте и проверка на наличие известных уязвимостей. Другой вариант — автоматическое сканирование кодовой базы на предмет отсутствия в ней уязвимостей в стиле OWASP.
Работоспособность	Многие приложения создаются, запускаются в производство, а затем начинают развиваться. И когда пользователи подключаются к ним, возникают проблемы. Принятие решения о минимальном наборе требований к эксплуатации платформы является ключевым фактором, обеспечивающим сохранение работоспособности системы. Для начала следует оценить наличие мониторинга и оповещения.

Создание ADR для желаемых функций приспособленности станет хорошим началом. Но сразу реализовать все, что представлено в этой таблице, может оказаться непросто.

Некоторые решения трудно отменить, и важно постараться определить такие решения заранее.



Необратимое решение — это не так уж и плохо! Однако необратимое решение без тщательного обдумывания — гораздо хуже. ADR помогает ответить на распространённый вопрос в стиле «О чем они думали...?» и поделиться историческим контекстом. Решения, принимаемые коллективно с помощью ADR и открытого обсуждения, приведут к созданию долговечной архитектуры.

Декомпозиция системы на модули

Приходилось ли вам работать над кодовой базой, получившей нарекания за монолитность (возможно, от вас или кого-то еще)? Один из авторов книги работал над кодовой базой в четыре миллиона строк, имевшей 24 года технического долга (по данным SonarQube). Структура кода, собранная для решения конкретной задачи из множества различных классов, создавала в приложении высокую степень неконтролируемой связанности. Рефакторинг любой части приложения был затруднен — часто исправление одной ошибки приводило к появлению других неожиданных ошибок. Но ни одна из проблем не была связана с монолитностью системы — скорее, с недостатками организации и проектирования кода.

Одним из подходов, позволяющих избежать спагетти-кода (комка грязи), стало разбиение программных приложений на модули. Проектирование модульных компонентов помогает определить в кодовой базе четкие границы и логическую группировку на основе связности функциональности. Модули призваны формировать четко очерченные границы, скрывающие детали реализации. Вопрос о том, где задается определение модуля, может быть сложным, — например, в Java существуют такие варианты, как метод, класс, пакет и модуль. Каждая из этих конструкций позволяет в разной степени скрывать информацию, что пересекается с объектно-ориентированными конструкциями инкапсуляции. Для целей нашего обсуждения мы будем рассматривать модуль как определение архитектурного разбиения в более широком масштабе, чем методы и отдельные классы.

Сэм Ньюмен дает несколько замечательных советов по ограничению того, с чего нужно начинать при разработке модулей. Книга Сэма «От монолита к микросервисам»¹⁰ представляет собой невероятно глубокое погружение эту в тему:

Лично я придерживаюсь подхода, при котором в границах модуля (или микросервиса) предоставляется доступ к наименьшему возможному объему информации. Как только что-то становится частью интерфейса модуля, от этого трудно отказаться. Но если вы спрячете эти данные сейчас, то всегда сможете решить поделиться ими позже.

Давайте разберемся, какие модули мы могли бы ввести в практический пример конференц-системы, которую начали создавать во *введении*. На рис. 8.1 представлен набор модулей для представления шаблонов контроллеров, сервисов и объектов доступа к данным (Data Access Object, DAO). Каждый контроллер обеспечивает доступ к конечным точкам RESTful, и к нему будет открыт доступ веб-сервером,

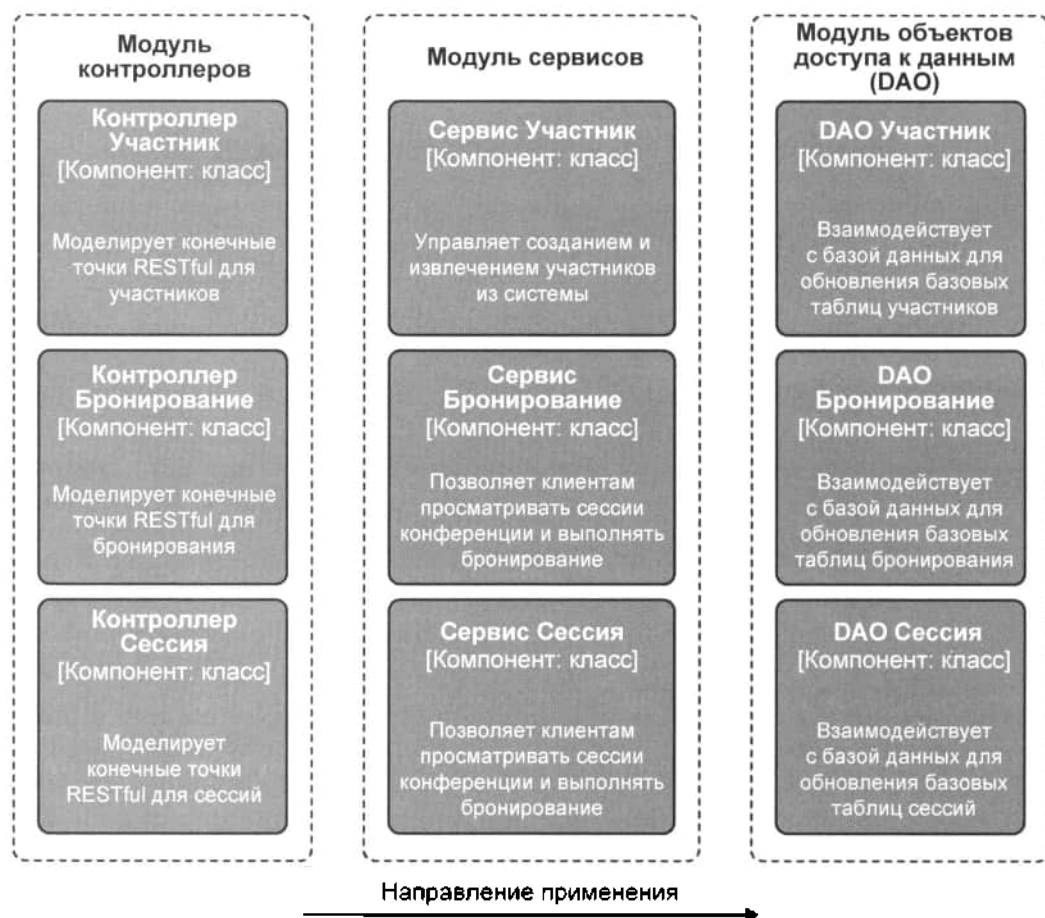


Рис. 8.1. Предложение по разработке модулей для практического примера конференц-системы

¹⁰ См. <https://bhy.ru/product/ot-monolita-k-mikroservisam/>.

хранящим приложение. Модуль сервисов — это место, где за контроллером располагается бизнес-логика, представляющая собой понятный интерфейс для контроллеров. Модуль DAO — это место, где объект доступа к данным выполняет работу после сервисов, открывая доступ интерфейсам к сервисам. Модульные слои встречаются довольно часто, и достижение такой точки, когда между модулями существует четкая однонаправленная зависимость, — это хорошее применение модульности.

Теперь, когда мы получили четкое разделение модулей, каждый из них может применить стратегию тестирования субъекта в отдельности. Еще одним преимуществом модульного подхода является возможность для разработчиков и проводить тестирование внутри модуля, и обсуждать его результаты.

В одном из недавних проектов один из авторов создал паттерн DAO для взаимодействия с базой данных в качестве модуля приложения. Интерфейс раскрывает функциональность для других модулей приложения, делая взаимодействие с модулем понятным. В дальнейшем было принято решение разделить бизнес-логику на три модуля, использующих DAO в качестве независимых сервисов, — первый этап эволюции монолита. Три новых модуля прекрасно обособились в отдельные сервисы, использующие модуль DAO в качестве библиотеки. Проектирование четко определенных модулей позволило обеспечить независимую эволюцию и принятие эволюционных решений в отношении всей системы.

Использование диаграмм C4 для описания программного обеспечения — это облегченный подход на компонентном уровне при определении связей между компонентами в системе. Диаграмма компонентов, впервые рассмотренная во *введении*, обеспечивает механизм просмотра взаимосвязей и помогает в определении модульных структур.

Определение модулей в приложениях — это хороший шаг в проектировании, хотя существует множество вариантов того, как достигается модульность. Используйте поддержку на уровне языка для внедрения модулей и договоритесь с командой о наиболее эффективном для вашего технического стека подходе.

Создание API как «швов» для расширения

Впервые понятие «швов» было представлено в вышедшей в 2004 году книге Майкла Физерса 2004 года «Эффективная работа с унаследованным кодом»¹¹. *Шов* — это место соединения (сшивания) функциональности. Его можно понимать как точку взаимодействия одного рассматриваемого объекта¹² с другим. Обычно это достигается с помощью таких приемов, как внедрение зависимостей, внедрение коллаборатора и выполнение на интерфейсе, допускающем заменяемость. Учет возможности заменяемости очень важен — он позволяет проводить эффективное тестирование без необходимости запуска всей системы (например, с помощью имитаций или тестовых дублеров).

¹¹ См. <https://www.williamspublishing.com/Books/978-5-8459-1530-6.html>.

¹² Здесь «рассматриваемый объект» — это класс или совокупность классов.

Если приложение создавалось не на должном уровне проектирования, то определение шва может оказаться сложным и затруднять понимание всего спектра поведения. При работе с унаследованным кодом могут возникнуть трудности с разбивкой и рефакторингом кода для более модульной работы. Николя Карло¹³ предложил полезный рецепт разделения по швам унаследованного кода, предполагающий, что тесты еще не существуют:

- ◆ определение точек изменения (швов);
- ◆ разрушение зависимостей;
- ◆ написание тестов;
- ◆ внесение изменений;
- ◆ рефакторинг.

При разработке изменений учитывайте возможность создания API, для того чтобы два (а возможно, и более) коллабораторов соединились друг с другом. Если существует вероятность того, что определение шва может быть использовано за пределами рассматриваемого объекта, то отличным выбором может стать межсервисный API. Например, если шов представляет собой одинаковое исполнение во многих различных частях кодовой базы, а цель состоит в том, чтобы разбить сервис на более мелкие архитектуры на основе сервисов, это дает возможность определить межсервисное повторное использование.

Определение точек приложения усилий в системе

Иногда архитекторам и разработчикам не составляет труда определить «точки приложения усилий» — т. е. код и сервисы, являющиеся очевидными кандидатами на рефакторинг и изменение с тем, чтобы сделать систему «лучше» в каком-либо отношении, — например, более производительной, расширяемой, безопасной. Если вы проработали в нашей отрасли более нескольких лет, вам наверняка приходилось иметь дело с системой, имеющей особенно сложный участок кодовой базы или постоянно изменяемый и «взбалтываемый» модуль (а зачастую эти две проблемы взаимосвязаны!), и вы думали о том, как найти правильное решение этой проблемы. Однако эти точки приложения усилий не всегда очевидны, особенно если вы унаследовали кодовую базу или систему. В этой ситуации — чтобы разобраться в таком коде и таких приложениях — вам было бы неплохо ознакомиться с книгой Адама Торнхилла «Your Code as a Crime Scene»¹⁴ издательства Pragmatic Bookshelf. Очень полезными могут стать и сопутствующие инструменты — например, утилиты для обнаружения «взбалтываний» в системе управления версиями, позволяющие находить постоянно изменяющиеся части кодовой базы, а также инструменты для измерения сложности программного обеспечения, анализирующие кодовую базу или сервис при каждом запуске конвейера сборки.

¹³ См. <https://oreil.ly/Xqzpm>.

¹⁴ См. <http://tinyurl.com/ysbmud2x>.

Непрерывная поставка и верификация

В главе 5 мы обратили внимание на важность автоматизации развертывания и релиза систем со слабой связанностью. Необходимость постоянной проверки систем по мере их развертывания критически важна для обеспечения эволюционной архитектуры.

Архитектурные паттерны для эволюционирующих систем с API

API обеспечивают мощную абстракцию для развития систем в направлении современной архитектуры, а также для внедрения новых функциональных возможностей и изменений. Как было отмечено в главах 3 и 4, шлюзы и конструкции на основе сервисных сетей позволяют осуществлять оперативную миграцию. При создании эволюционных изменений основной задачей в период развития системы станет снижение рисков, связанных с эволюцией, и достижение максимальной выгоды в кратчайшие сроки. Рассмотрим некоторые архитектурные паттерны, способные помочь при переходе на API.

Паттерн «Душитель»

Фигура-душитель (strangler fig) — это один из многочисленных видов тропических лиан, обвивающихся вокруг какого-либо дерева. Несмотря на то что фикус-душитель часто удушает и убивает своего хозяина, есть некоторые свидетельства того, что деревья, увитые такими фикусами, лучше переносят тропические ураганы. Это позволяет предположить, что подобные взаимоотношения могут быть в некоторой степени взаимовыгодными. В конечном счете это и есть цель эволюционной архитектуры — поддерживать изменяющуюся систему, что может привести к полному удалению всех старых разработок. Это достигается за счет внедрения новых компонентов приложения при сохранении старого механизма. Конечная цель — постепенный переход на новый подход на основе API.

В разд. «Практический пример: установка флагов функций» главы 5 было показано, как флаг функции может быть задействован для запроса устаревшего или вызова нового сервиса на основе API. На рис. 8.2 приведена диаграмма C4, демонстрирующая использование флагов функций. Этот подход хорошо работает для швов, ранее существовавших в виде внутривзаимодействий при внедрении в сервис новых API. Однако если существует множество потребителей, уже взаимодействующих с сервисом вне процесса, было бы нереалистично ожидать, что все они будут реализовывать и контролировать флаг функции.

Другая модель заключается в использовании прокси или шлюза для обеспечения взаимодействия с API, а также маршрутизации к старой или новой реализации. Она представляет собой разновидность фасада с использованием прокси, когда потребитель API использует тот же самый API и не знает о происходящем за кулисами переходе от одного сервиса к другому.

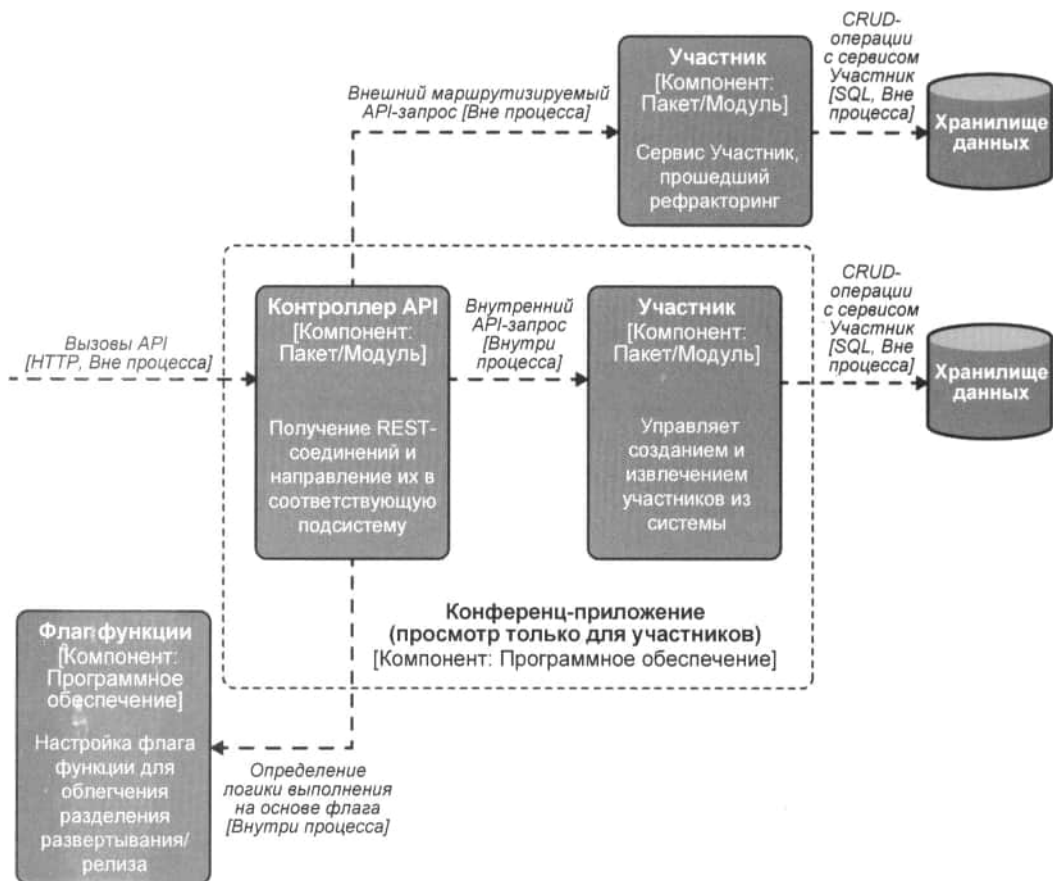


Рис. 8.2. Схема контейнера конференц-приложения для участников с применением флагов функций

Управлять паттерном «душитель» за кулисами может оказаться непросто, а внедрение нового компонента способно стать единой точкой отказа или узким местом, если проблемы такого типа не будут устранены. Прокси не должен брать на себя бизнес-логику, иначе его будет сложно удалить по окончании миграции. Управление унаследованными и современными процессами представляет собой сложную задачу для обеспечения связности данных между двумя сервисами. Более подробные рекомендации по преодолению этих проблем можно найти в книге «От монолита к микросервисам»¹⁵.

Паттерны «Фасад» и «Адаптер»

«Фасад» и «Адаптер» — широко распространенные паттерны, способные помочь при переходе на современные сервисы. Кстати, паттерн «Душитель» представляет собой разновидность «Фасада», перехватывающего вызовы API и скрывающего сложность за кулисами.

¹⁵ См. <https://bhv.ru/product/ot-monolita-k-mikroservisam/>.

Распространенная ситуация, с которой мы столкнулись, присутствует в существующих крупномасштабных распределенных приложениях, уже использующих ту или иную форму API. Возможно, межсервисное взаимодействие в них осуществляется по SOAP-RCP или другому старинному протоколу. Адаптеры могут способствовать развитию архитектуры, представляя собой компонент, преобразующий заданный SOAP-запрос в новый вызов RESTful API. Однако корректная реализация переписывания протоколов может оказаться весьма сложной задачей. При этом необходимо соблюдать осторожность, чтобы не допустить снижения связности или появления связанности.

Использование паттерна «Адаптер» может быть полезно не только в ситуациях с унаследованными системами. В *главе 1* мы рассмотрели использование gRPC как популярной и эффективной технологии для связи типа восток-запад. Используя проект gRPC-шлюза, можно представить конечную точку RESTful JSON, преобразуемую в фоновом режиме в представление gRPC.

Паттерны «Фасад» и «Адаптер» очень похожи, в том смысле, что они представляют собой *помеху*. Паттерн «Фасад» обычно менее сложен, чем «Адаптер», — традиционный API-шлюз просто направляет API-запросы, в то время как «Адаптер» отвечает за преобразование в представление, понятное целевому приложению.



Если API-шлюз переходит между паттернами от «Фасада» к «Адаптеру», то связанность сразу же возрастает. Обязательно поинтересуйтесь, правильно ли вы используете тот или иной компонент для решения этой задачи!

Паттерн «Слоеный пирог API»

Часто обсуждаемый в корпоративном контексте шаблон миграции API — это «Многослойные API», или «Слоеный пирог API». Он основан на модели «разделения ответственностей», присущей традиционным монолитным корпоративным приложениям. В 2000-х годах в корпоративных приложениях на Java или .NET считалось весьма удобным реализовывать функциональность приложения в виде нескольких слоев — например, слоев представления, приложений, доменов и хранилищ данных. Основная идея заключалась в том, что каждый запрос пользователя, поступающий в приложение, последовательно проходит вниз, а затем вверх по каждому уровню/слою. Этот паттерн позволял абстрагировать и повторно использовать специфичную для каждого слоя функциональность. При этом компромисс заключался в том, что для создания сквозного фрагмента функциональности часто требовалось модифицировать множество слоев — т. е. связность внутри каждого уровня достигалась за счет высокой связанности уровней для обеспечения единицы бизнес-функциональности.

Современный подход к этому паттерну, основанный на использовании API, представлен в стратегии под названием Pace-Layered Application Strategy¹⁶ компании

¹⁶ Более подробную информацию о стратегии Pace см. в статьях «Accelerating Innovation by Adopting a Pace-Layered Application Strategy» компании Gartner (<https://oreil.ly/EB1du>) и «A Pace-Layered Integration Architecture» Дэна Туми (<https://oreil.ly/dHaBE>).

Gartner. Для каждого уровня API или микросервиса используются новые названия, при этом слой представления переводится примерно как *система вовлечения* (Systems of Engagement, SoE), слой приложения — как *система дифференциации* (Systems of Differentiation, SoD), а слой хранилища данных — как *система записи* (Systems of Record, SoR).

Со временем этот паттерн приобрел плохую репутацию, особенно по мере того, как содержащие его унаследованные системы становились все более сложными для процесса эволюции. «Слоеный пирог API» побуждает архитекторов и разработчиков использовать короткие пути — например, дублировать функциональность между многими слоями, чтобы избежать вызова дополнительного слоя, или обходить уровни при работе с запросами — например, когда слой представления напрямую взаимодействует со слоем хранилища данных. В целом мы рекомендуем избегать использования этого паттерна.

Определение болевых точек и возможностей

Часто возникает соблазн избежать работы с частями системы или кодовой базы, заслужившими плохую репутацию из-за низкого качества кода, высокой сложности или просто происходящих с ними частых сбоев. Некоторые болевые точки не всегда очевидны до тех пор, пока не возникнут серьезные сбои в работе системы, требующие устранения. Однако никогда нельзя пускать кризис на самотек — выявление и каталогизация проблемных компонентов в системе поможет отследить и устранить известные проблемы. Давайте рассмотрим некоторые из типичных проблем, возникающих в распределенных системах на базе API, и попробуем решать их путем применения соответствующих изменений.

Вопросы модернизации и технического сопровождения

Определение мест, где в общей системе встречаются обновления и зарегистрированные ошибки, поможет вам составить их «черный список». Отслеживайте возникновение в системе следующих проблем:

- ♦ высокий процент отказов при изменении для конкретной подсистемы;
- ♦ большой объем вопросов, возникающих при поддержке системы;
- ♦ большие объемы «взбалтывания» определенной части системы или кодовой базы;
- ♦ высокая сложность (определяется с помощью статического анализа и цикломатической сложности);
- ♦ низкий уровень уверенности команд разработчиков при ответе на вопрос о легкости требуемых изменений.

Рассмотрите возможность добавления метрик качества кода, поскольку это может дать приблизительное представление о потенциальных глубинных проблемах. Проблемы с сопровождением или с подсистемой могут быть признаками того, что не соблюдались хорошие принципы кодирования. Но они также могут предоставить

хорошую возможность внедрить абстракцию API, чтобы вытащить функциональность наружу и использовать паттерн «душитель» для ее улучшения. В главе 9 мы рассмотрим подход к приложениям при переходе к архитектуре на основе API в новой инфраструктуре.

Проблемы с производительностью

Соглашения об уровне обслуживания (Service-Level Agreements, SLA) служат отличной верхней границей для отслеживания и мониторинга производительности. В главе 5 мы рассмотрели мониторинг и метрики, помогающие сигнализировать о проблемах в работе API-сервиса. Реальность такова, что во многих приложениях не предусмотрена проактивная защита от возникновения проблем. Если команда впервые узнает о проблеме производительности — например, о периферийной системе, исчерпавшей бюджет задержки запроса и ответа пользователя, — из прямых отзывов клиентов или из производственного мониторинга, она сразу же оказывается в невыгодном положении при реагировании на проблему.

Проблемы производительности могут иметь архитектурный характер — например, когда один сервис вызывает другой, расположенный в другом регионе, или через сеть Интернет? Когда речь идет о проблемах производительности, измерение и создание объективного плана их решения имеют решающее значение. Проведите измерения существующей системы, сформулируйте гипотезу о том, где можно улучшить ее работу, а затем проведите тестирование и проверку. Важно рассматривать систему в целом, а не пытаться оптимизировать конкретные компоненты по отдельности. Автоматизация процесса измерения позволяет внедрить измерение производительности в качестве части процесса сборки.

Разрушение зависимостей: API с сильной связанностью

Переход к распределенной архитектуре принесет свои плоды только в том случае, если ее части смогут развиваться независимо друг от друга. Особым антипаттерном, требующим пристального внимания, является синхронная координация релизов в различных частях системы. Это признак того, что API потенциально сильно связаны между собой и их разделение может стать возможностью для дальнейшего повторного использования и снижения трения при выполнении релиза.

Один из навыков, часто упускаемых из виду на учебных курсах и в командах, — это эффективная работа с унаследованным кодом. Зачастую разработчики не знают, как внедрить изменения, способствующие разрушению зависимостей. При поиске путей устранения зависимостей целесообразно рассмотреть две техники.

Техника «почкования» рассматривается в книге Майкла Физерса «Working Effectively with Legacy Code»¹⁷ («Эффективная работа с унаследованным кодом»). Зачастую очень сложно провести модульное тестирование не созданного с учетом требований тестирования кода. Техника «почкования» предполагает создание новой

¹⁷ См. <http://tinyurl.com/2pfh5dzf>.

функциональности в другом месте, ее тестирование и добавление в унаследованный метод, называемый *точкой вставки*. Другой прием — создать новый метод с тем же именем и сигнатурой, что и старый метод, и обернуть в него существующую функциональность. Старый метод тогда переименовывается и вызывается из нового, при этом перед вызовом старого метода выполняется дополнительная логика.

Если сервис преимущественно унаследован, то работа с его кодом — это критически важный навык, который необходимо развивать. Работа над кодированием катов или парным программированием в сложных областях кодовой базы поможет вам улучшить понимание в команде. Сандро Манкузо опубликовал на YouTube отличное видео¹⁸, весьма полезное для понимания практических подходов к работе с унаследованным кодом.

Заключение

В этой главе мы рассмотрели подход, основанный на использовании API для перехода от старых монолитных приложений к архитектуре на основе сервисов:

- ♦ API часто обеспечивают естественную абстракцию, или «шов», внутри систем, поддерживая декомпозицию сервисов и фасадов для проведения постепенных изменений. Поэтому они стали мощным оружием в арсенале архитектора, осуществляющего разработку системы;
- ♦ при создании системы с использованием API необходимо понимать такие ключевые понятия, как связанность и связность. Проектирование и создание систем с учетом этих универсальных архитектурных концепций позволит сделать эволюцию, тестирование и развертывание систем более простым и безопасным;
- ♦ при эволюции систем вам необходимо всегда ясно представлять себе текущие цели и ограничения. Без их четкого определения и обмена информацией миграция может затянуться на бесконечность, что приведет к истощению ресурсов и снижению эффективности, а также может негативно сказаться на моральном состоянии сотрудников;
- ♦ хорошо отработанные схемы — такие как паттерн «душитель», позволяют повысить скорость и безопасность эволюции системы и избежать необходимости «изобретать велосипед».

Следующая глава развивает основные положения текущей и расширяет сферу применения эволюционной архитектуры, включая миграцию в облачную инфраструктуру.

¹⁸ См. <https://oreil.ly/p2jWO>.

Использование инфраструктуры API для эволюции в сторону облачных платформ

В предыдущей главе вашему вниманию был представлен обзор архитектурных подходов, которые можно использовать при развитии API и сервисов на их основе. Не менее важной темой, рассматриваемой в рамках эволюции систем, является базовая инфраструктура, платформы и аппаратное обеспечение. Зачастую они меняются и развиваются в собственном ритме: по мере поломки оборудования, слияния или приобретения компаний и технологий, а также в соответствии с ИТ-политикой организации, диктующей необходимость модернизации инфраструктуры. Однако иногда программа развития API приводит к изменениям в инфраструктуре, особенно в части модернизации и перехода к более облачно-подобной (программно-определяемой) инфраструктуре. Осталось только узнать, как реализовать и управлять эволюционирующей системой и соответствующей инфраструктурой API.

Использование API-шлюза способно обеспечить прозрачность местоположения сервисов и API, что позволяет развертывать сервис в облаке и постепенно переводить трафик с существующего сервиса на новый с ограниченным (или нулевым) воздействием на потребителей. В этой главе, с опорой на представленные в предыдущих главах основы архитектуры, описывается применение инфраструктуры API: API-шлюзов, сервисных сетей и порталов разработчика — для развития системы при переходе к облачной среде. Вы разберетесь в том, чем отличается перенос приложений по методу *lift-and-shift* от «смены платформы» (*replatform*) и «рефакторинга или перестроения архитектуры» (*refactor or re-architecture*), и приобретете навыки, позволяющие определить, какой из этих подходов наиболее целесообразен в конкретном контексте. В развитие нашего практического примера мы покажем, как существующий API-шлюз и сервис Участник могут быть перенесены в облако. Также будут рассмотрены возможности миграции сервисов в облако с использованием мультилокационной/кластерной функциональности сервисной сети.

Практический пример: перенос сервиса Участник в облако

Следующий шаг в эволюции нашей конференц-системы заключается в переносе сервиса Участник в облачную инфраструктуру поставщика. Это, прежде всего, связано с тем, что владельцы конференц-систем хотят в конечном счете снять с себя бремя содержания собственного центра обработки данных. В итоге все новые сервисы: монолитное приложение, промежуточное ПО (например, API-шлюзы) и хранилища данных — будут перенесены в облако. Но в первую очередь мы решили

перенести сервис Участник, поскольку он представляет собой самый новый компонент, а также является одним из сервисов, получающих наибольший поток трафика. На рис. 9.1 показано, как извлеченный сервис Участник работает вне контекста основного приложения конференц-системы.

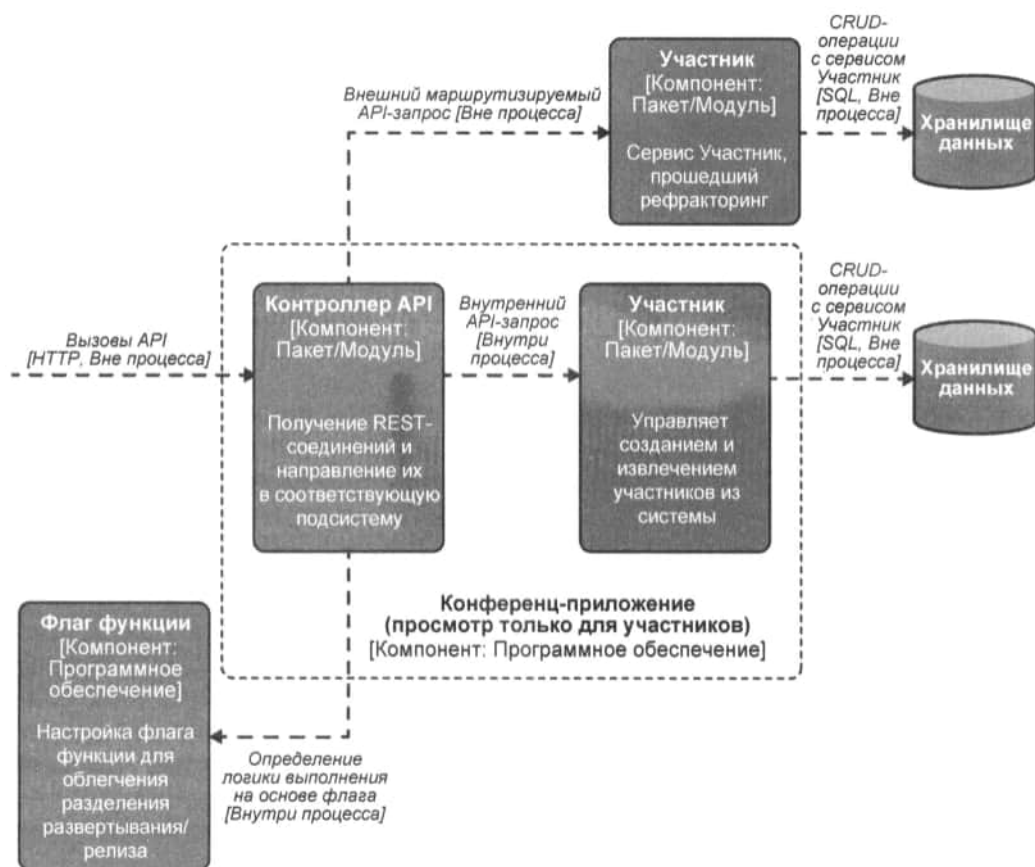


Рис. 9.1. Модель C4, демонстрирующая извлеченный сервис Участник

Далее в этой главе мы рассмотрим несколько подходов к переносу этого сервиса и поддерживающей его инфраструктуры в облако. Но прежде чем погрузиться в собственно механику процесса переноса, давайте сначала разберемся, какие варианты следует обсудить перед выбором стратегии миграции.

Выбор стратегии миграции в облако

Приступая к решению этого вопроса, возьмем за основу статью Gartner (2010 г.) под названием «Migrating Applications to the Cloud: Rehost, Refactor, Revise, Rebuild, or Replace?»¹. Кроме того, в 2016 году Amazon Web Services (AWS) опубликовала

¹ См. <https://oreil.ly/vHnc9>.

блог, описывающий шесть стратегий (six Rs, 6R) миграции в облако². Эти материалы представляют собой отличную отправную точку, если перед вами стоит задача оценить или возглавить миграцию существующей архитектуры и систем в облако. Поскольку API часто являются наиболее близким к пользователю компонентом, ориентированным на бизнес, и ключевой точкой входа для большинства запросов, то при выборе подхода к миграции следует обратить на них особое внимание. Упоминаемые в них шесть стратегий — это спектр вариантов от «ничего не делать» до полноценной сборки или вывода системы из эксплуатации:

- ◆ сохранение или пересмотр (Retain or Revisit);
- ◆ перенос (Rehost);
- ◆ смена платформы (Replatform);
- ◆ новая покупка (Repurchase);
- ◆ рефакторинг/смена архитектуры (Refactor/Re-architect);
- ◆ отключение (Retire).

Давайте рассмотрим шесть стратегий AWS более подробно и выясним, как можно использовать их для развития инфраструктуры API.

Сохранение или пересмотр

Это стратегия вида «ничего не делать» (пока). Хотя может возникнуть соблазн отказаться от такого подхода, многие архитекторы (в том числе и мы) советуют «выбирать битвы под себя», и иногда битва за миграцию API не окупает затрачиваемых усилий. Разумеется, это решение должно базироваться на обоснованной коммерческой и технической оценке, а о решении не предпринимать никаких действий следует сообщить сотрудникам компании и, при необходимости, за ее пределами. Именно в этом случае ADR становятся более эффективными — они обеспечивают «бумажный след» и обоснование этих решений для последующего использования.



Информирование об изменениях и депрекации

Деловая и техническая оценка текущего API или системы может привести вас к решению не осуществлять эволюцию системы в настоящее время. В этой ситуации все равно важно сообщить все известные даты, к которым должны быть приняты действия. Например, если бизнес-подразделение закрывается в известную дату, или система достигает конца срока службы (End-Of-Life, EOL), или срок действия лицензии на ключевое программное обеспечение или хранилище данных истекает в известное время, это должно быть доведено до сведения потребителей в виде предупреждения о депрекации (устаревании). Скорее всего, в договорах и соглашениях SLA будет прописано обязательное уведомление о депрекации, поэтому обязательно ознакомьтесь с ними.

Одним из ключевых факторов при рассмотрении вопроса о переносе приложений в облако является возможность возникновения задержек между двумя сервисами на этапах эволюции. Если у вас сервис с высоким трафиком, то при пересечении гра-

² См. статью «6 Strategies for Migrating Applications to the Cloud» (<https://oreil.ly/ilyJe>).

ниц сети каждый запрос может проходить с замедлением. При этом важно понять, насколько такая деградация мешает нормальному обмену, и если она нарушает SLA, то перенос сервиса может оказаться невозможным. В разд. «Моделирование обмена данными и выбор формата API» главы 1 было рассказано, как выбрать протокол и разработать API в рамках ограничений, что считается важным моментом при пересечении границ.

Сохранение нельзя считать жизнеспособной стратегией для нашей конференц-системы, поскольку это просто «откладывание на потом» и отсрочка выполнения цели миграции в облако.

Перенос

Перенос иначе называют методом lift-and-shift. Он предполагает перемещение систем и рабочих нагрузок на облачную платформу без перестройки архитектуры. Если вы хотите консолидировать рабочие нагрузки или просто вынуждены отказаться от существующей инфраструктуры, это может оказаться эффективной стратегией. Однако следует помнить, что облачная инфраструктура не всегда ведет себя так же, как локальное оборудование, поэтому необходимо выявить и подтвердить все сделанные допущения.



Будьте осторожны со специализированными системами и индивидуальным оборудованием

Хотя многие проекты организованные по методу lift-and-shift, работают как надо, некоторые из них не оправдывают ожиданий. Это особенно актуально для специализированных систем и индивидуального оборудования. Например, старые специализированные системы могут предполагать, что все коммуникации внутри системы происходят по локальной шине или выделенным сетевым соединениям (что не характерно для облака), а некоторые технологии хранилищ данных предполагают определенные аппаратные характеристики (или гарантии) базовой блочной системы хранения данных. Если у вас есть сомнения, проведите соответствующее исследование и разберитесь во всем этом досконально.

Перенос вполне может быть приемлемым подходом для нашего практического примера, хотя, как будет видно далее, мы все же выбрали вариант смены платформы, позволяющий воспользоваться некоторыми возможностями облака.

Смена платформы

Такой подход иногда называют lift-tinker-and-shift. Он очень похож на простой перенос, но при этом эксплуатирует преимущества некоторых облачных сервисов, требующих минимальной доработки. Например, существующее хранилище данных, работающее как системный компонент, может быть заменено на совместимый с протоколом облачный сервис. Можно заменить собственное хранилище данных MySQL на совместимое с MySQL хранилище AWS RDS, Azure Database или GCP Cloud SQL. Другой распространенной формой смены платформы является обновление или изменение сервера приложений или контейнера для определенного языка.

Именно такой подход мы выбрали для нашего примера конференц-системы, чтобы избежать значительных переделок и в то же время воспользоваться — при переходе от существующей локальной инфраструктуры — преимуществами новых облачных сервисов.

Новая покупка

Новая покупка в первую очередь подразумевает переход на другой продукт — например, подписку на сервис электронной рассылки на базе SaaS вместо того, чтобы продолжать использовать свой локальный почтовый сервер.

Поскольку наш пример конференц-системы состоит в основном из приложений, разработанных на заказ, и стандартных хранилищ данных, то возможность новой покупки отсутствует (за исключением, возможно, приобретения готовой системы управления конференциями, что не входит в задачи миграции).

Рефакторинг/смена архитектуры

Рефакторинг в нашем случае означает переосмысление архитектуры и разработку приложения, как правило, с использованием облачных функций. Как и при любом рефакторинге, основная (внешняя) функциональность приложения или системы не должна меняться, но то, как эта функциональность реализуется внутри их, обязательно изменится. Как правило, это обусловлено потребностью бизнеса в дополнительных возможностях, масштабировании или производительности, которые в противном случае было бы трудно реализовать в существующей среде приложения. Например, если организация приняла решение о разделении существующего монолитного приложения на микросервисы, то очень часто рассматривается и переход к использованию облачных паттернов. Эта модель, как правило, окажется наиболее дорогостоящей, но если вы добились хорошего соответствия продукта рынку и ограничены существующими технологическими возможностями, она может стать наиболее выгодной.

Мы специально не выбрали этот подход для нашего примера, поскольку на протяжении всей книги уже занимаемся перестройкой архитектуры конференц-системы. Инфраструктура и проект API, на которых основан наш пример, ведут к более облачному образу мышления, и это очень важно осознать. Определение и моделирование API, как это описывалось в *главе 1*, позволяет создать аккуратный механизм представления сервисов при рефакторинге/смене архитектуры. Ключевой момент состоит в том, что многие облачные сервисы или взаимодействия также основаны на API. Стратегия в отношении API при перестройке архитектуры тоже важна, как и планируемые к использованию в облаке сервисы и системы.

После завершения перестройки архитектуры наиболее целесообразно перенести платформу в облако, прежде чем вносить в архитектуру дополнительные изменения.

Отключение

Отключение систем и вывод их из эксплуатации в процессе миграции означает просто избавление от них. Мы принимали участие в процессах многих крупных миграций, и в них часто обнаруживалась как минимум одна неиспользуемая существующая система, о которой просто забыли. Когда потребность в ее функциональности отпадает, система может быть просто выведена из эксплуатации, а аппаратные ресурсы освобождены или утилизированы.

Разумеется, поскольку наша конференц-система относительно небольшая и целостная, в ней еще нет частей, готовых для отключения! Одной из общих целей у нас будет вывод из эксплуатации устаревшей конференц-системы, и после завершения смены платформы и рефакторинга мы сможем двигаться дальше.

Практический пример: смена платформы сервиса Участник для переноса его в облако

Учитывая рассуждения, приведенные в предыдущем разделе, мы решили реализовать для нашего сервиса Участник смену платформы в дополнение к переносу API-шлюза в облако. Все прочие варианты стратегий: сохранение или вывод из эксплуатации, простой перенос или новая покупка в рассматриваемом контексте также не имеют смысла. Поскольку в начале этой книги мы уже перестраивали архитектуру функциональности участников, вынося ее в сервис, выполнять рефакторинг/смену архитектуры для него сейчас также представляется нецелесообразным. Однако если в будущем в конференц-систему будут добавлены новые функциональные возможности, то перестройка архитектуры системы (возможно, извлечение сервиса) и перенос ее в облако — это вариант, требующий тщательного осмысления. Простой перенос также мог бы стать хорошей стратегией, но мы хотим воспользоваться преимуществами облачной базы данных как услуги, а не применять метод *lift and shift* к собственному экземпляру базы данных MySQL.

Показанный на рис. 9.2 подход «смена платформы» обеспечит хорошую основу для продолжения миграции по мере перевода большего количества сервисов в облако. Перенос в облако API-шлюза также поможет поддерживать постепенную маршрутизацию API-трафика в облако из существующего локального узла.

Теперь рассмотрим, как можно по мере развития конференц-системы реализовать другие требования — например, управление (менеджмент) API.

Роль API-менеджмента

Независимо от выбранной стратегии эволюции, API-менеджмент может играть ключевую роль в миграции, а также в раскрытии ценности API для организации и даже за ее пределами. API-менеджеры по сути своей являются надстроенным шлюзом, предоставляющим широкий спектр дополнительных возможностей для публикации и управления API. API-менеджеры также предоставляют политики, позво-

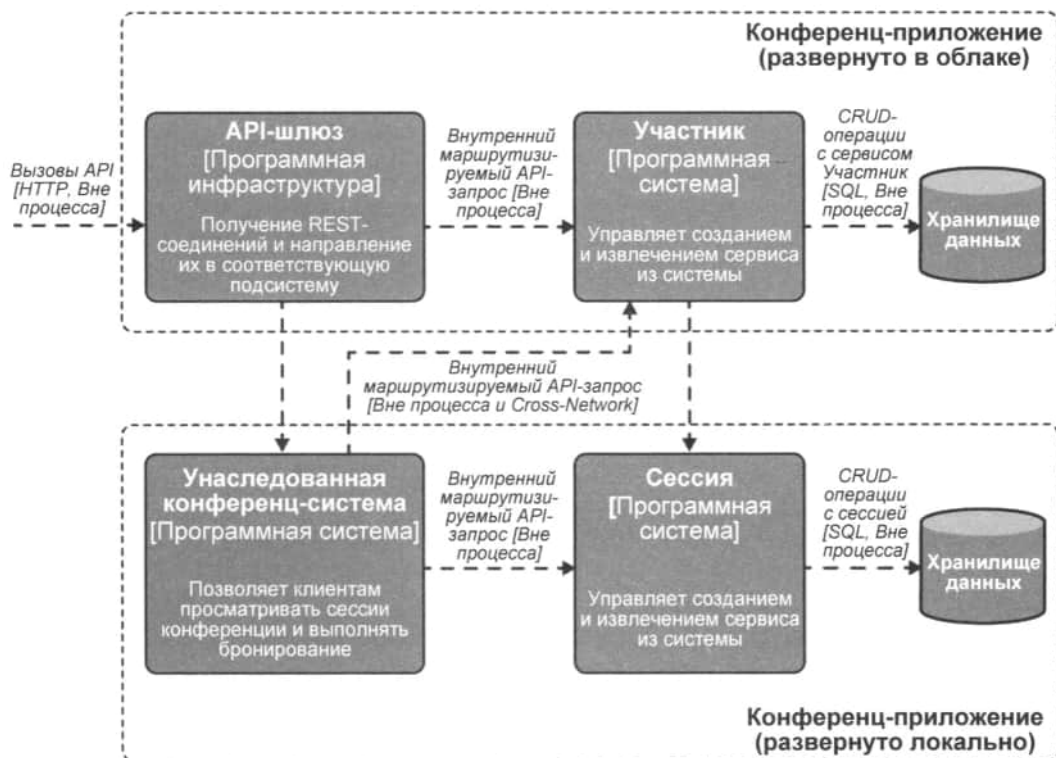


Рис. 9.2. Конечное состояние архитектуры конференц-системы после смены платформы: показаны извлеченные в облако API-шлюз и сервис Участник

ляющие решать задачи на периферии: вызовы OAuth2, проверки содержимого, ограничение количества запросов, троттлинг и многие другие функции, характерные для шлюзов. Кроме того, они могут открывать порталы разработчика, содержащие маркетплейс всех API. Разработчики смогут воспользоваться ими при создании систем для потребления предлагаемых API. Организации также смогут использовать API-менеджмент для монетизации доступа к API как для внешних клиентов, так и для внутренних «возвратных платежей», что часто встречается на предприятиях и в развертываниях между отделами. В разд. «Текущая таксономия API-шлюзов» главы 3 мы разделили таксономии для API-шлюзов и поместили управление API в категорию корпоративных шлюзов.

Возможно, наиболее важной составляющей API-менеджмента является возможность предложить центральную точку для обнаружения API, в то время как вы продолжаете вносить изменения за кулисами. Можно, например, опередить унаследованную конференц-систему с помощью API, одновременно предлагая новый разветвленный API Участник. Организаторы конференции, в свою очередь, могли бы предложить API, обеспечивающий «управление конференцией как услугой», что позволило бы другим конференциям использовать его или управляемым образом взаимодействовать с внешней CFP-системой. Если предположить, что контракты API не меняются, то можно осуществлять развитие за слоем API-менеджмента.

В организациях часто говорят о концепции API First, подразумевая, что все взаимодействия между системами тщательно продуманы и смоделированы как API. Эта концепция была описана в *главе 1*. Следование принципам проектирования и стремление к проекту API-First позволяет раскрыть его ценность как для внешних клиентов, так и внутри организации с помощью таких инструментов, как API-менеджмент.

Для модернизации и миграции приложений с целью получения преимуществ от использования внешнего API-менеджмента необходимо пересмотреть структуру трафика. По мере эволюции архитектуры в гибридную, охватывающую различные сети и развертывания, представления о трафике также должны быть пересмотрены.

Север-юг и восток-запад: размывание границ управления трафиком

Познакомившись с различными вариантами миграции инфраструктуры API, давайте посмотрим, как выбранный нами подход к смене платформы повлияет на управление трафиком API в развивающейся конференц-системе. Поскольку мы выбрали постепенную миграцию наших сервисов, а не «большой взрыв», использование сервисов в нескольких облачных средах и локальных центрах обработки данных может создать дополнительные трудности. Как и во многих других случаях постепенной миграции в облако, трафик должен будет пройти через несколько сетей, чтобы удовлетворить API-запрос пользователя.

Начните с периферии и работайте вглубь

В нашем примере мы решили начать с периферии, перенеся API-шлюз в облако вместе с одним сервисом. Это может дать команде миграции возможность первоначально настроить совершенно новую облачную среду, не нарушая работу существующей системы. Например, развернуть в облаке дубликат существующего API-шлюза, а существующий шлюз оставить в текущем рабочем состоянии. Это позволит минимизировать риски за счет постепенной настройки облачного API-шлюза без нарушения работы существующей эксплуатируемой системы.

Часто имеет смысл создать изолированную пробную версию концепции исключительно в облаке и только после ее проверки начать экспериментировать с маршрутизацией в облачную среду и из нее. Проектирование облачных архитектур часто представляет собой смену парадигмы с точки зрения проектирования. Не стоит недооценивать время, необходимое для изучения и понимания новой инфраструктуры.

Пересекая границы: маршрутизация по сетям

Перед началом миграции в облако часто необходимо обеспечить взаимодействие новых и старых систем в различных сетях. Как уже говорилось в *главах 3 и 4*, у вас есть ряд возможностей для реализации такой маршрутизации. Если имеется одно

монолитное приложение и небольшое количество простых маршрутов, то проще всего временно проложить маршрут от нового API-шлюза к старому, возможно, с помощью простого HTTP-перенаправления. Однако если существует большое количество маршрутов, пересекающих сети, или трафик не должен покидать сеть после того, как он в нее попал, необходимо предусмотреть другие варианты. Например, пиринговые виртуальные частные сети, конечные точки или мультикластерные сервисные сети.

Если трафик API проходит через несколько сетей, вам, скорее всего, придется проконсультироваться с командами InfoSec, поскольку такой трафик нарушит традиционный подход к защите периметра и зональной архитектуре. Давайте рассмотрим эту тему более подробно и узнаем, чем может помочь переход к сетям с нулевым доверием.

От зональной архитектуры к принципу нулевого доверия

Прежде чем узнать, как современные API-шлюзы и сервисные сети могут помочь в реализации сетей с нулевым доверием, давайте сначала взглянем на традиционный подход к архитектуре зональных сетей.

Попадание в зону

По мере роста популярности коммерческого Интернета все больше регулируемых отраслей стали предоставлять доступ к приложениям. Это означает, что как новые системы, так и существующие внутренние системы должны быть ориентированы на пользователя. Появление зональных архитектур стало лучшей практикой проектирования защищенных сетей. Разделение на зоны задается для снижения рисков, связанных с работой в полностью открытой или плоской сети, и организуется путем сегментирования инфраструктурных сервисов на логические группы, имеющие одинаковые политики сетевой безопасности и требования к безопасности. Злоумышленник, применив, например, эксплойт Log4Shell (CVE-2021-44228) — уязвимость нулевого дня, представляющую значительную опасность для Java-приложений, использующих пораженные библиотеки Log4J, может получить доступ к узлу в сети и начать выполнять вредоносные действия. Радиус поражения сервисов, попавших под влияние эксплойта, называется *радиусом атаки*. Но если все недоверенные запросы поступают в зону, имеющую доступ к очень небольшому количеству ценной информации, то радиус поражения минимизируется, и у служб безопасности остается время для принятия мер по предотвращению серьезных утечек информации. Зоны имеют тенденцию к каскадному построению, при этом на каждом переходе в следующую зону применяются более глубокие меры защиты для борьбы с входящим трафиком.

Зоны разделены периметрами (точками сопряжения зон), реализуемыми с помощью охранных и сетевых устройств. Разделение на зоны — это логический подход к проектированию, используемый для контроля и ограничения доступа и пото-

ков передачи данных только компонентам и пользователям, отвечающим политике безопасности.

Существует множество подходов к определению зон и соответствующих требований к безопасности, как стандартизированных (часто на уровне страны), так и индивидуальных. Однако, как показано на рис. 9.3, в большинстве зональных архитектур встречаются четыре типичные зоны³:

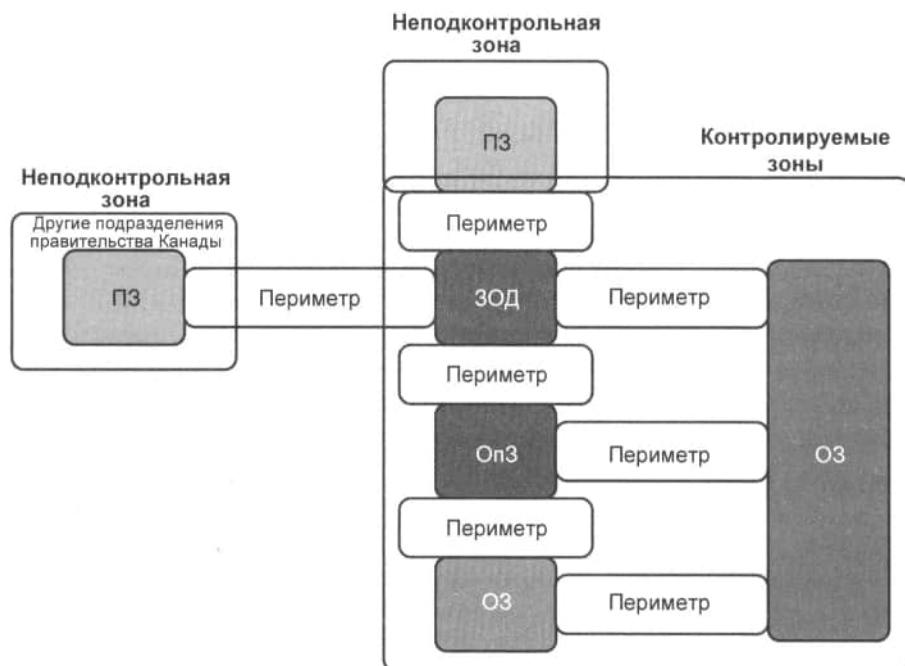


Рис. 9.3. Типичная зональная архитектура (взято из канадского правительственного документа ITSG-22)

◆ Публичная зона (ПЗ).

Эта зона полностью открыта и включает в себя такие сети общего пользования, как публичный Интернет, коммутируемая публичная телефонная сеть и другие магистральные сети и службы операторов связи общего пользования.

◆ Зона общего доступа (ЗОН).

- Эта зона служит посредником между операционными системами и публичной зоной и часто включает в себя демилитаризованную зону (ДМЗ).

◆ Операционная зона (ОпЗ).

Это стандартная среда для выполнения рутинных операций, обладающая соответствующими средствами контроля безопасности в конечных системах. Она может использоваться для обработки конфиденциальной информации. Однако,

³ Заинтересованные читатели могут получить более подробную информацию о зональных архитектурах на сайте Canadian government's Network Security Zoning (<https://oreil.ly/vYrDQ>).

как правило, операционная зона не подходит для больших хранилищ конфиденциальных данных или критически важных приложений без дополнительных надежных средств контроля безопасности.

◆ Ограниченная зона (ОЗ).

Эта зона обеспечивает контролируемую сетевую среду, обычно подходящую для критически важных ИТ-сервисов или крупных хранилищ конфиденциальной информации, и поддерживает доступ из систем в публичной зоне через ЗОД и ОпЗ.

Такой подход к проектированию сети на основе периметра напоминает старую защиту типа «крепость и ров», когда злоумышленник испытывает наибольшие трудности в точке проникновения, но, оказавшись внутри крепостных стен, уже легко ориентируется. Во многом это связано с тем, что в отношении любого сообщения, исходящего из периметра системы, сети или местоположения, делаются допущения. Однако облачная инфраструктура может поставить под сомнение эти допущения. Во многих облачных платформах географическое и сетевое расположение базовой инфраструктуры абстрагировано или недоступно. Но даже при наличии средств защиты, предоставляемых поставщиками кластеров, сохраняется риск атаки на цепочку поставок, когда программное обеспечение выполняет манипуляции с вредоносным содержимым на этапе сборки. Другой вариант — злоумышленник на сайте провайdera инфраструктуры, получающий доступ к информации с уровня платформы.

Использование различных типов защиты в различных вариантах развертывания возможно, однако более однородный подход снижает риск возникновения допущений и необходимость изучения этих типов защиты. В архитектуре, основанной на зонах, доверие заложено изначально, что и послужило толчком к разработке нового подхода — на основе нулевого доверия.

Никому не доверять и проверять

Модель безопасности с нулевым доверием, известная также как *архитектура с нулевым доверием*, или *безопасность без периметра*, описывает подход к проектированию и реализации современных сетевых систем. Основная концепция модели безопасности нулевого доверия: «никогда не доверяй, всегда проверяй». Это означает, что устройствам не следует доверять по умолчанию, даже если они подключены к разрешенной сети, такой как корпоративная локальная сеть, и даже если они были предварительно проверены. В пользу принципа нулевого доверия говорит тот факт, что традиционный подход — доверять устройствам в пределах условного «корпоративного периметра» или устройствам, подключенным через VPN, — в сложной среде корпоративной сети неактуален. Подход с нулевым доверием предполагает взаимную аутентификацию, включая проверку идентичности и целостности устройств без учета их местоположения, и предоставление доступа к приложениям и сервисам на основе уверенности в правильной идентификации устройства и его работоспособности в сочетании с аутентификацией пользователя.

Следующие восемь принципов помогут вам реализовать собственную архитектуру сети с нулевым доверием в корпоративной среде:

- ◆ найдите свою архитектуру, включая пользователей, устройства, сервисы и данные;
- ◆ найдите идентификаторы пользователей, сервисов и устройств;
- ◆ оценивайте поведение пользователей, состояние устройств и сервисов;
- ◆ используйте политики для авторизации запросов;
- ◆ повсеместно внедряйте аутентификацию и авторизацию;
- ◆ сосредоточьтесь мониторинг на всем, что касается доступа: пользователях, устройствах и сервисах;
- ◆ не доверяйте никаким сетям, включая свою собственную;
- ◆ выбирайте и проектируйте сервисы для работы по принципу нулевого доверия.

Эти восемь принципов вполне разумны, однако их очень сложно учесть в зональной архитектуре. Концепция доверительных допущений зоны ставит под сомнение многие из них. Например, при зональной архитектуре аутентификация пользователя часто производится только на периферии системы, а все сети внутри периметра по умолчанию считаются доверенными. Давайте рассмотрим, как можно осуществить эволюцию в сторону архитектуры, основанной на нулевом доверии.

Роль сервисной сети в архитектурах с нулевым доверием

Опубликованный NIST в 2020 году документ под названием *Zero Trust Architecture*⁴ (Архитектура с нулевым доверием) дает определение принципа нулевого доверия и предоставляет связанные с ним ключевые архитектурные соображения. Сервисная сеть и API-шлюзы в совокупности представляют собой вполне подходящую платформу для реализации архитектур, основанных на принципе нулевого доверия. Использование сервисной сети обеспечивает однородное моделирование представления компонентов вашей архитектуры и прохождения трафика между ними. Базовые технологии дают возможность создать конкретную модель идентификации в процессе работы, а управление сертификатами помогает идентифицировать и подтверждать личность. Интеграция активной трассировки и мониторинга позволяет проводить анализ во всех точках платформы, как для пользователей, так и для базовых сервисов и работоспособности подов Kubernetes. Весь входящий трафик подвергается серьезному исследованию. Часто запрос подтверждается с помощью OAuth2, как обсуждалось в *главе 7*, а трафик внутри кластера может использовать mTLS для строгих подтверждений аутентификации и авторизации.

Не доверять ни одной сети, включая свою собственную, — интересная задача для сервисных сетей. В большинстве моделей сервисной сети *sidcar*-прокси тесно связан с сервисом или приложением, обеспечивая управление трафиком и безопасность за счет маршрутизации через этот прокси. Однако простота такого развѣрты-

⁴ См. <https://oreil.ly/8r8aC>.

вания означает, что вы не можете делать конкретные допущения на используемой платформе. То, что находится под приложением и *sidecar*-прокси, должно быть надежно защищено так, чтобы не разрешать допущений, основанных на доверии.

Дополнение сервисной сети сетевыми политиками

Безопасность платформы зависит от любых допущений, сделанных на уровне приложений. Поэтому для достижения полного нулевого доверия необходимо спуститься на уровень ниже. В Kubernetes существует концепция *NetworkPolicies*⁵, позволяющая использовать сетевой плагин, — например, *Calico*⁶. Средства его управления дают возможность обеспечить изоляцию подов от платформы, на которой они работают.

Например, следующая политика заблокирует доступ всего входящего и исходящего трафика в указанный под. В архитектуре с нулевым доверием это могло бы стать стандартом для подов — с применением этого правила под становится полностью изолированным:

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes:
    - Egress
    - Ingress
```

Реализации сервисной сети часто ориентируются на поиск имен сервисов с помощью центральной системы DNS. Даже если она заблокирована. Для начала необходимо включить в платформу несколько контролируемых сценариев, чтобы система DNS оставалась заблокированной, но при этом позволяла работать сервисной сети. В следующей политике мы разрешаем поиск DNS для унаследованной конференц-системы, чтобы она могла найти сервис Участник:

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
spec:
  podSelector:
    matchLabels:
      app: legacy-conference
```

⁵ См. <https://oreil.ly/zxYkZ>.

⁶ См. <https://oreil.ly/5ds3n>.

```

policyTypes:
- Egress
egress:
- ports:
  # allow DNS resolution
- port: 53
  protocol: UDP
---
```

В этот момент сервисная сеть унаследованного сервиса конференций может через `sidecar` попытаться узнать, где находятся участники конференции, однако сам запрос будет заблокирован. Каждое правило маршрутизации в сервисной сети должно получить соответствующее разрешающее правило, определенное в адаптере сетевой политики. В следующем примере мы открываем правило для взаимодействия унаследованной конференц-системы с сервисом Участник:

```

---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-conference-egress
spec:
  podSelector:
    matchLabels:
      app: legacy-conference
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: attendees
```

Чтобы вход трафика работал, необходимо также добавить правила входа от шлюза сервисной сети к целевым сервисам. В разд. «Прикладные решения для эффективных релизов программного обеспечения» главы 5 мы описали решения на уровне приложений для субъективной платформы. Обеспечение последовательного применения правил и конфигурации во время релиза — еще одна причина, по которой стоит обратить внимание на субъективную платформу.

Используя сервисную сеть и сетевые политики, вы научились создавать микросегментированную архитектуру. Преимущество такого подхода заключается в том, что можно реализовать сценарий, при котором безопасность будет согласована в гибридной архитектуре как с облаком, так и со средами, ранее бывшими зональными. Распространенной схемой считается использование сервисной сети для объединения различных («мультикластерных») сетей. Это достигается за счет организации пиринговых сервисных сетей, объединяющих кластеры, в результате чего локальная и облачная плоскости данных оказываются под контролем объединенной плос-

кости управления⁷. На рис. 9.4 показано, что сервисная сеть отвечает за всю маршрутизацию и может обеспечить архитектуру нулевого доверия между сетями. Преимущество такого пути заключается в том, что он приводит к созданию безопасной эволюционной архитектуры и однородной среды безопасности. Заставив локальную систему работать как облако, можно легко перевести в облако и остальные сервисы.

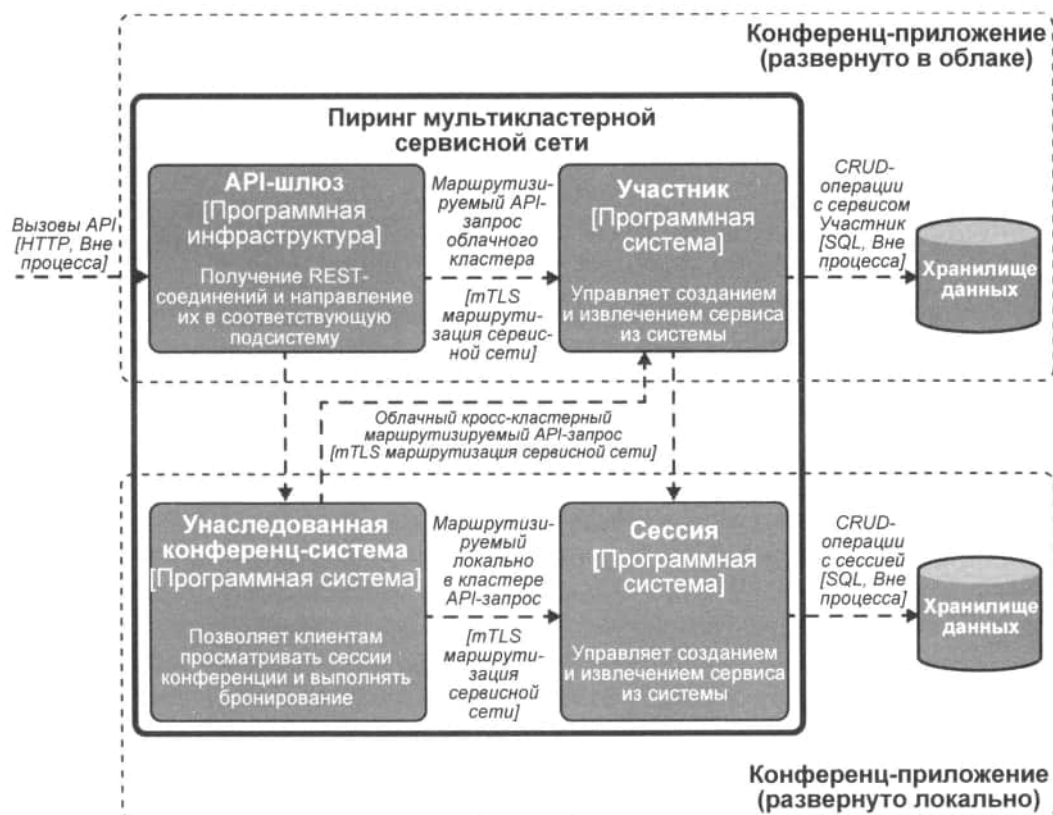


Рис. 9.4. Пиринг мультикластерной сервисной сети

Заключение

В этой главе показано, как использование инфраструктуры API: API-шлюзов, сервисных сетей и порталов разработчика — может применяться для развития системы при переходе к облачной среде:

- ♦ существует несколько подходов к эволюции или миграции системы на базе API в облако: от сохранения («ничего не делать») до простого переноса, смены платформы, новой покупки, рефакторинга/смены архитектуры (переписывания

⁷ Механика этого процесса различна для разных реализаций сервисной сети.

с целью использования преимуществ облачной инфраструктуры) и вывода из эксплуатации;

- ◆ при переносе API-приложения в облако часто возникает ситуация, когда стирается грань между управлением трафиком север-юг (входящий трафик) и восток-запад (трафик между сервисами);
- ◆ API-шлюз можно использовать в качестве инструмента миграции, поскольку он способен инкапсулировать функциональность и выступать в качестве фасада для нескольких бэкенд-систем, работающих в различных средах и сетях;
- ◆ индустрия ИТ переходит от зональных сетевых архитектур к системам с нулевым доверием, и технология сервисной сети может способствовать этому переходу;
- ◆ использование принципов нулевого доверия позволяет сочетать как архитектуру нулевого доверия, так и зональную, что помогает в период миграции объединить облачные и локальные системы.

Наше путешествие по миру архитектуры API практически завершено, и в следующей, заключительной главе мы подведем итог рассмотрению его ключевых концепций и заглянем в будущее этого мира.

Подведение итогов

В предыдущих девяти главах этой книги вы проделали путь от проектирования API до их внедрения, защиты и эксплуатации. Основное внимание в них уделялось архитектуре, но не менее важно и то, как вы применяете архитектуру в своей организации.

В этой заключительной главе книги мы рассмотрим новые технологии API, способные сыграть большую роль в будущем, и узнаем, как нам оставаться в курсе этих меняющихся лучших практик, инструментов и платформ.

Практический пример: оглянитесь на пройденный путь

На протяжении всей книги мы делали эволюционные шаги по обновлению и совершенствованию начального (исходного) варианта архитектуры конференц-системы (рис. 10.1).



Рис. 10.1. Исходная архитектура конференц-системы

Рассмотрим некоторые решения, принятые при извлечении сервиса Участник. Как показано на рис. 10.2 и описано во *введении*, мы приняли решение (на основании требований заинтересованных сторон конференц-системы) выделить функциональность для участников в сервис на базе API. Он будет работать как отдельный процесс, внешний по отношению к унаследованной конференц-системе.

В главах 1 и 2 архитектура оставалась статичной, пока мы изучали, как разрабатывать и тестировать API и сервис Участник. В главе 3 мы сделали первый большой эволюционный шаг, внедрив API-шлюз между конечным пользователем и существующей конференц-системой с новым сервисом.

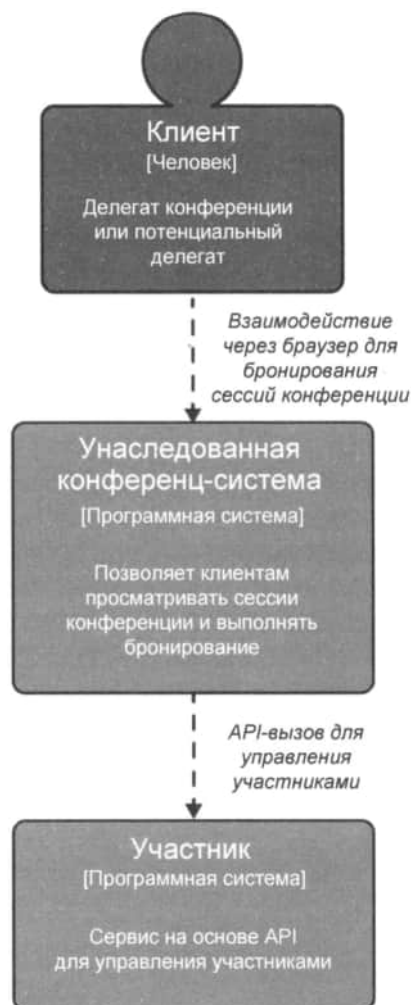


Рис. 10.2. Извлечение сервиса Участник из конференц-системы

Как показано на рис. 10.3, клиент теперь обращается к конференц-системе через API-шлюз, обеспечивающий абстракцию и единую точку входа для трафика, связанного как с унаследованной конференц-системой, так и с новым сервисом Участник. На этом этапе был внедрен паттерн Фасад, позволяющий контролировать, когда вызывается старый сервис, а когда обновленный.

Еще на один шаг мы продвинулись в главе 4, выделив функциональность сессии конференции из старой конференц-системы в новый сервис Сессия и внедрив сервисную сеть для обработки API-трафика между сервисами (рис. 10.4).

В главе 5, выполняя поэтапные релизы, мы создали внутреннюю и внешнюю версии сервиса Участник и использовали флаги функций для определения того, к какому сервису направляется запрос пользователя. На рис. 10.5 показаны два сервиса Участник, расположенные рядом на статической диаграмме архитектуры.

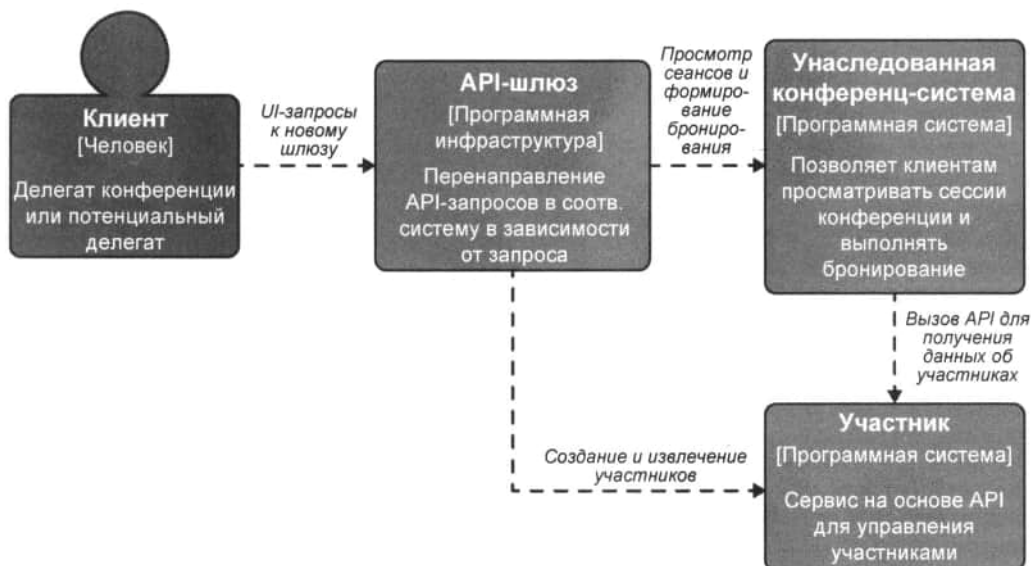


Рис. 10.3. Добавление API-шлюза в конференц-систему



Рис. 10.4. Модель C4, показывающая извлечение сервиса Сессия из конференц-системы

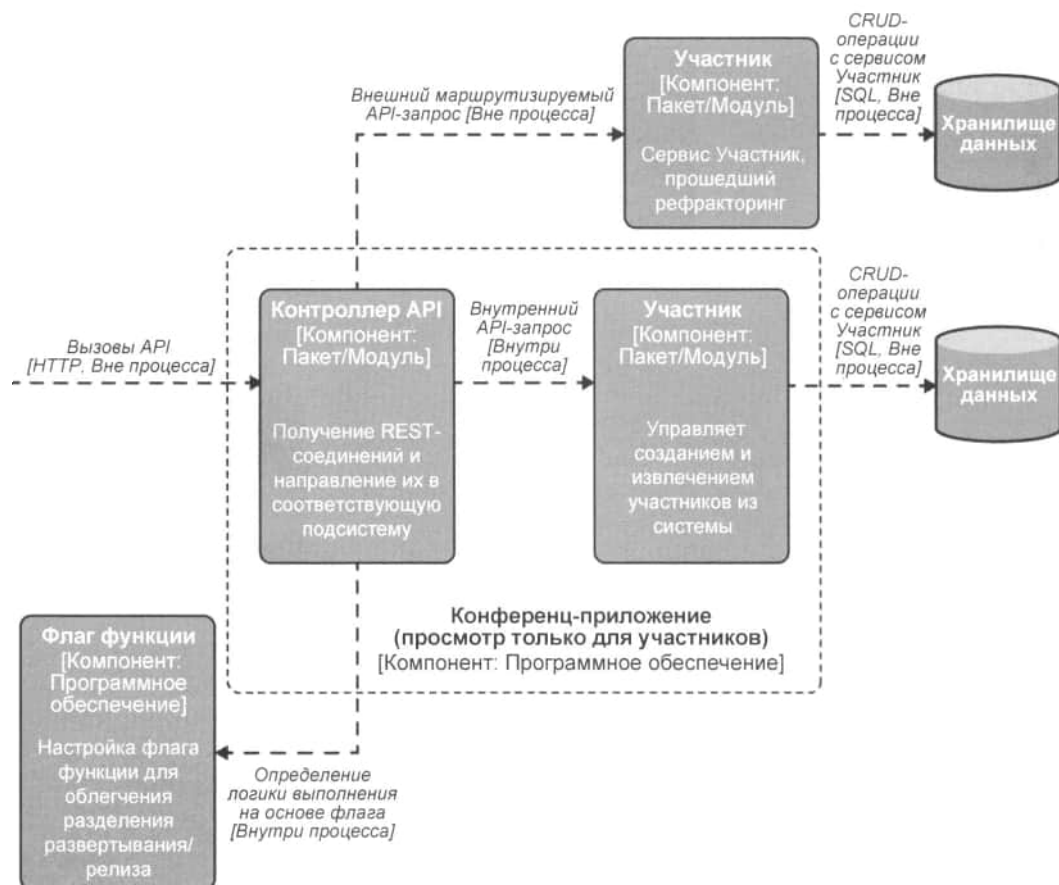


Рис. 10.5. Модель C4, демонстрирующая маршрутизацию к двум экземплярам сервиса Участник через флаги функций

В *главе 6* мы сосредоточились на безопасности, и хотя архитектура оставалась статичной, мы ввели понятие мобильного приложения, вызывающего конференц-систему, чтобы обеспечить реалистичный сценарий для проведения моделирования угроз (рис. 10.6).

В *главе 7* мы добавили в архитектуру внешнюю CFP-систему, что потребовало реализации внешней (обращенной к пользователю) аутентификации и авторизации (рис. 10.7).

В *главе 9* мы сосредоточились на переносе сервиса Участник и API-шлюза на облачную платформу, в результате чего была получена архитектура, представленная на рис. 10.8.

И в завершение в *главе 9* мы представили потенциальную модель перехода к архитектуре с нулевым доверием, обеспечивающую единый подход к развертыванию, маршрутизации и безопасности. На рис. 10.9 показан вариант такой гибридной архитектуры, но эволюционный путь к облаку остается открытым для продолжения.



Рис. 10.6. Архитектура С4, демонстрирующая взаимодействие мобильного приложения с конференц-системой

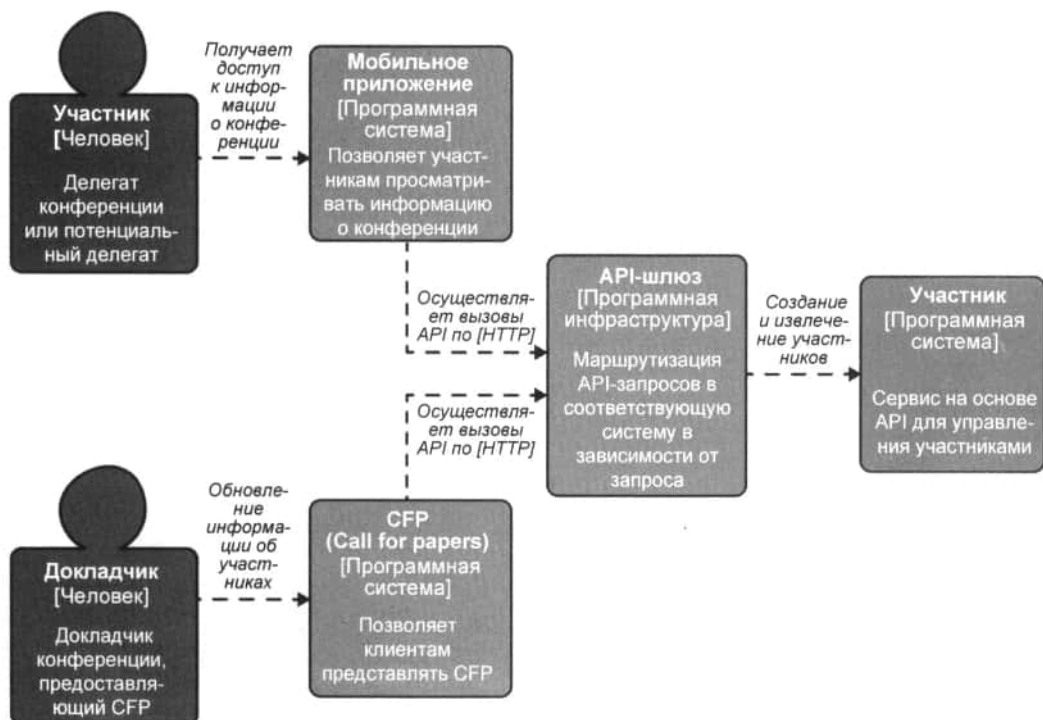


Рис. 10.7. Внешняя система, взаимодействующая с конференц-системой

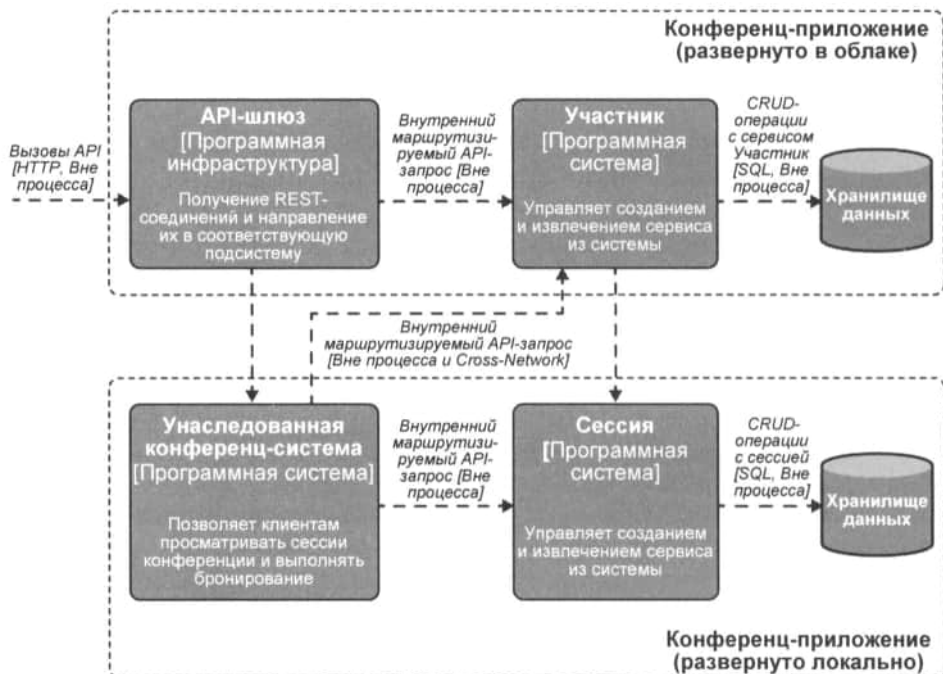


Рис. 10.8. Миграция конференц-системы в облако

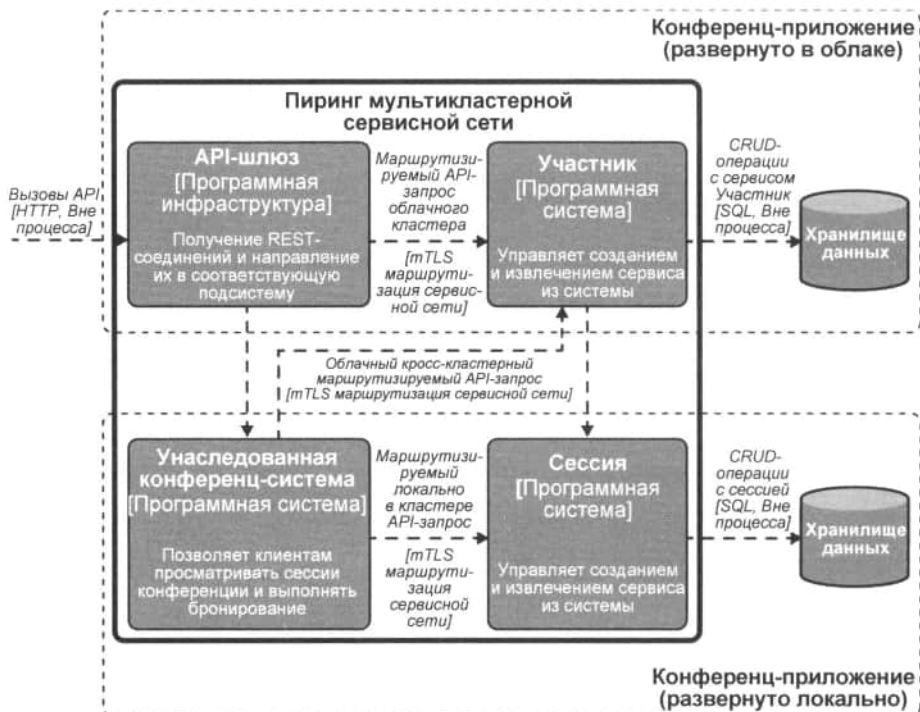


Рис. 10.9. Миграция конференц-системы в облако с использованием мультикластерной сервисной сети

Таким образом, на протяжении всех этапов рассмотрения нашего практического примера мы старались выделить ключевые вопросы, которые вам и вашей команде придется решать по мере эволюции типичной системы в сторону перехода на API или сервисы. Начав со скромного представления в виде одного сервиса и базы данных, работающих локально, и перейдя к мультисервисной системе, работающей и в облаке, и локально, вы должны были понять, что при большей гибкости, предоставленной этой конечной реализацией, в ней есть и компромиссы с архитектурой и сложностью инфраструктуры.

Хотя на этом развитие нашего примера завершено, мы призываем вас экспериментировать с созданием новых требований, проектированием новых API и извлечением дополнительных сервисов.

API, Закон Конвея и ваша организация

Эта книга не касается вопросов организационного дизайна, но мы хотели бы упомянуть о важности этого аспекта применительно к проектированию, созданию и эксплуатации API. Закон Конвея стал известен в сообществе микросервисов, и эта концепция также применима к архитектуре API:

Любая организация проектирует систему (в широком смысле этого слова), которая копирует структуру коммуникаций в этой организации.

Мелвин Конвей в статье «How Do Committees Invent?»¹

Или, говоря более кратко: «если над компилятором будут работать четыре группы, то получится четырехпроходной компилятор». Мы определенно наблюдаем это в мире API — как мы любим шутить: «если у вас есть четыре группы, работающие над микросервисной системой, вы получите четыре слоя API». Мы вряд ли могли бы охватить в этой книге тему организационного дизайна с той глубиной, которую он заслуживает, поэтому взамен хотим порекомендовать вам прочитать следующие книги:

- ♦ «Team Topologies» (IT Revolution Press)²;
- ♦ «Agile IT Organization Design: For Digital Transformation and Continuous Delivery» (Addison-Wesley)³;
- ♦ «The Art of Scalability» (Addison-Wesley)⁴.

Если вы собираетесь проводить серьезные организационные изменения или «цифровую трансформацию», мы настоятельно рекомендуем вам ознакомиться с этими работами. Любая система, основанная на API, по своей сути является социотехнической системой, поэтому социальный аспект должен учитываться в ней в той же степени, что и технический.

¹ См. <https://oreil.ly/FRAKR>.

² См. <https://oreil.ly/ch1eD>.

³ См. <https://oreil.ly/2acum>.

⁴ См. <https://oreil.ly/7CX01>.

Понимание типов решений

Джефф Безос, основатель компании Amazon, в одном из своих ежегодных писем акционерам высказался⁵ по поводу решений типа 1 и 2. Решения типа 1 нельзя легко отменить, и принимать их нужно очень осторожно. Решения типа 2 легко изменить: «как войти в дверь — если решение не нравится, всегда можно вернуться назад». Обычно эта концепция представляется в связи с путаницей между ними и использованием процессов типа 1 для принятия решений типа 2: «Конечным результатом такого решения станет медлительность, непродуманное неприятие риска, нежелание экспериментировать и, как следствие, снижение изобретательской активности. Мы должны придумать, как бороться с этой тенденцией». Однако в большинстве случаев — особенно в контексте крупного предприятия — выбор технологий, позволяющих использовать API, таких как API-шлюз или сервисная сеть, в значительной степени станет решением типа 1. Убедитесь, что ваша организация действует соответствующим образом!

Подготовка к будущему

При подготовке книги мы, естественно, фиксируем опыт и знания на определенный момент времени. Но постоянно появляются новые разработки. Далее представлены три темы, которые, по нашему мнению, пока не заслуживают отдельного раздела, но тем не менее им необходимо уделить внимание с точки зрения их будущего влияния на архитектуру API.

Асинхронное взаимодействие

Асинхронные API очень популярны и в целом делятся на две категории: клиент-сервер и клиент-брокер. Пример отношений клиент-сервер достигается с помощью таких технологий, как gRPC, а отношения клиент-брокер — с помощью промежуточных технологий, таких как Kafka. В *главе 1* вы узнали, что спецификации OpenAPI сыграли решающую роль в последовательном описании и спецификации REST API.

AsyncAPI⁶ — это интересный и развивающийся стандарт, призванный обеспечить спецификацию асинхронных API. Одной из проблем асинхронных API стала поддержка разнообразных форматов протоколов и различных технологий. За этим, безусловно, стоит следить, поскольку популярность событийно-ориентированных архитектур и необходимость определения обменов — это быстро растущее пространство.

⁵ См. <https://oreil.ly/7b3Mt>.

⁶ См. <https://www.asyncapi.com/>.

Протокол HTTP/3

HTTP/3⁷ — это наряду с HTTP/1.1 и HTTP/2 третья основная версия протокола передачи гипертекста, используемого для обмена информацией во Всемирной паутине. Семантика HTTP согласована между версиями — одни и те же методы запроса, коды состояния и поля сообщений, как правило, применимы ко всем версиям. Различия заключаются в отображении этих семантик на базовые транспортные протоколы. И HTTP/1.1, и HTTP/2 в качестве транспортного используют протокол TCP (как в TCP/IP). В HTTP/3 используется QUIC — сетевой протокол транспортного уровня, использующий UDP. Переход на QUIC призван устранить основную проблему HTTP/2, называемую «блокировкой начала строки». Она, в частности, затрагивает сайты, требующие загрузки нескольких ресурсов.

HTTP/3 обещает большой прирост скорости, но поскольку базовый транспортный протокол изменился, его внедрение потребует модернизации входных прокси и других сетевых компонентов. К хорошим новостям можно отнести тот факт, что на момент подготовки книги более чем 70% работающих веб-браузеров уже поддерживали HTTP/3.

Сеть на основе платформы

Как уже говорилось в *главе 4*, многие сигналы указывают на то, что сервисные сети будут объединены с современными платформенными предложениями. Если эта тенденция сохранится, то, возможно, будет целесообразно внедрить сеть, интегрированную в технологический стек выбранного вами поставщика. Точно так же, как большинство организаций, внедряющих облачный стек Kubernetes, не заменяют контейнерные (OCI) или сетевые (CNI) реализации, в будущем то же самое может произойти и с межсервисными коммуникациями. В рамках этой эволюции мы рекомендуем следить за смежными стандартами в этой области, такими как Service Mesh Interface (SMI)⁸. Появление и внедрение надежных интерфейсов, безусловно, указывает на придание однородности этому слою коммуникационного стека.

Что дальше: как продолжать изучать архитектуру API?

Мы начали эту книгу с упоминания о том, что все авторы вступили на путь, приведший к ее созданию, в феврале 2020 года на конференции O'Reilly Software Architecture Conference (SACON). Мы все любим учиться, и посещение мероприятий стало важной частью нашего постоянного стремления к приобретению новых навыков и знаний. Один из наиболее часто задаваемых нам общих вопросов касается того, как мы подходим к изучению и экспериментированию с новыми технологиями. В этом разделе главы мы поделимся с вами нашими практиками, знаниями и привычками.

⁷ См. <https://oreil.ly/g7jaE>.

⁸ См. <https://smi-spec.io/>.

Постоянное совершенствование базовых знаний

Мы все считаем, что жизненно важно постоянно обращаться к основам любого желаемого навыка. В особенности это относится к такой подверженной причудам отрасли, как разработка и эксплуатация программного обеспечения. Мы упомянем несколько мест, где можно искать такие знания, но хотим подчеркнуть, что при просмотре сайтов, чтении книг, посещении конференций и т. п. мы постоянно ведем поиск существующего и новейшего освещения фундаментальных вопросов. Например, на многих архитектурных конференциях организуются сессии, посвященные таким темам, как связность и связанность. Узнав там об этих концепциях и вникнув в них, мы потом принесли эти знания в свои рабочие офисы и сразу же применили. Помимо чтения новейших книг по облачным платформам, мы также не чураемся новых взглядов на традиционные темы, выраженных, например, в книге Грегора Хохпе «The Architect Elevator»⁹.

В нашей отрасли очень часто встречается клише: «новое — это хорошо забытое старое», и повторяющиеся в несколько иных формах циклы технологий могут быть гораздо более эффективно пройдены архитекторами-дизайнерами, не забывающими о базовых принципах.

Следите за новостями в отрасли

Мы рекомендуем составить и постоянно пополнять список сайтов и социальных сетей, освещающих последние новости в области архитектуры и API:

- ◆ InfoQ;
- ◆ Dzone;
- ◆ The New Stack;
- ◆ Software Architecture Reddit¹⁰.

Чтение этих и подобных ресурсов раз в неделю поможет вам не упустить возникновения новых тенденций и технологий. Возможно, вы захотите их изучить.

Помимо этих новостных сайтов и агрегаторов, вы также можете найти и другие организации или отдельных людей, пишущих полезные блоги на интересующие вас темы. Использование программы чтения RSS (например, Feedly) позволяет собирать эти источники и просматривать их на еженедельной или ежемесячной основе. Twitter также может стать мощным инструментом для получения информации и комментариев о появляющихся технологиях. Следить за публикациями специалистов, имеющих схожие интересы и участвующих в проектах с открытым исходным кодом, — хороший способ заблаговременно узнавать о новых возможностях.

⁹ См. <https://tinyurl.com/2vahkmzm>.

¹⁰ См. <https://oreil.ly/1c3K7>.

Радары, квадранты и трендовые отчеты

Хотя всегда следует проводить собственные эксперименты по проверке различных концепций, мы также рекомендуем следить за конкретными технологическими тенденциями с помощью аналитических сайтов. Этот тип контента может быть особенно полезен, когда вы боретесь с проблемой или уже нашли решение и нуждаетесь в конкретной технологии, предлагаемой многими поставщиками. Мы рекомендуем следующие источники информации для ознакомления с современным уровнем развития технологий в области API:

- ◆ ThoughtWorks Technology Radar¹¹;
- ◆ Gartner Magic Quadrant for Full Life Cycle API Management¹²;
- ◆ Cloud Native Computing Foundation (CNCF) Tech Radar¹³;
- ◆ InfoQ Trends Reports¹⁴.

Некоторые организации и частные лица также периодически публикуют таблицы сравнения технологий. Они могут быть полезны для простой «оценки на бумаге» при составлении короткого списка продуктов, выбираемых для экспериментов. Само собой разумеется, что при проведении таких сравнений необходимо проверить их на объективность (поставщики часто спонсируют подобные исследования) и убедиться в том, что дата публикации является относительно недавней.

Изучение лучших практик и примеров использования

Мы также рекомендуем постоянно следить за чужими лучшими практиками и примерами использования, связанными с выполняемой вами работой. Многие организации любят рассказывать о том, почему, что и как они делают. Важно понимать мотивы такого поведения, поскольку часто это сочетание альтруизма, хвастовства, информирования о продажах и рекрутинга. При изучении чужих примеров всегда следует проявлять осторожность, т. к. большинство из них создаются для формирования положительной направленности, в результате чего могут не быть описаны первые неудачные попытки, ситуации, когда что-то пошло нештатно или до сих пор работает не так, как задумано. Тем не менее подобный контекст может позволить вам соотнести чужие проблемы и решения с вашей организацией и командой. Иногда это может подтвердить правильность выбранного технологического стека или подхода, а в других случаях — заставить задуматься!

Варианты использования и лучшие практики часто можно найти как в форме публикаций, так и в формате презентации. Как правило, мы рекомендуем искать и то и другое, а преимущество презентаций на конференциях заключается в том, что после выступления можно пообщаться с докладчиками, чтобы узнать больше! Далее приводится список конференций, в которых мы регулярно участвуем:

¹¹ См. <https://oreil.ly/YHIRs>.

¹² См. <https://oreil.ly/HwvO5>.

¹³ См. <https://oreil.ly/G0B7a>.

¹⁴ См. <https://oreil.ly/aGY4Y>.

- ◆ QCon conference series¹⁵;
- ◆ CraftConf¹⁶;
- ◆ APIDays¹⁷ (основная тема — API);
- ◆ KubeCon¹⁸ (для специалистов по платформам);
- ◆ DevOxx¹⁹ / JavaOne (на определенной языковой базе);
- ◆ O'Reilly online events²⁰.

Обучение на практике

Мы считаем, что архитекторы должны оставаться практикующими инженерами-программистами. Возможно, вы не будете каждый день выводить код в производственную среду, но мы рекомендуем выделить в своем графике время, чтобы периодически работать в паре с инженером из вашей команды или проводить исследования и эксперименты с новейшими технологиями. Если не поступать так регулярно, то взаимоотношения архитектора и разработчиков могут серьезно осложниться. Выполнение этой работы также позволяет выявить новые точки трения или трудности, способные возникнуть в результате внедрения новых технологий. Например, мы заметили, что многие архитекторы изначально не понимали, какое влияние окажет контейнерная технология на инструментарий разработчика. И если у вас не будет опыта создания образов контейнеров и отправки их в удаленный реестр, вы не сможете учитывать влияние этих действий на ваш ежедневный рабочий процесс создания и поддержки API.

Обучение через преподавание

Надеемся, что из этой книги вам стало понятно, что мы также многому учимся в процессе преподавательской работы. Будь то написание книг, ведение курсов или выступление на конференциях, ничто не сравнится с опытом сбора информации, необходимой для преподавания той или иной концепции. Слишком часто в ходе этого процесса мы осознаем, что не совсем понимаем ту или иную концепцию, или, задав вопрос студенту, вдруг осознаем, что в наших знаниях есть пробел.

На наш взгляд, преподавание является одной из основных ролей архитектора. Что бы это ни было — обучение разработчиков основам или обмен передовым опытом — преподавание будет постоянно укреплять ваши навыки и повышать ваш авторитет в коллективе.

Желаем удачи на пути освоения архитектуры API.

¹⁵ См. <https://oreil.ly/YW6pB>.

¹⁶ См. <https://oreil.ly/trY4Y>.

¹⁷ См. <https://oreil.ly/eRSFK>.

¹⁸ См. <https://oreil.ly/fo6gU>.

¹⁹ См. <https://oreil.ly/1LSss>.

²⁰ См. <https://oreil.ly/rQKIk>.

Предметный указатель

6

6R (шесть стратегий миграции в облако)
257

A

Access-токены 217
API 26
API контроллер 28
API-First 26
API-менеджмент (APIM) 104
API-шлюз 96
◊ микросервисов 113
Application Delivery Controllers, ADC 109
Architecture Decision Record, ADR 25
Authorization Code Grant 223
Authorization Code Grant + PKCE 225

B

Behavior Driven Development, BDD 88
Broken Object Level Authorization, BOLA
231

C

Call for Papers (CFP) 30
Client Credentials Grant 227
Consumer-Driven Contracts, CDC 76
Content Delivery Network, CDN 103
Cross-Origin Request Sharing, CORS 207
Custom Resource Definition, CRD 178

D

Data Access Object, DAO 80
Data Flow Diagram, DFD 193
DeMilitarized Zone, DMZ 52

Denial of Service, DoS 139
Disaster Recovery and Business Continuity,
DR/BC 202
Domain-Driven Design, DDD 241

E

eBPF 150
Enterprise Service Bus, ESB 125

F

Function as a Service, FaaS 107

G

General Data Protection Regulation, GDPR 192
gRPC без прокси 148
GraphQL 45
gRPC 45

H

HTTP/2 60
HTTP/3 60

J

JavaScript Object Notation, JSON 218
JSON Web Encryption (JWE) 220
JSON Web Signatures (JWS) 220
JSON Web Token 218
JWT ID 220

K

Key Performance Indicators, KPI 104

M

Multi-Factor Authentication, MFA 213

N

Network Access Control List, NACL 139

O

OAuth2 217

Open Web Application Security Project,
OWASP 190

OpenAPI Specification (OAS) 51

OpenID Connect (OIDC) 232

P

Personally Identifiable Information, PII 47

Proof Key for Code Exchange, PKCE 225

Proxyless gRPC 148

R

Refresh-токены 227

Remote Procedure Call, RPC 44

REpresentation State Transfer, REST 42

Role Based Access Control, RBAC 231

S

Security Assertion Markup Language, SAML
234

Security Group, SG 139

Service-Level Agreement, SLA 119

Service-Level Indicators, SLI 65

Service-Level Objective, SLO 65, 119

Service-Oriented Architecture, SOA 241

Sidecar 131

Single Page Application, SPA 224

Site Reliability Engineering, SRE 183

Small-Medium Businesses, SMB 97

Software Development Kit, SDK 127

Software-Defined Network, SDN 139

T

Test-Driven Development, TDD 70

U

Unified Modeling Language, UML 33

User Interface, UI 30

W

Web Application Firewall, WAF 103

A

Архитектура 25

◊ зональных сетей 263

Асинхронные API 278

Аудитория 219

Аутентификация 213

Б

Балансировщик нагрузки 93

Брокер 78

В

В процессе, вызов API 27

Веб-токен JSON 218

Вне процесса, вызов API 27

Внедрение полезной нагрузки 199

Г

Группа безопасности 139

Д

Демилитаризованная зона, ДМЗ 52

Диагностический журнал 185

Диаграмма потоков данных 193

Ж

Журналы 182

З

Заданная цель уровня обслуживания 65

Записи архитектурных решений 25

Запросы без сохранения состояния 42

Зеркалирование трафика 176

И

Индикатор уровня обслуживания 65

Интеграционные тесты 82

К

Канареечный релиз 174

Квадрант тестирования 67

Ключевые показатели эффективности 104

Контрактное тестирование 72
 ◊ производителей 75
 Контракты, ориентированные
 на потребителя 76
 Контроллеры доставки приложений 109
 Контроль трафика 139

М

Маршрутизация на основе полезной
 нагрузки 118
 Массовое переназначение 200
 Межсервисный стиль взаимодействия 32
 Межсетевой экран веб-приложений 103
 Метаданные 42
 Метод RED 183
 Метрики 182
 Микросервисы 242
 Многослойные API 251
 Многофакторная аутентификация 213
 Моделирование угроз 194
 Модель
 ◊ C4 33
 ◊ DREAD 208
 ◊ STRIDE 197
 ◊ безопасности с нулевым доверием 265
 Модульные тесты 70
 Монолит 241

Н

Нарушенные авторизации на уровне
 объекта 231
 Неверное конфигурирование системы
 безопасности 206
 Некорректное управление активами 203

О

Области видимости OAuth2 229
 Общий регламент по защите данных 192
 Ограничение количества запросов 204
 Одностраничные приложения 224
 Отказ в обслуживании 197
 Открытый проект по безопасности
 веб-приложений 190
 Отрицание 197

П

Паттерн
 ◊ Душитель 249
 ◊ Адаптер 250
 ◊ Фасад 250
 Передача репрезентативного состояния 42
 Периферийный стек 98
 Персональные данные 47
 Пирамида тестирования 69
 Плоскость данных 96
 Подмена 197
 Политика
 ◊ fail open 205
 ◊ fail closed 205
 Полный прокси-сервер 132
 Пользовательское определение ресурса 178
 Предметно-ориентированное
 проектирование 241
 Придание формы трафику 138
 Программно-определяемые сети 139
 Прокси-сервер 96
 Протокол HTTP/3 279

Р

Разглашение сведений 197
 Разделение трафика 139
 Разработка
 ◊ через поведение 88
 ◊ через тестирование 70
 Расширение полномочий 198
 Регистрирующий журнал 185

С

Сброс нагрузки 205
 Семантическое версионирование 55
 Серверы-заглушки 82
 Сервисная сеть 130
 Сервисная шина предприятия 125
 Сервисные тесты 70
 Сервис-ориентированная архитектура 241
 Сеть доставки содержимого 103
 Сине-зеленое развертывание 177
 Сквозное тестирование 87
 Слоеный пирог API 251, 252
 Совместное использование ресурсов
 между разными источниками 207
 Соглашения об уровне обслуживания 119
 Спецификации OpenAPI 51

Списки контроля сетевого доступа 139
Список OWASP API Security Top 10 190
Спуфинг 197
Срок действия 219
Стандарт RFC-2119 47
Стандарты ошибок 49
Субъект 219
Субъективная платформа 186

Т

Терминирование TLS-соединений 207
Тестирование компонентов 79
Тесты пользовательского интерфейса 70
Техника «почкования» 253
Традиционный корпоративный API-шлюз 112
Трассировка 182
Трафик север-юг 31
Трафик-менеджмент 32

У

Удаленный вызов процедур 44
Унифицированный язык моделирования 33
Управление доступом на основе ролей 231
Усиление директив безопасности 208

Ф

Флаги функций 168
Функциональная архитектура 243
Функция
◊ как услуга 107
◊ приспособленности 244

Ц

Цели уровня обслуживания 119
ЦОД 30

Ч

Чрезмерное раскрытие данных 202

Ш

Шлюз сервисной сети 113
Шов системы 247

Э

Эволюционная архитектура 26
Эвристика зрелости Ричардсона 43
Эмитент 219

Я

Язык разметки декларации безопасности
234

Джеймс Гоф — известный инженер компании Morgan Stanley, занимающийся архитектурой и программами API. Он является чемпионом Java, входил в исполнительный комитет Java Community Process от имени лондонского Java-сообщества и участвовал в разработке OpenJDK. Джеймс также выступил в роли соавтора книги «Java. Оптимизация программ» и любит рассказывать об архитектуре и низкоуровневом Java.

Дэниэл Брайант — руководитель отдела по связям с разработчиками в Ambassador Labs. В вопросах выбора профессии он придерживается философии покемонов: «надо поймать их всех», в соответствии с чем он работал и в академической среде, а также был разработчиком, архитектором, инженером платформы, консультантом и техническим директором. Его технический опыт сосредоточен на инструментах DevOps, облачных/контейнерных платформах и внедрении микросервисов. Дэниэл является чемпионом Java и участвует в нескольких проектах с открытым исходным кодом. Он также пишет для InfoQ, O'Reilly и The New Stack, регулярно выступает на международных конференциях, таких как KubeCon, QCon и DevOxx. В свободное время он любит бегать, читать и путешествовать.

Мэтью Обери работал в компании Morgan Stanley над различными финансовыми системами. До Morgan Stanley он занимался созданием различных мобильных и веб-приложений. Магистерская диссертация Мэтью в основном была посвящена вопросам безопасности, что и поспособствовало его успешной работе в области безопасности при создании API.

Об изображении на обложке

Животное на обложке данной книги — это ящерица-броненосец (*Ouroborus cataphractus*), называемая также малым поясохвостом, поскольку ранее ее относили к роду *Cordylus* (поясохвосты).

Малые поясохвосты обитают в пустыне на западном побережье Южной Африки. По внешнему виду они напоминают миниатюрных драконов: светло- или темно-коричневая чешуя и желтое брюшко с черными узорами. Длина их тела без учета хвоста обычно составляет от 7,5 до 9 см.

Они живут группами и активны в течение дня, хотя большую часть времени проводят, принимая солнечные ванны. Их рацион состоит в основном из мелких насекомых (главным образом термитов), а зимой они впадают в частичную спячку. В отличие от большинства ящериц, откладывающих яйца, поясохвосты рожают живых молодых особей по одной или две примерно раз в год. Самки также могут кормить своих детенышей, что является еще одним примером необычного для ящериц поведения. Чтобы защититься от хищников, они сворачиваются в клубок и берут хвост в рот.

Природоохранный статус поясохвостов — «почти под угрозой». Как и все представители животного мира, изображенные на обложках книг издательства O'Reilly, они жизненно важны для нашей планеты.

Иллюстрация на обложке выполнена Карен Монтгомери на основе черно-белой гравюры из Музея естественной истории.

Джеймс Гоф, Дэниэл Брайант, Мэтью Оберн
Проектирование архитектуры API

Перевод с английского М. Райтмана

ТОО "АЛИСТ"
010000, Республика Казахстан,
г. Астана, пр. Сарыарка, д. 17, ВП 30

Подписано в печать 06.05.24.
Формат 70×100^{1/16}. Печать офсетная. Усл. печ. л. 23,22.
Тираж 1300 экз. Заказ № 9433.

Отпечатано с готового оригинал-макета
ООО "Принт-М", 142300, РФ, МО., г. Чехов, ул. Полиграфистов, д. 1

Проектирование архитектуры API

Большинству организаций, имеющих сайт или сервис в Интернете, приходится осваивать работу с API — программными интерфейсами приложений. API — это дверь, через которую пользователь находит путь к работе с сервисами и приложениями компании. Проектирование, выстраивание этих программных компонентов, а также управление ими касается всей организации, от разработчиков и администраторов до совета директоров и владельцев продукта. Но по-настоящему серьезный вызов для разработчиков и архитекторов решений — это необходимость собирать новый API с нуля.

В этой прикладной книге изложены стратегии создания REST API, рассказано, как на уровне шлюзов API сочетать самые разные функции, komponуя их в виде микросервисов. Авторы книги описывают, как небольшая корректировка инфраструктуры помогает перевести весь сайт или ресурс в облако и заново связать внутренние сервисы при помощи таких инновационных технологий, как сервисная сеть.

- Изучите основы работы с API и архитектурные паттерны, позволяющие собрать API и платформу с нуля
- На практических примерах разберитесь, как проектировать, собирать и тестировать системы на основе API
- Развертывайте, используйте, конфигурируйте ключевые компоненты API
- Пользуйтесь шлюзами API и сервисными сетями там, где они нужны
- Внедрите в компании ключевые практики обеспечения безопасности и научитесь находить в архитектуре API наиболее распространенные уязвимости
- Защищайте данные и API при помощи моделирования угроз, задействовав такие технологии, как OAuth2 и TLS
- Научитесь развивать существующие системы, подготавливая их на уровне API к миграции в облако

«Сегодня такое внимание уделяется контейнерам и микросервисам, что зачастую забываются фундаментальные вопросы, касающиеся базовой коммуникации сервисов. Данная книга заполняет именно этот пробел и подробно рассказывает, как правильно структурировать и развивать ваши API».

— Сэм Ньюмен,
автор книги «От монолита
к микросервисам»

«Книга отлично написана, полна советов, подсказок и практических примеров».

— Стефания Чаплин,
GitLab&DevStefOps

Джеймс Гоф — заслуженный инженер в компании Morgan Stanley, Java-чемпион, соавтор книги «Optimizing Java».

Дэниэл Брайант — глава отдела по взаимодействию с разработчиками в компании Ambassador Labs, Java-чемпион. Эксперт в инструментарии DevOps, облачных и контейнерных платформах, микросервисах.

Мэтью Оберн — вице-президент компании Morgan Stanley. Занимался разработкой финансовых систем, мобильных и веб-приложений, обеспечением безопасности API.

ISBN 978-601-09-5053-5



9 786010 950535